

PureMeTTa

Architecture for MeTTa-Pure, Elaborated MeTTa-Core,
Runtime Execution, and Overlap Certificates

Zar Oruži*

March 2026

Abstract

This document records the current MeTTa architecture as formalized in Lean 4. The central claim is that the project should converge to a single elaborated MeTTa-Core that compiles to two honest targets: a trusted proof-oriented kernel (MeTTa-Pure / PureKernel) and an efficient runtime execution seam (MORK/MM2-backed RuntimeExec).

The document is anchored in the code: every key claim cites the Lean file and theorem name where it is proved. The current proof-side center is no longer just an abstract certificate interface: it is a small closed-fragment semantic waist with canonical normalization by complete development, bidirectional checking, and parser-free checked canonical evaluation with preservation.

We describe the current proven overlap (artifact-only for Pure, direct execution for PeTTa/MM2-backed fragments), the runtime kernel classes (exec + query + space effect + oracle + meta), the restricted Pure certificate lane, the current ordinary-family spec/interface stack, and the concrete next steps toward a genuinely dual-target fragment.

Contents

1 Thesis	4
2 Current Architecture	5
2.1 System Diagram	5
2.2 The Proof Branch	5
2.3 The Current Closed Semantic Waist	6
2.4 What “Kernel Binders” Means Here	8
2.5 The Runtime Branch	8
2.6 Current Overlap Fact	9
3 Why the Current Fork Is Not Enough	10
4 Elaborated MeTTa-Core	10
4.1 Base Infrastructure	10
4.2 Current Proof-of-Concept Elaborator	10
4.3 What Should Change Next	11

*“Oruži” denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI).

5	Current Proven Overlap vs. Intended Overlap	11
5.1	Pure Overlap: Artifact-Only	11
5.2	Runtime Overlap: Direct Execution for PeTTa/MM2 Fragments	11
5.3	The Gap	12
5.4	Next Objective	12
6	Runtime Kernel Classes	12
6.1	Exec (Match-Rewrite)	12
6.2	Query	13
6.3	Space Effect	13
6.4	Oracle	13
6.5	Meta	13
7	Restricted Pure Certificate Lane	14
7.1	Surface Language	14
7.2	Embeddings	14
7.3	Certificates	14
7.4	Current Scope and Limitations	14
7.5	Judgment-Typed Checked Imports	15
8	Ordinary Families: Current Declaration Checker	15
8.1	Declaration Environment and Signature Discipline	15
8.2	Non-Parametric Checker	15
8.3	Parametric Checker and Telescope Operations	16
8.4	Honest Status	17
9	Why Legacy MeTTaFull Is Not the Primary Runtime Source	17
10	Resources as Generalized Spaces	17
11	The Pure Side	18
11.1	Current Status	18
11.2	Why Inductives Enter Through Declaration Checking, Not Term Constructors	18
12	How This Relates to Lean/F*/MetaRocq	18
13	Current Frontier Facts	19
14	Current Runtime-Real PeTTa Fragments	19
14.1	Rule Lane	19
14.2	Query Lane	19
14.3	Space-Effect Lane	20
14.4	WM Landing	20
14.5	Still Outside Current Proved Runtime Core	20
15	GodelClaw, VeriCore, and Why This Matters	20
16	Concrete Next Steps	20
17	Overall Vision and Road Map	21

18 Pros and Cons of the Main Routes	22
18.1 Fork-only architecture	22
18.2 Elaborated-core architecture	22
18.3 Runtime-first elaboration	22
18.4 Proof-first elaboration	22
19 Current Working Slogan	22
20 Open Technical Questions	22
21 Recursor Generation: Design Constraints and Pitfalls	23
21.1 Two immediate hygiene warnings	23
21.2 Recursor type generation: algorithmic shape	23
21.3 Indexed families: what changes in the checker	24
21.4 Substitution	24
21.5 Declaration semantics: iota rules belong in a separate layer	24
21.6 Proof strategy for generated declarations	25
22 Status Note	25
23 Hyperon Experimental (HE) Fragment Status	26
23.1 Three-Layer Fragment Hierarchy	26
23.2 Key Theorems	26
23.3 Additional HE Infrastructure	26
23.4 Relation to PeTTa	26
24 MeTTaIL as Shared Substrate	26
24.1 Core Formalization	27
24.2 Role in the Architecture	27
25 Runtime Reality: Spaces, Bindings, and Modularity	27
25.1 Current Runtime Space Models	27
25.2 Dynamic Binding Propagation in HE Semantics	28
25.3 Why Dynamic Bindings Matter Architecturally	29
25.4 Explicit Communication Boundaries	29
25.5 Why the Pure-Kernel Direction Helps	29
25.6 Near-Term Pragmatic Recommendation	30
A Appendix: Full Current PureKernel Listing	30
A.1 AlgorithmicTyping.lean	30
A.2 BoolDecl.lean	33
A.3 Confluence.lean	39
A.4 Context.lean	45
A.5 DeclarationEnv.lean	47
A.6 DeclarationSemantics.lean	49
A.7 DeclarationSpec.lean	81
A.8 CoreEmbedding.lean	88
A.9 DefEq.lean	99
A.10 Inst0BridgeDerived.lean	101

A.11 Inst0BridgeProof.lean	101
A.12 InductiveDecl.lean	181
A.13 NatDecl.lean	185
A.14 Parallel.lean	190
A.15 PatternBridge.lean	195
A.16 PatternBridgeSubst.lean	221
A.17 ProfileTheory.lean	222
A.18 RecursorDecl.lean	225
A.19 Reduction.lean	235
A.20 Renaming.lean	238
A.21 SubjectReduction.lean	241
A.22 Telescope.lean	248
A.23 Substitution.lean	253
A.24 Syntax.lean	261
A.25 UnitDecl.lean	262
A.26 TypedLangDef.lean	266
A.27 Typing.lean	267

1 Thesis

The working thesis is:

One surface MeTTa should elaborate to one shared MeTTa-Core, which then lowers to two distinct but connected targets:

1. a proof-oriented target: MeTTa-Pure / PureKernel;
2. a runtime target: MeTTaIL artifacts / RuntimeExec / MORK/MM2 / Rust execution.

This is intended to be closer to the Lean-style pattern

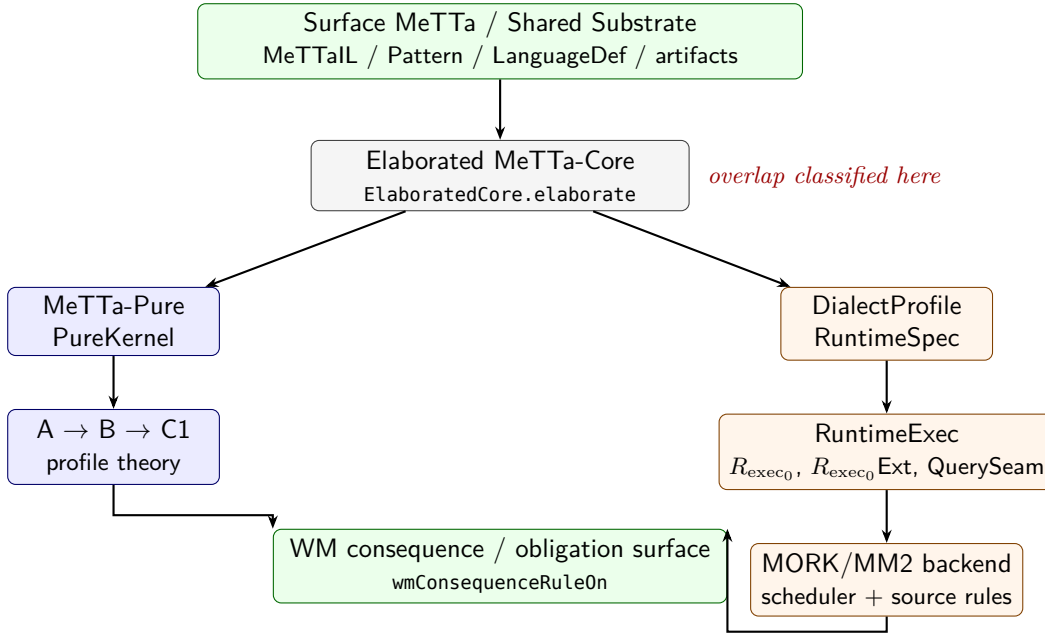
surface language $\rightarrow$ elaborated core $\rightarrow$ checked kernel and compiled runtime

than to either of the following extremes:

1. runtime *is* the proof kernel;
2. proof language and runtime language stay permanently separate and only meet in a semantic backstop such as world-model consequence.

2 Current Architecture

2.1 System Diagram



The important present facts are:

1. MeTTa-Pure and PureKernel are real and theorem-level strong, and the current closed proof-side waist is now packaged as a checking / normalization / evaluation service.
2. RuntimeExec is real and theorem-level strong over the current MORK/MM2 seam.
3. The current strongest shared semantic meeting point is the WM consequence / obligation surface.
4. The elaborated core classifies overlap honestly:
OverlapClass.artifactOnly vs. OverlapClass.directExec.
5. This is honest, but not yet the desired end state.

2.2 The Proof Branch

The current proof branch is:

$$\text{MeTTa-Pure} \rightarrow \text{PureKernel} \rightarrow A \rightarrow B \rightarrow C1$$

where:

- **A** is the tiny closed executable kernel fragment (operational steps via pureOpStep);
- **B** is kernel theory reduction (profile theory via PureKernel.ProfileTheory);
- **C1** is the quoted profile-theory closure (quoteClosedTm bridge).

Key theorems.

- `pureClosedTheoryBridge_to_star` — one-step bridge lifts to reflexive-transitive closure by induction.
- `profileStepStar_sound` — star closure transports to WM inequalities.
- `pureTheoryStep_to_wmStrengthObligation_default` — `PureKernel` reduction $\rightarrow$ WM obligation via the canonical bridge.

Primary files.

- `Mettapedia/Languages/MeTTa/Pure/Core.lean` — `PureTm`, `mettaPure` dialect definition.
- `Mettapedia/Languages/MeTTa/PureKernel/CoreEmbedding.lean` — kernel embedding, profile theory.
- `Mettapedia/Languages/MeTTa/PureKernel/PatternBridge.lean` — quotation, `quoteClosedTm`.
- `Mettapedia/Logic/PLNWorldModelPureKernelBridge.lean` — `PureJudgmentWMInterface`, WM landing.

This branch is not aspirational. It is already theorem-level packaged with 0 sorry.

2.3 The Current Closed Semantic Waist

The current proof-side reference model should not be identified with the CLI-level fuel evaluator alone. The authoritative center is the closed Pure semantic waist exposed above `PureCheckingBoundary`.

Canonical normalization and definitional equality. `Mettapedia/Languages/MeTTa/PureNormalizationService.lean` packages the current closed reference normalization service around complete development `cdev`. The central object is `CanonicalClosedPureTerm`, constructed by `canonicalizeClosedPureTerm`. It carries:

- the input closed term,
- its canonical development,
- a `RedStar` reduction to that canonical development,
- a conversion proof `Conv` to that canonical development.

The same module also exposes the current closed service operations:

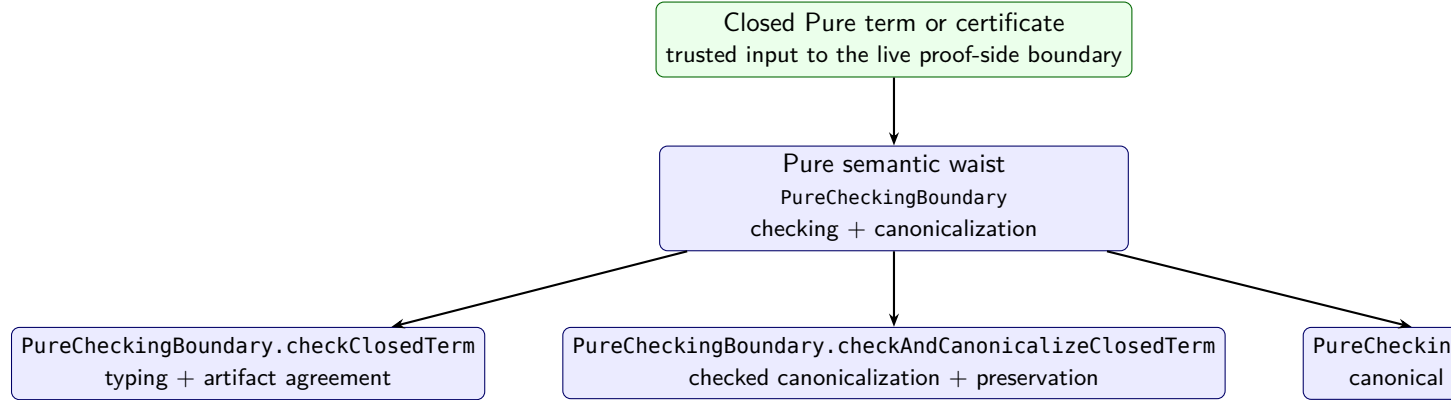
- `defEqClosed?`,
- `asPiClosed?`,
- `asSigmaClosed?`,
- boundary wrappers such as `PureCheckingBoundary.canonicalizeClosed`.

Bidirectional checking. `Mettapedia/Languages/MeTTa/PureBidirectionalChecking.lean` packages closed surface checking through `PureCheckSuccess`, `inferSurfacePure`, and `checkSurfacePure`. Parser-free checked canonical evaluation then lives in `Mettapedia/Languages/MeTTa/PureCanonicalEvaluation.lean`:

- `CheckedCanonicalEvaluation`,
- `PureCheckingBoundary.checkAndCanonicalizeClosedTerm`,
- `PureCheckingBoundary.checkAndEvalSurface`,
- `subjectReductionRedStar`.

Executable/canonical agreement. The executable stepper remains important, but now in the right role. In

Current service view. The current live Pure-facing proof services now land on the same semantic waist, but the live entry points are theoremic services rather than the archived prototype CLI tools:



This is now a deliberately parser-free proof-side service arrangement rather than a front-door evaluator. The archived prototype CLI was useful for exploration, but the live trusted path is the theoremic checking and canonicalization boundary itself.

Worked example. Consider the tiny closed Pure program represented directly as the closed kernel term `PureTm.app` of the identity lambda applied to `PureTm.u0`. The corresponding type judgment is the closed theorem that the input has type `Type1`.

```

input      := ((lambda x. x) Type0)
claimedType := Type1
inputTyping : HasType nil input claimedType
  
```

Applying `PureCheckingBoundary.checkAndCanonicalizeClosedTerm` to this input currently yields a theoremic record of the following shape:

```

input: ((lambda x. x) Type0)
claimedType: Type1
canonical-development: (Type0)
input-artifact: Pattern.apply "App" [...]
canonical-artifact: Pattern.apply "U0" []
proof: inputTyping : HasType nil input Type1
proof: outputTyping : HasType nil canonicalDevelopment Type1
proof: canonical-quoted-artifact-agreement
proof: profile-bridge
  
```

The key educational point is that `Type1` here is the *type of the input term*, not the evaluation result. The term evaluates to `Type0`, and the reason the check succeeds is that the current Pure kernel proves `Type0 : Type1`.

So even this tiny example already exhibits the live waist: closed checking, canonicalization by complete development, quotation into the shared artifact substrate, and profile-theory transport from the canonical reduction.

Why this already matters. This is enough to prevent the current Pure waist from being “only esoteric theory.” The present closed fragment already supports:

- trusted checking into `PureTm`,
- canonical normalization and definitional equality,
- parser-free checked canonical evaluation with preservation,
- quotation into the shared `MeTTaIL` artifact space.

What this *does not* yet mean is direct MM2/MORK operational overlap. The current sense in which it is already a strain of MeTTa is that the surface syntax, artifact substrate, and elaborated-core story are shared, while the proof-side operational account remains an honest closed-fragment kernel waist.

2.4 What “Kernel Binders” Means Here

The phrase “kernel binders” is doing important work in this paper. In the current Pure fragment, the binding forms belong to the trusted internal term language itself:

- Π for dependent function types,
- Σ for dependent pair types,
- `lam` for lambda abstraction.

These are not runtime closures or ad hoc higher-order macros. They are constructors of `PureTm` with typing, reduction, substitution, and conversion rules in the kernel theory. The preferred surface syntax (`lambda $x body`) is therefore a user-facing presentation of a real kernel binder, not a disguised `apply/let` encoding.

This distinction matters when comparing Pure against existing MeTTa practice:

- **Pure kernel binders** are part of the trusted proof-side syntax and can participate in checked typing, canonical normalization, and subject reduction.
- **Runtime closure forms** such as PeTTa’s `|->` are operationally real and useful, but they are not currently the semantic center of the proof kernel.
- **Encoded “lambda” tricks** in some HE/older MeTTa usage are even further away: they may be practical higher-order surface devices, but they are not the same thing as kernel-level binders with theoremic typing and conversion.

So when this paper says that Pure is already a real strain of MeTTa, it does not mean “Pure already subsumes PeTTa operationally.” It means that Pure already contributes a real typed binder language, expressed in MeTTa-shaped surface syntax, that lands in a trusted kernel and shared artifact substrate.

2.5 The Runtime Branch

The runtime branch passes through three layers:

$$\text{DialectProfile} / \text{RuntimeSpec} \rightarrow \text{RuntimeExec} \rightarrow \text{MORK/MM2 seam}$$

Three runtime surfaces. The central type is `MeTTaRuntimeExecSurface` (`RuntimeExec.lean`), a structure with 8 fields including an *embedded proof*:

- `noPremiseBridge : RuntimeExecNoPremiseSourceBridge` — proves that if a rule has `fvar`-headed LHS, translatable RHS, and no premises, then its template sinks fire correctly at the MORK source-rule level.

Three canonical instances are constructed:

Instance	Name	Capability
R_{exec0}	morkRuntimeExec0	No-premise source-rule bridge
$R_{\text{exec0}} \text{Ext}$	morkRuntimeExec0Ext	Multi-premise + guarded bridge
QuerySeam	morkRuntimeQueryExec0	Query surface with <i>soundness</i> + <i>completeness</i> proofs

Query surface detail. `MeTTaRuntimeQuerySurface` carries bidirectional correctness:

- `sourceFactorSound` — computable results inject into spec-level results;
- `sourceFactorComplete` — spec-level results extract from computable results.

Primary files.

- `Mettapedia/Languages/MeTTa/RuntimeExec.lean` (269 lines) — surfaces, instances.
- `Mettapedia/Languages/ProcessCalculi/MORK/ExecutionBoundary.lean` (509 lines) — 160+ re-exported theorems including `fireCorrespondence`, `schedulerProgress`.
- `Mettapedia/Languages/ProcessCalculi/MORK/DirectExecSurface.lean` (326 lines) — composition package: `pettaEvalStep_schedulerFires`, `addAtomCmd_schedulerFires`.

2.6 Current Overlap Fact

The current honest overlap is *artifact-only*, not direct runtime execution.

Fact 2.1 (Pure/runtime frontier). *No mettaPure rewrite rule currently fits the direct R_{exec0} source-rule bridge.*

Lean: `no_mettaPure_rewrite_fits_direct_runtimeExec0_source_bridge` in `PureRuntimeFrontier.lean`.

The real current overlap path is:

closed Pure surface terms $\rightarrow$ PureKernel term $\rightarrow$ quoted / shared artifact

proved by:

- `closedPure_overlap_via_abc` — artifact-only overlap via quotation.
- `closedPure_overlap_via_abc_to_wm` — extends to WM consequence.
- `surfacePureClosed_overlap_is_not_directExec` (`PureCertificateFragment.lean`).
- `surfaceCoreTypedAtom_overlap_is_not_directExec` (`ElaboratedCore.lean`).

The current live proof-side services sharpen this further. Canonicalization by `cdev` is exposed through `PureCheckingBoundary.canonicalizeClosed`, and checked canonical evaluation is exposed through `PureCheckingBoundary.checkAndCanonicalizeClosedTerm`. This is *not* the same as a theorem that Pure evaluation is already direct MORK/MM2 execution. The current honest statement remains:

Pure closed terms have a real canonicalization service and a real shared-artifact overlap witness, but they do not yet have direct proof/runtime operational overlap.

3 Why the Current Fork Is Not Enough

The current fork is a good bootstrap, but not the desired final user story.

- Positive: it prevents us from lying about overlap.
- Negative: it makes the system feel too much like “proof MeTTa” versus “runtime MeTTa”.

The missing piece is a genuine:

Elaborated MeTTa-Core

that is stronger than WM-only unification and earlier than backend execution.

4 Elaborated MeTTa-Core

4.1 Base Infrastructure

The file `ElaboratedCoreBase.lean` (62 lines) defines the neutral vocabulary shared by proof and runtime views:

- `ElaboratedRegion` (inductive, 4 constructors): `pureKernelRegion`, `runtimeExecRegion`, `oracleRegion`, `metaRegion`.
- `SharedArtifact` (structure): a `Pattern` from the MeTTaIL syntax layer.
- `RuntimeLowering` (inductive, 3 constructors): `exec`, `query`, `auditOnly`.
- `OverlapClass` (inductive, 2 constructors): `artifactOnly` (current honest state), `directExec` (reserved for future).

4.2 Current Proof-of-Concept Elaborator

The file `ElaboratedCore.lean` (414 lines, 0 sorry) contains the PoC classifier.

Surface language. `SurfaceNode` is an inductive with 11 constructors:

Constructor	Region	Certificate type
<code>surfacePureClosed</code>	<code>pureKernel</code>	<code>PureCertificate</code>
<code>coreTypedAtom</code>	<code>pureKernel</code>	<code>CoreTypedCertificate</code>
<code>pettaCoreRule</code>	<code>runtimeExec</code>	<code>PettaCoreRuleRuntimeCertificate</code>
<code>pettaSpaceCoreQuery</code>	<code>runtimeExec</code>	<code>PettaSpaceCoreRuntimeCertificate</code>
<code>heRuntimeRule/Query</code>	<code>runtimeExec</code>	<code>RuntimeCertificate</code>
<code>pettaRuntimeRule/Query</code>	<code>runtimeExec</code>	<code>RuntimeCertificate</code>
<code>fullLegacyRuntime</code>	<code>runtimeExec</code>	<code>RuntimeCertificate</code>
<code>oracleCall</code>	<code>oracle</code>	<code>OracleCertificate</code>
<code>metaQuoted</code>	<code>meta</code>	<code>MetaCertificate</code>

Elaborated output. `ElaboratedNode` (inductive, 7 constructors): `pureNode`, `coreTypedNode`, `pettaCoreRuleNode`, `pettaSpaceCoreNode`, `runtimeNode`, `oracleNode`, `metaNode`.

Main elaborator. `elaborate : SurfaceNode → ElaboratedNode` maps all 11 surface types to their classified outputs.

Preservation theorems (all proved).

- 7 region-preservation theorems (e.g. `elaborate_surfacePureClosed_region`).
- 5 artifact-preservation theorems (e.g. `elaborate_surfacePureClosed_artifact`).
- Backend agreement: `runtimeBackendAgreement` — HE and PeTTa target the same MORK/MM2 backend.

4.3 What Should Change Next

The elaborated core should gradually stop naming runtime fragments by witness dialect and instead classify them by semantic role.

The likely neutral runtime-facing classes are:

1. `ruleExecNode` (match-rewrite)
2. `queryNode` (read-only space query)
3. `spaceEffectNode` (stateful mutation)
4. `oracleNode` (external/grounded)
5. `metaNode` (elaboration-time)
6. `pureNode` (proof-side)

[TODO: Refactor the current dialect-specific runtime constructors in `SurfaceNode` to these neutral classes.]

5 Current Proven Overlap vs. Intended Overlap

This section states the overlap situation precisely.

5.1 Pure Overlap: Artifact-Only

The three `mettaPure` rewrite rules are classified exactly by `mettaPure_rewrites_exact` (rfl proof):

Rule	LHS shape	RHS translatable?	Fits R_{exec_0} ?
BetaPi	apply-headed	No (<code>.subst</code>)	No
BetaSigmaFst	apply-headed	Yes (atom-only)	No
BetaSigmaSnd	apply-headed	Yes (atom-only)	No

No rule has an `fvar`-headed LHS (`mettaPure_rewrite_lhs_not_fvar`), and BetaPi uses `.subst` which is not `morkTranslatable` (`betaPi_rhs_not_morkTranslatable`). Therefore:

Fact 5.1 (No direct Pure $\rightarrow$ runtime bridge). *no_mettaPure_rewrite_fits_direct_runtimeExec0_source_bridge: no mettaPure rule satisfies both fvar-LHS and morkTranslatable-RHS simultaneously.*

Current Pure overlap goes through quotation: `closedPure_overlap_via_abc` $\rightarrow$ `closedPure_overlap_via_abc_to_wm`.

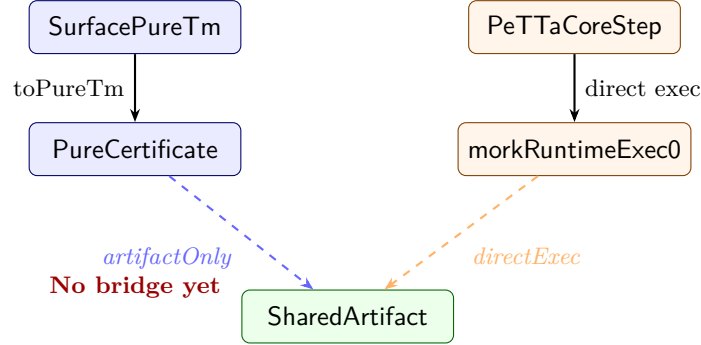
5.2 Runtime Overlap: Direct Execution for PeTTa/MM2 Fragments

Direct runtime execution *does* exist for the PeTTa/MM2-backed fragments:

- `evalStep_toMorkSourceFire` — core eval step lands on R_{exec_0} source-rule boundary.
- `evalcStep_toMorkSourceFire` — same for `evalc`.
- `pettaEvalStep_schedulerFires` — fires through the full MM2/MORK scheduler seam (3-phase refinement).

5.3 The Gap

Direct proof/runtime overlap is **not yet proved**. No surface fragment currently satisfies *both* PureKernel reduction *and* R_{exec_0} source-rule bridge simultaneously.

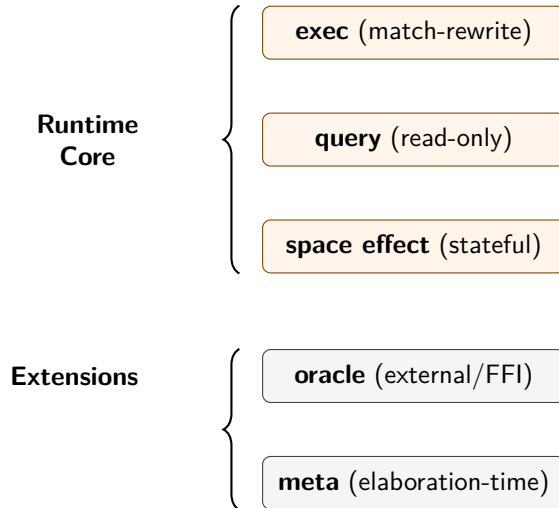


5.4 Next Objective

The next objective is a *genuinely dual-target fragment*: a small surface subset that elaborates to both a PureKernel certificate and a direct R_{exec_0} source-rule firing, sharing the same artifact. The likely candidate avoids `.subst` and uses symbolic atoms with top-rule rewrite patterns.

6 Runtime Kernel Classes

The runtime kernel should be understood not as “just exec” but as five semantic classes over one default space model:



6.1 Exec (Match-Rewrite)

The central runtime primitive is the scheduler/source rewrite seam over one space. The MORK/MM2 formalization covers:

1. find `exec` facts in the space (`execFactsOfSpace`);
2. pick one by scheduler order (`selectNextExec, WorkQueueOrder.lean`);
3. match its pattern against the read copy (`readCopy`);

4. apply template sinks to the live space (`fireExecFact, ThreePhaseRefinement.lean`).

Key theorems.

- `fireCorrespondence (ExecutionBoundary.lean) — DeclReduces → fireRule.`
- `schedulerProgress — selected exec fact ⇒ step exists.`
- `pettaEvalStep_schedulerFires, pettaEvalcStep_schedulerFires (DirectExecSurface.lean) — PeTTa steps fire through the full scheduler seam.`

6.2 Query

Queries are sibling primitives, not a disguised special case of `exec`. The currently formalized query seam includes:

- `(match &self $x tmpl) — anyFactMatch_toMorkSourceQuery, anyFactMatch_toComputableSourceQuery (SpaceCoreFragment.lean).`
- `(get-atoms &self) — getAtoms_toComputableSourceQuery.`

The query surface `morkRuntimeQueryExec0 (RuntimeExec.lean)` carries bidirectional correctness: soundness and completeness of computable matching against the spec-level semantics.

6.3 Space Effect

Space mutations are explicit, not proof-kernel rewrites:

- `(add-atom &self X) — addAtomSourceExecRule: template [.remove cmd, .add X, .add ()].`
- `(remove-atom &self X) — removeAtomSourceExecRule: template [.remove cmd, .remove X, .add ()].`

Proved at scheduler level. `addAtomCmd_schedulerFires, removeAtomCmd_schedulerFires (DirectExecSurface.lean) — commands fire through the full MORK/MM2 scheduler. Pattern matching round-trips via matchAtom_addAtomCmdAtom (SpaceEffectFragment.lean).`

6.4 Oracle

Grounded/FFI/resource calls are treated as *oracle* forms. The elaborated core already provides:

- `OracleCertificate (ElaboratedCore.lean) — fields: dialect, opName, resultDescriptor, args, artifact.`
- `Region: oracleRegion.`

Oracle calls are runtime-relevant and potentially effectful. They should remain explicit and not be silently treated as proof-kernel computation.

[TODO: Grow the oracle certificate lane toward explicit pre/post-condition contracts.]

6.5 Meta

Meta forms are elaboration-time/proof-time forms:

- `MetaCertificate (ElaboratedCore.lean) — fields: description, artifact.`
- `Region: metaRegion.`

These should stay separated from ordinary runtime execution.

7 Restricted Pure Certificate Lane

The Pure certificate lane is the first real dual-view fragment: a surface term maps to both a trusted PureKernel term and a shared MeTTaIL artifact, with proven agreement.

7.1 Surface Language

SurfacePureTm (PureCertificateFragment.lean, 191 lines) is an inductive parameterized by $n : \text{Nat}$ (de Bruijn depth) with 12 constructors:

Constructor	Meaning
var (Fin n)	de Bruijn variable
u0, u1	universe levels
pi, sigma	Π -type, Σ -type (bind with $n+1$)
id	identity type
lam	λ -abstraction (bind with $n+1$)
app, pair	application, pairing
fst, snd	projections
refl	reflexivity proof

7.2 Embeddings

Two maps out of SurfacePureTm:

1. `toPureTm : SurfacePureTm n → PureTm n` — structural embedding into the trusted kernel.
2. `toClosedPattern : SurfacePureTm 0 → Pattern` — quotation into the shared MeTTaIL artifact space.

Fact 7.1 (Quotation agreement). *toClosedPattern_eq_quoteClosedTm: the surface quotation agrees with the kernel quotation for all closed terms. Proved by structural induction over all 12 constructors (toPatternWith_eq_quoteTmWith).*

7.3 Certificates

- `PureCertificate` — fields: term (closed PureTm 0), artifact, abcSurface.
- `SharedPureOverlapCertificate` — the first real dual-view object: surface term, trusted Pure term, shared artifact, all proved to agree.
- `certifySurfacePure` — constructor that builds the overlap certificate from a surface term.
- `certifySurfacePure_overlapClass` — proves the result is artifactOnly.

7.4 Current Scope and Limitations

- ✓ Closed Pure terms (de Bruijn index 0).
- ✓ Closed type claims (Π , Σ , identity, universes).
- ✓ Explicit proof-relevant certificate objects.
- × No inductives yet.

- × No broad Lean-proof import yet.
- × No open terms (binder depth > 0).

This is the right first certificate/checking target: small enough to be auditable, rich enough to carry real type-theoretic content.

7.5 Judgment-Typed Checked Imports

The restricted lane is now stronger than a raw overlap wrapper. In `PureCertificateFragment.lean` the current checked import object is:

- `CheckedPureCertificate` — imported restricted certificate plus an explicit claimed closed type and an empty-context typing witness;
- `PureJudgmentKind` — currently `closedTyping` and `quotedArtifactAgreement`;
- `PureCertificateJudgment` — a judgment-typed view over a checked imported certificate.

The important theoremic point is that the proof lane now supports explicit judgment forms, not only ad hoc overlap objects:

- `PureCertificateJudgment.closedTyping_holds`,
- `PureCertificateJudgment.quotedArtifactAgreement_holds`.

This is still intentionally narrow: it is a real restricted import/check lane, but not broad Lean-proof import.

8 Ordinary Families: Current Declaration Checker

The kernel now contains the first real ordinary-family mechanism: a computable declaration checker that validates and emits `DeclSpecs` for ordinary strictly-positive inductive families, both non-parametric and parametric. This replaced a tower of interface scaffolding that had grown ahead of any real kernel implementation.

8.1 Declaration Environment and Signature Discipline

Three files establish the authoritative declaration layer:

- `DeclarationEnv.lean` — `DeclName`, `DeclEntry`, `DeclEnv`, lookup helpers `typeOf?` and `valueOf?`.
- `DeclarationSpec.lean` — `DeclSpec` (a closed name, type, and optional value), `envOfSpecs`, and the ordered signature discipline: `SignatureWellFormed`, `PrefixDeclSpecAdmissible`.
- `DeclarationSemantics.lean` — typed and well-formed declaration environments, `HasTypeDecl`, `RedDecl`, `DeclEnvWellFormed`.

The signature discipline ensures declarations are added in a well-founded order: every type and optional value in a spec may only refer to names already in the prefix.

8.2 Non-Parametric Checker

`InductiveDecl.lean` (205 lines, 0 sorry) provides the first Lean-`addInductive`-style checker for strictly-positive closed families:

- `CtorSpec` — a constructor name and closed type `PureTm 0`.
- `IndDecl` — a type name and list of `CtorSpecs`.
- Validation predicates: `notOccursConst`, `occursOnlyPositively`, `strictlyPositive`, `targetsFamily`, `namesDistinct` — all computable `Bool`.

- `checkIndDecl : IndDecl → Option (List DeclSpec)` — validates names, targets, and strict positivity; emits the type spec (type = U_0) plus constructor specs.
- Standard instances and their checker certificates (all proved by `decide`):

Value	Kind	Certificate
<code>unitDecl</code>	<code>IndDecl</code>	<code>check_unitDecl</code>
<code>boolDecl</code>	<code>IndDecl</code>	<code>check_boolDecl</code>
<code>natDecl</code>	<code>IndDecl</code>	<code>check_natDecl</code>

Agreement with the hand-written pilot specs is also kernel-checked (`unitDecl_specs_agree`, `boolDecl_specs_agree`, `natDecl_specs_agree`).

Negative examples (all by `decide`): duplicate names, negative occurrence, and wrong target each independently return `none`.

8.3 Parametric Checker and Telescope Operations

`Telescope.lean` (287 lines, 0 sorry) extends the checker to parametric families. The key insight is that `Ctx n` — already the canonical dependent context in `Context.lean` — is the right telescope type; no new inductive is needed.

Telescope operations.

- `telescopePi : Ctx n → PureTm n → PureTm 0` — wrap a body in Π -binders from a context.
- `telescopeLam` : same shape, produces λ -abstractions.

Checker predicates (all `Bool`, polymorphic in depth).

- `piAry` — count leading Π -binders.
- `resultIsU0` — after peeling k binders, body is U_0 .
- `matchingParamDomains` — first k Π -domains agree (by `BEq`).
- `isParamApp` — term is `const c` applied to consecutive de Bruijn variables.
- `targetsFamilyP` — after peeling all Π 's, head is the family applied to the parameter variables.

Parametric inductive declaration. `ParamIndDecl` stores the type-former type as a closed term (`typeFormerType : PureTm 0`) rather than a separate `numParams` field; the parameter count is derived as `piAry decl.typeFormerType`. This subsumes non-parametric families (`typeFormerType = U_0` gives `numParams = 0`).

Checker. `checkParamIndDecl : ParamIndDecl → Option (List DeclSpec)` validates names, type-former result, parameter domain agreement, target correctness, and strict positivity. The first parametric example — `List A` — is verified by `decide`:

Value	Certificate
<code>listDecl</code>	<code>check_listDecl</code>
<code>unitDeclP</code>	<code>check_unitDeclP</code>
<code>boolDeclP</code>	<code>check_boolDeclP</code>
<code>natDeclP</code>	<code>check_natDeclP</code>

Backward compatibility: `liftToParam` embeds every `IndDecl` into a `ParamIndDecl`, and the two checkers agree on the result (proved by `decide` for `Unit`, `Bool`, `Nat`).

8.4 Honest Status

The term syntax (`Syntax.lean`) is still: Π , Σ , Id , universes, λ , application, pairs, projections, and reflexivity. Declaration checking validates and emits `DeclSpec` entries; it does *not* add new term constructors for inductive types. Runtime overlap for these family declarations remains `artifactOnly`. Deliberately deferred to later kernel layers:

- recursor generation,
- indexed families (`Eq`, `Vec`, `Fin` — distinguishing uniform parameters from varying indices),
- induction-recursion,
- coinductives,
- structural fixpoints.

9 Why Legacy MeTTaFull Is Not the Primary Runtime Source

The `fullLegacyRuntime` constructor exists in `SurfaceNode` (one of 11 constructors), but it is *not* the primary runtime-core witness.

1. **It is a legacy spec-facing slice.** `RuntimeSpec` is deliberately dialect-facing and audit-facing: it records bindings, branching, context, native hooks, and collection control *without* defining execution machinery. It does not carry embedded proofs the way `MeTTaRuntimeExecSurface` does.
2. **Real execution witnesses are the PeTTa/MM2-backed fragments.** The files with embedded proof-carrying structures are:
 - `PeTTa/CoreFragment.lean` — `PeTTaCoreStep` (`eval/evalc` $\rightarrow$ `morkRuntimeExec0`).
 - `PeTTa/SpaceCoreFragment.lean` — `PeTTaSpaceCoreQuery` (`match/get-atoms` $\rightarrow$ `morkRuntimeQueryExec0`).
 - `PeTTa/SpaceEffectFragment.lean` — `add/remove` $\rightarrow$ scheduler seam.
3. **MeTTaFull is not the best current witness for runtime-core development.** It packages the whole dialect profile without isolating the operational kernel. The current architecture deliberately avoids treating it as the runtime definition.

10 Resources as Generalized Spaces

This section describes architectural direction, not proved theorems.

The runtime side should not stop at a single atomspace story. The right abstraction appears to be:

- `atomspace` = default resource;
- `resource` = generalized stateful/queryable backend.

All resources are classified under the existing runtime kernel classes (`query`, `effect`, `oracle`) rather than inventing a second runtime theory:

Resource kind	Dominant runtime classes	Examples
Atomspace	<code>exec</code> + <code>query</code> + <code>effect</code>	MORK, MM2, PeTTa atomspace
Map/dictionary	<code>query</code> + <code>effect</code>	hash maps, symbol tables
Queue/channel	<code>effect</code> + <code>query</code>	work queues, message buffers
Solver/service	<code>query</code> + <code>oracle</code>	Z3, ATP, external services

11 The Pure Side

11.1 Current Status

The Pure side is the trusted proof-oriented target, not the runtime. Current kernel ingredients:

- Π -types (dependent function)
- Σ -types (dependent pair)
- Identity types
- Universes (U_0 , U_1)
- The PureKernel reduction theory (`pureOpStep`)
- Declaration environment and signature discipline (`DeclEnv`, `SignatureWellFormed`)
- Computable ordinary-family checker (`checkIndDecl`, `checkParamIndDecl`)

11.2 Why Inductives Enter Through Declaration Checking, Not Term Constructors

The key architectural decision: ordinary families do *not* extend the `PureTm` term syntax. Instead they lower to `DeclSpec` entries via a computable checker. The checker validates strict positivity and target correctness but produces only the same generic constant declarations that any other kernel object uses.

This is exactly the current status:

- the declaration layer is live;
- non-parametric families (`Unit`, `Bool`, `Nat`) pass `checkIndDecl` and agree with hand-written pilot specs (kernel-checked by `decide`);
- the first parametric family (`List`) passes `checkParamIndDecl` (kernel-checked by `decide`);
- the next missing piece is recursor generation — emitting the eliminator `DeclSpec` from a validated `ParamIndDecl`.

So the next inductive step is *not* more interface wrapping; it is `generateRecursor` and then indexed families.

12 How This Relates to Lean/F*/MetaRocq

The architecture parallels the Lean compilation pipeline:

Role	Lean	MeTTa (this project)
Surface language	Lean 4	MeTTa surface
Elaborated middle	Lean elaborator	<code>ElaboratedCore</code>
Trusted proof target	Lean kernel	<code>PureKernel</code> (A/B/C1)
Efficient runtime	Lean compiler/IR	<code>RuntimeExec</code> / <code>MORK/MM2</code>
Overlap certificates	kernel + IR agree	<code>SharedPureOverlapCertificate</code>

Honest differences.

- MeTTa-Pure is currently much smaller than Lean’s kernel: Π , Σ , `Id`, universes, no inductives.
- `MORK/MM2` is not a general compiler but a pattern-rewrite engine over one space.
- Current overlap is artifact-only, not the deep kernel/IR agreement that Lean achieves.
- The elaborator is a PoC classifier (11 surface $\rightarrow$ 7 elaborated), not a full type-inference engine.

13 Current Frontier Facts

The Pure/runtime frontier is now theorem-level classified in `PureRuntimeFrontier.lean` (117 lines, 0 sorry).

1. The three `mettaPure` rewrites are known exactly: `mettaPure_rewrites_exact` (proof: `rfl`).
2. `BetaPi` has a non-morkTranslatable RHS because it uses `.subst: betaPi_rhs_not_morkTranslatable` (`rfl`).
3. `BetaSigmaFst` and `BetaSigmaSnd` have translatable RHSs: `betaSigmaFst_rhs_morkTranslatable`, `betaSigmaSnd_rhs_morkTranslatable` (both `rfl`).
4. None of the three rules has an `fvar`-headed LHS: `mettaPure_rewrite_lhs_not_fvar` (case analysis, constructor injectivity).
5. Therefore none fits the direct R_{exec_0} source-rule bridge: `no_mettaPure_rewrite_fits_direct_runtimeExec0_source_rule`.
6. The real current overlap is via quotation and the A/B/C1 bridge: `closedPure_overlap_via_abc`, `closedPure_overlap_via_abc_to_wm`.

14 Current Runtime-Real PeTTa Fragments

The current runtime-real fragments are intentionally narrow. Each lands on both the MORK/MM2 execution seam and the WM consequence surface.

14.1 Rule Lane

`PettaCoreStep` (`CoreFragment.lean`, 180 lines): 2 constructors.

- `evalStep` — core rule application via `eval`. Requires: `pettaCoreRule` (safe rule + `fvar` LHS).
- `evalcStep` — operational synonym via `evalc`.

Key theorems.

- `evalStep_toMorkSourceFire` — lands on R_{exec_0} .
- `pettaEvalStep_schedulerFires` (`DirectExecSurface.lean`) — fires through full scheduler.
- `pettaCoreStep_to_wmStrengthObligation` (`PLNWorldModelPettaCoreBridge.lean`) — WM landing.

14.2 Query Lane

`PettaSpaceCoreQuery` (`SpaceCoreFragment.lean`, 164 lines): 2 constructors.

- `anyFactMatch` — (`match &self $x tmpl`).
- `getAtoms` — (`get-atoms &self`).

Key theorems.

- `anyFactMatch_mem_spaceMatch` — core query $\subseteq$ `spaceMatch` semantics.
- `anyFactMatch_toComputableSourceQuery` — computable version on list spaces.
- `pettaSpaceCoreQuery_to_wmStrengthObligation` (`PLNWorldModelPettaSpaceCoreBridge.lean`) — WM landing.

14.3 Space-Effect Lane

`SpaceEffectFragment.lean` (194 lines):

- `addAtomSourceExecRule` — template: `[.remove cmd, .add X, .add ()]`.
- `removeAtomSourceExecRule` — template: `[.remove cmd, .remove X, .add ()]`.
- `addAtom_fireSourceRule_mem`, `removeAtom_fireSourceRule_mem` — fire at source level.
- `addAtomCmd_schedulerFires`, `removeAtomCmd_schedulerFires` (`DirectExecSurface.lean`) — through full scheduler.

14.4 WM Landing

All three lanes land on the WM consequence surface:

- `wmConsequenceRule0n_of_pettaCoreStep` (`PLNWorldModelPetTaCoreBridge.lean`).
- `wmConsequenceRule0n_of_pettaSpaceCoreQuery` (`PLNWorldModelPetTaSpaceCoreBridge.lean`).

14.5 Still Outside Current Proved Runtime Core

- general collection-pattern matching with rest variables
- beta/substitution-heavy runtime reduction
- broad scheduler metadata as part of the semantic core
- full HE congruence-level runtime propagation
- unrestricted grounded/oracle execution

15 GodelClaw, VeriCore, and Why This Matters

The project is not only about a generic language architecture. It is also about making practical agent systems reviewable and eventually certifiable.

The current relevant code already points toward:

- Lean-side architecture/formal semantics: `Mettapedia/CognitiveArchitecture/GodelClaw/`
- Rust-side runtime and policy core: `vericlaw/vericore-core/`
- policy-side Rust crate: `vericlaw/vericore-policy/`

The likely division of labor is:

1. Lean / `mettapedia` defines: elaborated core, proof certificates, semantic/runtime overlap theorems, reference specs.
2. Rust / runtime side implements: efficient execution, policy loops, backend integration, direct runners.

16 Concrete Next Steps

Done.

- ✓ Stable Pure waist (Π , Σ , Id , universes).
- ✓ Declaration environment and semantic layer.
- ✓ Non-parametric family checker: `Unit`, `Bool`, `Nat` as instances of `checkIndDecl`.
- ✓ Parametric family checker: `List` as first parametric instance of `checkParamIndDecl`; `Ctx n` as telescope.

Immediate next steps (in order).

1. **Recursor generation.** Implement `generateRecursor : ParamIndDecl → Option DeclSpec` that emits the eliminator type for a validated inductive family. This is the first honest step after declaration checking.
2. **Indexed families.** Extend `ParamIndDecl` to distinguish uniform parameters from varying indices (`Eq`, `Vec`, `Fin` require this). The current `typeFormerType` encoding subsumes non-parametric and parametric families; indexed families require tracking which leading `II`'s are uniform.
3. **Smallest structural fixpoints.** The first good target is `Nat.isZero` or `Nat.pred`: a tiny structurally recursive function *above* ordinary families, not before them. This requires deciding the fixpoint story (`W`-types, well-founded recursion, or sized types) before committing kernel term constructors.

Runtime connection. Once recursors are generated and well-typed, the runtime overlap class for constructor and eliminator applications can move from `artifactOnly` toward `directExec` — the point where proof-side reduction and MORK/MM2 pattern-rewriting agree on the same output.

Lockstep rule. Every widening step must add one honest semantic object, one explicit service or lowering target, and one honesty classification: does it extend the `PureKernel` waist, the runtime surface, or both? Is the overlap currently `artifactOnly`, canonical agreement only, or `directExec`?

17 Overall Vision and Road Map

Vision. MeTTa-Pure aims to be the first formally verified kernel for non-deterministic rewriting with dependent types: a Lean-like trusted proof target for the MeTTa language family, connected to the MORK/MM2 pattern-rewrite runtime through explicit overlap certificates. The full kernel will support ordinary strictly-positive inductive families with generated recursors, small structural fixpoints above that, and an elaborated surface language that classifies MeTTa programs as either kernel-provable or runtime-executable.

Road map.

Kernel step	Status	Notes
Pure waist ($\Pi/\Sigma/\text{Id}/\text{universes}$)	Done	<code>Syntax.lean</code> , <code>Typing.lean</code>
Reduction theory + confluence	Done	<code>Reduction.lean</code> , <code>Confluence.lean</code>
Declaration environment + semantics	Done	<code>DeclarationEnv.lean</code> , <code>DeclarationSemantics.lean</code>
Non-parametric family checker	Done	<code>InductiveDecl.lean</code> ; <code>Unit/Bool/Nat</code>
Parametric family checker + telescope	Done	<code>Telescope.lean</code> ; <code>List</code>
Recursor generation	Next	<code>generateRecursor</code>
Indexed families (<code>Eq</code> , <code>Vec</code> , <code>Fin</code>)	Planned	Distinguish params from indices
Structural fixpoints	Planned	Above indexed families
Runtime overlap: <code>directExec</code>	Partial	Currently <code>artifactOnly</code>
Elaborated core unification	Partial	PoC classifier in <code>ElaboratedCore.lean</code>

Why this ordering. Declaration checking must precede recursor generation because recursors are defined in terms of the validated family. Recursors must precede structural fixpoints because fixpoints consume eliminators. Runtime `directExec` overlap requires both sides (proof-side `RedDecl` and MORK/MM2 rewriting) to agree on the same reduction steps — that becomes tractable once recursors are generated and kernel-typed.

18 Pros and Cons of the Main Routes

18.1 Fork-only architecture

Pros. Honest. Already working. Keeps trust boundaries clear.

Cons. Too weak as a final user model. Feels like separate languages.

18.2 Elaborated-core architecture

Pros. One surface language. One classifier. Two targets. Lean-like.

Cons. Harder design work. Forces explicit phase distinctions.

18.3 Runtime-first elaboration

Pros. Grounded in real execution seams. Helps actual runtime work now.

Cons. Can drift into a runtime-only “core” if the proof lane is neglected.

18.4 Proof-first elaboration

Pros. Maximizes trust and certificate clarity.

Cons. Can remain artifact-only too long and miss the actual operational center.

19 Current Working Slogan

MeTTa-Pure proves. MORK/MM2 executes. Elaborated MeTTa-Core unifies. WM compares and composes them.

20 Open Technical Questions

1. What is the minimal PureKernel extension for ordinary families that preserves the cleanliness of the current semantic waist?
2. What theoremic support should remain live around canonicalization while evaluator and explorer tooling stay quarantined?
3. What is the first small checked executable recursor demo that feels unmistakably like a real application rather than only a kernel exercise?
4. At what point does an honest fragment move from `artifactOnly` to a stronger proof/runtime overlap class?

21 Recursor Generation: Design Constraints and Pitfalls

This section records design constraints and architectural pitfalls for the next high-risk development step: generating the dependent eliminator from a validated `ParamIndDecl`. The material here is informed by external review of the Mar 13 kernel snapshot.

21.1 Two immediate hygiene warnings

Warning 1: do not freeze the non-dependent pilot recursors. The current `RecursorDecl`.lean defines `unitRecType`, `boolRecType`, and `natRecType` as *non-dependent*, constant-motive recursors: the motive is a closed type $R : U_0$, not a dependent $P : F \rightarrow U_0$. These are acceptable *pilot programming recursors* and useful derived interfaces, but they must **not** become the canonical generator target. If `generateRecursor` is implemented by producing the non-dependent shape, it will silently under-specify the elimination principle for every family it generates.

The source-of-truth generator target is the **dependent eliminator**: for a p -parameter ordinary family F with constructors $c_1, \dots, c_k$ (in declaration order), the canonical closed recursor type has the outer shape

$$\Pi \text{params}, \Pi P : F \text{ params} \rightarrow U_0, \Pi \text{case}_1 : \text{CaseTy}_1, \dots, \Pi \text{case}_k : \text{CaseTy}_k, \Pi x : F \text{ params}, P x.$$

Case arms appear in *constructor declaration order* so that generated typing, generated iota rules, and future rule tables stay aligned. For each constructor $c_i : \Pi \text{params}, \Pi \Delta_i, F \text{ params}$, its case-arm type inserts an induction-hypothesis binder *immediately after every recursive argument* in Δ_i , then concludes with $P (c_i \text{ params args})$.

Reference: `Nat.rec` (dependent form). The canonical closed type for the dependent `Nat` recursor is:

$$\Pi (P : \text{Nat} \rightarrow U_0), P \text{ zero} \rightarrow (\Pi (n : \text{Nat}), P n \rightarrow P (\text{succ } n)) \rightarrow \Pi (n : \text{Nat}), P n.$$

The existing `natRecType` is the constant-motive specialization $P := \lambda _ . R$; it is a *derived* interface, not the generator source.

Warning 2: do not leave pilot family modules on the trusted import path. The current `PureKernel.lean` umbrella imports `UnitDecl`, `BoolDecl`, `NatDecl`, `RecursorDecl`, `InductiveDecl`, and `Telescope` alongside the pure semantic waist. This creates a trust-hygiene ambiguity: which pieces are the trusted minimal kernel, and which are pilot ordinary-family instances? The recommended split is:

- **PureKernelCore** — syntax, renaming, substitution, context, reduction, parallel reduction, confluence, definitional equality, typing, algorithmic typing, subject reduction;
- **PureKernelDecl** — declaration environment, spec, semantics;
- **PureKernelFamilies** — inductive declaration checker, telescope, `Unit/Bool/Nat/List` pilot instances and their recursors.

The first two layers form the trusted waist; the third is the pilot/generation lane, clearly visually and mechanically separate.

21.2 Recursor type generation: algorithmic shape

For the general parametric case:

- $p = \text{piArity decl.typeFormerType}$ (uniform parameters, appear before the motive);
- motive binds over F params: $P : F \text{ params} \rightarrow U_0$ (for non-indexed families);
- case arms in constructor declaration order, each inserting IH binders after recursive occurrences;
- major premise $x : F \text{ params}$, conclusion $P x$.

For **indexed families** (once supported), parameters still come before the motive; the motive abstracts over indices and the major premise. Parameters do *not* increase in number — indices belong *inside* the motive type, not before it.

21.3 Indexed families: what changes in the checker

Strict positivity must be checked over the *full* constructor type (including index arguments), not just the parametric prefix. The current `isParamApp/targetsFamilyP` design assumes all family arguments are bare de Bruijn parameter variables; this is too restrictive for indexed families such as $\text{Vec} : \text{Nat} \rightarrow U_0$ where targets like $\text{Vec}(\text{succ } n)$ must be allowed.

The needed replacement decomposes via an application-spine collector and validates:

- **head** = `const familyName`;
- first p arguments match expected parameter de Bruijn variables (uniform parameter check);
- remaining index arguments are arbitrary well-scoped terms.

21.4 Substitution

The current `Sub (Fin n -> PureTm m)` and parallel `subst` are semantically sufficient for recursor iota-reduction. The recommended addition is a thin ergonomic wrapper:

```
abbrev SubVec (k n : Nat) := Fin k -> PureTm n

def substVec (s : SubVec k n) : PureTm (n + k) -> PureTm n :=
  subst (fun i =>
    if h : i.val < k then s {i.val, }h
    else .var {i.val - k, by }omega)
```

Variable order: *innermost-first* (de Bruijn-aligned). `substVec` should be an ergonomic wrapper over `subst`, **not** a new semantic primitive.

21.5 Declaration semantics: iota rules belong in a separate layer

`DeclSpec` should *not* acquire a `computationRules` field. That would conflate δ -unfolding (constant lookup) with iota-reduction (reduction on a saturated eliminator spine), which are operationally distinct.

The recommended design is a separate generated-recursor layer above `SignatureWellFormed`:

```
structure IotaRule where
  recursorName : DeclName
  constructorName : DeclName
  rhs : PureTm 0 -- closed after substituting params, cases, args

structure GeneratedRecursorSemantics where
  recursorSpec : DeclSpec
  iotaRules : List IotaRule
```

`SignatureWellFormed` continues to own declaration order and δ correctness; a higher `FamilyDeclWellFormed` structure owns recursor generation correctness and iota typing preservation.

21.6 Proof strategy for generated declarations

A single monolithic `decide` over the full generated eliminator type is not recommended for non-trivial families. The recommended strategy (preserving kernel-checked proofs without `native_decide`):

1. Keep the generator functions small and transparent;
2. prove *shape lemmas* by `rfl` or `simp`: generated motive type, generated case-arm type, generated recursor type;
3. prove local boolean helpers separately (application-spine collector, parameter-prefix match, constructor-target decomposition);
4. use `decide` only on small closed subgoals after unfolding;
5. avoid a top-level `checkGeneratedRecursorDecl hugeDecl = some hugeOutput` as the main correctness story.

Staging. Follow the same $\text{Unit} \rightarrow \text{Bool} \rightarrow \text{Nat} \rightarrow \text{List}$ order for the dependent recursor generator: prove the simplest case first (Unit, one constructor, no recursive occurrences, trivial IH), then accumulate. Only after Unit/Bool/Nat are generated dependently should indexed families and fixpoints be addressed.

22 Status Note

Formalization inventory. The architecture described in this paper is backed by proven Lean 4 code with **0 sorry** and **0 axioms**:

Component	Lines
<i>Core architecture</i>	
ElaboratedCoreBase/Core/CertificateFragment	~680
PureRuntimeFrontier	117
RuntimeExec	269
PeTTa fragments (core/space/effect)	538
<i>PureKernel declaration mechanism</i>	
DeclarationEnv, DeclarationSpec, DeclarationSemantics	~2230
UnitDecl, BoolDecl, NatDecl, RecursorDecl	~1350
InductiveDecl, Telescope	~490
<i>HE formalization</i>	
HE/ (16 files)	5535
<i>MeTTaIL substrate</i>	
mettail-core (16 files)	3254
<i>MORK/MM2 boundary</i>	
ExecutionBoundary, DirectExecSurface, etc.	~1200
<i>WM bridges</i>	
PureKernel, Runtime, PeTTa, HE	~400
Total	~16,000

[TODO: Add references to external literature once the architecture stabilizes.]

23 Hyperon Experimental (HE) Fragment Status

The HE formalization is a parallel runtime witness to PeTTa, covering the Hyperon Experimental interpreter semantics. It lives in `Mettapedia/Languages/MeTTa/HE/` (5535 lines across 16 files, 0 sorry).

23.1 Three-Layer Fragment Hierarchy

HE defines three progressively richer fragment predicates:

Layer	Predicate	Capability
Core	<code>heCoreRule</code>	No-premise, safe, <code>fvar</code> -LHS
Premise	<code>hePremiseCoreRule</code>	+ relation query premises
Guarded	<code>heGuardedPremiseCoreRule</code>	+ extended translatable guards

Each layer has a corresponding step relation: `HECoreStep`, `HEPremiseCoreStep`, `HEGuardedPremiseCoreStep`.

23.2 Key Theorems

- `topRule_toMorkSourceFire` (`HE/CoreFragment.lean`) — no-premise HE rule fires at R_{exec_0} source-rule boundary.
- `topRuleWithPremises_toMorkSourceFire` — premise rules fire at the extended boundary.
- `topRuleWithExtPremises_toMorkSourceFire` — guarded rules fire at the guarded boundary.
- `hePremiseCoreStep_to_computableFire` — computable firing witness.
- `heCoreStep_to_wmStrengthObligation` (`PLNWorldModelHECoreBridge.lean`) — WM landing for core steps.

23.3 Additional HE Infrastructure

- `HE/Interpreter.lean` (526 lines) — core interpreter: 6 mutually recursive functions (`metta`, `interpretExpression`, `interpretFunction`, etc.), fuel-bounded termination.
- `HE/Conformance.lean` (480 lines) — 37 conformance theorems validating `HELanguageDef` against the interpreter spec (all kernel-checked via `rfl` or `decide`).
- `HE/HELanguageDef.lean` (1121 lines) — authoritative `LanguageDef` state machine specification, source of truth for Rust/Ascent backend generation.
- `HE/Types.lean` (311 lines) — error codes, bindings, result sets.
- `HE/Matching.lean` (249 lines) — pattern matching: `match_atoms`, `merge_bindings`.

23.4 Relation to PeTTa

Both PeTTa and HE are *witnesses* to the runtime kernel, not the definition. `runtimeBackendAgreement` (`ElaboratedCore.lean`) proves they target the same MORK/MM2 backend. The HE hierarchy is richer (3 layers vs. PeTTa’s 1), reflecting HE’s premise and guard machinery.

24 MeTTaIL as Shared Substrate

The MeTTaIL (MeTTa Intermediate Language) layer is the shared substrate on which both proof and runtime branches are built.

24.1 Core Formalization

The standalone formalization lives in `batteries/mettail-core/` (3254 lines, 16 files), mirrored in `Mettapedia/OSLF/MeTTaIL/` for use within Mettapedia.

Key types.

- `Pattern` — core syntax with `bvar`, `fvar`, `apply`, `lambda`, `multiLambda`, `subst`, `collection`.
- `LanguageDef` — structure specifying a dialect as a state machine (types, terms, rewrites).
- `RewriteRule` — LHS + RHS + premises.
- `Bindings` — pattern variable assignments.
- `RelationEnv` — pluggable environment for relation queries.
- `TransitionContract` — semantics contracts for instruction transitions (deterministic reduction, memoization safety, specialization safety, etc.).

Key modules.

- `Syntax.lean` (291 lines) — core syntax.
- `Engine.lean` (192 lines) — `rewriteStep`, `rewriteWithContext`.
- `Match.lean` (187 lines) — pattern matching and binding resolution.
- `Substitution.lean` (137 lines) — pattern substitution.
- `TransitionSpec.lean` (270 lines) — transition contracts.

24.2 Role in the Architecture

MeTTaIL is the *type* of patterns, rules, and artifacts that flow through the entire system:

- `SharedArtifact` wraps a `Pattern`.
- `MeTTaRuntimeExecSurface` translates `Pattern` to MORK Atom via `morkPatternToAtom`.
- `morkTranslatable` classifies which patterns the MORK/MM2 backend can handle (rejects `.subst` and rest-variables).
- Both PeTTa and HE specify their dialects as `LanguageDef` instances.

The MeTTaIL layer is *not* a runtime. It is the shared vocabulary that makes it possible for proof and runtime to agree on what a pattern, rule, or artifact *is*.

25 Runtime Reality: Spaces, Bindings, and Modularity

This section explains the current state of spaces and bindings across the MeTTa-family runtimes, why that state creates reasoning and modularity problems, and why the pure-kernel direction with explicit boundaries is the right long-term response.

The purpose is not to criticize any implementation—each represents a reasonable engineering trade-off for its stage of development. The purpose is to make the theory/runtime mismatch explicit, so that the MeTTa-Pure architecture can be understood as a principled response rather than a stylistic preference.

25.1 Current Runtime Space Models

MORK/MM2. The mainline MORK kernel centres on a single live mutable `btm` (binary trie-map) workspace. Additional data enters through a pluggable resource system with three source kinds: `BTM` (in-memory trie), `ACT` (archive-compressed trie, file-backed), and `Z3` (constraint-solver

integration). Each source kind becomes a factor in a dependent-product zipper that combines heterogeneous data sources into a single query interface.

The ACT sources are external file-backed artifacts, not additional live mutable workspaces. Many logical spaces can therefore be *simulated* by external source/sink patterns, but mainline MM2/MORK is not yet a polished built-in multi-live-space runtime.

MeTTa-Compiler (mettatron). The mettatron compiler is better engineered than PeTTa in software architecture, but it is not currently a richer many-space runtime. One `Environment` contains one MORK-backed `btm` (`Arc<RwLock<PathMap<()>>>`). One `MettaState` contains one `Environment`. The `match` evaluator accepts only the literal space name `"self"`; all other space names produce an error: `"match only supports 'self' as space name"`. The `create_space()` method clones the single `btm` via copy-on-write structural sharing, but this does not create a named independently addressable space.

Current mettatron is thus essentially one MORK-backed space per evaluation environment, not yet a true internal many-space runtime.

PeTTa. PeTTa is a single Prolog runtime backed by one global clause database. Its “spaces” are more than pure syntactic sugar: each space name becomes a Prolog predicate functor via `..` (univ), so `add_sexp(Space, Term)` and `match(Space, Pattern, Template, Result)` route to distinct predicate families. `get-atoms` enumerates clauses by `current_predicate(Space/Arity)`.

This is somewhat more real than mere tagging—distinct spaces inhabit distinct predicate namespaces—but it is still much weaker than true separate spaces in a systems sense. There is one Prolog process, one clause database, one unification context, and no isolation between space operations.

25.2 Dynamic Binding Propagation in HE Semantics

The Hyeron Experimental (HE) interpreter maintains an *interpretation plan*: a list of (`atom`, `bindings`) pairs. Each step of interpretation executes a single instruction and produces a new set of such pairs. The specification states:

“Bindings of the result are merged with the previous bindings.”
— *minimal-metta.md*, interpreter state description

The current HE-style operational semantics thus behaves like a dynamically threaded binding environment whose successful bindings are merged outward from nested evaluations, rather than a lexically sealed local substitution context. This is confirmed at the formal level by the `RuntimeSpec` surface (`Mettapedia/Languages/MeTTa/RuntimeSpec.lean`), which records:

Dialect	<code>bindingsSurface</code>	<code>contextSurface</code>
Pure	<code>.none</code>	<code>.none</code>
HE	<code>.explicit</code>	<code>.explicitStateAndSpace</code>
PeTTa	<code>.explicit</code>	<code>.explicitSpace</code>

Both HE and PeTTa thread explicit bindings through their result carriers (`List (Atom × Bindings)` and `List (Pattern × Bindings)` respectively), whereas the Pure kernel has no ambient bindings store at all.

25.3 Why Dynamic Bindings Matter Architecturally

Dynamic bindings as a feature. Outward binding propagation is genuinely useful for:

- opportunistic association during exploratory reasoning,
- fluid context carryover across chained evaluations,
- fast prototype development where binding plumbing is a burden.

These are legitimate cognitive and working-memory services for an AI reasoning layer.

Dynamic bindings as a liability. The same mechanism makes it difficult to know:

- what a subcomputation may affect (no confinement),
- whether a space boundary is semantically real or merely nominal,
- whether an external tool or oracle contaminated an intermediate result,
- whether a refactoring changes observable behaviour.

The core tension is that a fluid binding layer is a poor trusted semantic boundary. A proof or certificate that depends on the absence of certain bindings cannot be safely composed with a subcomputation that might introduce them.

Remark 25.1 (Positive and negative examples). A fast exploratory layer can legitimately use dynamic binding or a shared workspace for opportunistic reasoning—this is appropriate for a cognitive scratch layer. Conversely, a proof obligation, a certificate, or an authority boundary should not depend on ambient binding state or merely nominal space tags inside one monolithic runtime.

25.4 Explicit Communication Boundaries

The ρ -calculus and π -calculus offer a better conceptual model for boundaries: explicit send ($x!(m)$), receive ($\text{for } m \leftarrow x.P$), and private name creation ($(\text{new } x)P$). These primitives make every information transfer syntactically visible and confine binding propagation to explicitly declared channels.

This does not solve every engineering problem, but it gives a much better conceptual model for explicit information transfer than ambient binding propagation. The formalized rho-calculus layer in `Mettapedia/Languages/ProcessCalculi/` provides structural congruence, reduction, and alpha-equivalence, serving as a reference point for what “explicit boundaries” means in the MeTTa ecosystem.

25.5 Why the Pure-Kernel Direction Helps

The MeTTa-Pure kernel (`Mettapedia/Languages/MeTTa/PureKernel/`) provides exactly what is needed to prevent ambient binding leakage from becoming the trusted semantic core:

- **Scoped de Bruijn terms** (`PureKernel/Syntax.lean`): the type `PureTm n` is indexed by binding depth n , making scope statically visible in the type. Binders (Π , Σ , λ) increment n in their body argument.
- **Explicit substitution** (`PureKernel/Substitution.lean`): a substitution `Sub n m` is a function `Fin n → PureTm m` mapping indices in depth- n context to terms in depth- m context. Composition, identity, and interaction lemmas (`subst_comp`, `subst_ids`, `rename_subst`) are kernel-checked with zero sorries.
- **Scope contracts** (`ScopeContract.lean`): a shared artifact consumed by both the Lean formalization and the Rust compiler. Each binder form declares its `binderPositions`, `valuePositions`, and `bodyPositions`, so the compiler does not have to rediscover scoping rules.
- **No ambient bindings**: the Pure runtime spec declares `bindingsSurface = .none` and `contextSurface = .none`. There is no implicit binding store; all variable resolution is local.

The right long-term answer is not to pretend current runtimes already realize strong space or scope discipline, but to build a trusted lexical kernel with explicit substitution and then connect richer runtime layers to it through explicit contracts.

25.6 Near-Term Pragmatic Recommendation

Near-term AI systems built on MeTTa should use a layered strategy:

1. A **fluid scratch/workspace layer** for exploratory and associative reasoning, where dynamic binding and shared workspaces are acceptable.
2. **Explicit promotion boundaries** between the scratch layer and any trusted or persistent store.
3. **Explicit provenance and interface contracts**, so that the origin and authority of every atom crossing a boundary is visible.
4. A **small trusted lexical/scoped core** (MeTTa-Pure) for whatever must compose cleanly, carry proofs, or serve as a certificate target.

“Many spaces” as surface syntax is not enough. True many-space semantics requires either separate binding contexts, explicit communication/import-export primitives, or separate runtimes and stores. Until then, the honest strategy is: fluid scratchpad + explicit promotion + trusted lexical core + explicit communication boundaries.

The practical lesson is that fluid global context propagation may be useful as an exploratory cognitive mechanism, but it is a poor trusted semantic boundary. The long-term architectural answer is therefore not merely to add more nominal spaces, but to combine a fluid working layer with explicit communication and promotion boundaries, all grounded in a small lexical kernel with explicit substitution.

A Appendix: Full Current PureKernel Listing

This appendix includes the full current MeTTa-Pure kernel source for archival and audit purposes. The files are listed directly from the Lean sources so that readers can inspect the exact current trusted term language, typing rules, reduction theory, confluence infrastructure, quotation bridge, algorithmic checking layer, declaration mechanism, and family declaration checker in one place.

A.1 AlgorithmicTyping.lean

Mettapedia/Languages/MeTTa/PureKernel/AlgorithmicTyping.lean

```
import Mettapedia.Languages.MeTTa.PureKernel.DefEq

namespace Mettapedia.Languages.MeTTa.PureKernel

open Syntax
open Context
open Renaming
open Substitution
open Reduction
open Confluence
open Typing

structure InferredTyping  $\Gamma$  ( : Ctx n) (t : PureTm n) where
  type : PureTm n
```

```

typing : HasType  $\Gamma$  t type

structure CheckedTyping  $\Gamma$ ( : Ctx n) (t A : PureTm n) : Type where
  typing : HasType  $\Gamma$  t A

mutual

def inferPureType :  $\Gamma$ ( : Ctx n) -> (t : PureTm n) -> Except String (InferredTyping  $\Gamma$  t)
|  $\Gamma$ , .var i =>
  pure { type := lookup  $\Gamma$  i, typing := HasType.var  $\Gamma$ ( :=  $\Gamma$ ) i }
|  $\_$ , .const c =>
  throw s!"unknown declaration constant in current kernel fragment: {reprStr c}"
|  $\Gamma$ , .u0 =>
  pure { type := .u1, typing := HasType.u0_type  $\Gamma$  }
|  $\_$ , .u1 =>
  throw "Type1 has no type in the current Pure kernel"
|  $\Gamma$ , .pi A B => do
  let hA <- checkPureType  $\Gamma$  A .u1
  let hB <- checkPureType (.snoc  $\Gamma$  A) B .u1
  pure { type := .u1, typing := HasType.pi_form hA.typing hB.typing }
|  $\Gamma$ , .sigma A B => do
  let hA <- checkPureType  $\Gamma$  A .u1
  let hB <- checkPureType (.snoc  $\Gamma$  A) B .u1
  pure { type := .u1, typing := HasType.sigma_form hA.typing hB.typing }
|  $\Gamma$ , .id A a b => do
  let hA <- checkPureType  $\Gamma$  A .u1
  let ha <- checkPureType  $\Gamma$  a A
  let hb <- checkPureType  $\Gamma$  b A
  pure { type := .u1, typing := HasType.id_form hA.typing ha.typing hb.typing }
|  $\_$ , .lam _ =>
  throw "cannot infer a lambda without an expected Pi type; use `(: term type)``"
|  $\Gamma$ , .app (.lam body) a => do
  let arg <- inferPureType  $\Gamma$  a
  let bodyTy <- inferPureType (.snoc  $\Gamma$  arg.type) body
  pure
    { type := inst0 a bodyTy.type
      typing := HasType.app_elim (HasType.lam_intro bodyTy.typing) arg.typing }
|  $\Gamma$ , .app f a => do
  let funInfo <- inferPureType  $\Gamma$  f
  match asPi? funInfo.type with
  | some piInfo =>
    let hfPi : HasType  $\Gamma$  f (.pi piInfo.dom piInfo.cod) :=
      HasType.conv funInfo.typing piInfo.conv
    let ha <- checkPureType  $\Gamma$  a piInfo.dom
    pure
      { type := inst0 a piInfo.cod
        typing := HasType.app_elim hfPi ha.typing }
  | none =>
    throw "application expects a function whose type normalizes to Pi"
|  $\Gamma$ , .pair a b => do
  let leftTy <- inferPureType  $\Gamma$  a
  let rightTy <- inferPureType  $\Gamma$  b
  have hb' : HasType  $\Gamma$  b (inst0 a (rename wk rightTy.type)) := by
    simp [inst0_rename_wk_cancel a rightTy.type] using rightTy.typing

```

```

    pure
      { type := .sigma leftTy.type (rename wk rightTy.type)
        typing := HasType.pair_intro leftTy.typing hb' }
|  $\Gamma$ , .fst p => do
  let pairInfo <- inferPureType  $\Gamma$  p
  match asSigma? pairInfo.type with
  | some sigmaInfo =>
    let hpSigma : HasType  $\Gamma$  p (.sigma sigmaInfo.dom sigmaInfo.cod) :=
      HasType.conv pairInfo.typing sigmaInfo.conv
    pure { type := sigmaInfo.dom, typing := HasType.fst_elim hpSigma }
  | none =>
    throw "fst expects a term whose type normalizes to Sigma"
|  $\Gamma$ , .snd p => do
  let pairInfo <- inferPureType  $\Gamma$  p
  match asSigma? pairInfo.type with
  | some sigmaInfo =>
    let hpSigma : HasType  $\Gamma$  p (.sigma sigmaInfo.dom sigmaInfo.cod) :=
      HasType.conv pairInfo.typing sigmaInfo.conv
    pure
      { type := inst0 (.fst p) sigmaInfo.cod
        typing := HasType.snd_elim hpSigma }
  | none =>
    throw "snd expects a term whose type normalizes to Sigma"
|  $\Gamma$ , .refl a => do
  let info <- inferPureType  $\Gamma$  a
  pure
    { type := .id info.type a a
      typing := HasType.refl_intro info.typing }
termination_by _ t => 2 * sizeOf t
decreasing_by
  all_goals
    simp_wf <;> try omega

def checkPureType :  $\Gamma$  ( : Ctx n) -> (t A : PureTm n) -> Except String (CheckedTyping  $\Gamma$  t A)
|  $\Gamma$ , .lam body, expected =>
  match asPi? expected with
  | some piInfo => do
    let hBody <- checkPureType (.snoc  $\Gamma$  piInfo.dom) body piInfo.cod
    have hconv : Conv (.pi piInfo.dom piInfo.cod) expected :=
      Relation.EqvGen.symm _ _ piInfo.conv
    pure { typing := HasType.conv (HasType.lam_intro hBody.typing) hconv }
  | none =>
    throw "lambda requires an expected Pi type"
|  $\Gamma$ , .pair a b, expected =>
  match asSigma? expected with
  | some sigmaInfo => do
    let ha <- checkPureType  $\Gamma$  a sigmaInfo.dom
    let hb <- checkPureType  $\Gamma$  b (inst0 a sigmaInfo.cod)
    have hconv : Conv (.sigma sigmaInfo.dom sigmaInfo.cod) expected :=
      Relation.EqvGen.symm _ _ sigmaInfo.conv
    pure { typing := HasType.conv (HasType.pair_intro ha.typing hb.typing) hconv }
  | none => do
    let inferred <- inferPureType  $\Gamma$  (.pair a b)
    match defEqByNormalization? inferred.type expected with

```



```

      | some hconv => pure { typing := HasType.conv inferred.typing hconv.conv }
      | none =>
        throw s!"type mismatch: inferred {reprStr (cdev inferred.type)} but expected {
reprStr (cdev expected)}"
|  $\Gamma$ , t, expected => do
  let inferred <- inferPureType  $\Gamma$  t
  match defEqByNormalization? inferred.type expected with
  | some hconv => pure { typing := HasType.conv inferred.typing hconv.conv }
  | none =>
    throw s!"type mismatch: inferred {reprStr (cdev inferred.type)} but expected {reprStr (
cdev expected)}"
termination_by _ t _ => 2 * sizeof t + 1
decreasing_by
  all_goals
    simp_wf <;> try omega

end

def checkIsPureType ( $\Gamma$  : Ctx n) (A : PureTm n) : Except String Unit := do
  match A with
  | .u1 => pure ()
  | _ =>
    let _ <- checkPureType  $\Gamma$  A .u1
    pure ()

def inferClosedPureType (t : PureTm 0) : Except String (InferredTyping .nil t) :=
  inferPureType .nil t

def checkClosedPureType (t A : PureTm 0) : Except String (CheckedTyping .nil t A) :=
  checkPureType .nil t A

end Mettapedia.Languages.MeTTa.PureKernel

```

A.2 BoolDecl.lean

Mettapedia/Languages/MeTTa/PureKernel/BoolDecl.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

namespace Mettapedia.Languages.MeTTa.PureKernel.BoolDecl

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

/-- Pilot family/type constant name for Bool in the declaration environment. -/
def boolTyName : DeclName := `Bool

/-- Pilot constructor constant names for Bool. -/
def boolTrueName : DeclName := `Bool.true
def boolFalseName : DeclName := `Bool.false

```

```

/-- Pilot alias constant used to witness declaration-aware unfolding 6(''). -/
def boolAliasName : DeclName := `Bool.alias

/-- `Bool` family/type declaration spec. -/
def boolTySpec : DeclSpec :=
  { name := boolTyName, type := .u0 }

/-- `Bool.true` constructor declaration spec. -/
def boolTrueSpec : DeclSpec :=
  { name := boolTrueName, type := .const boolTyName }

/-- `Bool.false` constructor declaration spec. -/
def boolFalseSpec : DeclSpec :=
  { name := boolFalseName, type := .const boolTyName }

/-- `Bool.alias` declaration spec used for the first δ`` witness. -/
def boolAliasSpec : DeclSpec :=
  { name := boolAliasName, type := .const boolTyName, value? := some (.const boolTrueName) }

/-- `Bool` pilot declarations expressed through the shared scaffold. -/
def boolSpecs : List DeclSpec :=
  [boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec]

/-- Minimal declaration environment for the Bool pilot.
`Bool`, `Bool.true`, and `Bool.false` are declared; `Bool.alias` unfolds to `Bool.true`. -/
def boolDeclEnv : DeclEnv := envOfSpecs boolSpecs

@[simp] theorem typeOf_boolTy :
  typeOf? boolDeclEnv boolTyName = some .u0 := by
  decide

@[simp] theorem typeOf_boolTrue :
  typeOf? boolDeclEnv boolTrueName = some (.const boolTyName) := by
  decide

@[simp] theorem typeOf_boolFalse :
  typeOf? boolDeclEnv boolFalseName = some (.const boolTyName) := by
  decide

@[simp] theorem typeOf_boolAlias :
  typeOf? boolDeclEnv boolAliasName = some (.const boolTyName) := by
  decide

@[simp] theorem valueOf_boolAlias :
  valueOf? boolDeclEnv boolAliasName = some (.const boolTrueName) := by
  decide

/-- Typed declaration witness:  $\vdash \text{Bool} : U0$ . -/
theorem hasType_boolTy :
  HasTypeDecl boolDeclEnv .nil ((.const boolTyName : PureTm 0)) .u0 :=
  hasType_const_from_lookup (E := boolDeclEnv)  $\Gamma$ ( := .nil) (c := boolTyName) (A0 := .u0) (by
    simp)

/-- Typed declaration witness:  $\vdash \text{Bool.true} : \text{Bool}$ . -/

```

```

theorem hasType_boolTrue :
  HasTypeDecl boolDeclEnv .nil ((.const boolTrueName : PureTm 0)) (.const boolTyName) :=
  hasType_const_from_lookup (E := boolDeclEnv)  $\Gamma$ ( := .nil) (c := boolTrueName) (A0 := .const
    boolTyName) (by
      simp)

/-- Typed declaration witness:  $\vdash` Bool.false : Bool`. -/
theorem hasType_boolFalse :
  HasTypeDecl boolDeclEnv .nil ((.const boolFalseName : PureTm 0)) (.const boolTyName) :=
  hasType_const_from_lookup (E := boolDeclEnv)  $\Gamma$ ( := .nil) (c := boolFalseName) (A0 := .const
    boolTyName) (by
      simp)

/-- Typed declaration witness:  $\vdash` Bool.alias : Bool`. -/
theorem hasType_boolAlias :
  HasTypeDecl boolDeclEnv .nil ((.const boolAliasName : PureTm 0)) (.const boolTyName) :=
  hasType_const_from_lookup (E := boolDeclEnv)  $\Gamma$ ( := .nil) (c := boolAliasName) (A0 := .const
    boolTyName) (by
      simp)

/-- Reduction witness: `Bool.alias` unfolds to `Bool.true`. -/
theorem red_boolAlias_to_true :
  RedDecl boolDeclEnv
    ((.const boolAliasName : PureTm 0))
    ((.const boolTrueName : PureTm 0)) := by
  simp using
    red_const_from_unfold0 (E := boolDeclEnv) (c := boolAliasName) (v := (.const boolTrueName)) (
      by
        simp)

/-- Small checked pilot package: typed source, one declaration-aware step, typed target. -/
theorem boolAlias_checked_step :
   $\exists$  A : PureTm 0,
    HasTypeDecl boolDeclEnv .nil ((.const boolAliasName : PureTm 0)) A  $\wedge$ 
    RedDecl boolDeclEnv ((.const boolAliasName : PureTm 0)) ((.const boolTrueName : PureTm 0))
     $\wedge$ 
    HasTypeDecl boolDeclEnv .nil ((.const boolTrueName : PureTm 0)) A := by
  simp using
    (checked_delta_step_closed_of_typed_value
      (E := boolDeclEnv)
      (c := boolAliasName)
      (A0 := (.const boolTyName))
      (v0 := (.const boolTrueName))
      (by simp)
      (by simp)
      (by simp using hasType_boolTrue))

/-- Reusable typed-value obligations for `boolSpecs` in any declaration
environment where `Bool.true : Bool` is available. -/
theorem boolSpecs_hTyped_in
  (E : DeclEnv)
  (hTrueType : typeOf? E boolTrueName = some (.const boolTyName)) :
   $\forall$  s  $\in$  boolSpecs,  $\forall$  v0 : PureTm 0,
    s.value? = some v0  $\rightarrow$$$ 
```

```

      HasTypeDecl E .nil (liftClosed v0) (liftClosed s.type) := by
intro s hs v0 hVal
simp [boolSpecs, boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec] at hs
rcases hs with rfl | rfl | rfl | rfl
· simp at hVal
· simp at hVal
· simp at hVal
· have hv : v0 = (.const boolTrueName) := by
  simp [boolAliasSpec] using hVal.symm
  subst hv
  simp [liftClosed_zero] using
    (hasType_const_from_lookup
      (E := E)
      (Γ := .nil)
      (c := boolTrueName)
      (A0 := (.const boolTyName))
      hTrueType)

/-- Reusable no-self-unfolding obligations for `boolSpecs`. -/
theorem boolSpecs_hNoSelf :
  ∀ s ∈ boolSpecs, ∀ v0 : PureTm 0,
  s.value? = some v0 →
  v0 ≠ (.const s.name) := by
intro s hs v0 hVal
simp [boolSpecs, boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec] at hs
rcases hs with rfl | rfl | rfl | rfl
· simp at hVal
· simp at hVal
· simp at hVal
· have hv : v0 = (.const boolTrueName) := by
  simp [boolAliasSpec] using hVal.symm
  subst hv
  intro hEq
  have hNe : ((.const boolTrueName : PureTm 0)) ≠ (.const boolAliasName) := by
    decide
  exact hNe hEq

def boolSpecs_obligations : DeclSpecObligations boolSpecs where
  valuesWellTyped :=
    boolSpecs_hTyped_in
    (E := envOfSpecs boolSpecs)
    (hTrueType := by decide)
  noSelfDelta := boolSpecs_hNoSelf

def boolSpecs_signatureWellFormed : SignatureWellFormed boolSpecs where
  noShadowing := by decide
  obligations := boolSpecs_obligations

theorem boolTySpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [] boolTySpec where
  fresh := by simp [boolTySpec]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames, boolTySpec]
  valueUsesEarlier := by

```

```

    intro v0 hVal
    simp [boolTySpec] at hVal
valueWellTyped := by
    intro v0 hVal
    simp [boolTySpec] at hVal
noSelfDelta := by
    intro v0 hVal
    simp [boolTySpec] at hVal

theorem boolTrueSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [boolTySpec] boolTrueSpec where
fresh := by simp [boolTySpec, boolTrueSpec, boolTyName, boolTrueName]
typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames, boolTySpec, boolTrueSpec, boolTyName, boolTrueName]
valueUsesEarlier := by
  intro v0 hVal
  simp [boolTrueSpec] at hVal
valueWellTyped := by
  intro v0 hVal
  simp [boolTrueSpec] at hVal
noSelfDelta := by
  intro v0 hVal
  simp [boolTrueSpec] at hVal

theorem boolFalseSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [boolTySpec, boolTrueSpec] boolFalseSpec where
fresh := by
  simp [boolTySpec, boolTrueSpec, boolFalseSpec, boolTyName, boolTrueName, boolFalseName]
typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames,
    boolTySpec, boolTrueSpec, boolFalseSpec, boolTyName, boolTrueName, boolFalseName]
valueUsesEarlier := by
  intro v0 hVal
  simp [boolFalseSpec] at hVal
valueWellTyped := by
  intro v0 hVal
  simp [boolFalseSpec] at hVal
noSelfDelta := by
  intro v0 hVal
  simp [boolFalseSpec] at hVal

theorem boolAliasSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [boolTySpec, boolTrueSpec, boolFalseSpec] boolAliasSpec where
fresh := by
  simp [boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec,
    boolTyName, boolTrueName, boolFalseName, boolAliasName]
typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames,
    boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec,
    boolTyName, boolTrueName, boolFalseName, boolAliasName]
valueUsesEarlier := by
  intro v0 hVal
  have hv : v0 = (.const boolTrueName) := by
    simpa [boolAliasSpec] using hVal.symm

```

```

    subst hv
    simp [UsesOnlyDeclNamesFrom, prefixNames,
          boolTySpec, boolTrueSpec, boolFalseSpec, boolTrueName]
valueWellTyped := by
  intro v0 hVal
  have hv : v0 = (.const boolTrueName) := by
    simpa [boolAliasSpec] using hVal.symm
  subst hv
  have hLookup :
    typeOf? (envOfSpecs [boolTySpec, boolTrueSpec, boolFalseSpec]) boolTrueName =
      some (.const boolTyName) :=
    typeOf_envOfSpecs_eq_of_mem_of_nodup
      (specs := [boolTySpec, boolTrueSpec, boolFalseSpec])
      (s := boolTrueSpec)
      (hNodup := by decide)
      (hs := by simp [boolTySpec, boolTrueSpec, boolFalseSpec])
  simpa [liftClosed_zero] using
    (hasType_const_from_lookup
      (E := envOfSpecs [boolTySpec, boolTrueSpec, boolFalseSpec])
      Γ( := .nil)
      (c := boolTrueName)
      (A0 := (.const boolTyName))
      hLookup)
noSelfDelta := by
  intro v0 hVal
  have hv : v0 = (.const boolTrueName) := by
    simpa [boolAliasSpec] using hVal.symm
  subst hv
  intro hEq
  have hNe : ((.const boolTrueName : PureTm 0)) ≠ (.const boolAliasName) := by
    decide
  exact hNe hEq

theorem boolSpecs_prefixWellFormed :
  SignatureWellFormedPrefix boolSpecs := by
  refine And.intro boolTySpec_prefixAdmissible ?_
  refine And.intro ?_ ?_
  · simpa [boolSpecs, boolTySpec, boolTrueSpec]
    using boolTrueSpec_prefixAdmissible
  · refine And.intro ?_ ?_
    · simpa [PrefixSignatureWellFormed, boolSpecs, boolTySpec, boolTrueSpec, boolFalseSpec]
      using boolFalseSpec_prefixAdmissible
    · refine And.intro ?_ trivial
      simpa [PrefixSignatureWellFormed, boolSpecs, boolTySpec, boolTrueSpec, boolFalseSpec,
            boolAliasSpec]
        using boolAliasSpec_prefixAdmissible

theorem boolSpecs_signatureWellFormed_fromPrefix :
  SignatureWellFormed boolSpecs :=
  SignatureWellFormed.ofPrefix boolSpecs_prefixWellFormed boolSpecs_obligations

theorem boolDeclEnv_wellFormed : DeclEnvWellFormed boolDeclEnv := by
  unfold boolDeclEnv
  exact boolSpecs_signatureWellFormed.toDeclEnvWellFormed

```

```
end Mettapedia.Languages.MeTTa.PureKernel.BoolDecl
```

A.3 Confluence.lean

Mettapedia/Languages/MeTTa/PureKernel/Confluence.lean

```
import Mathlib.Logic.Relation
import Mettapedia.Languages.MeTTa.PureKernel.Parallel
import Mettapedia.Languages.MeTTa.PureKernel.Typing

namespace Mettapedia.Languages.MeTTa.PureKernel.Confluence

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.Reduction
open Mettapedia.Languages.MeTTa.PureKernel.Parallel
open Mettapedia.Languages.MeTTa.PureKernel.Typing

/-- Complete development for parallel reduction. -/
def cdev : PureTm n → PureTm n
| .var i => .var i
| .const c => .const c
| .u0 => .u0
| .u1 => .u1
| .pi A B => .pi (cdev A) (cdev B)
| .sigma A B => .sigma (cdev A) (cdev B)
| .id A a b => .id (cdev A) (cdev a) (cdev b)
| .lam b => .lam (cdev b)
| .app (.lam b) a => inst0 (cdev a) (cdev b)
| .app f a => .app (cdev f) (cdev a)
| .pair a b => .pair (cdev a) (cdev b)
| .fst (.pair a _) => cdev a
| .fst p => .fst (cdev p)
| .snd (.pair _ b) => cdev b
| .snd p => .snd (cdev p)
| .refl a => .refl (cdev a)

/-- Parallel reduction always reaches complete development. -/
theorem par_to_cdev : ∀ {t u : PureTm n}, ParRed t u → ParRed u (cdev t) := by
  intro t u h
  induction h with
  | var i =>
    simp [cdev]
  | const c =>
    simp [cdev]
  | u0 =>
    simp [cdev]
  | u1 =>
    simp [cdev]
  | pi hA hB ihA ihB =>
    simp [cdev] using ParRed.pi ihA ihB
  | sigma hA hB ihA ihB =>
```

```

    simpa [cdev] using ParRed.sigma ihA ihB
| id hA ha hb ihA iha ihb =>
    simpa [cdev] using ParRed.id ihA iha ihb
| lam hb ih =>
    simpa [cdev] using ParRed.lam ih
| @app _ f f' a a' hf ha ihf iha =>
    cases f with
    | var i =>
        simpa [cdev] using ParRed.app ihf iha
    | const c =>
        simpa [cdev] using ParRed.app ihf iha
    | u0 =>
        simpa [cdev] using ParRed.app ihf iha
    | u1 =>
        simpa [cdev] using ParRed.app ihf iha
    | pi A B =>
        simpa [cdev] using ParRed.app ihf iha
    | sigma A B =>
        simpa [cdev] using ParRed.app ihf iha
    | id A a b =>
        simpa [cdev] using ParRed.app ihf iha
    | lam b =>
        cases hf with
        | lam hb =>
            cases ihf with
            | lam hbc =>
                simpa [cdev] using (ParRed.betaPi hbc iha)
    | app f a =>
        simpa [cdev] using ParRed.app ihf iha
    | pair a b =>
        simpa [cdev] using ParRed.app ihf iha
    | fst p =>
        simpa [cdev] using ParRed.app ihf iha
    | snd p =>
        simpa [cdev] using ParRed.app ihf iha
    | refl a =>
        simpa [cdev] using ParRed.app ihf iha
| pair ha hb iha ihb =>
    simpa [cdev] using ParRed.pair iha ihb
| @fst _ p p' hp ih =>
    cases p with
    | var i =>
        simpa [cdev] using ParRed.fst ih
    | const c =>
        simpa [cdev] using ParRed.fst ih
    | u0 =>
        simpa [cdev] using ParRed.fst ih
    | u1 =>
        simpa [cdev] using ParRed.fst ih
    | pi A B =>
        simpa [cdev] using ParRed.fst ih
    | sigma A B =>
        simpa [cdev] using ParRed.fst ih
    | id A a b =>

```



```

      simpa [cdev] using ParRed.fst ih
| lam b =>
      simpa [cdev] using ParRed.fst ih
| app f a =>
      simpa [cdev] using ParRed.fst ih
| pair a b =>
      cases hp with
      | pair ha hb =>
        cases ih with
        | pair ha' hb' =>
          simpa [cdev] using (ParRed.betaSigmaFst ha' hb')
| fst q =>
      simpa [cdev] using ParRed.fst ih
| snd q =>
      simpa [cdev] using ParRed.fst ih
| refl a =>
      simpa [cdev] using ParRed.fst ih
| @snd _ p p' hp ih =>
      cases p with
      | var i =>
        simpa [cdev] using ParRed.snd ih
      | const c =>
        simpa [cdev] using ParRed.snd ih
      | u0 =>
        simpa [cdev] using ParRed.snd ih
      | u1 =>
        simpa [cdev] using ParRed.snd ih
      | pi A B =>
        simpa [cdev] using ParRed.snd ih
      | sigma A B =>
        simpa [cdev] using ParRed.snd ih
      | id A a b =>
        simpa [cdev] using ParRed.snd ih
      | lam b =>
        simpa [cdev] using ParRed.snd ih
      | app f a =>
        simpa [cdev] using ParRed.snd ih
      | pair a b =>
        cases hp with
        | pair ha hb =>
          cases ih with
          | pair ha' hb' =>
            simpa [cdev] using (ParRed.betaSigmaSnd ha' hb')
      | fst q =>
        simpa [cdev] using ParRed.snd ih
      | snd q =>
        simpa [cdev] using ParRed.snd ih
      | refl a =>
        simpa [cdev] using ParRed.snd ih
| refl ha iha =>
      simpa [cdev] using ParRed.refl iha
| betaPi hbody ha ihbody iha =>
      simpa [cdev] using par_inst0 iha ihbody
| betaSigmaFst ha hb iha ihb =>

```

```

    simp [cdev] using iha
| betaSigmaSnd ha hb iha ihb =>
    simp [cdev] using ihb

theorem par_to_cdev_self (t : PureTm n) : ParRed t (cdev t) :=
  par_to_cdev (par_refl t)

/-- Diamond property for parallel reduction via complete development. -/
theorem diamond_parRed {s 1t 2t : PureTm n}
  (1h : ParRed s 1t) (2h : ParRed s 2t) :
  ∃ u, ParRed 1t u ∧ ParRed 2t u :=
  (cdev s, par_to_cdev 1h, par_to_cdev 2)h

abbrev ParStar (t u : PureTm n) : Prop := Relation.ReflTransGen ParRed t u

theorem redStar_to_parStar {t u : PureTm n} (h : RedStar t u) : ParStar t u := by
  induction h with
  | refl =>
    exact Relation.ReflTransGen.refl
  | tail hxy hyz ih =>
    exact Relation.ReflTransGen.tail ih (red_to_par hyz)

theorem parStar_to_redStar {t u : PureTm n} (h : ParStar t u) : RedStar t u := by
  induction h with
  | refl =>
    exact Relation.ReflTransGen.refl
  | tail hxy hyz ih =>
    exact RedStar.trans ih (par_to_redStar hyz)

/-- Confluence of parallel-star by local diamond. -/
theorem parStar_confluence {s 1t 2t : PureTm n}
  (1h : ParStar s 1t) (2h : ParStar s 2t) :
  ∃ u, ParStar 1t u ∧ ParStar 2t u := by
  have hlocal : ∀ (a b c : PureTm n), ParRed a b → ParRed a c →
    ∃ d : PureTm n, Relation.ReflGen ParRed b d ∧ Relation.ReflTransGen ParRed c d := by
    intro a b c hab hac
    rcases diamond_parRed hab hac with (d, hbd, )hcd
    exact (d, Relation.ReflGen.single hbd,
      Relation.ReflTransGen.tail Relation.ReflTransGen.refl )hcd
  rcases Relation.church_rosser (r := ParRed) hlocal 1h 2h with (u, 1hu, 2)hu
  exact (u, 1hu, 2)hu

/-- Confluence of multi-step  $\beta$ -reduction. -/
theorem redStar_confluence {s 1t 2t : PureTm n}
  (1h : RedStar s 1t) (2h : RedStar s 2t) :
  ∃ u, RedStar 1t u ∧ RedStar 2t u := by
  rcases parStar_confluence (redStar_to_parStar 1h) (redStar_to_parStar 2h) with
    (u, 1hu, 2)hu
  exact (u, parStar_to_redStar 1hu, parStar_to_redStar 2)hu

theorem redStar_implies_conv {t u : PureTm n} (h : RedStar t u) : Conv t u := by
  induction h with
  | refl =>
    exact Relation.EqvGen.refl _

```

```

| tail hxy hyz ih =>
  exact Relation.EqvGen.trans _ _ _ ih (red_implies_conv hyz)

/-- Church-Rosser for conversion generated by one-step reduction. -/
theorem church_rosser_conv {s t : PureTm n} (h : Conv s t) :
  ∃ u, RedStar s u ∧ RedStar t u := by
  refine Relation.EqvGen.rec ?hrel ?hrefl ?hsymm ?htrans h
  · intro a b hab
    exact ⟨b, red_to_redStar hab, RedStar.refl _⟩
  · intro a
    exact ⟨a, RedStar.refl _, RedStar.refl _⟩
  · intro a b hab ih
    rcases ih with ⟨u, ha, hb⟩
    exact ⟨u, hb, ha⟩
  · intro a b c hab hbc ihab ihbc
    rcases ihab with ⟨u1, ha_u1, hb_u1⟩
    rcases ihbc with ⟨u2, hb_u2, hc_u2⟩
    rcases redStar_confluence ha_u1 hb_u1 with ⟨w, hu_w, hu_w⟩
    exact ⟨w, RedStar.trans ha_u1 hu_w, RedStar.trans hc_u2 hu_w⟩

/-- One-step reduction from Pi keeps Pi head. -/
private theorem red_pi_head {A : PureTm n} {B : PureTm (n + 1)} {t : PureTm n}
  (h : Red (.pi A B) t) :
  ∃ (A' : PureTm n) (B' : PureTm (n + 1)), t = .pi A' B' := by
  cases h with
  | congPiDom hA => exact ⟨_, _, rfl⟩
  | congPiCod hB => exact ⟨_, _, rfl⟩

/-- One-step reduction from Sigma keeps Sigma head. -/
private theorem red_sigma_head {A : PureTm n} {B : PureTm (n + 1)} {t : PureTm n}
  (h : Red (.sigma A B) t) :
  ∃ (A' : PureTm n) (B' : PureTm (n + 1)), t = .sigma A' B' := by
  cases h with
  | congSigmaDom hA => exact ⟨_, _, rfl⟩
  | congSigmaCod hB => exact ⟨_, _, rfl⟩

theorem redStar_pi_head {A : PureTm n} {B : PureTm (n + 1)} {t : PureTm n}
  (h : RedStar (.pi A B) t) :
  ∃ (A' : PureTm n) (B' : PureTm (n + 1)), t = .pi A' B' :=
  by
  induction h with
  | refl =>
    exact ⟨A, B, rfl⟩
  | tail hxy hyz ih =>
    rcases ih with ⟨Am, Bm, hm⟩
    subst hm
    exact red_pi_head hyz

theorem redStar_sigma_head {A : PureTm n} {B : PureTm (n + 1)} {t : PureTm n}
  (h : RedStar (.sigma A B) t) :
  ∃ (A' : PureTm n) (B' : PureTm (n + 1)), t = .sigma A' B' := by
  induction h with
  | refl =>
    exact ⟨A, B, rfl⟩

```

```

| tail hxy hyz ih =>
  rcases ih with ⟨Am, Bm, ⟩hm
  subst hm
  exact red_sigma_head hyz

private theorem redStar_pi_decomp_full {A : PureTm n} {B : PureTm (n + 1)}
  {t : PureTm n} (h : RedStar (.pi A B) t) :
  ∃ (A' : PureTm n) (B' : PureTm (n + 1)), t = .pi A' B' ∧
    RedStar A A' ∧ RedStar B B' := by
  induction h with
  | refl =>
    exact ⟨A, B, rfl, RedStar.refl _, RedStar.refl ⟩_
  | tail hxy hyz ih =>
    rcases ih with ⟨Am, Bm, hm, hA, ⟩hB
    subst hm
    cases hyz with
    | congPiDom hAstep =>
      exact ⟨_, _, rfl, RedStar.tail hA hAstep, ⟩hB
    | congPiCod hBstep =>
      exact ⟨_, _, rfl, hA, RedStar.tail hB ⟩hBstep

theorem redStar_pi_decomp {A A' : PureTm n} {B B' : PureTm (n + 1)}
  (h : RedStar (.pi A B) (.pi A' B')) :
  RedStar A A' ∧ RedStar B B' := by
  rcases redStar_pi_decomp_full h with ⟨₁A, ₁B, ht, hA, ⟩hB
  simp at ht
  obtain ⟨rfl, ⟩rfl := ht
  exact ⟨hA, ⟩hB

private theorem redStar_sigma_decomp_full {A : PureTm n} {B : PureTm (n + 1)}
  {t : PureTm n} (h : RedStar (.sigma A B) t) :
  ∃ (A' : PureTm n) (B' : PureTm (n + 1)), t = .sigma A' B' ∧
    RedStar A A' ∧ RedStar B B' := by
  induction h with
  | refl =>
    exact ⟨A, B, rfl, RedStar.refl _, RedStar.refl ⟩_
  | tail hxy hyz ih =>
    rcases ih with ⟨Am, Bm, hm, hA, ⟩hB
    subst hm
    cases hyz with
    | congSigmaDom hAstep =>
      exact ⟨_, _, rfl, RedStar.tail hA hAstep, ⟩hB
    | congSigmaCod hBstep =>
      exact ⟨_, _, rfl, hA, RedStar.tail hB ⟩hBstep

theorem redStar_sigma_decomp {A A' : PureTm n} {B B' : PureTm (n + 1)}
  (h : RedStar (.sigma A B) (.sigma A' B')) :
  RedStar A A' ∧ RedStar B B' := by
  rcases redStar_sigma_decomp_full h with ⟨₁A, ₁B, ht, hA, ⟩hB
  simp at ht
  obtain ⟨rfl, ⟩rfl := ht
  exact ⟨hA, ⟩hB

/-- Pi injectivity under conversion, via Church-Rosser + decomposition. -/

```

```

theorem pi_injectivity {A A' : PureTm n} {B B' : PureTm (n + 1)}
  (h : Conv (.pi A B) (.pi A' B')) :
  Conv A A'  $\wedge$  Conv B B' := by
  rcases church_rosser_conv h with (u,  $\text{ih}$ ,  $\text{zh}$ )h
  rcases redStar_pi_head  $\text{ih}$  with (U, V, )hu
  have  $\text{zh}'$  : RedStar (.pi A' B') (.pi U V) := by simpa [hu] using  $\text{zh}$ 
  have  $\text{ih}'$  : RedStar (.pi A B) (.pi U V) := by simpa [hu] using  $\text{ih}$ 
  have  $\text{ihdec}$  := redStar_pi_decomp  $\text{ih}'$ 
  have  $\text{zhdec}$  := redStar_pi_decomp  $\text{zh}'$ 
  have hAU : Conv A U := redStar_implies_conv  $\text{ihdec}$ .1
  have hA'U : Conv A' U := redStar_implies_conv  $\text{zhdec}$ .1
  have hBV : Conv B V := redStar_implies_conv  $\text{ihdec}$ .2
  have hB'V : Conv B' V := redStar_implies_conv  $\text{zhdec}$ .2
  have hUA' : Conv U A' := Relation.EqvGen.symm _ _ hA'U
  have hVB' : Conv V B' := Relation.EqvGen.symm _ _ hB'V
  exact (Relation.EqvGen.trans _ _ _ hAU hUA', Relation.EqvGen.trans _ _ _ hBV hVB)'

/-- Sigma injectivity under conversion, via Church-Rosser + decomposition. -/
theorem sigma_injectivity {A A' : PureTm n} {B B' : PureTm (n + 1)}
  (h : Conv (.sigma A B) (.sigma A' B')) :
  Conv A A'  $\wedge$  Conv B B' := by
  rcases church_rosser_conv h with (u,  $\text{ih}$ ,  $\text{zh}$ )h
  rcases redStar_sigma_head  $\text{ih}$  with (U, V, )hu
  have  $\text{zh}'$  : RedStar (.sigma A' B') (.sigma U V) := by simpa [hu] using  $\text{zh}$ 
  have  $\text{ih}'$  : RedStar (.sigma A B) (.sigma U V) := by simpa [hu] using  $\text{ih}$ 
  have  $\text{ihdec}$  := redStar_sigma_decomp  $\text{ih}'$ 
  have  $\text{zhdec}$  := redStar_sigma_decomp  $\text{zh}'$ 
  have hAU : Conv A U := redStar_implies_conv  $\text{ihdec}$ .1
  have hA'U : Conv A' U := redStar_implies_conv  $\text{zhdec}$ .1
  have hBV : Conv B V := redStar_implies_conv  $\text{ihdec}$ .2
  have hB'V : Conv B' V := redStar_implies_conv  $\text{zhdec}$ .2
  have hUA' : Conv U A' := Relation.EqvGen.symm _ _ hA'U
  have hVB' : Conv V B' := Relation.EqvGen.symm _ _ hB'V
  exact (Relation.EqvGen.trans _ _ _ hAU hUA', Relation.EqvGen.trans _ _ _ hBV hVB)'

end Mettapedia.Languages.MeTTa.PureKernel.Confluence

```

A.4 Context.lean

Mettapedia/Languages/MeTTa/PureKernel/Context.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.Renaming

namespace Mettapedia.Languages.MeTTa.PureKernel.Context

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming

/-- Telescope-style contexts, indexed by number of bound variables. -/
inductive Ctx : Nat  $\rightarrow$  Type where
  | nil : Ctx 0
  | snoc : Ctx n  $\rightarrow$  PureTm n  $\rightarrow$  Ctx (n + 1)
deriving Repr

```

```

def lookup : Ctx n → Fin n → PureTm n
| .nil, i => nomatch i
| .snoc Γ A, i =>
  Fin.cases (rename wk A) (fun j => rename wk (lookup Γ j)) i

@[simp] theorem lookup_snoc_zero Γ( : Ctx n) (A : PureTm n) :
  lookup (.snoc Γ A) 0 = rename wk A := rfl

@[simp] theorem lookup_snoc_succ Γ( : Ctx n) (A : PureTm n) (i : Fin n) :
  lookup (.snoc Γ A) i.succ = rename wk (lookup Γ i) := rfl

/-- Context morphism induced by a renaming, stated as lookup compatibility. -/
def CtxRen Γ( : Ctx n) Δ( : Ctx m) ρ( : Ren n m) : Prop :=
  ∀ i : Fin n, lookup Δ ρ( i) = rename ρ (lookup Γ i)

/-- Lift a context renaming morphism through one binder extension. -/
theorem CtxRen.snoc Γ{ : Ctx n} Δ{ : Ctx m} ρ{ : Ren n m}
  (ph : CtxRen Γ Δ ρ) (A : PureTm n) :
  CtxRen (.snoc Γ A) (.snoc Δ (rename ρ A)) (liftRen ρ) := by
  intro i
  refine Fin.cases ?_ ?_ i
  ·
    calc
      lookup (.snoc Δ (rename ρ A)) (liftRen ρ 0)
      = rename wk (rename ρ A) := by
        simp [lookup_snoc_zero, liftRen]
      _ = rename (fun x => wk ρ( x)) A := by
        simp [rename_comp]
      _ = rename (fun x => liftRen ρ (wk x)) A := by
        apply rename_ext
        intro x
        simp [wk, liftRen]
      _ = rename (liftRen ρ) (rename wk A) := by
        simp [rename_comp]
      _ = rename (liftRen ρ) (lookup (.snoc Γ A) 0) := by
        simp [lookup_snoc_zero]
  · intro j
    calc
      lookup (.snoc Δ (rename ρ A)) (liftRen ρ j.succ)
      = rename wk (lookup Δ ρ( j)) := by
        simp [lookup_snoc_succ, liftRen]
      _ = rename wk (rename ρ (lookup Γ j)) := by
        simp [ph j]
      _ = rename (fun x => wk ρ( x)) (lookup Γ j) := by
        simp [rename_comp]
      _ = rename (fun x => liftRen ρ (wk x)) (lookup Γ j) := by
        apply rename_ext
        intro x
        simp [wk, liftRen]
      _ = rename (liftRen ρ) (rename wk (lookup Γ j)) := by
        simp [rename_comp]
      _ = rename (liftRen ρ) (lookup (.snoc Γ A) j.succ) := by
        simp [lookup_snoc_succ]

```

```
end Mettapedia.Languages.MeTTa.PureKernel.Context
```

A.5 DeclarationEnv.lean

Mettapedia/Languages/MeTTa/PureKernel/DeclarationEnv.lean

```
import Mettapedia.Languages.MeTTa.PureKernel.Renaming

namespace Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming

/-- A closed declaration entry for a global kernel constant. -/
structure DeclEntry where
  type : PureTm 0
  value? : Option (PureTm 0) := none
deriving Repr

/-- Global declaration environment for kernel constants. -/
structure DeclEnv where
  entries : DeclName → Option DeclEntry

/-- `Efull` preserves all declaration entries already present in `Epre`. -/
def Extends (Epre Efull : DeclEnv) : Prop :=
  ∀ {c : DeclName} {entry : DeclEntry},
    Epre.entries c = some entry →
      Efull.entries c = some entry

/-- Empty declaration environment (fail-closed). -/
def empty : DeclEnv := { entries := fun _ => none }

/-- Lookup type of a constant. -/
def typeOf? (E : DeclEnv) (c : DeclName) : Option (PureTm 0) :=
  (E.entries c).map (fun e => e.type)

/-- Lookup optional definitional unfolding of a constant. -/
def valueOf? (E : DeclEnv) (c : DeclName) : Option (PureTm 0) :=
  (E.entries c).bind (fun e => e.value?)

/-- Insert/update a constant entry. -/
def insert (E : DeclEnv) (c : DeclName) (entry : DeclEntry) : DeclEnv :=
  { entries := fun d => if d == c then some entry else E.entries d }

/-- Build a declaration environment from a list of named entries. Earlier entries shadow later ones. -/
def ofList : List (DeclName × DeclEntry) → DeclEnv
| [] => empty
| (c, e) :: rest => insert (ofList rest) c e

/-- Lift a closed term into any de Bruijn depth. -/
def liftClosed (t : PureTm 0) : PureTm n :=
  rename (fun i : Fin 0 => nomatch i) t
```

```

@[simp] theorem typeOf_empty (c : DeclName) : typeOf? empty c = none := rfl

@[simp] theorem valueOf_empty (c : DeclName) : valueOf? empty c = none := rfl

@[simp] theorem typeOf_insert_eq (E : DeclEnv) (c : DeclName) (entry : DeclEntry) :
  typeOf? (insert E c entry) c = some entry.type := by
  simp [typeOf?, insert]

@[simp] theorem valueOf_insert_eq (E : DeclEnv) (c : DeclName) (entry : DeclEntry) :
  valueOf? (insert E c entry) c = entry.value? := by
  simp [valueOf?, insert]

theorem Extends.typeOf {Epre Efull : DeclEnv} (hExt : Extends Epre Efull)
  {c : DeclName} {A0 : PureTm 0}
  (hTy : typeOf? Epre c = some A0) :
  typeOf? Efull c = some A0 := by
  unfold typeOf? at hTy ⊢
  cases hEntries : Epre.entries c with
  | none =>
    simp [hEntries] at hTy
  | some entry =>
    have hType : entry.type = A0 := by
      simpa [hEntries] using hTy
    have hFull : Efull.entries c = some entry := hExt hEntries
    simp [hFull, hType]

theorem Extends.valueOf {Epre Efull : DeclEnv} (hExt : Extends Epre Efull)
  {c : DeclName} {v0 : PureTm 0}
  (hVal : valueOf? Epre c = some v0) :
  valueOf? Efull c = some v0 := by
  unfold valueOf? at hVal ⊢
  cases hEntries : Epre.entries c with
  | none =>
    simp [hEntries] at hVal
  | some entry =>
    have hValue : entry.value? = some v0 := by
      simpa [hEntries] using hVal
    have hFull : Efull.entries c = some entry := hExt hEntries
    simp [hFull, hValue]

theorem Extends.refl (E : DeclEnv) : Extends E E := by
  intro c entry h
  exact h

theorem Extends.trans {1E 2E 3E : DeclEnv}
  (12h : Extends 1E 2E)
  (23h : Extends 2E 3E) :
  Extends 1E 3E := by
  intro c entry h
  exact 23h (12h h)

@[simp] theorem liftClosed_u0 : liftClosed (.u0 : PureTm 0) = (.u0 : PureTm n) := rfl
@[simp] theorem liftClosed_u1 : liftClosed (.u1 : PureTm 0) = (.u1 : PureTm n) := rfl
@[simp] theorem liftClosed_const (c : DeclName) : liftClosed (.const c : PureTm 0) = (.const c :

```



```

    PureTm n) := rfl
@[simp] theorem liftClosed_zero (t : PureTm 0) : liftClosed (n := 0) t = t := by
  unfold liftClosed
  calc
    rename (fun i : Fin 0 => nomatch i) t = rename (idRen (n := 0)) t := by
      apply rename_ext
      intro i
      nomatch i
    _ = t := by
      exact rename_id (t := t)

@[simp] theorem rename_liftClosed {n m : Nat} ρ( : Ren n m) (t : PureTm 0) :
  rename ρ (liftClosed (n := n) t) = liftClosed (n := m) t := by
  unfold liftClosed
  calc
    rename ρ (rename (fun i : Fin 0 => nomatch i) t)
      = rename (fun i : Fin 0 => ρ ((fun j : Fin 0 => nomatch j) i)) t := by
        exact rename_comp ρ₂( := ρ) ρ₁( := (fun i : Fin 0 => nomatch i)) t
    _ = rename (fun i : Fin 0 => nomatch i) t := by
      apply rename_ext
      intro i
      nomatch i
    _ = liftClosed (n := m) t := rfl

end Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv

```

A.6 DeclarationSemantics.lean

Mettapedia/Languages/MeTTa/PureKernel/DeclarationSemantics.lean

```

import Mathlib.Logic.Relation
import Mettapedia.Languages.MeTTa.PureKernel.Typing
import Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv

namespace Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.Reduction
open Mettapedia.Languages.MeTTa.PureKernel.Typing
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv

/-- Declaration-aware one-step reduction.
Includes the base Pure reduction plus fail-closed constant unfolding from `DeclEnv`. -/
inductive RedDecl (E : DeclEnv) : PureTm n → PureTm n → Prop where
| core {t u : PureTm n} :
  Red t u → RedDecl E t u
| deltaConst {c : DeclName} {v : PureTm 0} :
  valueOf? E c = some v →
  RedDecl E (.const c) (liftClosed v)
| congPiDom {A A' : PureTm n} {B : PureTm (n + 1)} :
  RedDecl E A A' → RedDecl E (.pi A B) (.pi A' B)

```

```

| congPiCod {A : PureTm n} {B B' : PureTm (n + 1)} :
  RedDecl E B B' → RedDecl E (.pi A B) (.pi A B')
| congSigmaDom {A A' : PureTm n} {B : PureTm (n + 1)} :
  RedDecl E A A' → RedDecl E (.sigma A B) (.sigma A' B)
| congSigmaCod {A : PureTm n} {B B' : PureTm (n + 1)} :
  RedDecl E B B' → RedDecl E (.sigma A B) (.sigma A B')
| congIdTy {A A' a b : PureTm n} :
  RedDecl E A A' → RedDecl E (.id A a b) (.id A' a b)
| congIdLeft {A a a' b : PureTm n} :
  RedDecl E a a' → RedDecl E (.id A a b) (.id A a' b)
| congIdRight {A a b b' : PureTm n} :
  RedDecl E b b' → RedDecl E (.id A a b) (.id A a b')
| congLam {b b' : PureTm (n + 1)} :
  RedDecl E b b' → RedDecl E (.lam b) (.lam b')
| congAppFun {f f' a : PureTm n} :
  RedDecl E f f' → RedDecl E (.app f a) (.app f' a)
| congAppArg {f a a' : PureTm n} :
  RedDecl E a a' → RedDecl E (.app f a) (.app f a')
| congPairFst {a a' b : PureTm n} :
  RedDecl E a a' → RedDecl E (.pair a b) (.pair a' b)
| congPairSnd {a b b' : PureTm n} :
  RedDecl E b b' → RedDecl E (.pair a b) (.pair a b')
| congFst {p p' : PureTm n} :
  RedDecl E p p' → RedDecl E (.fst p) (.fst p')
| congSnd {p p' : PureTm n} :
  RedDecl E p p' → RedDecl E (.snd p) (.snd p')
| congRefl {a a' : PureTm n} :
  RedDecl E a a' → RedDecl E (.refl a) (.refl a')

```

abbrev RedStarDecl (E : DeclEnv) (t u : PureTm n) : **Prop** :=
 Relation.ReflTransGen (RedDecl E) t u

abbrev ConvDecl (E : DeclEnv) (t u : PureTm n) : **Prop** :=
 Relation.EqvGen (RedDecl E) t u

theorem redDecl_implies_conv {E : DeclEnv} {t u : PureTm n} (h : RedDecl E t u) :
 ConvDecl E t u :=
 Relation.EqvGen.rel _ _ h

theorem conv_core_to_decl {E : DeclEnv} {t u : PureTm n} (h : Conv t u) :
 ConvDecl E t u := **by**
 induction h **with**
 | rel x y hred =>
exact .rel _ _ (.core hred)
 | refl x =>
exact .refl x
 | symm x y hxy ih =>
exact .symm _ _ ih
 | trans x y z hxy hyz ihxy ihyz =>
exact .trans _ _ _ ihxy ihyz

theorem redDecl_monotone {Epre Efull : DeclEnv}
 (hExt : Extends Epre Efull)
 {t u : PureTm n}

```

(h : RedDecl Epre t u) :
RedDecl Efull t u := by
induction h with
| core hred =>
  exact .core hred
| deltaConst hVal =>
  exact .deltaConst (Extends.valueOf hExt hVal)
| congPiDom hred ih =>
  exact .congPiDom ih
| congPiCod hred ih =>
  exact .congPiCod ih
| congSigmaDom hred ih =>
  exact .congSigmaDom ih
| congSigmaCod hred ih =>
  exact .congSigmaCod ih
| congIdTy hred ih =>
  exact .congIdTy ih
| congIdLeft hred ih =>
  exact .congIdLeft ih
| congIdRight hred ih =>
  exact .congIdRight ih
| congLam hred ih =>
  exact .congLam ih
| congAppFun hred ih =>
  exact .congAppFun ih
| congAppArg hred ih =>
  exact .congAppArg ih
| congPairFst hred ih =>
  exact .congPairFst ih
| congPairSnd hred ih =>
  exact .congPairSnd ih
| congFst hred ih =>
  exact .congFst ih
| congSnd hred ih =>
  exact .congSnd ih
| congRefl hred ih =>
  exact .congRefl ih

theorem convDecl_monotone {Epre Efull : DeclEnv}
(hExt : Extends Epre Efull)
{t u : PureTm n}
(h : ConvDecl Epre t u) :
ConvDecl Efull t u := by
induction h with
| rel x y hred =>
  exact .rel _ _ (redDecl_monotone hExt hred)
| refl x =>
  exact .refl x
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

theorem redDecl_rename {E : DeclEnv} {t u : PureTm n} (h : RedDecl E t u) :

```

```

  ∀ {m : Nat} p( : Ren n m), RedDecl E (rename p t) (rename p u) := by
induction h with
| core hred =>
  intro m p
  exact .core (red_rename hred p)
| @deltaConst n c v hVal =>
  intro m p
  have hdelta : RedDecl E ((.const c : PureTm m)) (liftClosed (n := m) v) := .deltaConst hVal
  simp [rename, rename_liftClosed] using hdelta
| congPiDom hred ih =>
  intro m p
  exact .congPiDom (ih p)
| congPiCod hred ih =>
  intro m p
  exact .congPiCod (ih (liftRen p))
| congSigmaDom hred ih =>
  intro m p
  exact .congSigmaDom (ih p)
| congSigmaCod hred ih =>
  intro m p
  exact .congSigmaCod (ih (liftRen p))
| congIdTy hred ih =>
  intro m p
  exact .congIdTy (ih p)
| congIdLeft hred ih =>
  intro m p
  exact .congIdLeft (ih p)
| congIdRight hred ih =>
  intro m p
  exact .congIdRight (ih p)
| congLam hred ih =>
  intro m p
  exact .congLam (ih (liftRen p))
| congAppFun hred ih =>
  intro m p
  exact .congAppFun (ih p)
| congAppArg hred ih =>
  intro m p
  exact .congAppArg (ih p)
| congPairFst hred ih =>
  intro m p
  exact .congPairFst (ih p)
| congPairSnd hred ih =>
  intro m p
  exact .congPairSnd (ih p)
| congFst hred ih =>
  intro m p
  exact .congFst (ih p)
| congSnd hred ih =>
  intro m p
  exact .congSnd (ih p)
| congRefl hred ih =>
  intro m p
  exact .congRefl (ih p)

```

```

theorem convDecl_rename {E : DeclEnv} ρ( : Ren n m) {t u : PureTm n}
  (h : ConvDecl E t u) :
  ConvDecl E (rename ρ t) (rename ρ u) := by
induction h with
| rel x y hred =>
  exact .rel _ _ (redDecl_rename hred ρ)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem subst_vars_eq_rename ρ( : Ren n m) (t : PureTm n) :
  subst (fun i => (.var ρ( i) : PureTm m)) t = rename ρ t := by
calc
  subst (fun i => (.var ρ( i) : PureTm m)) t
    = subst (ids (n := m)) (rename ρ t) := by
      symm
      simpa using (subst_rename σ( := ids (n := m)) ρ( := ρ) (t := t))
  _ = rename ρ t := by
    exact subst_ids (t := rename ρ t)

@[simp] theorem subst_liftClosed {n m : Nat} σ( : Sub n m) (t : PureTm 0) :
  subst σ (liftClosed (n := n) t) = liftClosed (n := m) t := by
let p0 : Ren 0 m := fun i => nomatch i
unfold liftClosed
calc
  subst σ (rename (fun i : Fin 0 => nomatch i) t)
    = subst (fun i : Fin 0 => σ ((fun j : Fin 0 => nomatch j) i)) t := by
      simpa using (subst_rename σ( := σ) ρ( := (fun i : Fin 0 => nomatch i)) (t := t))
  _ = subst (fun i : Fin 0 => nomatch i) t := by
    apply subst_ext
    intro i
    nomatch i
  _ = subst (fun i : Fin 0 => (.var ρ(0 i) : PureTm m)) t := by
    apply subst_ext
    intro i
    nomatch i
  _ = rename p0 t := subst_vars_eq_rename p0 t

theorem redDecl_subst {E : DeclEnv} {t u : PureTm n} (h : RedDecl E t u) :
  ∀ {m : Nat} σ( : Sub n m), RedDecl E (subst σ t) (subst σ u) := by
induction h with
| core hred =>
  intro m σ
  exact .core (red_subst hred σ)
| @deltaConst n c v hVal =>
  intro m σ
  have hdelta : RedDecl E ((.const c : PureTm m)) (liftClosed (n := m) v) := .deltaConst hVal
  simp [subst, subst_liftClosed] using hdelta
| congPiDom hred ih =>
  intro m σ

```

```

    exact .congPiDom (ih σ)
| congPiCod hred ih =>
    intro m σ
    exact .congPiCod (ih (liftSub σ))
| congSigmaDom hred ih =>
    intro m σ
    exact .congSigmaDom (ih σ)
| congSigmaCod hred ih =>
    intro m σ
    exact .congSigmaCod (ih (liftSub σ))
| congIdTy hred ih =>
    intro m σ
    exact .congIdTy (ih σ)
| congIdLeft hred ih =>
    intro m σ
    exact .congIdLeft (ih σ)
| congIdRight hred ih =>
    intro m σ
    exact .congIdRight (ih σ)
| congLam hred ih =>
    intro m σ
    exact .congLam (ih (liftSub σ))
| congAppFun hred ih =>
    intro m σ
    exact .congAppFun (ih σ)
| congAppArg hred ih =>
    intro m σ
    exact .congAppArg (ih σ)
| congPairFst hred ih =>
    intro m σ
    exact .congPairFst (ih σ)
| congPairSnd hred ih =>
    intro m σ
    exact .congPairSnd (ih σ)
| congFst hred ih =>
    intro m σ
    exact .congFst (ih σ)
| congSnd hred ih =>
    intro m σ
    exact .congSnd (ih σ)
| congRefl hred ih =>
    intro m σ
    exact .congRefl (ih σ)

theorem convDecl_subst {E : DeclEnv} σ( : Sub n m) {t u : PureTm n}
  (h : ConvDecl E t u) :
  ConvDecl E (subst σ t) (subst σ u) := by
induction h with
| rel x y hred =>
    exact .rel _ _ (redDecl_subst hred σ)
| refl x =>
    exact .refl _
| symm x y hxy ih =>
    exact .symm _ _ ih

```

```

| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congPiDom {E : DeclEnv} {A A' : PureTm n} {B : PureTm (n + 1)}
  (h : ConvDecl E A A') :
  ConvDecl E (.pi A B) (.pi A' B) := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congPiDom hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congPiCod {E : DeclEnv} {A : PureTm n} {B B' : PureTm (n + 1)}
  (h : ConvDecl E B B') :
  ConvDecl E (.pi A B) (.pi A B') := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congPiCod hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congSigmaDom {E : DeclEnv} {A A' : PureTm n} {B : PureTm (n + 1)}
  (h : ConvDecl E A A') :
  ConvDecl E (.sigma A B) (.sigma A' B) := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congSigmaDom hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congSigmaCod {E : DeclEnv} {A : PureTm n} {B B' : PureTm (n + 1)}
  (h : ConvDecl E B B') :
  ConvDecl E (.sigma A B) (.sigma A B') := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congSigmaCod hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

```

```

private theorem convDecl_congIdTy {E : DeclEnv} {A A' a b : PureTm n}
  (h : ConvDecl E A A') :
  ConvDecl E (.id A a b) (.id A' a b) := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congIdTy hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congIdLeft {E : DeclEnv} {A a a' b : PureTm n}
  (h : ConvDecl E a a') :
  ConvDecl E (.id A a b) (.id A a' b) := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congIdLeft hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congIdRight {E : DeclEnv} {A a b b' : PureTm n}
  (h : ConvDecl E b b') :
  ConvDecl E (.id A a b) (.id A a b') := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congIdRight hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congLam {E : DeclEnv} {b b' : PureTm (n + 1)}
  (h : ConvDecl E b b') :
  ConvDecl E (.lam b) (.lam b') := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congLam hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congAppFun {E : DeclEnv} {f f' a : PureTm n}

```



```

    (h : ConvDecl E f f') :
    ConvDecl E (.app f a) (.app f' a) := by
induction h with
| rel _ _ hred =>
    exact .rel _ _ (.congAppFun hred)
| refl x =>
    exact .refl _
| symm x y hxy ih =>
    exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
    exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congAppArg {E : DeclEnv} {f a a' : PureTm n}
    (h : ConvDecl E a a') :
    ConvDecl E (.app f a) (.app f a') := by
induction h with
| rel _ _ hred =>
    exact .rel _ _ (.congAppArg hred)
| refl x =>
    exact .refl _
| symm x y hxy ih =>
    exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
    exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congPairFst {E : DeclEnv} {a a' b : PureTm n}
    (h : ConvDecl E a a') :
    ConvDecl E (.pair a b) (.pair a' b) := by
induction h with
| rel _ _ hred =>
    exact .rel _ _ (.congPairFst hred)
| refl x =>
    exact .refl _
| symm x y hxy ih =>
    exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
    exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congPairSnd {E : DeclEnv} {a b b' : PureTm n}
    (h : ConvDecl E b b') :
    ConvDecl E (.pair a b) (.pair a b') := by
induction h with
| rel _ _ hred =>
    exact .rel _ _ (.congPairSnd hred)
| refl x =>
    exact .refl _
| symm x y hxy ih =>
    exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
    exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congFst {E : DeclEnv} {p p' : PureTm n}
    (h : ConvDecl E p p') :
    ConvDecl E (.fst p) (.fst p') := by

```

```

induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congFst hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congSnd {E : DeclEnv} {p p' : PureTm n}
  (h : ConvDecl E p p') :
  ConvDecl E (.snd p) (.snd p') := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congSnd hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_congRefl {E : DeclEnv} {a a' : PureTm n}
  (h : ConvDecl E a a') :
  ConvDecl E (.refl a) (.refl a') := by
induction h with
| rel _ _ hred =>
  exact .rel _ _ (.congRefl hred)
| refl x =>
  exact .refl _
| symm x y hxy ih =>
  exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
  exact .trans _ _ _ ihxy ihyz

private theorem convDecl_liftSub_congr {E : DeclEnv} σ {τ : Sub n m}
  (σth : ∀ i : Fin n, ConvDecl E σ( i) τ( i)) :
  ∀ i : Fin (n + 1), ConvDecl E (liftSub σ i) (liftSub τ i) := by
intro i
refine Fin.cases ?_ ?_ i
· exact .refl _
· intro j
  simp [liftSub] using (convDecl_rename (E := E) ρ( := wk) (σth j))

/- Conversion congruence under pointwise-convertible substitutions. -/
theorem convDecl_subst_congr {E : DeclEnv} σ {τ : Sub n m}
  (σth : ∀ i : Fin n, ConvDecl E σ( i) τ( i)) :
  ∀ t : PureTm n, ConvDecl E (subst σ t) (subst τ t) := by
intro t
induction t generalizing m with
| var i =>
  simp [subst] using σth i
| const c =>

```

```

    exact .refl _
| u0 =>
    exact .refl _
| u1 =>
    exact .refl _
| pi A B ihA ihB =>
    have hA : ConvDecl E (subst σ A) (subst τ A) := ihA σth
    have hB : ConvDecl E (subst (liftSub σ) B) (subst (liftSub τ) B) :=
      ihB (convDecl_liftSub_congr (E := E) σth)
    exact Relation.EqvGen.trans _ _ _ (convDecl_congPiDom hA) (convDecl_congPiCod hB)
| sigma A B ihA ihB =>
    have hA : ConvDecl E (subst σ A) (subst τ A) := ihA σth
    have hB : ConvDecl E (subst (liftSub σ) B) (subst (liftSub τ) B) :=
      ihB (convDecl_liftSub_congr (E := E) σth)
    exact Relation.EqvGen.trans _ _ _ (convDecl_congSigmaDom hA) (convDecl_congSigmaCod hB)
| id A a b ihA iha ihb =>
    have hA : ConvDecl E (subst σ A) (subst τ A) := ihA σth
    have ha : ConvDecl E (subst σ a) (subst τ a) := iha σth
    have hb : ConvDecl E (subst σ b) (subst τ b) := ihb σth
    exact Relation.EqvGen.trans _ _ _ (convDecl_congIdTy hA)
      (Relation.EqvGen.trans _ _ _ (convDecl_congIdLeft ha) (convDecl_congIdRight hb))
| lam b ih =>
    have hb : ConvDecl E (subst (liftSub σ) b) (subst (liftSub τ) b) :=
      ih (convDecl_liftSub_congr (E := E) σth)
    exact convDecl_congLam hb
| app f a ihf iha =>
    have hf : ConvDecl E (subst σ f) (subst τ f) := ihf σth
    have ha : ConvDecl E (subst σ a) (subst τ a) := iha σth
    exact Relation.EqvGen.trans _ _ _ (convDecl_congAppFun hf) (convDecl_congAppArg ha)
| pair a b iha ihb =>
    have ha : ConvDecl E (subst σ a) (subst τ a) := iha σth
    have hb : ConvDecl E (subst σ b) (subst τ b) := ihb σth
    exact Relation.EqvGen.trans _ _ _ (convDecl_congPairFst ha) (convDecl_congPairSnd hb)
| fst p ih =>
    exact convDecl_congFst (ih σth)
| snd p ih =>
    exact convDecl_congSnd (ih σth)
| refl a iha =>
    exact convDecl_congRefl (iha σth)

/- Argument convertibility implies convertibility after single-variable instantiation. -/
theorem inst0_arg_conv_decl {E : DeclEnv} {a a' : PureTm n} {B : PureTm (n + 1)}
  (h : ConvDecl E a a') :
  ConvDecl E (inst0 a B) (inst0 a' B) := by
  have σth : ∀ i : Fin (n + 1), ConvDecl E (subst0 a i) (subst0 a' i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · simp [subst0]
      using h
    · intro j
      exact .refl _
  simp [inst0] using
    (convDecl_subst_congr (E := E) σ( := subst0 a) τ( := subst0 a') σth B)

```

```

/-- Declaration-aware typing judgment.
This extends the kernel typing rules with global constants looked up from `DeclEnv`. -/
inductive HasTypeDecl (E : DeclEnv) : Ctx n → PureTm n → PureTm n → Prop where
| u0_type Γ ( : Ctx n) :
  HasTypeDecl E Γ .u0 .u1
| var Γ{ : Ctx n} (i : Fin n) :
  HasTypeDecl E Γ (.var i) (lookup Γ i)
| const Γ{ : Ctx n} {c : DeclName} {A0 : PureTm 0} :
  typeOf? E c = some A0 →
  HasTypeDecl E Γ (.const c) (liftClosed A0)
| pi_form Γ{ : Ctx n} {A : PureTm n} {B : PureTm (n + 1)} :
  HasTypeDecl E Γ A .u1 →
  HasTypeDecl E (.snoc Γ A) B .u1 →
  HasTypeDecl E Γ (.pi A B) .u1
| sigma_form Γ{ : Ctx n} {A : PureTm n} {B : PureTm (n + 1)} :
  HasTypeDecl E Γ A .u1 →
  HasTypeDecl E (.snoc Γ A) B .u1 →
  HasTypeDecl E Γ (.sigma A B) .u1
| lam_intro Γ{ : Ctx n} {A : PureTm n} {body B : PureTm (n + 1)} :
  HasTypeDecl E (.snoc Γ A) body B →
  HasTypeDecl E Γ (.lam body) (.pi A B)
| app_elim Γ{ : Ctx n} {f a A : PureTm n} {B : PureTm (n + 1)} :
  HasTypeDecl E Γ f (.pi A B) →
  HasTypeDecl E Γ a A →
  HasTypeDecl E Γ (.app f a) (inst0 a B)
| pair_intro Γ{ : Ctx n} {a b A : PureTm n} {B : PureTm (n + 1)} :
  HasTypeDecl E Γ a A →
  HasTypeDecl E Γ b (inst0 a B) →
  HasTypeDecl E Γ (.pair a b) (.sigma A B)
| fst_elim Γ{ : Ctx n} {p A : PureTm n} {B : PureTm (n + 1)} :
  HasTypeDecl E Γ p (.sigma A B) →
  HasTypeDecl E Γ (.fst p) A
| snd_elim Γ{ : Ctx n} {p A : PureTm n} {B : PureTm (n + 1)} :
  HasTypeDecl E Γ p (.sigma A B) →
  HasTypeDecl E Γ (.snd p) (inst0 (.fst p) B)
| id_form Γ{ : Ctx n} {A a b : PureTm n} :
  HasTypeDecl E Γ A .u1 →
  HasTypeDecl E Γ a A →
  HasTypeDecl E Γ b A →
  HasTypeDecl E Γ (.id A a b) .u1
| refl_intro Γ{ : Ctx n} {a A : PureTm n} :
  HasTypeDecl E Γ a A →
  HasTypeDecl E Γ (.refl a) (.id A a a)
| conv Γ{ : Ctx n} {t A B : PureTm n} :
  HasTypeDecl E Γ t A →
  ConvDecl E A B →
  HasTypeDecl E Γ t B

theorem hasTypeDecl_monotone {Epre Efull : DeclEnv}
  (hExt : Extends Epre Efull)
  Γ{ : Ctx n} {t A : PureTm n}
  (ht : HasTypeDecl Epre Γ t A) :
  HasTypeDecl Efull Γ t A := by
induction ht with

```

```

| u0_type  $\Gamma \Rightarrow$ 
  exact .u0_type  $\Gamma$ 
| var i  $\Rightarrow$ 
  exact .var i
| @const n  $\Gamma$  c A0 hLookup  $\Rightarrow$ 
  exact .const (Extends.typeOf hExt hLookup)
| pi_form hA hB ihA ihB  $\Rightarrow$ 
  exact .pi_form ihA ihB
| sigma_form hA hB ihA ihB  $\Rightarrow$ 
  exact .sigma_form ihA ihB
| lam_intro hBody ihBody  $\Rightarrow$ 
  exact .lam_intro ihBody
| app_elim hf ha ihf iha  $\Rightarrow$ 
  exact .app_elim ihf iha
| pair_intro ha hb iha ihb  $\Rightarrow$ 
  exact .pair_intro iha ihb
| fst_elim hp ihp  $\Rightarrow$ 
  exact .fst_elim ihp
| snd_elim hp ihp  $\Rightarrow$ 
  exact .snd_elim ihp
| id_form hA ha hb ihA iha ihb  $\Rightarrow$ 
  exact .id_form ihA iha ihb
| refl_intro ha iha  $\Rightarrow$ 
  exact .refl_intro iha
| conv ht hAB iht  $\Rightarrow$ 
  exact .conv iht (convDecl_monotone hExt hAB)

```

theorem hasType_core_to_decl {E : DeclEnv} $\Gamma\{ : \text{Ctx } n\}$ {t A : PureTm n}

(h : HasType Γ t A) : HasTypeDecl E Γ t A := **by**

induction h **with**

```

| u0_type  $\Gamma \Rightarrow$ 
  exact .u0_type  $\Gamma$ 
| var i  $\Rightarrow$ 
  exact .var i
| pi_form hA hB ihA ihB  $\Rightarrow$ 
  exact .pi_form ihA ihB
| sigma_form hA hB ihA ihB  $\Rightarrow$ 
  exact .sigma_form ihA ihB
| lam_intro hBody ihBody  $\Rightarrow$ 
  exact .lam_intro ihBody
| app_elim hf ha ihf iha  $\Rightarrow$ 
  exact .app_elim ihf iha
| pair_intro ha hb iha ihb  $\Rightarrow$ 
  exact .pair_intro iha ihb
| fst_elim hp ihp  $\Rightarrow$ 
  exact .fst_elim ihp
| snd_elim hp ihp  $\Rightarrow$ 
  exact .snd_elim ihp
| id_form hA ha hb ihA iha ihb  $\Rightarrow$ 
  exact .id_form ihA iha ihb
| refl_intro ha iha  $\Rightarrow$ 
  exact .refl_intro iha
| conv ht hAB iht  $\Rightarrow$ 
  exact .conv iht (conv_core_to_decl hAB)

```

```

/-- Typed context morphism for declaration-aware simultaneous substitution. -/
def CtxMorDecl (E : DeclEnv)  $\Gamma$ ( : Ctx n)  $\Delta$ ( : Ctx m)  $\sigma$ ( : Sub n m) : Prop :=
   $\forall$  i : Fin n, HasTypeDecl E  $\Delta$   $\sigma$ ( i) (subst  $\sigma$  (lookup  $\Gamma$  i))

theorem typing_rename_decl {E : DeclEnv}  $\Gamma$ { : Ctx n} {t A : PureTm n}
  (ht : HasTypeDecl E  $\Gamma$  t A) :
   $\forall$  {m : Nat}  $\Delta$ { : Ctx m}  $\rho$ { : Ren n m},
    CtxRen  $\Gamma$   $\Delta$   $\rho$   $\rightarrow$ 
    HasTypeDecl E  $\Delta$  (rename  $\rho$  t) (rename  $\rho$  A) := by
  induction ht with
  | u0_type  $\Gamma$  =>
    intro m  $\Delta$   $\rho$  ph
    simpa [rename] using (HasTypeDecl.u0_type (E := E)  $\Gamma$ ( :=  $\Delta$ ))
  | var i =>
    intro m  $\Delta$   $\rho$  ph
    simpa [ph i] using (HasTypeDecl.var (E := E)  $\Gamma$ ( :=  $\Delta$ ) (i :=  $\rho$  i))
  | @const n  $\Gamma$  c A0 hLookup =>
    intro m  $\Delta$   $\rho$  ph
    simpa [rename, rename_liftClosed] using
      (HasTypeDecl.const (E := E)  $\Gamma$ ( :=  $\Delta$ ) (c := c) (A0 := A0) hLookup)
  | @pi_form n  $\Gamma$  A B hA hB ihA ihB =>
    intro m  $\Delta$   $\rho$  ph
    have hA' := ihA (m := m)  $\Delta$ ( :=  $\Delta$ )  $\rho$ ( :=  $\rho$ ) ph
    have hB' := ihB (m := m + 1)  $\Delta$ ( := .snoc  $\Delta$  (rename  $\rho$  A))  $\rho$ ( := liftRen  $\rho$ )
      (CtxRen.snoc ph A)
    simpa [rename] using (HasTypeDecl.pi_form hA' hB')
  | @sigma_form n  $\Gamma$  A B hA hB ihA ihB =>
    intro m  $\Delta$   $\rho$  ph
    have hA' := ihA (m := m)  $\Delta$ ( :=  $\Delta$ )  $\rho$ ( :=  $\rho$ ) ph
    have hB' := ihB (m := m + 1)  $\Delta$ ( := .snoc  $\Delta$  (rename  $\rho$  A))  $\rho$ ( := liftRen  $\rho$ )
      (CtxRen.snoc ph A)
    simpa [rename] using (HasTypeDecl.sigma_form hA' hB')
  | @lam_intro n  $\Gamma$  A body B hBody ihBody =>
    intro m  $\Delta$   $\rho$  ph
    have hBody' := ihBody (m := m + 1)  $\Delta$ ( := .snoc  $\Delta$  (rename  $\rho$  A))  $\rho$ ( := liftRen  $\rho$ )
      (CtxRen.snoc ph A)
    simpa [rename] using (HasTypeDecl.lam_intro hBody')
  | @app_elim n  $\Gamma$  f a A B hf ha ihf iha =>
    intro m  $\Delta$   $\rho$  ph
    simpa [rename, rename_inst0] using
      (HasTypeDecl.app_elim
        (ihf (m := m)  $\Delta$ ( :=  $\Delta$ )  $\rho$ ( :=  $\rho$ ) ph)
        (iha (m := m)  $\Delta$ ( :=  $\Delta$ )  $\rho$ ( :=  $\rho$ ) ph))
  | @pair_intro n  $\Gamma$  a b A B ha hb iha ihb =>
    intro m  $\Delta$   $\rho$  ph
    have ha' := iha (m := m)  $\Delta$ ( :=  $\Delta$ )  $\rho$ ( :=  $\rho$ ) ph
    have hb' :
      HasTypeDecl E  $\Delta$  (rename  $\rho$  b) (inst0 (rename  $\rho$  a) (rename (liftRen  $\rho$ ) B)) := by
        simpa [rename_inst0] using (ihb (m := m)  $\Delta$ ( :=  $\Delta$ )  $\rho$ ( :=  $\rho$ ) ph)
        simpa [rename] using (HasTypeDecl.pair_intro ha' hb')
  | @fst_elim n  $\Gamma$  p A B hp ihp =>
    intro m  $\Delta$   $\rho$  ph
    simpa [rename] using

```

```

      (HasTypeDecl.fst_elim (ihp (m := m) Δ( := Δ) ρ( := ρ) ph))
| @snd_elim n Γ p A B hp ihp =>
  intro m Δ p ph
  simp [rename, rename_inst0] using
    (HasTypeDecl.snd_elim (ihp (m := m) Δ( := Δ) ρ( := ρ) ph))
| @id_form n Γ A a b hA ha hb ihA iha ihb =>
  intro m Δ p ph
  simp [rename] using
    (HasTypeDecl.id_form
      (ihA (m := m) Δ( := Δ) ρ( := ρ) ph)
      (iha (m := m) Δ( := Δ) ρ( := ρ) ph)
      (ihb (m := m) Δ( := Δ) ρ( := ρ) ph))
| @refl_intro n Γ a A ha iha =>
  intro m Δ p ph
  simp [rename] using
    (HasTypeDecl.refl_intro (iha (m := m) Δ( := Δ) ρ( := ρ) ph))
| @conv n Γ t A B ht hAB iht =>
  intro m Δ p ph
  exact HasTypeDecl.conv
    (iht (m := m) Δ( := Δ) ρ( := ρ) ph)
    (convDecl_rename ρ hAB)

theorem weakening_decl {E : DeclEnv} Γ{ : Ctx n} {t A U : PureTm n}
  (ht : HasTypeDecl E Γ t A) :
  HasTypeDecl E (.snoc Γ U) (rename wk t) (rename wk A) := by
have hwk : CtxRen Γ (.snoc Γ U) wk := by
  intro i
  simp [wk]
  simp using
    (typing_rename_decl ht (m := n + 1) Δ( := .snoc Γ U) ρ( := wk) hwk)

theorem CtxMorDecl.lift {E : DeclEnv} Γ{ : Ctx n} Δ{ : Ctx m} σ{ : Sub n m}
  (oh : CtxMorDecl E Γ Δ σ) (A : PureTm n) :
  CtxMorDecl E (.snoc Γ A) (.snoc Δ (subst σ A)) (liftSub σ) := by
intro i
  refine Fin.cases ?_ ?_ i
  · simp [CtxMorDecl, lookup_snoc_zero, liftSub, subst_liftSub_wk] using
    (HasTypeDecl.var (E := E) Γ( := .snoc Δ (subst σ A)) (i := (0 : Fin (m + 1))))
  · intro j
    have hwk : CtxRen Δ (.snoc Δ (subst σ A)) wk := by
      intro i
      simp [wk]
    have hj : HasTypeDecl E (.snoc Δ (subst σ A))
      (rename wk σ( j))
      (rename wk (subst σ (lookup Γ j))) := by
      exact typing_rename_decl (ht := oh j) (m := m + 1) Δ( := .snoc Δ (subst σ A))
        ρ( := wk) hwk
    simp [CtxMorDecl, lookup_snoc_succ, liftSub, subst_liftSub_wk] using hj

/-- Generic declaration-aware simultaneous typing substitution along a typed context morphism. -/
theorem typing_subst_decl {E : DeclEnv} Γ{ : Ctx n} {t A : PureTm n} (ht : HasTypeDecl E Γ t A) :
  ∀ {m : Nat} Δ{ : Ctx m} σ{ : Sub n m},
  CtxMorDecl E Γ Δ σ → HasTypeDecl E Δ (subst σ t) (subst σ A) := by
  induction ht with

```

```

| u0_type  $\Gamma \Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [subst] using (HasTypeDecl.u0_type (E := E)  $\Gamma$  (:=  $\Delta$ ))
| var i  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [CtxMorDecl] using oh i
| @const n  $\Gamma$  c A0 hLookup  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [subst, subst_liftClosed] using
    (HasTypeDecl.const (E := E)  $\Gamma$  (:=  $\Delta$ ) (c := c) (A0 := A0) hLookup)
| @pi_form n  $\Gamma$  A B hA hB ihA ihB  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  have hA' := ihA (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh
  have hB' := ihB (m := m + 1)  $\Delta$  (:= .snoc  $\Delta$  (subst  $\sigma$  A))  $\sigma$  (:= liftSub  $\sigma$ )
    (CtxMorDecl.lift oh A)
  simpa [subst] using (HasTypeDecl.pi_form hA' hB')
| @sigma_form n  $\Gamma$  A B hA hB ihA ihB  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  have hA' := ihA (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh
  have hB' := ihB (m := m + 1)  $\Delta$  (:= .snoc  $\Delta$  (subst  $\sigma$  A))  $\sigma$  (:= liftSub  $\sigma$ )
    (CtxMorDecl.lift oh A)
  simpa [subst] using (HasTypeDecl.sigma_form hA' hB')
| @lam_intro n  $\Gamma$  A body B hBody ihBody  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  have hBody' := ihBody (m := m + 1)  $\Delta$  (:= .snoc  $\Delta$  (subst  $\sigma$  A))  $\sigma$  (:= liftSub  $\sigma$ )
    (CtxMorDecl.lift oh A)
  simpa [subst] using (HasTypeDecl.lam_intro hBody')
| @app_elim n  $\Gamma$  f a A B hf ha ihf iha  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [subst, subst_inst0] using
    (HasTypeDecl.app_elim
      (ihf (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh)
      (iha (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh))
| @pair_intro n  $\Gamma$  a b A B ha hb iha ihb  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  have ha' := iha (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh
  have hb' :
    HasTypeDecl E  $\Delta$  (subst  $\sigma$  b) (inst0 (subst  $\sigma$  a) (subst (liftSub  $\sigma$ ) B)) := by
      simpa [subst_inst0] using (ihb (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh)
  simpa [subst] using (HasTypeDecl.pair_intro ha' hb')
| @fst_elim n  $\Gamma$  p A B hp ihp  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [subst] using
    (HasTypeDecl.fst_elim (ihp (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh))
| @snd_elim n  $\Gamma$  p A B hp ihp  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [subst, subst_inst0] using
    (HasTypeDecl.snd_elim (ihp (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh))
| @id_form n  $\Gamma$  A a b hA ha hb ihA iha ihb  $\Rightarrow$ 
  intro m  $\Delta$   $\sigma$  oh
  simpa [subst] using
    (HasTypeDecl.id_form
      (ihA (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh)
      (iha (m := m)  $\Delta$  (:=  $\Delta$ )  $\sigma$  (:=  $\sigma$ ) oh))

```



```

      (ihb (m := m) Δ( := Δ) σ( := σ) oh))
| @refl_intro n Γ a A ha iha =>
  intro m Δ σ oh
  simp [subst] using
    (HasTypeDecl.refl_intro (iha (m := m) Δ( := Δ) σ( := σ) oh))
| @conv n Γ t A B ht hAB iht =>
  intro m Δ σ oh
  exact HasTypeDecl.conv
    (iht (m := m) Δ( := Δ) σ( := σ) oh)
    (convDecl_subst σ hAB)

/-- Transport declaration-aware typing along pointwise context conversion. -/
theorem context_conv_decl {E : DeclEnv} Γ{ Δ : Ctx n} {t T : PureTm n}
  (hctx : ∀ i : Fin n, ConvDecl E (lookup Γ i) (lookup Δ i))
  (ht : HasTypeDecl E Γ t T) :
  HasTypeDecl E Δ t T := by
induction ht with
| u0_type Γ =>
  exact .u0_type _
| var i =>
  exact .conv
    (HasTypeDecl.var (E := E) Γ( := Δ) (i := i))
    (Relation.EqvGen.symm _ _ (hctx i))
| @const n Γ c A0 hLookup =>
  exact .const hLookup
| @pi_form n Γ A B hA hB ihA ihB =>
  have hA' : HasTypeDecl E Δ A .u1 := ihA hctx
  have hctx' : ∀ i : Fin (n + 1), ConvDecl E (lookup (.snoc Γ A) i) (lookup (.snoc Δ A) i) :=
  by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .refl _
    · intro j
      simp [lookup_snoc_succ] using (convDecl_rename wk (hctx j))
  have hB' : HasTypeDecl E (.snoc Δ A) B .u1 := ihB hctx'
  exact .pi_form hA' hB'
| @sigma_form n Γ A B hA hB ihA ihB =>
  have hA' : HasTypeDecl E Δ A .u1 := ihA hctx
  have hctx' : ∀ i : Fin (n + 1), ConvDecl E (lookup (.snoc Γ A) i) (lookup (.snoc Δ A) i) :=
  by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .refl _
    · intro j
      simp [lookup_snoc_succ] using (convDecl_rename wk (hctx j))
  have hB' : HasTypeDecl E (.snoc Δ A) B .u1 := ihB hctx'
  exact .sigma_form hA' hB'
| @lam_intro n Γ A body B hBody ihBody =>
  have hctx' : ∀ i : Fin (n + 1), ConvDecl E (lookup (.snoc Γ A) i) (lookup (.snoc Δ A) i) :=
  by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .refl _
    · intro j

```

```

      simpa [lookup_snoc_succ] using (convDecl_rename wk (hctx j))
    exact .lam_intro (ihBody hctx')
| @app_elim n  $\Gamma$  f a A B hf ha ihf iha =>
    exact .app_elim (ihf hctx) (iha hctx)
| @pair_intro n  $\Gamma$  a b A B ha hb iha ihb =>
    exact .pair_intro (iha hctx) (ihb hctx)
| @fst_elim n  $\Gamma$  p A B hp ihp =>
    exact .fst_elim (ihp hctx)
| @snd_elim n  $\Gamma$  p A B hp ihp =>
    exact .snd_elim (ihp hctx)
| @id_form n  $\Gamma$  A a b hA ha hb iha iha ihb =>
    exact .id_form (ihA hctx) (iha hctx) (ihb hctx)
| @refl_intro n  $\Gamma$  a A ha iha =>
    exact .refl_intro (iha hctx)
| @conv n  $\Gamma$  t A B ht hAB iht =>
    exact .conv (iht hctx) hAB

/-- Specialization of `context_conv_decl` to changing only the head binder type. -/
theorem context_conv_head_decl {E : DeclEnv}  $\Gamma$ { : Ctx n} {A A' : PureTm n} {t T : PureTm (n + 1)}
  (hAA' : ConvDecl E A A') (ht : HasTypeDecl E (.snoc  $\Gamma$  A) t T) :
  HasTypeDecl E (.snoc  $\Gamma$  A') t T := by
have hctx :  $\forall$  i : Fin (n + 1),
  ConvDecl E (lookup (.snoc  $\Gamma$  A) i) (lookup (.snoc  $\Gamma$  A') i) := by
  intro i
  refine Fin.cases ?_ ?_ i
  · simp [lookup_snoc_zero] using (convDecl_rename p( := wk) hAA')
  · intro j
    simp [lookup_snoc_succ] using (Relation.EqvGen.refl (lookup (.snoc  $\Gamma$  A) j.succ))
exact context_conv_decl hctx ht

private theorem app_generation_decl_aux {E : DeclEnv}  $\Gamma$ { : Ctx n} {t C : PureTm n}
  (ht : HasTypeDecl E  $\Gamma$  t C) :
   $\forall$  {f a : PureTm n}, t = .app f a  $\rightarrow$ 
   $\exists$  A B, HasTypeDecl E  $\Gamma$  f (.pi A B)  $\wedge$  HasTypeDecl E  $\Gamma$  a A  $\wedge$  ConvDecl E (inst0 a B) C := by
induction ht with
| app_elim hf ha =>
  intro f a hEq
  cases hEq
  exact (_, _, hf, ha, .refl )_
| conv ht hconv ih =>
  intro f a hEq
  rcases ih hEq with (A, B, hf, ha, )hC
  exact (A, B, hf, ha, .trans _ _ _ hC )hconv
| u0_type =>
  intro f a hEq
  cases hEq
| var =>
  intro f a hEq
  cases hEq
| const =>
  intro f a hEq
  cases hEq
| pi_form hA hB ihA ihB =>
  intro f a hEq

```

```

      cases hEq
| sigma_form hA hB ihA ihB =>
  intro f a hEq
  cases hEq
| lam_intro hBody ihBody =>
  intro f a hEq
  cases hEq
| pair_intro ha hb iha ihb =>
  intro f a hEq
  cases hEq
| fst_elim hp ihp =>
  intro f a hEq
  cases hEq
| snd_elim hp ihp =>
  intro f a hEq
  cases hEq
| id_form hA ha hb ihA iha ihb =>
  intro f a hEq
  cases hEq
| refl_intro ha iha =>
  intro f a hEq
  cases hEq

theorem app_generation_decl {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\} \{f a C : \text{PureTm } n\}$ 
  (ht : HasTypeDecl E  $\Gamma$  (.app f a) C) :
   $\exists A B, \text{HasTypeDecl } E \Gamma f (.pi A B) \wedge \text{HasTypeDecl } E \Gamma a A \wedge \text{ConvDecl } E (\text{inst0 } a B) C :=$ 
  app_generation_decl_aux ht rfl

private theorem lam_generation_decl_aux {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\} \{t C : \text{PureTm } n\}$ 
  (ht : HasTypeDecl E  $\Gamma$  t C) :
   $\forall \{body : \text{PureTm } (n + 1)\}, t = .lam \text{ body} \rightarrow$ 
   $\exists A B, \text{HasTypeDecl } E (.snoc \Gamma A) \text{ body } B \wedge \text{ConvDecl } E (.pi A B) C :=$  by
  induction ht with
| lam_intro hBody =>
  intro body hEq
  cases hEq
  exact (_, _, hBody, .refl) _
| conv ht hconv ih =>
  intro body hEq
  rcases ih hEq with (A, B, hBody, )hPi
  exact (A, B, hBody, .trans _ _ _ hPi) hconv
| u0_type =>
  intro body hEq
  cases hEq
| var =>
  intro body hEq
  cases hEq
| const =>
  intro body hEq
  cases hEq
| pi_form hA hB ihA ihB =>
  intro body hEq
  cases hEq
| sigma_form hA hB ihA ihB =>

```

```

      intro body hEq
      cases hEq
| app_elim hf ha ihf iha =>
      intro body hEq
      cases hEq
| pair_intro ha hb iha ihb =>
      intro body hEq
      cases hEq
| fst_elim hp ihp =>
      intro body hEq
      cases hEq
| snd_elim hp ihp =>
      intro body hEq
      cases hEq
| id_form hA ha hb ihA iha ihb =>
      intro body hEq
      cases hEq
| refl_intro ha iha =>
      intro body hEq
      cases hEq

theorem lam_generation_decl {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\}$  {body : PureTm (n + 1)} {C : PureTm n}
  (ht : HasTypeDecl E  $\Gamma$  (.lam body) C) :
   $\exists A B, \text{HasTypeDecl } E \text{ (.snoc } \Gamma A) \text{ body } B \wedge \text{ConvDecl } E \text{ (.pi } A B) C :=$ 
  lam_generation_decl_aux ht rfl

private theorem pair_generation_decl_aux {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\}$  {t C : PureTm n}
  (ht : HasTypeDecl E  $\Gamma$  t C) :
   $\forall \{a b : \text{PureTm } n\}, t = \text{.pair } a b \rightarrow$ 
   $\exists A B, \text{HasTypeDecl } E \Gamma a A \wedge \text{HasTypeDecl } E \Gamma b (\text{inst0 } a B) \wedge \text{ConvDecl } E \text{ (.sigma } A B) C :=$ 
  by
  induction ht with
| pair_intro ha hb =>
      intro a b hEq
      cases hEq
      exact ( $\_, \_, ha, hb, \text{.refl}$ )_
| conv ht hconv ih =>
      intro a b hEq
      rcases ih hEq with (A, B, ha, hb, )hSigma
      exact (A, B, ha, hb, .trans _ _ _ hSigma )hconv
| u0_type =>
      intro a b hEq
      cases hEq
| var =>
      intro a b hEq
      cases hEq
| const =>
      intro a b hEq
      cases hEq
| pi_form hA hB ihA ihB =>
      intro a b hEq
      cases hEq
| sigma_form hA hB ihA ihB =>
      intro a b hEq

```

```

      cases hEq
| lam_intro hBody ihBody =>
  intro a b hEq
  cases hEq
| app_elim hf ha ihf iha =>
  intro a b hEq
  cases hEq
| fst_elim hp ihp =>
  intro a b hEq
  cases hEq
| snd_elim hp ihp =>
  intro a b hEq
  cases hEq
| id_form hA ha hb ihA iha ihb =>
  intro a b hEq
  cases hEq
| refl_intro ha iha =>
  intro a b hEq
  cases hEq

theorem pair_generation_decl {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\} \{a \ b \ C : \text{PureTm } n\}$ 
  (ht : HasTypeDecl E  $\Gamma$  (.pair a b) C) :
   $\exists \ A \ B, \text{HasTypeDecl } E \ \Gamma \ a \ A \wedge \text{HasTypeDecl } E \ \Gamma \ b \ (inst0 \ a \ B) \wedge \text{ConvDecl } E \ (.sigma \ A \ B) \ C :=$ 
  pair_generation_decl_aux ht rfl

private theorem subst0_wk_cancel_decl (a t : PureTm n) :
  subst (subst0 a) (rename wk t) = t := by
calc
  subst (subst0 a) (rename wk t)
    = subst (fun i => subst0 a (wk i)) t := by
      simpa using (subst_rename  $\sigma$ ( := subst0 a)  $\rho$ ( := wk) (t := t))
  _ = subst ids t := by
    apply subst_ext
    intro i
    rfl
  _ = t := by
    exact subst_ids (t := t)

private theorem ctxMorDecl_subst0 {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\} \{A \ a : \text{PureTm } n\}$ 
  (ha : HasTypeDecl E  $\Gamma$  a A) :
  CtxMorDecl E (.snoc  $\Gamma$  A)  $\Gamma$  (subst0 a) := by
intro i
refine Fin.cases ?_ ?_ i
· simpa [CtxMorDecl, lookup_snoc_zero, subst0_wk_cancel_decl] using ha
· intro j
  simpa [CtxMorDecl, lookup_snoc_succ, subst0_wk_cancel_decl] using
    (HasTypeDecl.var (E := E)  $\Gamma$ ( :=  $\Gamma$ ) (i := j))

private theorem typing_inst0_decl {E : DeclEnv}  $\Gamma\{ : \text{Ctx } n\} \{A \ a : \text{PureTm } n\}$ 
  {body B : PureTm (n + 1)}
  (hBody : HasTypeDecl E (.snoc  $\Gamma$  A) body B) (ha : HasTypeDecl E  $\Gamma$  a A) :
  HasTypeDecl E  $\Gamma$  (inst0 a body) (inst0 a B) := by
simpa [inst0] using
  (typing_subst_decl (E := E)  $\Gamma$ ( := .snoc  $\Gamma$  A)  $\Delta$ ( :=  $\Gamma$ )  $\sigma$ ( := subst0 a) hBody

```

```

      (ctxMorDecl_subst0 (E := E) ha))

private def tmOfDecl {E : DeclEnv} {Γ : Ctx n} {t A : PureTm n}
  (_ : HasTypeDecl E Γ t A) : PureTm n := t

/-- Core-kernel one-step preservation inside declaration-aware typing, assuming
Pi/Sigma convertibility injectivity and argument-instantiation compatibility. -/
theorem core_step_preserves_type_decl_of_assumptions
  {E : DeclEnv}
  (piInjective :
    ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
    ConvDecl E (.pi A B) (.pi A' B') →
    ConvDecl E A A' ∧ ConvDecl E B B')
  (sigmaInjective :
    ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
    ConvDecl E (.sigma A B) (.sigma A' B') →
    ConvDecl E A A' ∧ ConvDecl E B B')
  (inst0ArgConv :
    ∀ {k : Nat} {a a' : PureTm k} {B : PureTm (k + 1)},
    ConvDecl E a a' → ConvDecl E (inst0 a B) (inst0 a' B))
  {Γ : Ctx n} {t t' A : PureTm n}
  (ht : HasTypeDecl E Γ t A) (hr : Red t t') :
  HasTypeDecl E Γ t' A := by
induction ht with
| u0_type =>
  cases hr
| var =>
  cases hr
| const =>
  cases hr
| @pi_form n Γ A B hA hB ihA ihB =>
  cases hr with
  | congPiDom hred =>
    have hA' : HasTypeDecl E Γ _ .u1 := ihA hred
    have hB' : HasTypeDecl E (.snoc Γ _) B .u1 :=
      context_conv_head_decl
      (hAA' := redDecl_implies_conv (.core hred)) hB
    exact .pi_form hA' hB'
  | congPiCod hred =>
    exact .pi_form hA (ihB hred)
| @sigma_form n Γ A B hA hB ihA ihB =>
  cases hr with
  | congSigmaDom hred =>
    have hA' : HasTypeDecl E Γ _ .u1 := ihA hred
    have hB' : HasTypeDecl E (.snoc Γ _) B .u1 :=
      context_conv_head_decl
      (hAA' := redDecl_implies_conv (.core hred)) hB
    exact .sigma_form hA' hB'
  | congSigmaCod hred =>
    exact .sigma_form hA (ihB hred)
| @lam_intro n Γ A body B hBody ihBody =>
  cases hr with
  | congLam hred =>
    exact .lam_intro (ihBody hred)

```

```

| @app_elim n  $\Gamma$  f a A B hf ha ihf iha =>
  cases hr with
  | betaPi body _ =>
    have hfLam : HasTypeDecl E  $\Gamma$  (.lam body) (.pi A B) := by
      simpa using hf
    have haArg : HasTypeDecl E  $\Gamma$  a A := by
      simpa using ha
    rcases lam_generation_decl hfLam with (A1, B1, hBody, )hPiConv
    have hAB : ConvDecl E A1 A  $\wedge$  ConvDecl E B1 B := piInjective hPiConv
    have ha1 : HasTypeDecl E  $\Gamma$  a A1 :=
      .conv haArg (Relation.EqvGen.symm _ _ hAB.1)
    have hSub : HasTypeDecl E  $\Gamma$  (inst0 a body) (inst0 a B1) :=
      typing_inst0_decl hBody ha1
    have hTy : ConvDecl E (inst0 a B1) (inst0 a B) :=
      convDecl_subst (E := E)  $\sigma$ ( := subst0 a) hAB.2
    have hRes : HasTypeDecl E  $\Gamma$  (inst0 a body) (inst0 a B) :=
      .conv hSub hTy
    simpa using hRes
  | congAppFun hred =>
    exact .app_elim (ihf hred) ha
  | congAppArg hred =>
    have ha' : HasTypeDecl E  $\Gamma$  _ A := iha hred
    have hApp' : HasTypeDecl E  $\Gamma$  (.app f _) (inst0 _ B) := .app_elim hf ha'
    have hArgConv : ConvDecl E _ a :=
      Relation.EqvGen.symm _ _ (redDecl_implies_conv (.core hred))
    have hTy : ConvDecl E (inst0 _ B) (inst0 a B) := inst0ArgConv hArgConv
    exact .conv hApp' hTy
| @pair_intro n  $\Gamma$  a b A B ha hb iha ihb =>
  cases hr with
  | congPairFst hred =>
    have ha' : HasTypeDecl E  $\Gamma$  _ A := iha hred
    have hArgConv : ConvDecl E a _ := redDecl_implies_conv (.core hred)
    have hTy : ConvDecl E (inst0 a B) (inst0 _ B) := inst0ArgConv hArgConv
    have hb' : HasTypeDecl E  $\Gamma$  b (inst0 _ B) := .conv hb hTy
    exact .pair_intro ha' hb'
  | congPairSnd hred =>
    exact .pair_intro ha (ihb hred)
| @fst_elim n  $\Gamma$  p A B hp ihp =>
  cases hr with
  | betaSigmaFst _ _ =>
    rcases pair_generation_decl hp with (A1, B1, ha1, hb1, )hSigma
    have hAB : ConvDecl E A1 A  $\wedge$  ConvDecl E B1 B := sigmaInjective hSigma
    exact .conv ha1 hAB.1
  | congFst hred =>
    exact .fst_elim (ihp hred)
| @snd_elim n  $\Gamma$  p A B hp ihp =>
  cases hr with
  | betaSigmaSnd _ _ =>
    rcases pair_generation_decl hp with (A1, B1, ha1, hb1, )hSigma
    have hAB : ConvDecl E A1 A  $\wedge$  ConvDecl E B1 B := sigmaInjective hSigma
    let a0 : PureTm n := tmOfDecl ha1
    let b0 : PureTm n := tmOfDecl hb1
    have hb1' : HasTypeDecl E  $\Gamma$  b0 (inst0 a0 B1) := by
      simpa [a0, b0, tmOfDecl] using hb1

```

```

    have hCod : ConvDecl E (inst0 a0 B1) (inst0 a0 B) :=
      convDecl_subst (E := E) σ( := subst0 a0) hAB.2
  have hFst : ConvDecl E a0 (.fst (.pair a0 b0)) :=
    Relation.EqvGen.symm _ _ (redDecl_implies_conv (.core (Red.betaSigmaFst a0 b0)))
  have hArg : ConvDecl E (inst0 a0 B) (inst0 (.fst (.pair a0 b0)) B) :=
    inst0ArgConv hFst
  exact .conv hb1' (Relation.EqvGen.trans _ _ _ hCod hArg)
| congSnd hred =>
  have hp' : HasTypeDecl E Γ _ (.sigma A B) := ihp hred
  have hsd' : HasTypeDecl E Γ (.snd _) (inst0 (.fst _) B) := .snd_elim hp'
  have hFst : ConvDecl E (.fst _) (.fst p) :=
    Relation.EqvGen.symm _ _ (redDecl_implies_conv (.core (Red.congFst hred)))
  have hTy : ConvDecl E (inst0 (.fst _) B) (inst0 (.fst p) B) :=
    inst0ArgConv hFst
  exact .conv hsd' hTy
| @id_form n Γ A a b hA ha hb ihA iha ihb =>
  cases hr with
  | congIdTy hred =>
    have hA' : HasTypeDecl E Γ _ .u1 := ihA hred
    have hAA' : ConvDecl E A _ := redDecl_implies_conv (.core hred)
    have ha' : HasTypeDecl E Γ a _ := .conv ha hAA'
    have hb' : HasTypeDecl E Γ b _ := .conv hb hAA'
    exact .id_form hA' ha' hb'
  | congIdLeft hred =>
    exact .id_form hA (iha hred) hb
  | congIdRight hred =>
    exact .id_form hA ha (ihb hred)
| @refl_intro n Γ a A ha iha =>
  cases hr with
  | congRefl hred =>
    have ha' : HasTypeDecl E Γ _ A := iha hred
    let a' : PureTm n := tmOfDecl ha'
    have hidL : ConvDecl E (.id A a a) (.id A a' a) :=
      redDecl_implies_conv (.core (Red.congIdLeft hred))
    have hidR : ConvDecl E (.id A a' a) (.id A a' a') :=
      redDecl_implies_conv (.core (Red.congIdRight (A := A) (a := a') hred))
    have hid : ConvDecl E (.id A a a) (.id A a' a') :=
      Relation.EqvGen.trans _ _ _ hidL hidR
    exact .conv (.refl_intro ha') (Relation.EqvGen.symm _ _ hid)
| @conv n Γ t A B ht hAB ih =>
  exact .conv (ih hr) hAB

/-- Core-kernel one-step preservation with only Pi/Sigma injectivity assumptions;
the instantiation compatibility premise is discharged by `inst0_arg_conv_decl`. -/
theorem core_step_preserves_type_decl_of_injective
{E : DeclEnv}
(piInjective :
  ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl E (.pi A B) (.pi A' B') →
  ConvDecl E A A' ∧ ConvDecl E B B')
(sigmaInjective :
  ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl E (.sigma A B) (.sigma A' B') →
  ConvDecl E A A' ∧ ConvDecl E B B')

```



```

Γ{ : Ctx n} {t t' A : PureTm n}
(ht : HasTypeDecl E Γ t A) (hr : Red t t') :
HasTypeDecl E Γ t' A := by
exact core_step_preserves_type_decl_of_assumptions
(piInjective := piInjective)
(sigmaInjective := sigmaInjective)
(inst0ArgConv := inst0_arg_conv_decl)
ht hr

private theorem const_generation_aux {E : DeclEnv} Γ{ : Ctx n} {t C : PureTm n}
(ht : HasTypeDecl E Γ t C) :
  ∀ {c : DeclName}, t = .const c →
    ∃ A0 : PureTm 0, typeOf? E c = some A0 ∧ ConvDecl E (liftClosed (n := n) A0) C := by
induction ht with
| @const n Γ c A0 hLookup =>
  intro c' hEq
  cases hEq
  exact (A0, hLookup, .refl )_
| conv ht hconv ih =>
  intro c hEq
  rcases ih hEq with (A0, hType, )hConv
  exact (A0, hType, .trans _ _ _ hConv )hconv
| u0_type =>
  intro c hEq
  cases hEq
| var =>
  intro c hEq
  cases hEq
| pi_form hA hB ihA ihB =>
  intro c hEq
  cases hEq
| sigma_form hA hB ihA ihB =>
  intro c hEq
  cases hEq
| lam_intro hBody ihBody =>
  intro c hEq
  cases hEq
| app_elim hf ha ihf iha =>
  intro c hEq
  cases hEq
| pair_intro ha hb iha ihb =>
  intro c hEq
  cases hEq
| fst_elim hp ihp =>
  intro c hEq
  cases hEq
| snd_elim hp ihp =>
  intro c hEq
  cases hEq
| id_form hA ha hb ihA iha ihb =>
  intro c hEq
  cases hEq
| refl_intro ha iha =>
  intro c hEq

```

```

cases hEq

theorem const_generation {E : DeclEnv}  $\Gamma$ { : Ctx n} {c : DeclName} {C : PureTm n}
  (ht : HasTypeDecl E  $\Gamma$  (.const c) C) :
   $\exists$  A0 : PureTm 0, typeOf? E c = some A0  $\wedge$  ConvDecl E (liftClosed (n := n) A0) C :=
  const_generation_aux ht rfl

-- Soundness invariant for declaration-level unfolding:
every definitional value stored in 'DeclEnv' is typed at its declared type. -/
def DeclValuesWellTyped (E : DeclEnv) : Prop :=
   $\forall$  {c : DeclName} {A0 v0 : PureTm 0},
    typeOf? E c = some A0  $\rightarrow$ 
    valueOf? E c = some v0  $\rightarrow$ 
    HasTypeDecl E .nil (liftClosed v0) (liftClosed A0)

-- Packaged declaration-environment well-formedness for the current
declaration-aware kernel layer.
Closedness of stored declaration types and values is already enforced by
'DeclEntry' using 'PureTm 0'; the packaged proof obligations here are the
semantic invariants used by declaration-aware reduction and preservation. -/
structure DeclEnvWellFormed (E : DeclEnv) : Prop where
  valuesWellTyped : DeclValuesWellTyped E
  noSelfDelta :  $\forall$  {c : DeclName} {v0 : PureTm 0},
    valueOf? E c = some v0  $\rightarrow$  v0  $\neq$  (.const c)

theorem DeclEnvWellFormed.toDeclValuesWellTyped {E : DeclEnv}
  (hWf : DeclEnvWellFormed E) :
  DeclValuesWellTyped E :=
  hWf.valuesWellTyped

private theorem redDecl_const_step_inv {E : DeclEnv} {c : DeclName} {v : PureTm 0}
  (h : RedDecl E ((.const c : PureTm 0)) v) :
   $\exists$  v0 : PureTm 0, v = liftClosed v0  $\wedge$  valueOf? E c = some v0 := by
cases h with
| core hred =>
  cases hred
| deltaConst hVal =>
  exact (_, rfl, )hVal

private theorem redDecl_const_step_inv_any {E : DeclEnv} {n : Nat} {c : DeclName} {u : PureTm n}
  (h : RedDecl E ((.const c : PureTm n)) u) :
   $\exists$  v0 : PureTm 0, u = liftClosed (n := n) v0  $\wedge$  valueOf? E c = some v0 := by
cases h with
| core hred =>
  cases hred
| deltaConst hVal =>
  exact (_, rfl, )hVal

private theorem closed_typing_lift_to_ctx {E : DeclEnv}  $\Gamma$ { : Ctx n} {t A : PureTm 0}
  (h : HasTypeDecl E .nil (liftClosed t) (liftClosed A)) :
  HasTypeDecl E  $\Gamma$  (liftClosed (n := n) t) (liftClosed (n := n) A) := by
let p0 : Ren 0 n := fun i => nomatch i
have ph : CtxRen .nil  $\Gamma$  p0 := by
intro i

```

```

    nomatch i
  simpa [rename_liftClosed] using
    (typing_rename_decl (ht := h) (m := n)  $\Delta$ ( :=  $\Gamma$ )  $\rho$ ( :=  $\rho_0$ ) ph)

/-- Declaration-aware one-step preservation for  $\delta$ -unfolding from a successful
value lookup in a well-formed declaration environment. -/
theorem deltaConst_preserves_type_of_lookup {E : DeclEnv}  $\Gamma$ { : Ctx n} {c : DeclName} {v0 : PureTm
  0} {C : PureTm n}
  (hVal : valueOf? E c = some v0)
  (hTy : HasTypeDecl E  $\Gamma$  ((.const c : PureTm n)) C)
  (hWf : DeclEnvWellFormed E) :
  HasTypeDecl E  $\Gamma$  (liftClosed (n := n) v0) C := by
  rcases const_generation hTy with (A0, hType, )hConv
  have hClosed : HasTypeDecl E .nil (liftClosed v0) (liftClosed A0) :=
    hWf.valuesWellTyped hType hVal
  have hLifted : HasTypeDecl E  $\Gamma$  (liftClosed (n := n) v0) (liftClosed (n := n) A0) :=
    closed_typing_lift_to_ctx  $\Gamma$ ( :=  $\Gamma$ ) hClosed
  exact .conv hLifted hConv

theorem deltaConst_preserves_type {E : DeclEnv}  $\Gamma$ { : Ctx n} {c : DeclName} {v0 : PureTm 0} {C :
  PureTm n}
  (hStep : RedDecl E ((.const c : PureTm n)) (liftClosed (n := n) v0))
  (hTy : HasTypeDecl E  $\Gamma$  ((.const c : PureTm n)) C)
  (hWf : DeclEnvWellFormed E) :
  HasTypeDecl E  $\Gamma$  (liftClosed (n := n) v0) C := by
  rcases redDecl_const_step_inv_any hStep with (v1, hEq, )hVal
  have hTyped : HasTypeDecl E  $\Gamma$  (liftClosed (n := n) v1) C :=
    deltaConst_preserves_type_of_lookup  $\Gamma$ ( :=  $\Gamma$ ) (c := c) (v0 := v1) (C := C)
    (hVal := hVal) (hTy := hTy) (hWf := hWf)
  simpa [hEq] using hTyped

theorem const_head_step_preserves_type {E : DeclEnv}  $\Gamma$ { : Ctx n} {c : DeclName} {u C : PureTm n}
  (hStep : RedDecl E ((.const c : PureTm n)) u)
  (hTy : HasTypeDecl E  $\Gamma$  ((.const c : PureTm n)) C)
  (hWf : DeclEnvWellFormed E) :
  HasTypeDecl E  $\Gamma$  u C := by
  rcases redDecl_const_step_inv_any hStep with (v0, hEq, )hVal
  have hDelta : HasTypeDecl E  $\Gamma$  (liftClosed (n := n) v0) C :=
    deltaConst_preserves_type_of_lookup
       $\Gamma$ ( :=  $\Gamma$ ) (c := c) (v0 := v0) (C := C)
      (hVal := hVal) (hTy := hTy) (hWf := hWf)
  simpa [hEq] using hDelta

/-- One-step subject reduction for declaration-aware reduction, under the same
Pi/Sigma conversion injectivity assumptions as the core theorem plus a well-formed
declaration environment for  $\delta$ -unfolding. -/
theorem redDecl_step_preserves_type_of_assumptions
  {E : DeclEnv}
  (piInjective :
     $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
    ConvDecl E (.pi A B) (.pi A' B')  $\rightarrow$ 
    ConvDecl E A A'  $\wedge$  ConvDecl E B B')
  (sigmaInjective :
     $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},

```

```

      ConvDecl E (.sigma A B) (.sigma A' B') →
        ConvDecl E A A' ∧ ConvDecl E B B')
(inst0ArgConv :
  ∀ {k : Nat} {a a' : PureTm k} {B : PureTm (k + 1)},
    ConvDecl E a a' → ConvDecl E (inst0 a B) (inst0 a' B))
(hWf : DeclEnvWellFormed E)
(Γ : Ctx n) {t t' A : PureTm n}
(ht : HasTypeDecl E Γ t A) (hr : RedDecl E t t') :
  HasTypeDecl E Γ t' A := by
induction ht with
| u0_type =>
  cases hr with
  | core hred =>
    cases hred
| var =>
  cases hr with
  | core hred =>
    cases hred
| @const n Γ c A0 hLookup =>
  cases hr with
  | core hred =>
    cases hred
  | deltaConst hVal =>
    exact const_head_step_preserves_type
      (hStep := .deltaConst hVal)
      (hTy := .const hLookup)
      (hWf := hWf)
| @pi_form n Γ A B hA hB ihA ihB =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .pi_form hA hB)
      (hr := hred)
  | congPiDom hred =>
    have hA' : HasTypeDecl E Γ _ .u1 := ihA hred
    have hB' : HasTypeDecl E (.snoc Γ _) B .u1 :=
      context_conv_head_decl (hAA' := redDecl_implies_conv hred) hB
    exact .pi_form hA' hB'
  | congPiCod hred =>
    exact .pi_form hA (ihB hred)
| @sigma_form n Γ A B hA hB ihA ihB =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .sigma_form hA hB)
      (hr := hred)
  | congSigmaDom hred =>
    have hA' : HasTypeDecl E Γ _ .u1 := ihA hred

```

```

    have hB' : HasTypeDecl E (.snoc  $\Gamma$  _) B .u1 :=
      context_conv_head_decl (hAA' := redDecl_implies_conv hred) hB
    exact .sigma_form hA' hB'
  | congSigmaCod hred =>
    exact .sigma_form hA (ihB hred)
| @lam_intro n  $\Gamma$  A body B hBody ihBody =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .lam_intro hBody)
      (hr := hred)
  | congLam hred =>
    exact .lam_intro (ihBody hred)
| @app_elim n  $\Gamma$  f a A B hf ha ihf iha =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .app_elim hf ha)
      (hr := hred)
  | congAppFun hred =>
    exact .app_elim (ihf hred) ha
  | congAppArg hred =>
    have ha' : HasTypeDecl E  $\Gamma$  _ A := iha hred
    have hApp' : HasTypeDecl E  $\Gamma$  (.app f _) (inst0 _ B) := .app_elim hf ha'
    have hArgConv : ConvDecl E _ a :=
      Relation.EqvGen.symm _ _ (redDecl_implies_conv hred)
    have hTy : ConvDecl E (inst0 _ B) (inst0 a B) := inst0ArgConv hArgConv
    exact .conv hApp' hTy
| @pair_intro n  $\Gamma$  a b A B ha hb iha ihb =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .pair_intro ha hb)
      (hr := hred)
  | congPairFst hred =>
    have ha' : HasTypeDecl E  $\Gamma$  _ A := iha hred
    have hArgConv : ConvDecl E a _ := redDecl_implies_conv hred
    have hTy : ConvDecl E (inst0 a B) (inst0 _ B) := inst0ArgConv hArgConv
    have hb' : HasTypeDecl E  $\Gamma$  b (inst0 _ B) := .conv hb hTy
    exact .pair_intro ha' hb'
  | congPairSnd hred =>
    exact .pair_intro ha (ihb hred)
| @fst_elim n  $\Gamma$  p A B hp ihp =>
  cases hr with
  | core hred =>

```

```

    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .fst_elim hp)
      (hr := hred)
  | congFst hred =>
    exact .fst_elim (ihp hred)
| @snd_elim n  $\Gamma$  p A B hp ihp =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .snd_elim hp)
      (hr := hred)
  | congSnd hred =>
    have hp' : HasTypeDecl E  $\Gamma$  _ (.sigma A B) := ihp hred
    have hsnd' : HasTypeDecl E  $\Gamma$  (.snd _) (inst0 (.fst _) B) := .snd_elim hp'
    have hFst : ConvDecl E (.fst _) (.fst p) :=
      Relation.EqvGen.symm _ _ (redDecl_implies_conv (.congFst hred))
    have hTy : ConvDecl E (inst0 (.fst _) B) (inst0 (.fst p) B) :=
      inst0ArgConv hFst
    exact .conv hsnd' hTy
| @id_form n  $\Gamma$  A a b hA ha hb ihA iha ihb =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .id_form hA ha hb)
      (hr := hred)
  | congIdTy hred =>
    have hA' : HasTypeDecl E  $\Gamma$  _ .u1 := ihA hred
    have hAA' : ConvDecl E A _ := redDecl_implies_conv hred
    have ha' : HasTypeDecl E  $\Gamma$  a _ := .conv ha hAA'
    have hb' : HasTypeDecl E  $\Gamma$  b _ := .conv hb hAA'
    exact .id_form hA' ha' hb'
  | congIdLeft hred =>
    exact .id_form hA (iha hred) hb
  | congIdRight hred =>
    exact .id_form hA ha (ihb hred)
| @refl_intro n  $\Gamma$  a A ha iha =>
  cases hr with
  | core hred =>
    exact core_step_preserves_type_decl_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (ht := .refl_intro ha)
      (hr := hred)
  | congRefl hred =>

```

```

have ha' : HasTypeDecl E  $\Gamma$  _ A := iha hred
let a' : PureTm n := tmOfDecl ha'
have hidL : ConvDecl E (.id A a a) (.id A a' a) :=
  redDecl_implies_conv (.congIdLeft hred)
have hidR : ConvDecl E (.id A a' a) (.id A a' a') :=
  redDecl_implies_conv (.congIdRight (A := A) (a := a') hred)
have hid : ConvDecl E (.id A a a) (.id A a' a') :=
  Relation.EqvGen.trans _ _ _ hidL hidR
exact .conv (.refl_intro ha') (Relation.EqvGen.symm _ _ hid)
| @conv n  $\Gamma$  t A B ht hAB iht =>
  exact .conv (iht hr) hAB

/-- One-step declaration-aware preservation with only Pi/Sigma injectivity
assumptions; argument-instantiation compatibility is discharged by
`inst0_arg_conv_decl`. -/
theorem redDecl_step_preserves_type_of_injective
{E : DeclEnv}
(piInjective :
   $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl E (.pi A B) (.pi A' B')  $\rightarrow$ 
  ConvDecl E A A'  $\wedge$  ConvDecl E B B')
(sigmaInjective :
   $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl E (.sigma A B) (.sigma A' B')  $\rightarrow$ 
  ConvDecl E A A'  $\wedge$  ConvDecl E B B')
(hWf : DeclEnvWellFormed E)
 $\Gamma$ { : Ctx n} {t t' A : PureTm n}
(ht : HasTypeDecl E  $\Gamma$  t A) (hr : RedDecl E t t') :
HasTypeDecl E  $\Gamma$  t' A := by
exact redDecl_step_preserves_type_of_assumptions
(piInjective := piInjective)
(sigmaInjective := sigmaInjective)
(inst0ArgConv := inst0_arg_conv_decl)
(hWf := hWf)
ht hr

/-- Star-closure subject reduction for declaration-aware reduction under the
same assumptions as `redDecl_step_preserves_type_of_assumptions`. -/
theorem redStarDecl_preserves_type_of_assumptions
{E : DeclEnv}
(piInjective :
   $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl E (.pi A B) (.pi A' B')  $\rightarrow$ 
  ConvDecl E A A'  $\wedge$  ConvDecl E B B')
(sigmaInjective :
   $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl E (.sigma A B) (.sigma A' B')  $\rightarrow$ 
  ConvDecl E A A'  $\wedge$  ConvDecl E B B')
(inst0ArgConv :
   $\forall$  {k : Nat} {a a' : PureTm k} {B : PureTm (k + 1)},
  ConvDecl E a a'  $\rightarrow$  ConvDecl E (inst0 a B) (inst0 a' B))
(hWf : DeclEnvWellFormed E)
 $\Gamma$ { : Ctx n} {t u A : PureTm n}
(ht : HasTypeDecl E  $\Gamma$  t A) (hs : RedStarDecl E t u) :

```

```

    HasTypeDecl E  $\Gamma$  u A := by
induction hs with
| refl =>
    simpa using ht
| tail hxy hyz ih =>
    exact redDecl_step_preserves_type_of_assumptions
      (piInjective := piInjective)
      (sigmaInjective := sigmaInjective)
      (inst0ArgConv := inst0ArgConv)
      (hWf := hWf)
      (ht := ih)
      (hr := hyz)

/-- Star-closure declaration-aware preservation with only Pi/Sigma injectivity
assumptions; argument-instantiation compatibility is discharged by
`inst0_arg_conv_decl`. -/
theorem redStarDecl_preserves_type_of_injective
  {E : DeclEnv}
  (piInjective :
     $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
    ConvDecl E (.pi A B) (.pi A' B')  $\rightarrow$ 
    ConvDecl E A A'  $\wedge$  ConvDecl E B B')
  (sigmaInjective :
     $\forall$  {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
    ConvDecl E (.sigma A B) (.sigma A' B')  $\rightarrow$ 
    ConvDecl E A A'  $\wedge$  ConvDecl E B B')
  (hWf : DeclEnvWellFormed E)
  ( $\Gamma$  : Ctx n) {t u A : PureTm n}
  (ht : HasTypeDecl E  $\Gamma$  t A) (hs : RedStarDecl E t u) :
  HasTypeDecl E  $\Gamma$  u A := by
exact redStarDecl_preserves_type_of_assumptions
  (piInjective := piInjective)
  (sigmaInjective := sigmaInjective)
  (inst0ArgConv := inst0_arg_conv_decl)
  (hWf := hWf)
  ht hs

theorem deltaConst_preserves_type_closed {E : DeclEnv} {c : DeclName} {v0 C : PureTm 0}
  (hStep : RedDecl E ((.const c : PureTm 0)) (liftClosed v0))
  (hTy : HasTypeDecl E .nil ((.const c : PureTm 0)) C)
  (hWf : DeclEnvWellFormed E) :
  HasTypeDecl E .nil (liftClosed v0) C := by
exact deltaConst_preserves_type
  (hStep := hStep)
  (hTy := hTy)
  (hWf := hWf)

/-- Generic closed checked-step for declaration unfolding  $\delta(\cdot)$  from an explicit typed-value
witness. -/
theorem checked_delta_step_closed_of_typed_value {E : DeclEnv} {c : DeclName} {A0 v0 : PureTm 0}
  (hType : typeOf? E c = some A0)
  (hVal : valueOf? E c = some v0)
  (hValTy : HasTypeDecl E .nil (liftClosed v0) (liftClosed A0)) :
   $\exists$  A : PureTm 0,

```



```

      HasTypeDecl E .nil ((.const c : PureTm 0)) A ∧
      RedDecl E ((.const c : PureTm 0)) (liftClosed v0) ∧
      HasTypeDecl E .nil (liftClosed v0) A := by
exact (liftClosed A0, .const hType, .deltaConst hVal, )hValTy

/-- Closed checked-step for declaration unfolding 5(`) under the global declaration typed-value
invariant. -/
theorem checked_delta_step_closed {E : DeclEnv} {c : DeclName} {A0 v0 : PureTm 0}
  (hType : typeOf? E c = some A0)
  (hVal : valueOf? E c = some v0)
  (hWf : DeclValuesWellTyped E) :
  ∃ A : PureTm 0,
    HasTypeDecl E .nil ((.const c : PureTm 0)) A ∧
    RedDecl E ((.const c : PureTm 0)) (liftClosed v0) ∧
    HasTypeDecl E .nil (liftClosed v0) A := by
exact checked_delta_step_closed_of_typed_value hType hVal (hWf hType hVal)

theorem checked_delta_step_closed_wf {E : DeclEnv} {c : DeclName} {A0 v0 : PureTm 0}
  (hType : typeOf? E c = some A0)
  (hVal : valueOf? E c = some v0)
  (hWf : DeclEnvWellFormed E) :
  ∃ A : PureTm 0,
    HasTypeDecl E .nil ((.const c : PureTm 0)) A ∧
    RedDecl E ((.const c : PureTm 0)) (liftClosed v0) ∧
    HasTypeDecl E .nil (liftClosed v0) A := by
exact checked_delta_step_closed hType hVal hWf.valuesWellTyped

end Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics

```

A.7 DeclarationSpec.lean

Mettapedia/Languages/MeTTa/PureKernel/DeclarationSpec.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics

namespace Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics

/-- Small declarative spec used by pilot family modules. -/
structure DeclSpec where
  name : DeclName
  type : PureTm 0
  value? : Option (PureTm 0) := none
deriving DecidableEq, Repr

/-- Reusable proof obligations for pilot declaration specs.
Closedness of spec types and values is already enforced by the `PureTm 0` fields. -/
def DeclSpec.toPair (s : DeclSpec) : DeclName × DeclEntry :=
  (s.name, { type := s.type, value? := s.value? })

```

```

/-- Build a declaration environment from concise declaration specs. -/
def envOfSpecs (specs : List DeclSpec) : DeclEnv :=
  ofList (specs.map DeclSpec.toPair)

/-- Reusable proof obligations for pilot declaration specs.
Closedness of spec types and values is already enforced by the `PureTm 0` fields. -/
structure DeclSpecObligations (specs : List DeclSpec) : Prop where
  valuesWellTyped :
    ∀ s ∈ specs, ∀ v0 : PureTm 0,
      s.value? = some v0 →
        HasTypeDecl (envOfSpecs specs) .nil (liftClosed v0) (liftClosed s.type)
  noSelfDelta :
    ∀ s ∈ specs, ∀ v0 : PureTm 0,
      s.value? = some v0 →
        v0 ≠ (.const s.name)

/-- Ordered declaration-signature discipline above the operational declaration
environment. This is the right place to add stronger signature theory later
without changing the reduction-preservation boundary. -/
structure SignatureWellFormed (specs : List DeclSpec) : Prop where
  noShadowing : (specs.map DeclSpec.name).Nodup
  obligations : DeclSpecObligations specs

theorem SignatureWellFormed.toDeclSpecObligations {specs : List DeclSpec}
  (hSig : SignatureWellFormed specs) :
  DeclSpecObligations specs :=
  hSig.obligations

/-- Closed declaration types and unfolding values should only mention names
already available in the current signature prefix. -/
def UsesOnlyDeclNamesFrom (allowed : List DeclName) : PureTm n → Prop
| .var _ => True
| .const c => c ∈ allowed
| .u0 => True
| .u1 => True
| .pi A B => UsesOnlyDeclNamesFrom allowed A ∧ UsesOnlyDeclNamesFrom allowed B
| .sigma A B => UsesOnlyDeclNamesFrom allowed A ∧ UsesOnlyDeclNamesFrom allowed B
| .id A a b =>
  UsesOnlyDeclNamesFrom allowed A ∧
  UsesOnlyDeclNamesFrom allowed a ∧
  UsesOnlyDeclNamesFrom allowed b
| .lam body => UsesOnlyDeclNamesFrom allowed body
| .app f a => UsesOnlyDeclNamesFrom allowed f ∧ UsesOnlyDeclNamesFrom allowed a
| .pair a b => UsesOnlyDeclNamesFrom allowed a ∧ UsesOnlyDeclNamesFrom allowed b
| .fst p => UsesOnlyDeclNamesFrom allowed p
| .snd p => UsesOnlyDeclNamesFrom allowed p
| .refl a => UsesOnlyDeclNamesFrom allowed a

def prefixNames (pre : List DeclSpec) : List DeclName :=
  pre.map DeclSpec.name

/-- Prefix-aware admissibility for one declaration spec.
This is the first semantic use of signature order: any unfolding value must
already typecheck against the earlier signature prefix. -/

```

```

structure PrefixDeclSpecAdmissible (pre : List DeclSpec) (s : DeclSpec) : Prop where
  fresh : s.name ∉ pre.map DeclSpec.name
  typeUsesEarlier :
    UsesOnlyDeclNamesFrom (prefixNames pre) s.type
  valueUsesEarlier :
    ∀ {v0 : PureTm 0},
      s.value? = some v0 →
        UsesOnlyDeclNamesFrom (prefixNames pre) v0
  valueWellTyped :
    ∀ {v0 : PureTm 0},
      s.value? = some v0 →
        HasTypeDecl (envOfSpecs pre) .nil (liftClosed v0) (liftClosed s.type)
  noSelfDelta :
    ∀ {v0 : PureTm 0},
      s.value? = some v0 →
        v0 ≠ (.const s.name)

-- Ordered signature checking, left to right, against earlier declarations only. --
def PrefixSignatureWellFormed : List DeclSpec → List DeclSpec → Prop
| _, [] => True
| pre, s :: rest =>
  PrefixDeclSpecAdmissible pre s ∧
  PrefixSignatureWellFormed (pre ++ [s]) rest

abbrev SignatureWellFormedPrefix (specs : List DeclSpec) : Prop :=
  PrefixSignatureWellFormed [] specs

theorem prefixSignatureWellFormed_nodup
  {pre specs : List DeclSpec}
  (hPre : (prefixNames pre).Nodup)
  (hSig : PrefixSignatureWellFormed pre specs) :
  (prefixNames (pre ++ specs)).Nodup := by
  induction specs generalizing pre with
  | nil =>
    simpa [PrefixSignatureWellFormed] using hPre
  | cons s rest ih =>
    rcases hSig with (hs, )hrest
    have hPreSnoc : (prefixNames (pre ++ [s])).Nodup := by
      have hAppend : (prefixNames pre ++ [s.name]).Nodup := by
        exact List.nodup_append.mpr (hPre, by simp, by simpa [prefixNames] using hs.)fresh
        simpa [prefixNames] using hAppend
      simpa [prefixNames, List.append_assoc] using ih hPreSnoc hrest

theorem SignatureWellFormedPrefix.noShadowing
  {specs : List DeclSpec}
  (hSig : SignatureWellFormedPrefix specs) :
  (prefixNames specs).Nodup := by
  simpa [SignatureWellFormedPrefix] using
    prefixSignatureWellFormed_nodup (pre := []) (specs := specs) (by simp [prefixNames]) hSig

theorem SignatureWellFormed.ofPrefix
  {specs : List DeclSpec}
  (hPrefix : SignatureWellFormedPrefix specs)
  (hObligations : DeclSpecObligations specs) :

```

```

SignatureWellFormed specs where
noShadowing := hPrefix.noShadowing
obligations := hObligations

theorem entries_envOfSpecs_eq_of_mem_of_nodup
  {specs : List DeclSpec} {s : DeclSpec}
  (hNodup : (specs.map DeclSpec.name).Nodup)
  (hs : s ∈ specs) :
  (envOfSpecs specs).entries s.name = some { type := s.type, value? := s.value? } := by
induction specs generalizing s with
| nil =>
  cases hs
| cons head tail ih =>
  rcases List.nodup_cons.mp hNodup with {hHeadFresh, }hTailNodup
  simp at hs
  rcases hs with rfl | hTail
  · simp [envOfSpecs, DeclarationEnv.ofList, DeclarationEnv.insert, DeclSpec.toPair]
  · have hMemName : s.name ∈ tail.map DeclSpec.name := by
    exact List.mem_map.mpr {s, hTail, }rfl
    have hNameNe : s.name ≠ head.name := by
      intro hEq
      apply hHeadFresh
      simp [hEq] using hMemName
    have hTail := ih hTailNodup hTail
    simp [envOfSpecs, DeclarationEnv.ofList, DeclarationEnv.insert, DeclSpec.toPair, hNameNe]
  ]
  using hTail

theorem typeOf_envOfSpecs_eq_of_mem_of_nodup
  {specs : List DeclSpec} {s : DeclSpec}
  (hNodup : (specs.map DeclSpec.name).Nodup)
  (hs : s ∈ specs) :
  typeOf? (envOfSpecs specs) s.name = some s.type := by
unfold DeclarationEnv.typeOf?
simp [entries_envOfSpecs_eq_of_mem_of_nodup hNodup hs]

theorem valueOf_envOfSpecs_eq_of_mem_some_of_nodup
  {specs : List DeclSpec} {s : DeclSpec} {v0 : PureTm 0}
  (hNodup : (specs.map DeclSpec.name).Nodup)
  (hs : s ∈ specs)
  (hVal : s.value? = some v0) :
  valueOf? (envOfSpecs specs) s.name = some v0 := by
unfold DeclarationEnv.valueOf?
simp [hVal, entries_envOfSpecs_eq_of_mem_of_nodup hNodup hs]

theorem valueOf_envOfSpecs_eq_none_of_mem_none_of_nodup
  {specs : List DeclSpec} {s : DeclSpec}
  (hNodup : (specs.map DeclSpec.name).Nodup)
  (hs : s ∈ specs)
  (hVal : s.value? = none) :
  valueOf? (envOfSpecs specs) s.name = none := by
unfold DeclarationEnv.valueOf?
simp [hVal, entries_envOfSpecs_eq_of_mem_of_nodup hNodup hs]

```

```

theorem SignatureWellFormed.entries_eq_of_mem
  {specs : List DeclSpec} (hSig : SignatureWellFormed specs) {s : DeclSpec}
  (hs : s ∈ specs) :
  (envOfSpecs specs).entries s.name = some { type := s.type, value? := s.value? } :=
  entries_envOfSpecs_eq_of_mem_of_nodup hSig.noShadowing hs

theorem SignatureWellFormed.typeOf_eq_of_mem
  {specs : List DeclSpec} (hSig : SignatureWellFormed specs) {s : DeclSpec}
  (hs : s ∈ specs) :
  typeOf? (envOfSpecs specs) s.name = some s.type :=
  typeOf_envOfSpecs_eq_of_mem_of_nodup hSig.noShadowing hs

theorem SignatureWellFormed.valueOf_eq_of_mem_some
  {specs : List DeclSpec} (hSig : SignatureWellFormed specs) {s : DeclSpec} {v0 : PureTm 0}
  (hs : s ∈ specs) (hVal : s.value? = some v0) :
  valueOf? (envOfSpecs specs) s.name = some v0 :=
  valueOf_envOfSpecs_eq_of_mem_some_of_nodup hSig.noShadowing hs hVal

theorem SignatureWellFormed.valueOf_eq_of_mem_none
  {specs : List DeclSpec} (hSig : SignatureWellFormed specs) {s : DeclSpec}
  (hs : s ∈ specs) (hVal : s.value? = none) :
  valueOf? (envOfSpecs specs) s.name = none :=
  valueOf_envOfSpecs_eq_of_mem_none_of_nodup hSig.noShadowing hs hVal

/-- Generic typing helper for declared constants. -/
theorem hasType_const_from_lookup {E : DeclEnv} {Γ : Ctx n} {c : DeclName} {A0 : PureTm 0}
  (h : typeOf? E c = some A0) :
  HasTypeDecl E Γ (.const c) (liftClosed A0) :=
  .const h

/-- Generic unfolding helper for declared constants at depth `0`. -/
theorem red_const_from_unfold0 {E : DeclEnv} {c : DeclName} {v : PureTm 0}
  (h : valueOf? E c = some v) :
  RedDecl E ((.const c : PureTm 0)) (liftClosed v) := by
exact .deltaConst h

/-- If all declaration specs are non-unfolding (`value? = none`), the
generated declaration environment is fail-closed for `valueOf?`. -/
theorem entries_envOfSpecs_some_implies_valueNone
  (specs : List DeclSpec)
  (hNone : ∀ s ∈ specs, s.value? = none) :
  ∀ {c : DeclName} {a : DeclEntry}, (envOfSpecs specs).entries c = some a → a.value? = none :=
by
induction specs with
| nil =>
  intro c a h
  simp [envOfSpecs, DeclarationEnv.ofList, DeclarationEnv.empty] at h
| cons s rest ih =>
  intro c a h
  have hs : s.value? = none := hNone s (by simp)
  have hRest : ∀ r ∈ rest, r.value? = none := by
    intro r hr
    exact hNone r (by simp [hr])
  unfold envOfSpecs at h

```

```

    simp [DeclarationEnv.ofList] at h
    unfold DeclarationEnv.insert at h
    by_cases hEq : c = s.name
    · simp [hEq, DeclSpec.toPair] at h
      cases h
      simp [hs]
    · simp [hEq, DeclSpec.toPair] at h
      exact ih hRest h

/-- Any successful lookup in `envOfSpecs` comes from some source spec with
matching name/type/value fields. -/
theorem entries_envOfSpecs_some_implies_from_specs
  (specs : List DeclSpec) :
  ∀ {c : DeclName} {a : DeclEntry},
  (envOfSpecs specs).entries c = some a →
  ∃ s ∈ specs, s.name = c ∧ s.type = a.type ∧ s.value? = a.value? := by
  induction specs with
  | nil =>
    intro c a h
    simp [envOfSpecs, DeclarationEnv.ofList, DeclarationEnv.empty] at h
  | cons s rest ih =>
    intro c a h
    unfold envOfSpecs at h
    simp [DeclarationEnv.ofList] at h
    unfold DeclarationEnv.insert at h
    by_cases hEq : c = s.name
    · simp [hEq, DeclSpec.toPair] at h
      cases h
      refine {s, by simp, ?}_
      simp [hEq]
    · simp [hEq, DeclSpec.toPair] at h
      rcases ih h with {s', hs', hsName, hsType, }hsValue
      exact {s', by simp [hs'], hsName, hsType, }hsValue

theorem valueOf_envOfSpecs_eq_none_of_all_none
  (specs : List DeclSpec)
  (hNone : ∀ s ∈ specs, s.value? = none) :
  ∀ c : DeclName, valueOf? (envOfSpecs specs) c = none := by
  intro c
  have hEntryValueNone :
    ∀ {a : DeclEntry}, (envOfSpecs specs).entries c = some a → a.value? = none :=
    entries_envOfSpecs_some_implies_valueNone specs hNone
  unfold DeclarationEnv.valueOf?
  cases hEntries : (envOfSpecs specs).entries c with
  | none =>
    rfl
  | some a =>
    have ha : a.value? = none := hEntryValueNone hEntries
    simp [ha]

/-- Generic typed-value soundness for declaration environments built from specs:
if every `some` value in the specs is typed at its declared type, then the
resulting environment satisfies `DeclValuesWellTyped`. -/
theorem envOfSpecs_declValuesWellTyped_of_specValues

```

```

(specs : List DeclSpec)
(hTyped :
  ∀ s ∈ specs, ∀ v0 : PureTm 0,
    s.value? = some v0 →
      HasTypeDecl (envOfSpecs specs) .nil (liftClosed v0) (liftClosed s.type)) :
DeclValuesWellTyped (envOfSpecs specs) := by
intro c A0 v0 hType hVal
unfold DeclarationEnv.typeOf? at hType
unfold DeclarationEnv.valueOf? at hVal
cases hEntries : (envOfSpecs specs).entries c with
| none =>
  simp [hEntries] at hType
| some a =>
  have hA : a.type = A0 := by
    simpa [hEntries] using hType
  have hv : a.value? = some v0 := by
    simpa [hEntries] using hVal
  rcases entries_envOfSpecs_some_implies_from_specs specs hEntries with
    (s, hsMem, _, hsType, )hsValue
  have hsValSome : s.value? = some v0 := by
    calc
      s.value? = a.value? := hsValue
      _ = some v0 := hv
  have hsv : HasTypeDecl (envOfSpecs specs) .nil (liftClosed v0) (liftClosed s.type) :=
    hTyped s hsMem v0 hsValSome
  have hsTypeA0 : s.type = A0 := by
    calc
      s.type = a.type := hsType
      _ = A0 := hA
  simpa [hsTypeA0] using hsv

/-- Generic acyclicity/no-self-unfolding invariant for declaration environments
built from specs: if each unfolding value is not the defining constant itself
at the spec level, the generated env satisfies `noSelfDelta`. -/
theorem envOfSpecs_noSelfDelta_of_specNoSelf
(specs : List DeclSpec)
(hNoSelf :
  ∀ s ∈ specs, ∀ v0 : PureTm 0,
    s.value? = some v0 →
      v0 ≠ (.const s.name)) :
  ∀ {c : DeclName} {v0 : PureTm 0},
    valueOf? (envOfSpecs specs) c = some v0 →
      v0 ≠ (.const c) := by
intro c v0 hVal
unfold DeclarationEnv.valueOf? at hVal
cases hEntries : (envOfSpecs specs).entries c with
| none =>
  simp [hEntries] at hVal
| some a =>
  have hv : a.value? = some v0 := by
    simpa [hEntries] using hVal
  rcases entries_envOfSpecs_some_implies_from_specs specs hEntries with
    (s, hsMem, hsName, _, )hsValue
  have hsValSome : s.value? = some v0 := by

```

```

    calc
      s.value? = a.value? := hsValue
      _ = some v0 := hv
    have hNo : v0 ≠ (.const s.name) := hNoSelf s hsMem v0 hsValSome
    intro hEq
    apply hNo
    simp [hsName] using hEq

/-- Generic well-formedness constructor from per-spec typed-value and
no-self-unfolding obligations. -/
theorem envOfSpecs_wellFormed_of_specObligations
  (specs : List DeclSpec)
  (hObligations : DeclSpecObligations specs) :
  DeclEnvWellFormed (envOfSpecs specs) := by
  refine {?, ?}_
  · exact envOfSpecs_declValuesWellTyped_of_specValues specs hObligations.valuesWellTyped
  · exact envOfSpecs_noSelfDelta_of_specNoSelf specs hObligations.noSelfDelta

theorem SignatureWellFormed.toDeclEnvWellFormed {specs : List DeclSpec}
  (hSig : SignatureWellFormed specs) :
  DeclEnvWellFormed (envOfSpecs specs) :=
  envOfSpecs_wellFormed_of_specObligations specs hSig.obligations

/-- Generic well-formedness for declaration environments built from
non-unfolding declaration specs. -/
theorem envOfSpecs_wellFormed_of_all_none
  (specs : List DeclSpec)
  (hNone : ∀ s ∈ specs, s.value? = none) :
  DeclEnvWellFormed (envOfSpecs specs) := by
  refine {?, ?}_
  · intro c A0 v0 _ hVal
    have hNoneAt : valueOf? (envOfSpecs specs) c = none :=
      valueOf_envOfSpecs_eq_none_of_all_none specs hNone c
    rw [hNoneAt] at hVal
    cases hVal
  · intro c v0 hVal
    have hNoneAt : valueOf? (envOfSpecs specs) c = none :=
      valueOf_envOfSpecs_eq_none_of_all_none specs hNone c
    rw [hNoneAt] at hVal
    cases hVal

end Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

```

A.8 CoreEmbedding.lean

Mettapedia/Languages/MeTTa/PureKernel/CoreEmbedding.lean

```

import Mettapedia.Languages.MeTTa.CoreProfile
import Mettapedia.Languages.MeTTa.PureKernel.TypedLangDef
import Mettapedia.Languages.MeTTa.PureKernel.PatternBridge
import Mettapedia.Languages.MeTTa.PureKernel.Reduction
import Mettapedia.Languages.MeTTa.PureKernel.Renaming
import Mettapedia.Languages.MeTTa.PureKernel.Substitution
import Mettapedia.Languages.MeTTa.PureKernel.ProfileTheory

```



```

import Mettapedia.OSLF.Framework.TypeSynthesis

/-!
# PureKernel Embedding Into MeTTa Core Profiles

Scaffold linking the trusted scoped kernel (`PureTm`) with the MeTTa profile
interface through contextual quotation.
-/

namespace Mettapedia.Languages.MeTTa.PureKernel.CoreEmbedding

open Mettapedia.OSLF.MeTTaIL.Syntax
open Mettapedia.OSLF.MeTTaIL.Match
open Mettapedia.OSLF.MeTTaIL.Engine
open Mettapedia.OSLF.Framework.TypeSynthesis
open Mettapedia.OSLF.MeTTaIL.Substitution
open Mettapedia.Languages.MeTTa.CoreProfile
open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Reduction
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.PatternBridge
open Mettapedia.Languages.MeTTa.PureKernel.ProfileTheory
open Mettapedia.Languages.MeTTa.PureKernel.Assembly
open Mettapedia.Languages.MeTTa.Pure.Core

/-- Minimal embedding contract for closed PureKernel terms into a MeTTa profile. -/
structure KernelEmbedding where
  profile : MeTTaCoreProfile
  kernel : TypedKernelDef
  quoteClosed : PureTm 0 → Pattern
  quoteClosed_lc : ∀ t, lc_at 0 (quoteClosed t) = true

/-- Canonical embedding of PureKernel into the Pure core profile. -/
def pureKernelIntoPureProfile : KernelEmbedding where
  profile := pureProfile
  kernel := mettaPureKernelTyped
  quoteClosed := quoteClosedTm
  quoteClosed_lc := by
    intro t
    -- `quoteClosedTm` is `quoteTmWith defaultBinderName 0 emptyEnv`.
    simpa [quoteClosedTm, quoteTm, emptyEnv] using
      lc_quoteTmWith defaultBinderName 0 emptyEnv t

theorem pureKernel_embedding_target :
  pureKernelIntoPureProfile.profile.lang.name = "MeTTaPure" := by
  rfl

private def betaPiRule : RewriteRule :=
{ name := "BetaPi",
  typeContext := [("body", .base "Tm"), ("a", .base "Tm")],
  premises := [],
  left := .apply "App" [.apply "Lam" [.lambda (.fvar "body")], .fvar "a"],
  right := .subst (.fvar "body") (.fvar "a") }

```

```

private def betaSigmaFstRule : RewriteRule :=
{ name := "BetaSigmaFst",
  typeContext := [("a", .base "Tm"), ("b", .base "Tm")],
  premises := [],
  left := .apply "Fst" [.apply "Pair" [.fvar "a", .fvar "b"]],
  right := .fvar "a" }

private def betaSigmaSndRule : RewriteRule :=
{ name := "BetaSigmaSnd",
  typeContext := [("a", .base "Tm"), ("b", .base "Tm")],
  premises := [],
  left := .apply "Snd" [.apply "Pair" [.fvar "a", .fvar "b"]],
  right := .fvar "b" }

/-- Positive bridge example  $\beta\pi$ ( identity): quoted kernel step is a `langReduces` step. -/
theorem langReduces_quoteClosed_betaPi_id (a : PureTm 0) :
  langReduces mettaPure (quoteClosedTm (.app (.lam (.var 0)) a)) (quoteClosedTm a) := by
  apply exec_to_langReducesUsing (relEnv := RelationEnv.empty) (lang := mettaPure)
  unfold langReducesExecUsing rewriteWithContextWithPremisesUsing rewriteStepWithPremisesUsing
  apply List.mem_append.mpr
  left
  rw [List.mem_flatMap]
  refine (betaPiRule, ?_, ?)_
  · simp [betaPiRule, mettaPure]
  · unfold applyRuleWithPremisesUsing
    rw [List.mem_flatMap]
    let qbody : Pattern :=
      closeFVar 0 (defaultBinderName 0) (.fvar (defaultBinderName 0))
    let bs : Bindings := [("a", quoteClosedTm a), ("body", qbody)]
    refine (bs, ?_, ?)_
    · simp [bs, qbody, betaPiRule, quoteClosedTm, quoteTm, quoteTmWith,
      mkApp, mkLam, matchPattern, matchArgs, mergeBindings, envCons, emptyEnv]
    · rw [List.mem_map]
      refine (bs, ?_, ?)_
      · simp [applyPremisesWithEnv, bs, betaPiRule]
      · simp [bs, qbody, betaPiRule, applyBindings, closeFVar, openBVar]

/-- Positive bridge example  $\beta\Sigma$ ( fst): quoted kernel step is a `langReduces` step. -/
theorem langReduces_quoteClosed_betaSigmaFst (a b : PureTm 0) :
  langReduces mettaPure (quoteClosedTm (.fst (.pair a b))) (quoteClosedTm a) := by
  apply exec_to_langReducesUsing (relEnv := RelationEnv.empty) (lang := mettaPure)
  unfold langReducesExecUsing rewriteWithContextWithPremisesUsing rewriteStepWithPremisesUsing
  apply List.mem_append.mpr
  left
  rw [List.mem_flatMap]
  refine (betaSigmaFstRule, ?_, ?)_
  · simp [betaSigmaFstRule, mettaPure]
  · unfold applyRuleWithPremisesUsing
    rw [List.mem_flatMap]
    let bs : Bindings := [("b", quoteClosedTm b), ("a", quoteClosedTm a)]
    refine (bs, ?_, ?)_
    · simp [bs, betaSigmaFstRule, quoteClosedTm, quoteTm, quoteTmWith,
      mkFst, mkPair, matchPattern, matchArgs, mergeBindings]

```

```

· rw [List.mem_map]
  refine (bs, ?_, ?)_
· simp [applyPremisesWithEnv, bs, betaSigmaFstRule]
· simp [bs, betaSigmaFstRule, applyBindings]

/-- Positive bridge example  $\beta\Sigma$ ( snd): quoted kernel step is a `langReduces` step. -/
theorem langReduces_quoteClosed_betaSigmaSnd (a b : PureTm 0) :
  langReduces mettaPure (quoteClosedTm (.snd (.pair a b))) (quoteClosedTm b) := by
  apply exec_to_langReducesUsing (relEnv := RelationEnv.empty) (lang := mettaPure)
  unfold langReducesExecUsing rewriteWithContextWithPremisesUsing rewriteStepWithPremisesUsing
  apply List.mem_append.mpr
  left
  rw [List.mem_flatMap]
  refine (betaSigmaSndRule, ?_, ?)_
· simp [betaSigmaSndRule, mettaPure]
· unfold applyRuleWithPremisesUsing
  rw [List.mem_flatMap]
  let bs : Bindings := [("b", quoteClosedTm b), ("a", quoteClosedTm a)]
  refine (bs, ?_, ?)_
· simp [bs, betaSigmaSndRule, quoteClosedTm, quoteTm, quoteTmWith,
  mkSnd, mkPair, matchPattern, matchArgs, mergeBindings]
· rw [List.mem_map]
  refine (bs, ?_, ?)_
· simp [applyPremisesWithEnv, bs, betaSigmaSndRule]
· simp [bs, betaSigmaSndRule, applyBindings]

/-- **A-layer (operational)**: the closed computational fragment executed as one
`langReduces` step by current `mettaPure` profile semantics. -/
inductive PureOpStep : PureTm 0 → PureTm 0 → Prop where
| betaPiId (a : PureTm 0) :
  PureOpStep (.app (.lam (.var 0)) a) a
| betaSigmaFst (a b : PureTm 0) :
  PureOpStep (.fst (.pair a b)) a
| betaSigmaSnd (a b : PureTm 0) :
  PureOpStep (.snd (.pair a b)) b

/-- **B-layer (theory)**: congruence-theoretic kernel step relation. -/
abbrev PureTheoryStep : PureTm 0 → PureTm 0 → Prop := Red

/-- A-layer steps are B-layer steps. -/
theorem pureOpStep_to_pureTheoryStep {t u : PureTm 0}
  (h : PureOpStep t u) : PureTheoryStep t u :=
  match h with
| .betaPiId a => by
  simpa [PureTheoryStep] using (Red.betaPi (.var 0) a)
| .betaSigmaFst a b => by
  simpa [PureTheoryStep] using (Red.betaSigmaFst a b)
| .betaSigmaSnd a b => by
  simpa [PureTheoryStep] using (Red.betaSigmaSnd a b)

/-- A-layer one-step soundness into declarative `langReduces`. -/
theorem pureOpStep_sound_langReduces_quoteClosed {t u : PureTm 0}
  (h : PureOpStep t u) :
  langReduces mettaPure (quoteClosedTm t) (quoteClosedTm u) :=

```

```

match h with
| .betaPiId a => by
  simpa using langReduces_quoteClosed_betaPi_id a
| .betaSigmaFst a b => langReduces_quoteClosed_betaSigmaFst a b
| .betaSigmaSnd a b => langReduces_quoteClosed_betaSigmaSnd a b

/-- Reflexive-transitive closure of the operational A-layer. -/
abbrev PureOpStepStar (t u : PureTm 0) : Prop := Relation.ReflTransGen PureOpStep t u

/-- Reflexive-transitive closure of the theory B-layer. -/
abbrev PureTheoryStepStar (t u : PureTm 0) : Prop := RedStar t u

/-- A-layer stars embed into B-layer stars. -/
theorem pureOpStepStar_to_pureTheoryStepStar {t u : PureTm 0}
  (h : PureOpStepStar t u) : PureTheoryStepStar t u := by
  induction h with
  | refl =>
    simpa [PureTheoryStepStar] using RedStar.refl t
  | tail hxy hyz ih =>
    exact RedStar.tail ih (pureOpStep_to_pureTheoryStep hyz)

/-- A-layer multi-step soundness into profile-level multi-step reduction. -/
theorem pureOpStepStar_sound_langReduces_quoteClosed {t u : PureTm 0}
  (h : PureOpStepStar t u) :
  PureProfileStepStar (quoteClosedTm t) (quoteClosedTm u) := by
  induction h with
  | refl =>
    exact Relation.ReflTransGen.refl
  | tail hxy hyz ih =>
    exact Relation.ReflTransGen.tail ih
      (pureOpStep_sound_langReduces_quoteClosed hyz)

/-- A-layer one-step soundness into the sealed C1 base. -/
theorem pureOpStep_sound_pureProfileBaseStep_quoteClosed {t u : PureTm 0}
  (h : PureOpStep t u) :
  PureProfileBaseStep (quoteClosedTm t) (quoteClosedTm u) := by
  match h with
  | .betaPiId a =>
    let qbody : Pattern := closeFVar 0 (defaultBinderName 0) (.fvar (defaultBinderName 0))
    have hbase :
      PureProfileBaseStep (quoteClosedTm (.app (.lam (.var 0)) a))
        (openBVar 0 (quoteClosedTm a) qbody) := by
      simpa [qbody, quoteClosedTm, quoteTm, quoteTmWith, emptyEnv, envCons, mkApp, mkLam] using
        (PureProfileBaseStep.betaPi qbody (quoteClosedTm a))
    have hopen : openBVar 0 (quoteClosedTm a) qbody = quoteClosedTm a := by
      simp [qbody, closeFVar, openBVar]
    exact hopen ▸ hbase
  | .betaSigmaFst a b =>
    simpa [quoteClosedTm, quoteTm, quoteTmWith, mkFst, mkPair] using
      (PureProfileBaseStep.betaSigmaFst (quoteClosedTm a) (quoteClosedTm b))
  | .betaSigmaSnd a b =>
    simpa [quoteClosedTm, quoteTm, quoteTmWith, mkSnd, mkPair] using
      (PureProfileBaseStep.betaSigmaSnd (quoteClosedTm a) (quoteClosedTm b))

```

```

/-- A-layer one-step soundness into C1. -/
theorem pureOpStep_sound_pureProfileTheoryStep_quoteClosed {t u : PureTm 0}
  (h : PureOpStep t u) :
  PureProfileTheoryStep (quoteClosedTm t) (quoteClosedTm u) := by
  exact .base (pureOpStep_sound_pureProfileBaseStep_quoteClosed h)

/-- A-layer multi-step soundness into C1 star closure. -/
theorem pureOpStepStar_sound_pureProfileTheoryStep_quoteClosed {t u : PureTm 0}
  (h : PureOpStepStar t u) :
  PureProfileTheoryStepStar (quoteClosedTm t) (quoteClosedTm u) := by
  induction h with
  | refl =>
    exact Relation.ReflTransGen.refl
  | tail hxy hyz ih =>
    exact Relation.ReflTransGen.tail ih
    (pureOpStep_sound_pureProfileTheoryStep_quoteClosed hyz)

/-- **B -> C1 (parametric)**:
If kernel `inst0` commutes with quotation/opening for a naming policy v``,
then every kernel one-step reduction is sound into C1 at that quotation. -/
theorem pureTheoryStep_sound_pureProfileTheoryStep_quoteTmWith_assuming_inst0
  v( : Nat → String)
  (hinst0 : Inst0OpenBridgeCompat v)
  {n : Nat} (k : Nat) ρ( : QuoteEnv n) {t u : PureTm n}
  (hcompat : QuoteCompat v k ρ)
  (h : Red t u) :
  PureProfileTheoryStep (quoteTmWith v k ρ t) (quoteTmWith v k ρ u) := by
  induction h generalizing k with
  | @betaPi n body a =>
    have hbase :
      PureProfileTheoryStep
        (quoteTmWith v k ρ (.app (.lam body) a))
        (openBVar 0 (quoteTmWith v k ρ a)
          (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) body))) := by
    exact .base (PureProfileBaseStep.betaPi
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) body))
      (quoteTmWith v k ρ a))
    have hq := hinst0 k ρ a body hcompat
    simpa [quoteTmWith, mkApp, mkLam, hq] using hbase
  | @betaSigmaFst n a b =>
    simpa [quoteTmWith, mkFst, mkPair] using
      (PureProfileTheoryStep.base
        (PureProfileBaseStep.betaSigmaFst (quoteTmWith v k ρ a) (quoteTmWith v k ρ b)))
  | @betaSigmaSnd n a b =>
    simpa [quoteTmWith, mkSnd, mkPair] using
      (PureProfileTheoryStep.base
        (PureProfileBaseStep.betaSigmaSnd (quoteTmWith v k ρ a) (quoteTmWith v k ρ b)))
  | @congPiDom n A A' B hAA' ih =>
    have hstep := ih (k := k) ρ( := ρ) hcompat
    have hctx :
      PureProfileTheoryStep
        (mkPi (quoteTmWith v k ρ A)
          (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B)))
        (mkPi (quoteTmWith v k ρ A')

```

```

      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B))) := by
    exact .ctx (K := .piDom .hole
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B))) hstep
    simp [quoteTmWith, mkPi] using hctx
| @congPiCod n A B B' hBB' ih =>
  have hstep := ih (k := k + 1) ρ( := envCons v( k) ρ)
    (QuoteCompat.envCons hcompat.1 hcompat)
  have hclose :
    PureProfileTheoryStep
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B))
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B')) := by
    exact .ctx (K := .close v( k) .hole) hstep
  have hctx :
    PureProfileTheoryStep
      (mkPi (quoteTmWith v k ρ A)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B)))
      (mkPi (quoteTmWith v k ρ A)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B')))) := by
    exact .ctx (K := .piCod (quoteTmWith v k ρ A) .hole) hclose
    simp [quoteTmWith, mkPi] using hctx
| @congSigmaDom n A A' B hAA' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx :
    PureProfileTheoryStep
      (mkSigma (quoteTmWith v k ρ A)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B)))
      (mkSigma (quoteTmWith v k ρ A')
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B)))) := by
    exact .ctx (K := .sigmaDom .hole
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B))) hstep
    simp [quoteTmWith, mkSigma] using hctx
| @congSigmaCod n A B B' hBB' ih =>
  have hstep := ih (k := k + 1) ρ( := envCons v( k) ρ)
    (QuoteCompat.envCons hcompat.1 hcompat)
  have hclose :
    PureProfileTheoryStep
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B))
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B')) := by
    exact .ctx (K := .close v( k) .hole) hstep
  have hctx :
    PureProfileTheoryStep
      (mkSigma (quoteTmWith v k ρ A)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B)))
      (mkSigma (quoteTmWith v k ρ A)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B')))) := by
    exact .ctx (K := .sigmaCod (quoteTmWith v k ρ A) .hole) hclose
    simp [quoteTmWith, mkSigma] using hctx
| @congIdTy n A A' a b hAA' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkId (quoteTmWith v k ρ A) (quoteTmWith v k ρ a) (quoteTmWith v k ρ b))
    (mkId (quoteTmWith v k ρ A') (quoteTmWith v k ρ a) (quoteTmWith v k ρ b)) := by
    exact .ctx (K := .idTy .hole (quoteTmWith v k ρ a) (quoteTmWith v k ρ b)) hstep
    simp [quoteTmWith, mkId] using hctx

```

```

| @congIdLeft n A a a' b haa' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkId (quoteTmWith v k ρ A) (quoteTmWith v k ρ a) (quoteTmWith v k ρ b))
    (mkId (quoteTmWith v k ρ A) (quoteTmWith v k ρ a') (quoteTmWith v k ρ b)) := by
    exact .ctx (K := .idLeft (quoteTmWith v k ρ A) .hole (quoteTmWith v k ρ b)) hstep
  simpa [quoteTmWith, mkId] using hctx
| @congIdRight n A a b b' hbb' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkId (quoteTmWith v k ρ A) (quoteTmWith v k ρ a) (quoteTmWith v k ρ b))
    (mkId (quoteTmWith v k ρ A) (quoteTmWith v k ρ a) (quoteTmWith v k ρ b')) := by
    exact .ctx (K := .idRight (quoteTmWith v k ρ A) (quoteTmWith v k ρ a) .hole) hstep
  simpa [quoteTmWith, mkId] using hctx
| @congLam n b b' hbb' ih =>
  have hstep := ih (k := k + 1) ρ( := envCons v( k) ρ)
    (QuoteCompat.envCons hcompat.1 hcompat)
  have hclose :
    PureProfileTheoryStep
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) b))
      (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) b')) := by
    exact .ctx (K := .close v( k) .hole) hstep
  have hctx :
    PureProfileTheoryStep
      (mkLam (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) b)))
      (mkLam (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) b')))) := by
    exact .ctx (K := .lam .hole) hclose
  simpa [quoteTmWith, mkLam] using hctx
| @congAppFun n f f' a hff' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkApp (quoteTmWith v k ρ f) (quoteTmWith v k ρ a))
    (mkApp (quoteTmWith v k ρ f') (quoteTmWith v k ρ a)) := by
    exact .ctx (K := .appFun .hole (quoteTmWith v k ρ a)) hstep
  simpa [quoteTmWith, mkApp] using hctx
| @congAppArg n f a a' haa' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkApp (quoteTmWith v k ρ f) (quoteTmWith v k ρ a))
    (mkApp (quoteTmWith v k ρ f) (quoteTmWith v k ρ a')) := by
    exact .ctx (K := .appArg (quoteTmWith v k ρ f) .hole) hstep
  simpa [quoteTmWith, mkApp] using hctx
| @congPairFst n a a' b haa' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkPair (quoteTmWith v k ρ a) (quoteTmWith v k ρ b))
    (mkPair (quoteTmWith v k ρ a') (quoteTmWith v k ρ b)) := by
    exact .ctx (K := .pairFst .hole (quoteTmWith v k ρ b)) hstep
  simpa [quoteTmWith, mkPair] using hctx
| @congPairSnd n a b b' hbb' ih =>
  have hstep := ih (k := k) ρ( := ρ) hcompat
  have hctx : PureProfileTheoryStep
    (mkPair (quoteTmWith v k ρ a) (quoteTmWith v k ρ b))
    (mkPair (quoteTmWith v k ρ a) (quoteTmWith v k ρ b')) := by

```

```

    exact .ctx (K := .pairSnd (quoteTmWith v k p a) .hole) hstep
    simpa [quoteTmWith, mkPair] using hctx
| @congFst n p p' hpp' ih =>
    have hstep := ih (k := k) p( := p) hcompat
    have hctx : PureProfileTheoryStep
      (mkFst (quoteTmWith v k p p))
      (mkFst (quoteTmWith v k p p')) := by
    exact .ctx (K := .fst .hole) hstep
    simpa [quoteTmWith, mkFst] using hctx
| @congSnd n p p' hpp' ih =>
    have hstep := ih (k := k) p( := p) hcompat
    have hctx : PureProfileTheoryStep
      (mkSnd (quoteTmWith v k p p))
      (mkSnd (quoteTmWith v k p p')) := by
    exact .ctx (K := .snd .hole) hstep
    simpa [quoteTmWith, mkSnd] using hctx
| @congRefl n a a' haa' ih =>
    have hstep := ih (k := k) p( := p) hcompat
    have hctx : PureProfileTheoryStep
      (mkRefl (quoteTmWith v k p a))
      (mkRefl (quoteTmWith v k p a')) := by
    exact .ctx (K := .refl .hole) hstep
    simpa [quoteTmWith, mkRefl] using hctx

/-- Closed specialization of `
    pureTheoryStep_sound_pureProfileTheoryStep_quoteTmWith_assuming_inst0`
for the default binder naming policy. -/
theorem pureTheoryStep_sound_pureProfileTheoryStep_quoteClosed_assuming_inst0
  (hinst0 : Inst0OpenBridgeCompat defaultBinderName)
  (hcompat0 : QuoteCompat defaultBinderName 0 emptyEnv)
  {t u : PureTm 0} (h : PureTheoryStep t u) :
  PureProfileTheoryStep (quoteClosedTm t) (quoteClosedTm u) := by
  simpa [quoteClosedTm, quoteTm, emptyEnv] using
    pureTheoryStep_sound_pureProfileTheoryStep_quoteTmWith_assuming_inst0
    v( := defaultBinderName) hinst0 (k := 0) p( := emptyEnv) hcompat0 h

/-- Star-closure transport for B -> C1 under the same `inst0` bridge assumption. -/
theorem pureTheoryStepStar_sound_pureProfileTheoryStepStar_quoteClosed_assuming_inst0
  (hinst0 : Inst0OpenBridgeCompat defaultBinderName)
  (hcompat0 : QuoteCompat defaultBinderName 0 emptyEnv)
  {t u : PureTm 0} (h : PureTheoryStepStar t u) :
  PureProfileTheoryStepStar (quoteClosedTm t) (quoteClosedTm u) := by
  induction h with
  | refl =>
    exact Relation.ReflTransGen.refl
  | tail hxy hyz ih =>
    exact Relation.ReflTransGen.tail ih
    (pureTheoryStep_sound_pureProfileTheoryStep_quoteClosed_assuming_inst0 hinst0 hcompat0
    hyz)

/-- B -> C1 transport (parameterized by an `inst0` bridge witness). -/
theorem pureTheoryStep_sound_pureProfileTheoryStep_quoteTmWith
  v( : Nat → String) (hinst0 : Inst0OpenBridgeCompat v)
  {n : Nat} (k : Nat) p( : QuoteEnv n) {t u : PureTm n}

```



```

(hcompat : QuoteCompat v k p)
(h : Red t u) :
  PureProfileTheoryStep (quoteTmWith v k p t) (quoteTmWith v k p u) :=
pureTheoryStep_sound_pureProfileTheoryStep_quoteTmWith_assuming_inst0
  v( := v) hinst0 (k := k) p( := p) hcompat h

/-- Closed B -> C1 transport (parameterized by an `inst0` bridge witness). -/
theorem pureTheoryStep_sound_pureProfileTheoryStep_quoteClosed
  (hinst0 : Inst0OpenBridgeCompat defaultBinderName)
  (hcompat0 : QuoteCompat defaultBinderName 0 emptyEnv)
  {t u : PureTm 0} (h : PureTheoryStep t u) :
  PureProfileTheoryStep (quoteClosedTm t) (quoteClosedTm u) :=
pureTheoryStep_sound_pureProfileTheoryStep_quoteClosed_assuming_inst0 hinst0 hcompat0 h

/-- Closed star transport B* -> C1* (parameterized by an `inst0` bridge witness). -/
theorem pureTheoryStepStar_sound_pureProfileTheoryStepStar_quoteClosed
  (hinst0 : Inst0OpenBridgeCompat defaultBinderName)
  (hcompat0 : QuoteCompat defaultBinderName 0 emptyEnv)
  {t u : PureTm 0} (h : PureTheoryStepStar t u) :
  PureProfileTheoryStepStar (quoteClosedTm t) (quoteClosedTm u) :=
pureTheoryStepStar_sound_pureProfileTheoryStepStar_quoteClosed_assuming_inst0 hinst0 hcompat0 h

private def betaPiOneNestedLamRedex : PureTm 0 :=
  .app (.lam (.lam (.var (Fin.succ (0 : Fin 1))))) .u0

private def betaPiOneNestedLamContractum : PureTm 0 :=
  .lam .u0

private def betaPiTwoNestedLamRedex : PureTm 0 :=
  .app (.lam (.lam (.lam (.var (Fin.succ (Fin.succ (0 : Fin 1))))))) .u0

private def betaPiTwoNestedLamContractum : PureTm 0 :=
  .lam (.lam .u0)

/-- Regression: one nested binder in  $\beta\eta$  body still transports to C1. -/
theorem betaPi_bridge_regression_one_nestedLam_assuming_inst0
  (hinst0 : Inst0OpenBridgeCompat defaultBinderName)
  (hcompat0 : QuoteCompat defaultBinderName 0 emptyEnv) :
  PureProfileTheoryStep
    (quoteClosedTm betaPiOneNestedLamRedex)
    (quoteClosedTm betaPiOneNestedLamContractum) := by
have hred : Red betaPiOneNestedLamRedex betaPiOneNestedLamContractum := by
  simpa [betaPiOneNestedLamRedex, betaPiOneNestedLamContractum, inst0, subst, subst0, liftSub,
    rename, wk] using
    (Red.betaPi (.lam (.var (Fin.succ (0 : Fin 1))))) (.u0))
exact pureTheoryStep_sound_pureProfileTheoryStep_quoteClosed_assuming_inst0 hinst0 hcompat0
  hred

/-- Regression: two nested binders in  $\beta\eta$  body still transports to C1. -/
theorem betaPi_bridge_regression_two_nestedLam_assuming_inst0
  (hinst0 : Inst0OpenBridgeCompat defaultBinderName)
  (hcompat0 : QuoteCompat defaultBinderName 0 emptyEnv) :
  PureProfileTheoryStep
    (quoteClosedTm betaPiTwoNestedLamRedex)

```

```

      (quoteClosedTm betaPiTwoNestedLamContractum) := by
have hred : Red betaPiTwoNestedLamRedex betaPiTwoNestedLamContractum := by
  simpa [betaPiTwoNestedLamRedex, betaPiTwoNestedLamContractum, inst0, subst, subst0, liftSub,
    rename, wk] using
    (Red.betaPi (.lam (.lam (.var (Fin.succ (Fin.succ (0 : Fin 1)))))) (.u0))
exact pureTheoryStep_sound_pureProfileTheoryStep_quoteClosed_assuming_inst0 hinst0 hcompat0
hred

/-- Backwards-compatible name for the A-layer step relation. -/
abbrev ClosedComputationStep : PureTm 0 → PureTm 0 → Prop := PureOpStep

/-- Backwards-compatible name for A-layer star closure. -/
abbrev ClosedComputationStepStar (t u : PureTm 0) : Prop := PureOpStepStar t u

/-- Backwards-compatible theorem alias. -/
theorem closedComputationStep_to_red {t u : PureTm 0}
  (h : ClosedComputationStep t u) : Red t u := by
  simpa [ClosedComputationStep, PureTheoryStep] using
    (pureOpStep_to_pureTheoryStep (t := t) (u := u) h)

/-- Backwards-compatible theorem alias. -/
theorem closedComputationStepStar_to_redStar {t u : PureTm 0}
  (h : ClosedComputationStepStar t u) : RedStar t u := by
  simpa [ClosedComputationStepStar, PureTheoryStepStar] using
    (pureOpStepStar_to_pureTheoryStepStar (t := t) (u := u) h)

/-- Backwards-compatible theorem alias. -/
theorem closedComputationStep_sound_langReduces_quoteClosed {t u : PureTm 0}
  (h : ClosedComputationStep t u) :
  langReduces mettaPure (quoteClosedTm t) (quoteClosedTm u) := by
  simpa [ClosedComputationStep] using
    (pureOpStep_sound_langReduces_quoteClosed (t := t) (u := u) h)

/-- Backwards-compatible theorem alias. -/
theorem closedComputationStepStar_sound_langReduces_quoteClosed {t u : PureTm 0}
  (h : ClosedComputationStepStar t u) :
  PureProfileStepStar (quoteClosedTm t) (quoteClosedTm u) := by
  simpa [ClosedComputationStepStar] using
    (pureOpStepStar_sound_langReduces_quoteClosed (t := t) (u := u) h)

/-- Negative bridge example:
the full kernel step relation (`Red`, with constructor congruence) is strictly
stronger than current `langReduces mettaPure` (top-level rewrite in this profile). -/
theorem not_all_pureTheoryStep_sound_to_langReduces_quoteClosed :
  ¬ ∀( {t u : PureTm 0}, PureTheoryStep t u →
    langReduces mettaPure (quoteClosedTm t) (quoteClosedTm u)) := by
intro hsound
let t : PureTm 0 := .app .u1 (.fst (.pair .u0 .u1))
let u : PureTm 0 := .app .u1 .u0
have hred : PureTheoryStep t u := by
  simpa [PureTheoryStep] using (Red.congAppArg (Red.betaSigmaFst _ _))
have hlang : langReduces mettaPure (quoteClosedTm t) (quoteClosedTm u) := hsound hred
have hexec : langReducesExecUsing RelationEnv.empty mettaPure (quoteClosedTm t) (quoteClosedTm
  u) :=

```

```

langReducesUsing_to_exec (relEnv := RelationEnv.empty) (lang := mettaPure) hlang
have hempty : rewriteWithContextWithPremisesUsing RelationEnv.empty mettaPure (quoteClosedTm t)
  = [] := by
simp [t, quoteClosedTm, quoteTm, quoteTmWith, mkApp, mkFst, mkPair,
  rewriteWithContextWithPremisesUsing, rewriteStepWithPremisesUsing,
  applyRuleWithPremisesUsing, mettaPure, applyPremisesWithEnv]
constructor
· intro x hx
  simp [ul, matchArgs, matchPattern] at hx
constructor
· intro x hx
  simp [ul, matchPattern] at hx
· intro x hx
  simp [ul, matchPattern] at hx
have hfalse : False := by
  simp [langReducesExecUsing, hempty] at hexec
exact False.elim hfalse

/-- Backwards-compatible name for the same mismatch fact. -/
theorem not_all_red_steps_sound_to_langReduces_quoteClosed :
  ¬ ∀( {t u : PureTm 0}, Red t u →
    langReduces mettaPure (quoteClosedTm t) (quoteClosedTm u)) := by
intro h
exact not_all_pureTheoryStep_sound_to_langReduces_quoteClosed (fun hred => h hred)

end Mettapedia.Languages.MeTTa.PureKernel.CoreEmbedding

```

A.9 DefEq.lean

Mettapedia/Languages/MeTTa/PureKernel/DefEq.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.Confluence
import Mettapedia.Languages.MeTTa.PureKernel.Parallel
import Mettapedia.Languages.MeTTa.PureKernel.Typing

namespace Mettapedia.Languages.MeTTa.PureKernel

open Syntax
open Renaming
open Substitution
open Reduction
open Confluence
open Parallel
open Typing

theorem inst0_rename_wk_cancel (a t : PureTm n) :
  inst0 a (rename wk t) = t := by
calc
  inst0 a (rename wk t)
    = subst (subst0 a) (rename wk t) := by
      rfl
  _ = subst (fun i => subst0 a (wk i)) t := by
      simp using (subst_rename σ( := subst0 a) ρ( := wk) (t := t))
  _ = subst ids t := by

```

```

    apply subst_ext
    intro i
    rfl
  _ = t := by
    exact subst_ids (t := t)

theorem conv_to_cdev (t : PureTm n) : Conv t (cdev t) :=
  redStar_implies_conv (par_to_redStar (par_to_cdev_self t))

theorem conv_of_cdev_eq {A B : PureTm n} (h : cdev A = cdev B) :
  Conv A B := by
  have hA : Conv A (cdev A) := conv_to_cdev A
  have hB : Conv B (cdev B) := conv_to_cdev B
  exact Relation.EqvGen.trans _ _ hA
    (Relation.EqvGen.symm _ _ (by simpa [h] using hB))

structure DefEqWitness (A B : PureTm n) : Type where
  conv : Conv A B

def defEqByNormalization? (A B : PureTm n) : Option (DefEqWitness A B) :=
  if h : cdev A = cdev B then
    some (conv_of_cdev_eq )h
  else
    none

structure PiView (t : PureTm n) : Type where
  dom : PureTm n
  cod : PureTm (n + 1)
  conv : Conv t (.pi dom cod)

structure SigmaView (t : PureTm n) : Type where
  dom : PureTm n
  cod : PureTm (n + 1)
  conv : Conv t (.sigma dom cod)

def asPi? (t : PureTm n) : Option (PiView t) :=
  match hnorm : cdev t with
  | .pi A B =>
    some
      { dom := A
        cod := B
        conv := by
          simpa [hnorm] using conv_to_cdev t }
  | _ => none

def asSigma? (t : PureTm n) : Option (SigmaView t) :=
  match hnorm : cdev t with
  | .sigma A B =>
    some
      { dom := A
        cod := B
        conv := by
          simpa [hnorm] using conv_to_cdev t }
  | _ => none

```

```
end Mettapedia.Languages.MeTTa.PureKernel
```

A.10 Inst0BridgeDerived.lean

Mettapedia/Languages/MeTTa/PureKernel/Inst0BridgeDerived.lean

```
import Mettapedia.Languages.MeTTa.PureKernel.Inst0BridgeProof

namespace Mettapedia.Languages.MeTTa.PureKernel.PatternBridge

open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.OSLF.MeTTaIL.Substitution

/-- Default-binder compatibility bridge derived from the staged ambient proof family. -/
theorem inst0ApplyBridgeCompat_defaultBinderName :
  Inst0ApplyBridgeCompat defaultBinderName := by
  intro n k p a body hcompat
  have hambient :
    Inst0BridgeProof.inst0AmbientDistinctTargetEq 0 defaultBinderName 0 k p a body :=
    Inst0BridgeProof.inst0AmbientDistinctTargetEq_all defaultBinderName 0 0 k p a body hcompat
  have hbinder :
    Inst0BridgeProof.inst0BinderTargetEqDistinct defaultBinderName 0 k p a body :=
    (Inst0BridgeProof.inst0AmbientDistinctTargetEq_zero_iff defaultBinderName 0 k p a body).mp
    hambient
  simp [Inst0BridgeProof.inst0BinderTargetEqDistinct,
    Inst0BridgeProof.inst0BinderClosedLhsDistinct,
    Inst0BridgeProof.inst0BinderClosedTargetDistinct, Inst0BridgeProof.closeAmbient,
    Inst0BridgeProof.buildEnv, Inst0BridgeProof.liftSubN, inst0, subst, subst0]
  using hbinder

/-- Default-binder open-form bridge derived from the staged apply-form theorem. -/
theorem inst0OpenBridgeCompat_defaultBinderName :
  Inst0OpenBridgeCompat defaultBinderName :=
  inst0ApplyBridgeCompat_to_openBridgeCompat inst0ApplyBridgeCompat_defaultBinderName

/-- Default-binder apply-form `inst0` theorem without external bridge arguments. -/
theorem quoteTmWith_defaultBinderName_inst0_apply
  (k : Nat) p( : QuoteEnv n) (hcompat : QuoteCompat defaultBinderName k p)
  (a : PureTm n) (body : PureTm (n + 1)) :
  quoteTmWith defaultBinderName k p (inst0 a body) =
  applySubst (SubstEnv.extend SubstEnv.empty (defaultBinderName k)
    (quoteTmWith defaultBinderName k p a))
    (quoteTmWith defaultBinderName (k + 1)
      (envCons (defaultBinderName k) p) body) :=
  inst0ApplyBridgeCompat_defaultBinderName k p a body hcompat

end Mettapedia.Languages.MeTTa.PureKernel.PatternBridge
```

A.11 Inst0BridgeProof.lean

Mettapedia/Languages/MeTTa/PureKernel/Inst0BridgeProof.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.PatternBridge
import Mettapedia.Languages.MeTTa.PureKernel.Substitution
import Mettapedia.OSLF.MeTTaIL.Substitution

/-!
# Inst0 Bridge Proof Core

This module contains the internal proof core for the theoremic default-binder
`inst0` bridge.

It is not the user-facing API surface. The durable public theorems live in
`PatternBridge` and `Inst0BridgeDerived`. This file keeps the ambient/distinct
proof family and its support lemmas in one place so downstream modules can rely
on a stable theoremic interface without threading bridge assumptions.
-/

namespace Mettapedia.Languages.MeTTa.PureKernel.PatternBridge.Inst0BridgeProof

open Mettapedia.Languages.MeTTa.PureKernel.PatternBridge
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.OSLF.MeTTaIL.Syntax
open Mettapedia.OSLF.MeTTaIL.Substitution

def liftSubN : (i : Nat) -> Sub n m -> Sub (n + i) (m + i)
| 0, σ => σ
| i + 1, σ => liftSub (liftSubN i σ)

@[simp] theorem liftSubN_zero σ( : Sub n m) :
  liftSubN 0 σ = σ := rfl

@[simp] theorem liftSubN_succ (i : Nat) σ( : Sub n m) :
  liftSubN (i + 1) σ = liftSub (liftSubN i σ) := rfl

def buildEnv v( : Nat → String) (k : Nat) : (i : Nat) -> QuoteEnv n -> QuoteEnv (n + i)
| 0, ρ => ρ
| i + 1, ρ => envCons v( (k + i)) (buildEnv v k i ρ)

@[simp] theorem buildEnv_zero v( : Nat → String) (k : Nat) ρ( : QuoteEnv n) :
  buildEnv v k 0 ρ = ρ := rfl

@[simp] theorem buildEnv_succ v( : Nat → String) (k i : Nat) ρ( : QuoteEnv n) :
  buildEnv v k (i + 1) ρ = envCons v( (k + i)) (buildEnv v k i ρ) := rfl

def closeRange v( : Nat → String) (k : Nat) : Nat -> Pattern -> Pattern
| 0, p => p
| i + 1, p => closeRange v k i (closeFVar 0 v( (k + i)) p)

@[simp] theorem closeRange_zero v( : Nat → String) (k : Nat) (p : Pattern) :
  closeRange v k 0 p = p := rfl

@[simp] theorem closeRange_succ v( : Nat → String) (k i : Nat) (p : Pattern) :

```

```

    closeRange v k (i + 1) p = closeRange v k i (closeFVar 0 v( (k + i)) p) := rfl

def closeRangeAt (d : Nat) v( : Nat → String) (k : Nat) : Nat -> Pattern -> Pattern
| 0, p => p
| i + 1, p => closeRangeAt d v k i (closeFVar d v( (k + i)) p)

@[simp] theorem closeRangeAt_zero (d : Nat) v( : Nat → String) (k : Nat) (p : Pattern) :
  closeRangeAt d v k 0 p = p := rfl

@[simp] theorem closeRangeAt_succ
  (d : Nat) v( : Nat → String) (k i : Nat) (p : Pattern) :
  closeRangeAt d v k (i + 1) p = closeRangeAt d v k i (closeFVar d v( (k + i)) p) := rfl

/-- Close `e` ambient binder names v` k, ..., v (k+e-1)` so the newest closes to
`bvar d` and older ones shift outwards. -/
def closeAmbient (d : Nat) v( : Nat → String) (k : Nat) : Nat -> Pattern -> Pattern
| 0, p => p
| i + 1, p => closeAmbient (d + 1) v k i (closeFVar d v( (k + i)) p)

@[simp] theorem closeAmbient_zero
  (d : Nat) v( : Nat → String) (k : Nat) (p : Pattern) :
  closeAmbient d v k 0 p = p := rfl

@[simp] theorem closeAmbient_succ
  (d : Nat) v( : Nat → String) (k i : Nat) (p : Pattern) :
  closeAmbient d v k (i + 1) p = closeAmbient (d + 1) v k i (closeFVar d v( (k + i)) p) := rfl

@[simp] theorem closeAmbient_one
  (d : Nat) v( : Nat → String) (k : Nat) (p : Pattern) :
  closeAmbient d v k 1 p = closeFVar d v( k) p := by
  simp [closeAmbient]

@[simp] theorem closeAmbient_bvar
  (d : Nat) v( : Nat → String) (k m j : Nat) :
  closeAmbient d v k m (.bvar j) = .bvar j := by
  induction m generalizing d with
  | zero =>
    rfl
  | succ m ih =>
    simp [closeAmbient, ih, closeFVar]

theorem closeAmbient_mkLam
  (d : Nat) v( : Nat → String) (k e : Nat) (body : Pattern) :
  closeAmbient d v k e (Mettapedia.Languages.MeTTa.Pure.Core.mkLam body)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (closeAmbient (d + 1) v k e body) := by
  induction e generalizing d k body with
  | zero =>
    simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkLam]
  | succ e ih =>
    simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkLam, Nat.add_assoc]
    using
      ih (d := d + 1) (k := k) (body := closeFVar (d + 1) v( (k + e)) body)

```

```

theorem closeAmbient_mkPi
  (d : Nat) v( : Nat → String) (k e : Nat) (A B : Pattern) :
  closeAmbient d v k e (Mettapedia.Languages.MeTTa.Pure.Core.mkPi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (closeAmbient d v k e A)
    (closeAmbient (d + 1) v k e B) := by
induction e generalizing d k A B with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkPi]
| succ e ih =>
  simp [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPi, Nat.add_assoc]
  using
    ih (d := d + 1) (k := k)
    (A := closeFVar d v( (k + e)) A)
    (B := closeFVar (d + 1) v( (k + e)) B)

theorem closeAmbient_mkSigma
  (d : Nat) v( : Nat → String) (k e : Nat) (A B : Pattern) :
  closeAmbient d v k e (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (closeAmbient d v k e A)
    (closeAmbient (d + 1) v k e B) := by
induction e generalizing d k A B with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkSigma]
| succ e ih =>
  simp [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSigma, Nat.add_assoc]
  using
    ih (d := d + 1) (k := k)
    (A := closeFVar d v( (k + e)) A)
    (B := closeFVar (d + 1) v( (k + e)) B)

theorem closeAmbient_push_next
  (d : Nat) v( : Nat → String) (k e : Nat) (p : Pattern) :
  closeAmbient (d + 1) v k e (closeFVar d v( (k + e)) p)
  =
  closeAmbient d v k (e + 1) p := by
simp [closeAmbient]

theorem closeAmbient_split
  (d : Nat) v( : Nat → String) (k m e : Nat) (p : Pattern) :
  closeAmbient d v k (m + e) p =
  closeAmbient (d + e) v k m
    (closeAmbient d v (k + m) e p) := by
induction e generalizing d k p with
| zero =>
  simp
| succ e ih =>
  calc
    closeAmbient d v k (m + (e + 1)) p
    = closeAmbient d v k ((m + e) + 1) p := by

```



```

      simp [Nat.add_assoc]
    _ = closeAmbient (d + 1) v k (m + e)
      (closeFVar d v( (k + (m + e))) p) := by
      simp [closeAmbient]
    _ = closeAmbient ((d + 1) + e) v k m
      (closeAmbient (d + 1) v (k + m) e
        (closeFVar d v( (k + (m + e))) p)) := by
      rw [ih (d := d + 1) (k := k)
        (p := closeFVar d v( (k + (m + e))) p)]
    _ = closeAmbient (d + (e + 1)) v k m
      (closeAmbient d v (k + m) (e + 1) p) := by
      simp [closeAmbient, Nat.add_left_comm, Nat.add_comm]

theorem closeRange_eq_closeRangeAt_zero
  v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeRange v k m p = closeRangeAt 0 v k m p := by
  induction m generalizing p with
  | zero =>
    rfl
  | succ m ih =>
    simp [closeRange, closeRangeAt, ih]

def renameWkN : (i : Nat) -> PureTm n -> PureTm (n + i)
| 0, t => t
| i + 1, t => rename wk (renameWkN i t)

@[simp] theorem renameWkN_zero (t : PureTm n) :
  renameWkN 0 t = t := rfl

@[simp] theorem renameWkN_succ (i : Nat) (t : PureTm n) :
  renameWkN (i + 1) t = rename wk (renameWkN i t) := rfl

def castPureTm {n m : Nat} (h : n = m) (t : PureTm n) : PureTm m := by
  subst h
  exact t

@[simp] theorem castPureTm_rfl (t : PureTm n) :
  castPureTm rfl t = t := rfl

def castAmbientBody
  {n m e : Nat}
  (b : PureTm (((n + 1) + (m + e)) + 1)) :
  PureTm ((n + 1) + (m + (e + 1))) :=
  castPureTm (by omega) b

def wkN : (i : Nat) -> Ren n (n + i)
| 0 => idRen
| i + 1 => fun x => wk (wkN i x)

@[simp] theorem wkN_zero :
  wkN (n := n) 0 = idRen := rfl

@[simp] theorem wkN_succ (i : Nat) :
  wkN (n := n) (i + 1) = fun x => wk (wkN i x) := rfl

```

```

@[simp] theorem wkN_val (m : Nat) (i : Fin n) :
  (wkN (n := n) m i).1 = m + i.1 := by
  induction m generalizing n with
  | zero =>
    simp [wkN, idRen]
  | succ m ih =>
    simp [wk, wkN, ih, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm]

def prefixIdx (tail : Nat) {m : Nat} (i : Fin m) : Fin (tail + m) :=
  (i.1, by )omega

@[simp] theorem prefixIdx_val (tail : Nat) {m : Nat} (i : Fin m) :
  (prefixIdx tail i).1 = i.1 := rfl

@[simp] theorem renameWkN_eq_rename_wkN (i : Nat) (t : PureTm n) :
  renameWkN i t = rename (wkN (n := n) i) t := by
  induction i generalizing n with
  | zero =>
    simp [renameWkN, wkN]
  | succ i ih =>
    simp [renameWkN, wkN, ih, rename_comp]

@[simp] theorem buildEnv_wkN_apply
  v( : Nat → String) (k m : Nat) p( : QuoteEnv n) (i : Fin n) :
  buildEnv v k m p (wkN (n := n) m i) = p i := by
  induction m generalizing n k p with
  | zero =>
    simp [buildEnv, wkN, idRen]
  | succ m ih =>
    simp [buildEnv, wkN, envCons] using ih (k := k) p( := p) i

theorem quoteTmWith_renameWkN_buildEnv
  v( : Nat → String) (k m : Nat) p( : QuoteEnv n) (t : PureTm n) :
  quoteTmWith v (k + m) (buildEnv v k m p) (renameWkN m t) =
  quoteTmWith v (k + m) p t := by
  rw [renameWkN_eq_rename_wkN]
  calc
  quoteTmWith v (k + m) (buildEnv v k m p) (rename (wkN (n := n) m) t)
  =
  quoteTmWith v (k + m)
  (fun i => buildEnv v k m p (wkN (n := n) m i)) t := by
  simp using
  (quoteTmWith_rename v (k := k + m)
  p(dst := buildEnv v k m p) p( := wkN (n := n) m) (t := t))
  _ = quoteTmWith v (k + m) p t := by
  congr
  funext i
  exact buildEnv_wkN_apply v k m p i

theorem quoteTmWith_renameWkN_buildEnv_any
  v( : Nat → String) (l k m : Nat) p( : QuoteEnv n) (t : PureTm n) :
  quoteTmWith v l (buildEnv v k m p) (renameWkN m t) =
  quoteTmWith v l p t := by

```

```

rw [renameWkN_eq_rename_wkN]
calc
  quoteTmWith v l (buildEnv v k m p) (rename (wkN (n := n) m) t)
    =
  quoteTmWith v l
    (fun i => buildEnv v k m p (wkN (n := n) m i)) t := by
      simpa using
        (quoteTmWith_rename v (k := l)
          p(dst := buildEnv v k m p) p( := wkN (n := n) m) (t := t))
  _ = quoteTmWith v l p t := by
    congr
    funext i
    exact buildEnv_wkN_apply v k m p i

theorem quoteCompat_mono
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) {j : Nat} (hkj : k ≤ j) :
  QuoteCompat v j p := by
  refine (hcompat.1, ?)_
intro i j' hj'
exact hcompat.2 i j' (le_trans hkj hj')

theorem buildEnv_compat
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) :
  ∀ m : Nat, QuoteCompat v (k + m) (buildEnv v k m p) := by
intro m
induction m with
| zero =>
  simpa using hcompat
| succ m ih =>
  simpa [buildEnv, Nat.add_assoc] using
    (QuoteCompat.envCons hcompat.1 ih)

theorem isFresh_quoteTmWith_of_compat
  v{ : Nat → String} {j : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v j p) (l : Nat) (t : PureTm n) :
  isFresh v(j) (quoteTmWith v l p t) = true := by
  unfold isFresh
  rw [Bool.not_eq_true']
  apply Bool.eq_false_iff.mpr
intro hcontains
have hz : v j ∈ freeVars (quoteTmWith v l p t) := List.contains_iff_mem.mp hcontains
rcases freeVars_quoteTmWith_mem_env v l p t hz with {i, }hi
exact (hcompat.2 i j (by omega)) hi

theorem closeRange_quoteTmWith_id
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) :
  ∀ (m l : Nat) (t : PureTm n),
  closeRange v k m (quoteTmWith v l p t) = quoteTmWith v l p t := by
intro m
induction m with
| zero =>

```

```

    intro l t
    simp [closeRange]
| succ m ih =>
    intro l t
    have hcompat' : QuoteCompat v (k + m) p :=
      quoteCompat_mono hcompat (j := k + m) (by omega)
    have hfresh :
      isFresh v( (k + m)) (quoteTmWith v l p t) = true :=
      isFresh_quoteTmWith_of_compat hcompat' l t
    calc
      closeRange v k (m + 1) (quoteTmWith v l p t)
        = closeRange v k m
          (closeFVar 0 v( (k + m)) (quoteTmWith v l p t)) := by
            simp [closeRange]
      _ = closeRange v k m (quoteTmWith v l p t) := by
        simp [closeFVar_fresh_id 0 v( (k + m)) (quoteTmWith v l p t) hfresh]
      _ = quoteTmWith v l p t := ih l t

theorem closeRangeAt_quoteTmWith_id
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (d : Nat) :
  ∀ (m l : Nat) (t : PureTm n),
    closeRangeAt d v k m (quoteTmWith v l p t) = quoteTmWith v l p t := by
intro m
induction m with
| zero =>
    intro l t
    simp [closeRangeAt]
| succ m ih =>
    intro l t
    have hcompat' : QuoteCompat v (k + m) p :=
      quoteCompat_mono hcompat (j := k + m) (by omega)
    have hfresh :
      isFresh v( (k + m)) (quoteTmWith v l p t) = true :=
      isFresh_quoteTmWith_of_compat hcompat' l t
    calc
      closeRangeAt d v k (m + 1) (quoteTmWith v l p t)
        = closeRangeAt d v k m
          (closeFVar d v( (k + m)) (quoteTmWith v l p t)) := by
            simp [closeRangeAt]
      _ = closeRangeAt d v k m (quoteTmWith v l p t) := by
        simp [closeFVar_fresh_id d v( (k + m)) (quoteTmWith v l p t) hfresh]
      _ = quoteTmWith v l p t := ih l t

theorem closeAmbient_quoteTmWith_id
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (d : Nat) :
  ∀ (m l : Nat) (t : PureTm n),
    closeAmbient d v k m (quoteTmWith v l p t) = quoteTmWith v l p t := by
intro m
induction m generalizing d with
| zero =>
    intro l t
    simp [closeAmbient]

```

```

| succ m ih =>
  intro l t
  have hcompat' : QuoteCompat v (k + m) p :=
    quoteCompat_mono hcompat (j := k + m) (by omega)
  have hfresh :
    isFresh v( (k + m)) (quoteTmWith v l p t) = true :=
    isFresh_quoteTmWith_of_compat hcompat' l t
  calc
    closeAmbient d v k (m + 1) (quoteTmWith v l p t)
      = closeAmbient (d + 1) v k m
        (closeFVar d v( (k + m)) (quoteTmWith v l p t)) := by
          simp [closeAmbient]
    _ = closeAmbient (d + 1) v k m (quoteTmWith v l p t) := by
          simp [closeFVar_fresh_id d v( (k + m)) (quoteTmWith v l p t) hfresh]
    _ = quoteTmWith v l p t := ih (d := d + 1) l t

/-- Staging-only multi-open used to recover plain depth-independence. -/
def multiOpenAtStaging (d : Nat) {n : Nat} ρ ( : QuoteEnv n) : Pattern → Pattern
| .bvar j =>
  if h : d ≤ j ∧ j < d + n then
    .fvar ρ( {j - d, by }omega)
  else
    .bvar j
| .fvar x => .fvar x
| .apply c args => .apply c (args.map (multiOpenAtStaging d ρ))
| .lambda body => .lambda (multiOpenAtStaging (d + 1) ρ body)
| .multiLambda m body => .multiLambda m (multiOpenAtStaging (d + m) ρ body)
| .subst body repl => .subst (multiOpenAtStaging (d + 1) ρ body) (multiOpenAtStaging d ρ repl)
| .collection ct elems rest =>
  .collection ct (elems.map (multiOpenAtStaging d ρ)) rest
termination_by p => sizeOf p

theorem freeVars_quoteRaw_staging {n : Nat} (t : PureTm n) : freeVars (quoteRaw t) = [] := by
  induction t with
  | var i => simp [quoteRaw, freeVars]
  | const c =>
    simp [quoteRaw, quoteConst, freeVars]
  | u0 => simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.u0, freeVars]
  | u1 => simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.u1, freeVars]
  | pi A B ihA ihB =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkPi, freeVars, List.flatMap, ihA, ihB]
  | sigma A B ihA ihB =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkSigma, freeVars, List.flatMap, ihA, ihB]
  | id A a b ihA iha ihb =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkId, freeVars, List.flatMap, ihA, iha, ihb]
  | lam b ih =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkLam, freeVars, List.flatMap, ih]
  | app f a ihf iha =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkApp, freeVars, List.flatMap, ihf, iha]
  | pair a b iha ihb =>

```

```

    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkPair, freeVars, List.flatMap, iha,
ihb]
| fst p ih =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkFst, freeVars, List.flatMap, ih]
| snd p ih =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd, freeVars, List.flatMap, ih]
| refl a ih =>
    simp [quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl, freeVars, List.flatMap, ih]

theorem closeFVar_multiOpenAt_envCons_staging
  {n : Nat} (d : Nat) (x : String) p( : QuoteEnv n) (p : Pattern)
  (hfresh : freeVars p = []) (ph : ∀ i : Fin n, p i ≠ x) :
  closeFVar d x (multiOpenAtStaging d (envCons x p) p) = multiOpenAtStaging (d + 1) p p := by
  induction p using Pattern.inductionOn generalizing d with
  | hbvar j =>
    simp only [multiOpenAtStaging]
    rcases Nat.lt_or_ge j d with hjd | hjd
    · rw [dif_neg (by omega), dif_neg (by omega)]
      simp [closeFVar]
    · rcases Nat.eq_or_lt_of_le hjd with rfl | hjd
      · rw [dif_pos (by omega : d ≤ d ∧ d < d + (n + 1)),
        dif_neg (by omega : ¬(d + 1 ≤ d ∧ d < d + 1 + n))]
        have hfin0 : ((d - d, by )omega : Fin (n + 1)) = (0 : Fin (n + 1)) := by
          apply Fin.ext
          simp
        rw [hfin0]
        simp [envCons, closeFVar]
      · rcases Nat.lt_or_ge j (d + (n + 1)) with hjn | hjn
        · rw [dif_pos (by omega), dif_pos (by omega)]
          have hfin_succ :
            ((j - d, by )omega : Fin (n + 1)) =
              Fin.succ (j - d - 1, by )omega := by
            ext
            simp
            omega
          have phneq : p (j - d - 1, by )omega ≠ x := ph (j - d - 1, by )omega
          have hidx :
            ((j - d - 1, by )omega : Fin n) =
              (j - (d + 1), by )omega := by
            apply Fin.ext
            have hnats : j - d - 1 = j - (d + 1) := by omega
            simp using hnats
          rw [hfin_succ]
          simp [envCons, closeFVar, phneq, hidx]
        · rw [dif_neg (by omega), dif_neg (by omega)]
          simp [closeFVar]
  | hfvar y =>
    simp [freeVars] at hfresh
  | happly c args ih =>
    simp only [multiOpenAtStaging, closeFVar, List.map_map]
    congr 1
    have hargs : ∀ a ∈ args, freeVars a = [] :=
      List.flatMap_eq_nil_iff.mp (show args.flatMap freeVars = [] by simp [freeVars] using
hfresh)

```

```

    exact List.map_eq_map_iff.mpr fun a ha => ih a ha d (hargs a ha)
| hlambda body ih =>
    simp only [multiOpenAtStaging, closeFVar]
    congr 1
    exact ih (d + 1) (by simpa [freeVars] using hfresh)
| hmultiLambda m body ih =>
    simp only [multiOpenAtStaging, closeFVar]
    have heq : d + 1 + m = d + m + 1 := by omega
    rw [heq]
    congr 1
    exact ih (d + m) (by simpa [freeVars] using hfresh)
| hsubst body repl ihb ihr =>
    simp only [multiOpenAtStaging, closeFVar]
    have hparts : freeVars body = [] ∧ freeVars repl = [] := by
      simpa [freeVars, List.append_eq_nil_iff] using hfresh
    have heq : d + 1 + 1 = (d + 1) + 1 := by omega
    congr
    · rw [heq]
      exact ihb (d + 1) hparts.1
    · exact ihr d hparts.2
| hcollection ct elems rest ih =>
    simp only [multiOpenAtStaging, closeFVar, List.map_map]
    congr 1
    have helems : ∀ a ∈ elems, freeVars a = [] :=
      List.flatMap_eq_nil_iff.mp (show elems.flatMap freeVars = [] by simpa [freeVars] using
hfresh)
    exact List.map_eq_map_iff.mpr fun a ha => ih a ha d (helems a ha)

theorem quoteTmWith_eq_multiOpenAt_quoteRaw_staging
  (v : Nat → String) (k : Nat) {n : Nat} ρ ( : QuoteEnv n) (t : PureTm n)
  (hcompat : QuoteCompat v k ρ) :
  quoteTmWith v k ρ t = multiOpenAtStaging 0 ρ (quoteRaw t) := by
induction t generalizing k with
| var i =>
  simp only [quoteTmWith, quoteRaw, multiOpenAtStaging]
  split
  · rename_i h
    simp
  · exfalso
    rename_i h
    apply h
    constructor
    · exact Nat.zero_le _
    · simp [i.isLt]
| const c =>
  simp [quoteTmWith, quoteRaw, quoteConst, multiOpenAtStaging]
| u0 =>
  simp [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.u0, multiOpenAtStaging,
List.map]
| u1 =>
  simp [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.u1, multiOpenAtStaging,
List.map]
| id A a b ihA iha ihb =>
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkId,

```

```

multiOpenAtStaging, List.map]
  rw [ihA k p hcompat, iha k p hcompat, ihb k p hcompat]
| app f a ihf iha =>
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkApp,
multiOpenAtStaging, List.map]
  rw [ihf k p hcompat, iha k p hcompat]
| pair a b iha ihb =>
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkPair,
multiOpenAtStaging, List.map]
  rw [iha k p hcompat, ihb k p hcompat]
| fst p ih =>
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkFst,
multiOpenAtStaging, List.map]
  rw [ih k p hcompat]
| snd p ih =>
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd,
multiOpenAtStaging, List.map]
  rw [ih k p hcompat]
| refl a ih =>
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl,
multiOpenAtStaging, List.map]
  rw [ih k p hcompat]
| lam b ih =>
  have hcompat' := QuoteCompat.envCons hcompat.1 hcompat
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkLam,
multiOpenAtStaging, List.map]
  congr 1
  congr 1
  congr 1
  rw [ih (k + 1) (envCons v( k) p) hcompat']
  simpa using congrArg Pattern.lambda
    (closeFVar_multiOpenAt_envCons_staging 0 v( k) p (quoteRaw b)
      (freeVars_quoteRaw_staging b)
      (fun i => hcompat.2 i k (by omega)))
| pi A B ihA ihB =>
  have hcompat' := QuoteCompat.envCons hcompat.1 hcompat
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkPi,
multiOpenAtStaging, List.map]
  rw [ihA k p hcompat]
  congr 1
  congr 1
  congr 1
  rw [ihB (k + 1) (envCons v( k) p) hcompat']
  simpa using congrArg Pattern.lambda
    (closeFVar_multiOpenAt_envCons_staging 0 v( k) p (quoteRaw B)
      (freeVars_quoteRaw_staging B)
      (fun i => hcompat.2 i k (by omega)))
| sigma A B ihA ihB =>
  have hcompat' := QuoteCompat.envCons hcompat.1 hcompat
  simp only [quoteTmWith, quoteRaw, Mettapedia.Languages.MeTTa.Pure.Core.mkSigma,
multiOpenAtStaging, List.map]
  rw [ihA k p hcompat]
  congr 1
  congr 1

```



```

    congr 1
    rw [ihB (k + 1) (envCons v( k) p) hcompat']
    simpa using congrArg Pattern.lambda
      (closeFVar_multiOpenAt_envCons_staging 0 v( k) p (quoteRaw B)
        (freeVars_quoteRaw_staging B)
        (fun i => hcompat.2 i k (by omega)))

theorem quoteTmWith_depth_indep_staging
  v{ : Nat → String} {n : Nat} (k k' : Nat) p{ : QuoteEnv n} (t : PureTm n)
  (hcompat : QuoteCompat v k p) (hcompat' : QuoteCompat v k' p) :
  quoteTmWith v k p t = quoteTmWith v k' p t := by
  rw [quoteTmWith_eq_multiOpenAt_quoteRaw_staging v k p t hcompat,
    quoteTmWith_eq_multiOpenAt_quoteRaw_staging v k' p t hcompat']

theorem closeRange_quoteTmWith_renameWkN_buildEnv
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (m : Nat) (t : PureTm n) :
  closeRange v k m
    (quoteTmWith v (k + m) (buildEnv v k m p) (renameWkN m t))
  =
  quoteTmWith v (k + m) p t := by
  rw [quoteTmWith_renameWkN_buildEnv]
exact closeRange_quoteTmWith_id hcompat m (k + m) t

theorem closeRangeAt_quoteTmWith_renameWkN_buildEnv
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (d m : Nat) (t : PureTm n) :
  closeRangeAt d v k m
    (quoteTmWith v (k + m) (buildEnv v k m p) (renameWkN m t))
  =
  quoteTmWith v (k + m) p t := by
  rw [quoteTmWith_renameWkN_buildEnv]
exact closeRangeAt_quoteTmWith_id hcompat d m (k + m) t

theorem closeRangeAt_quoteTmWith_renameWkN_buildEnv_any
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (d m l : Nat) (t : PureTm n) :
  closeRangeAt d v k m
    (quoteTmWith v l (buildEnv v k m p) (renameWkN m t))
  =
  quoteTmWith v l p t := by
  rw [quoteTmWith_renameWkN_buildEnv_any]
exact closeRangeAt_quoteTmWith_id hcompat d m l t

theorem closeAmbient_quoteTmWith_renameWkN_buildEnv_any
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (d m l : Nat) (t : PureTm n) :
  closeAmbient d v k m
    (quoteTmWith v l (buildEnv v k m p) (renameWkN m t))
  =
  quoteTmWith v l p t := by
  rw [quoteTmWith_renameWkN_buildEnv_any]
exact closeAmbient_quoteTmWith_id hcompat d m l t

```

```

theorem closeFVar_quoteTmWith_renameWkN_buildEnv_head
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ) (m : Nat) (t : PureTm n) :
  closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) ρ) (renameWkN (m + 1) t))
  =
    quoteTmWith v (k + m + 1) (buildEnv v k m ρ) (renameWkN m t) := by
have hbuild : QuoteCompat v (k + m) (buildEnv v k m ρ) :=
  buildEnv_compat hcompat m
simp [buildEnv, renameWkN, Nat.add_assoc] using
  (closeFVar_quoteTmWith_rename_wk_envCons
    v( := v) (k := k + m) ρ( := buildEnv v k m ρ)
    (hcompat := hbuild) (t := renameWkN m t))

@[simp] theorem liftSubN_wkN_apply
  (m : Nat) σ( : Sub n k) (i : Fin n) :
  liftSubN m σ (wkN (n := n) m i) = renameWkN m σ( i) := by
induction m generalizing n k with
| zero =>
  simp [liftSubN, wkN, renameWkN, idRen]
| succ m ih =>
  calc
    liftSubN (m + 1) σ (wkN (n := n) (m + 1) i)
      = liftSub (liftSubN m σ) (wk (wkN (n := n) m i)) := by
        simp [liftSubN, wkN]
    _ = rename wk (liftSubN m σ (wkN (n := n) m i)) := by
      rfl
    _ = rename wk (renameWkN m σ( i)) := by
      rw [ih]
    _ = renameWkN (m + 1) σ( i) := by
      simp [renameWkN]

@[simp] theorem liftSubN_prefix_apply
  (m : Nat) σ( : Sub n k) (i : Fin m) :
  liftSubN m σ (prefixIdx n i) = .var (prefixIdx k i) := by
induction m generalizing n k with
| zero =>
  exact Fin.elim0 i
| succ m ih =>
  cases i using Fin.cases with
| zero =>
  simp [liftSubN, prefixIdx]
| succ j =>
  simp [liftSubN, prefixIdx, wk] using congrArg (rename wk) (ih (n := n) (k := k) σ j)

@[simp] theorem closeRange_bvar v( : Nat → String) (k m j : Nat) :
  closeRange v k m (.bvar j) = .bvar j := by
induction m with
| zero =>
  simp [closeRange]
| succ m ih =>
  simp [closeRange, closeFVar, ih]

@[simp] theorem closeRangeAt_bvar (d : Nat) v( : Nat → String) (k m j : Nat) :

```

```

    closeRangeAt d v k m (.bvar j) = .bvar j := by
induction m with
| zero =>
    simp [closeRangeAt]
| succ m ih =>
    simp [closeRangeAt, closeFVar, ih]

theorem closeRange_fvar_of_ne
  v( : Nat → String) (k m : Nat) (x : String)
  (hneq : ∀ r, r < m → x ≠ v (k + r)) :
  closeRange v k m (.fvar x) = .fvar x := by
induction m with
| zero =>
    simp [closeRange]
| succ m ih =>
    calc
      closeRange v k (m + 1) (.fvar x)
        = closeRange v k m (closeFVar 0 v( (k + m)) (.fvar x)) := by
          simp [closeRange]
    _ = closeRange v k m (.fvar x) := by
      simp [closeFVar, hneq m (by omega)]
    _ = .fvar x := by
      apply ih
      intro r hr
      exact hneq r (by omega)

theorem closeRangeAt_fvar_of_ne
  (d : Nat) v( : Nat → String) (k m : Nat) (x : String)
  (hneq : ∀ r, r < m → x ≠ v (k + r)) :
  closeRangeAt d v k m (.fvar x) = .fvar x := by
induction m with
| zero =>
    simp [closeRangeAt]
| succ m ih =>
    calc
      closeRangeAt d v k (m + 1) (.fvar x)
        = closeRangeAt d v k m (closeFVar d v( (k + m)) (.fvar x)) := by
          simp [closeRangeAt]
    _ = closeRangeAt d v k m (.fvar x) := by
      simp [closeFVar, hneq m (by omega)]
    _ = .fvar x := by
      apply ih
      intro r hr
      exact hneq r (by omega)

theorem closeAmbient_fvar_of_ne
  (d : Nat) v( : Nat → String) (k m : Nat) (x : String)
  (hneq : ∀ r, r < m → x ≠ v (k + r)) :
  closeAmbient d v k m (.fvar x) = .fvar x := by
induction m generalizing d with
| zero =>
    simp [closeAmbient]
| succ m ih =>
    calc

```

```

    closeAmbient d v k (m + 1) (.fvar x)
      = closeAmbient (d + 1) v k m (closeFVar d v (k + m)) (.fvar x) := by
        simp [closeAmbient]
  _ = closeAmbient (d + 1) v k m (.fvar x) := by
    simp [closeFVar, hneq m (by omega)]
  _ = .fvar x := by
    apply ih
    intro r hr
    exact hneq r (by omega)

theorem buildEnv_prefix_name
  v( : Nat → String) (k : Nat) p( : QuoteEnv n) :
  ∀ {m : Nat} (i : Fin m), ∃ r, r < m ∧ buildEnv v k m p (prefixIdx n i) = v (k + r) := by
intro m
induction m generalizing n k p with
| zero =>
  intro i
  exact Fin.elim0 i
| succ m ih =>
  intro i
  cases i using Fin.cases with
  | zero =>
    refine {m, by omega, ?}_
    simp [buildEnv, prefixIdx, envCons]
  | succ j =>
    rcases ih (n := n) (k := k) p( := p) j with {r, hr, }hEq
    refine {r, by omega, ?}_
    simp [buildEnv, prefixIdx, envCons] using hEq

theorem buildEnv_prefix_name_exact
  v( : Nat → String) (k : Nat) p( : QuoteEnv n) :
  ∀ {m : Nat} (i : Fin m),
    buildEnv v k m p (prefixIdx n i) = v (k + (m - 1 - i.1)) := by
intro m
induction m generalizing n k p with
| zero =>
  intro i
  exact Fin.elim0 i
| succ m ih =>
  intro i
  cases i using Fin.cases with
  | zero =>
    simp [buildEnv, prefixIdx, envCons]
  | succ j =>
    have hEq := ih (n := n) (k := k) p( := p) j
    calc
      buildEnv v k (m + 1) p (prefixIdx n (Fin.succ j))
      = buildEnv v k m p (prefixIdx n j) := by
        simp [buildEnv, prefixIdx, envCons]
      _ = v (k + (m - 1 - j.1)) := hEq
      _ = v (k + ((m + 1) - 1 - (Fin.succ j).1)) := by
        have hsucc : (Fin.succ j).1 = j.1 + 1 := rfl
        rw [hsucc]
        congr 1

```

omega

theorem closeRange_bound_name

```
v{ : Nat → String} (hij : Function.Injective v) (k : Nat) :  
  ∀ (m r : Nat), r < m → closeRange v k m (.fvar v( (k + r))) = .bvar 0 := by  
intro m  
induction m with  
| zero =>  
  intro r hr  
  omega  
| succ m ih =>  
  intro r hr  
  by_cases hrm : r = m  
  · subst hrm  
    simp [closeRange, closeFVar, closeRange_bvar]  
  · have hr' : r < m := by omega  
    have hneq : v (k + r) ≠ v (k + m) := by  
      intro hEq  
      have : k + r = k + m := hij hEq  
      omega  
    calc  
      closeRange v k (m + 1) (.fvar v( (k + r)))  
        = closeRange v k m (closeFVar 0 v( (k + m)) (.fvar v( (k + r)))) := by  
          simp [closeRange]  
      _ = closeRange v k m (.fvar v( (k + r))) := by  
          simp [closeFVar, hneq]  
      _ = .bvar 0 := ih r hr'
```

theorem closeRangeAt_bound_name

```
v{ : Nat → String} (hij : Function.Injective v) (d k : Nat) :  
  ∀ (m r : Nat), r < m → closeRangeAt d v k m (.fvar v( (k + r))) = .bvar d := by  
intro m  
induction m with  
| zero =>  
  intro r hr  
  omega  
| succ m ih =>  
  intro r hr  
  by_cases hrm : r = m  
  · subst hrm  
    simp [closeRangeAt, closeFVar, closeRangeAt_bvar]  
  · have hr' : r < m := by omega  
    have hneq : v (k + r) ≠ v (k + m) := by  
      intro hEq  
      have : k + r = k + m := hij hEq  
      omega  
    calc  
      closeRangeAt d v k (m + 1) (.fvar v( (k + r)))  
        = closeRangeAt d v k m (closeFVar d v( (k + m)) (.fvar v( (k + r)))) := by  
          simp [closeRangeAt]  
      _ = closeRangeAt d v k m (.fvar v( (k + r))) := by  
          simp [closeFVar, hneq]  
      _ = .bvar d := ih r hr'
```

```

theorem closeAmbient_bound_name
  v{ : Nat → String} (hij : Function.Injective v) (d k : Nat) :
  ∀ (m r : Nat), r < m →
    closeAmbient d v k m (.fvar v( (k + r))) = .bvar (d + (m - 1 - r)) := by
intro m
induction m generalizing d k with
| zero =>
  intro r hr
  omega
| succ m ih =>
  intro r hr
  by_cases hrm : r = m
  · subst r
    have hb :
      closeAmbient (d + 1) v k m (.bvar d) = .bvar d :=
      closeAmbient_bvar (d := d + 1) v( := v) (k := k) (m := m) (j := d)
    calc
      closeAmbient d v k (m + 1) (.fvar v( (k + m)))
      = closeAmbient (d + 1) v k m (.bvar d) := by
        simp [closeAmbient, closeFVar]
    _ = .bvar d := hb
    _ = .bvar (d + ((m + 1) - 1 - m)) := by
      congr 1
      omega
  · have hr' : r < m := by omega
    have hneq : v (k + r) ≠ v (k + m) := by
      intro hEq
      have : k + r = k + m := hij hEq
      omega
    have hrec :
      closeAmbient (d + 1) v k m (.fvar v( (k + r))) =
      .bvar ((d + 1) + (m - 1 - r)) :=
      ih (d := d + 1) (k := k) r hr'
    have hidx :
      (d + 1) + (m - 1 - r) = d + ((m + 1) - 1 - r) := by
      omega
    calc
      closeAmbient d v k (m + 1) (.fvar v( (k + r)))
      = closeAmbient (d + 1) v k m (.fvar v( (k + r))) := by
        simp [closeAmbient, closeFVar, hneq]
    _ = .bvar ((d + 1) + (m - 1 - r)) := hrec
    _ = .bvar (d + ((m + 1) - 1 - r)) := by
      simp [hidx]

def inst0BinderClosedTarget
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + m)) : Pattern :=
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeRange v (k + 1) m
    (quoteTmWith v (k + m + 1)
      (buildEnv v (k + 1) m (envCons v( k) p)) t))

def inst0BinderClosedLhs
  v( : Nat → String) {n : Nat}

```

```

    (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + m)) : Pattern :=
closeRange v k m
    (quoteTmWith v (k + m) (buildEnv v k m p)
      (subst (liftSubN m (subst0 a)) t))

def inst0BinderTargetEq
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + m)) : Prop :=
inst0BinderClosedLhs v m k p a t = inst0BinderClosedTarget v m k p a t

/-- Generalized closed-codomain family with `e` ambient binders beyond the
preserved prefix `m`. `e = 0` recovers the top-level term family, `e = 1`
recovers the current body family. -/
def inst0AmbientClosedLhs
  (e : Nat) v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + (m + e))) : Pattern :=
closeRangeAt e v k m
  (closeAmbient 0 v (k + m) e
    (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
      (subst (liftSubN (m + e) (subst0 a)) t))))

def inst0AmbientClosedTarget
  (e : Nat) v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + (m + e))) : Pattern :=
applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeRangeAt e v (k + 1) m
    (closeAmbient 0 v (k + m + 1) e
      (quoteTmWith v (k + m + e + 1)
        (buildEnv v (k + 1) (m + e) (envCons v( k) p)) t))))

def inst0AmbientTargetEq
  (e : Nat) v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + (m + e))) : Prop :=
inst0AmbientClosedLhs e v m k p a t = inst0AmbientClosedTarget e v m k p a t

/-- Variant of the ambient family that preserves the `m` prefix binders as distinct
de Bruijn indices rather than collapsing them all to depth `e`. -/
def inst0AmbientDistinctClosedLhs
  (e : Nat) v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + (m + e))) : Pattern :=
closeAmbient e v k m
  (closeAmbient 0 v (k + m) e
    (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
      (subst (liftSubN (m + e) (subst0 a)) t))))

def inst0AmbientDistinctClosedTarget
  (e : Nat) v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + (m + e))) : Pattern :=
applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeAmbient e v (k + 1) m
    (closeAmbient 0 v (k + m + 1) e
      (quoteTmWith v (k + m + e + 1)
        (buildEnv v (k + 1) (m + e) (envCons v( k) p)) t))))

```

```

def inst0AmbientDistinctTargetEq
  (e : Nat) v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + (m + e))) : Prop :=
  inst0AmbientDistinctClosedLhs e v m k p a t =
  inst0AmbientDistinctClosedTarget e v m k p a t

def inst0BinderClosedLhsDistinct
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + m)) : Pattern :=
  closeAmbient 0 v k m
  (quoteTmWith v (k + m) (buildEnv v k m p)
   (subst (liftSubN m (subst0 a)) t))

def inst0BinderClosedTargetDistinct
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + m)) : Pattern :=
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeAmbient 0 v (k + 1) m
   (quoteTmWith v (k + m + 1)
    (buildEnv v (k + 1) m (envCons v( k) p)) t))

def inst0BinderTargetEqDistinct
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (t : PureTm ((n + 1) + m)) : Prop :=
  inst0BinderClosedLhsDistinct v m k p a t =
  inst0BinderClosedTargetDistinct v m k p a t

theorem inst0AmbientDistinctClosedLhs_zero_eq
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + m)) :
  inst0AmbientDistinctClosedLhs 0 v m k p a t =
  inst0BinderClosedLhsDistinct v m k p a t := by
simp [inst0AmbientDistinctClosedLhs, inst0BinderClosedLhsDistinct]

theorem inst0AmbientDistinctClosedTarget_zero_eq
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + m)) :
  inst0AmbientDistinctClosedTarget 0 v m k p a t =
  inst0BinderClosedTargetDistinct v m k p a t := by
simp [inst0AmbientDistinctClosedTarget, inst0BinderClosedTargetDistinct]

theorem inst0AmbientDistinctTargetEq_zero_iff
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + m)) :
  inst0AmbientDistinctTargetEq 0 v m k p a t ↔
  inst0BinderTargetEqDistinct v m k p a t := by
simp [inst0AmbientDistinctTargetEq, inst0BinderTargetEqDistinct,
  inst0AmbientDistinctClosedLhs_zero_eq, inst0AmbientDistinctClosedTarget_zero_eq]

theorem inst0AmbientDistinctClosedLhs_eq_total
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + (m + e))) :
  inst0AmbientDistinctClosedLhs e v m k p a t =
  inst0BinderClosedLhsDistinct v (m + e) k p a t := by

```



```

unfold inst0AmbientDistinctClosedLhs inst0BinderClosedLhsDistinct
rw [closeAmbient_split]
simp [Nat.add_left_comm, Nat.add_comm]

theorem inst0AmbientDistinctClosedTarget_eq_total
  (v : Nat → String) (e m k : Nat) (p : QuoteEnv n) (a : PureTm n)
  (hcompat : QuoteCompat v k p)
  (t : PureTm ((n + 1) + (m + e))) :
  inst0AmbientDistinctClosedTarget e v m k p a t =
    inst0BinderClosedTargetDistinct v (m + e) k p a t := by
  unfold inst0AmbientDistinctClosedTarget inst0BinderClosedTargetDistinct
  have hq :
    quoteTmWith v (k + m) p a = quoteTmWith v (k + (m + e)) p a := by
  apply quoteTmWith_depth_indep_staging
  · exact quoteCompat_mono hcompat (j := k + m) (by omega)
  · exact quoteCompat_mono hcompat (j := k + (m + e)) (by omega)
  rw [hq, closeAmbient_split]
  simp [Nat.add_assoc, Nat.add_left_comm, Nat.add_comm]

theorem inst0AmbientDistinctTargetEq_total_iff
  (v : Nat → String) (e m k : Nat) (p : QuoteEnv n) (a : PureTm n)
  (hcompat : QuoteCompat v k p)
  (t : PureTm ((n + 1) + (m + e))) :
  inst0AmbientDistinctTargetEq e v m k p a t ↔
    inst0BinderTargetEqDistinct v (m + e) k p a t := by
  simp [inst0AmbientDistinctTargetEq, inst0BinderTargetEqDistinct,
    inst0AmbientDistinctClosedLhs_eq_total, inst0AmbientDistinctClosedTarget_eq_total,
    hcompat]

theorem inst0AmbientClosedLhs_zero_eq
  (v : Nat → String) (m k : Nat) (p : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + m)) :
  inst0AmbientClosedLhs 0 v m k p a t = inst0BinderClosedLhs v m k p a t := by
  simp [inst0AmbientClosedLhs, inst0BinderClosedLhs, closeRange_eq_closeRangeAt_zero]

theorem inst0AmbientClosedTarget_zero_eq
  (v : Nat → String) (m k : Nat) (p : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + m)) :
  inst0AmbientClosedTarget 0 v m k p a t = inst0BinderClosedTarget v m k p a t := by
  simp [inst0AmbientClosedTarget, inst0BinderClosedTarget, closeRange_eq_closeRangeAt_zero]

theorem inst0AmbientTargetEq_zero_iff
  (v : Nat → String) (m k : Nat) (p : QuoteEnv n) (a : PureTm n)
  (t : PureTm ((n + 1) + m)) :
  inst0AmbientTargetEq 0 v m k p a t ↔ inst0BinderTargetEq v m k p a t := by
  simp [inst0AmbientTargetEq, inst0BinderTargetEq, inst0AmbientClosedLhs_zero_eq,
    inst0AmbientClosedTarget_zero_eq]

theorem inst0BinderTargetEq_wkN_var
  (v : Nat → String) {k : Nat} (p : QuoteEnv n)
  (hcompat : QuoteCompat v k p) (m : Nat) (a : PureTm n) (i : Fin (n + 1)) :
  inst0BinderClosedLhs v m k p a (.var (wkN (n := n + 1) m i))
  =
  inst0BinderClosedTarget v m k p a (.var (wkN (n := n + 1) m i)) := by

```

```

have hL :
  inst0BinderClosedLhs v m k p a (.var (wkN (n := n + 1) m i))
  =
  quoteTmWith v (k + m) p (subst0 a i) := by
calc
  inst0BinderClosedLhs v m k p a (.var (wkN (n := n + 1) m i))
  =
  closeRange v k m
    (quoteTmWith v (k + m) (buildEnv v k m p)
      (liftSubN m (subst0 a) (wkN (n := n + 1) m i))) := by
    simp [inst0BinderClosedLhs]
  - =
  closeRange v k m
    (quoteTmWith v (k + m) (buildEnv v k m p)
      (renameWkN m (subst0 a i))) := by
    simp [liftSubN_wkN_apply]
  _ = quoteTmWith v (k + m) p (subst0 a i) := by
    exact closeRange_quoteTmWith_renameWkN_buildEnv hcompat m (subst0 a i)
cases i using Fin.cases with
| zero =>
  have hclose :
    closeRange v (k + 1) m (.fvar v( k)) = .fvar v( k) := by
    apply closeRange_fvar_of_ne
    intro r hr hEq
    have hk : k ≠ k + 1 + r := by omega
    exact hk (hcompat.1 hEq)
  calc
    inst0BinderClosedLhs v m k p a (.var (wkN (n := n + 1) m 0))
      = quoteTmWith v (k + m) p a := by
        simpa using hL
  - =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (closeRange v (k + 1) m
      (quoteTmWith v (k + m + 1)
        (buildEnv v (k + 1) m (envCons v( k) p))
        (.var (wkN (n := n + 1) m 0)))) := by
    simp [quoteTmWith, buildEnv_wkN_apply, envCons]
    rw [hclose]
    simp [applySubst, SubstEnv.find_extend_eq]
  _ = inst0BinderClosedTarget v m k p a (.var (wkN (n := n + 1) m 0)) := by
    rfl
| succ j =>
  have hclose :
    closeRange v (k + 1) m (.fvar p( j)) = .fvar p( j) := by
    apply closeRange_fvar_of_ne
    intro r hr
    exact hcompat.2 j (k + 1 + r) (by omega)
  have hne : p j ≠ v k := hcompat.2 j k (by omega)
  calc
    inst0BinderClosedLhs v m k p a (.var (wkN (n := n + 1) m j.succ))
      = quoteTmWith v (k + m) p (.var j) := by
        simpa using hL
  - =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))

```

```

      (closeRange v (k + 1) m
        (quoteTmWith v (k + m + 1)
          (buildEnv v (k + 1) m (envCons v( k) p))
          (.var (wkN (n := n + 1) m j.succ)))) := by
          simp [quoteTmWith, buildEnv_wkN_apply, envCons]
          rw [hclose]
          have hfind :
            (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a)).find p(
j)
            = none :=
            SubstEnv.find_extend_ne hne.symm
            rw [applySubst, hfind]
      _ = inst0BinderClosedTarget v m k p a (.var (wkN (n := n + 1) m j.succ)) := by
        rfl

```

theorem inst0BinderTargetEq_preserved_zero

```

v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
(_hcompat : QuoteCompat v k p) (m : Nat) (a : PureTm n) :
inst0BinderClosedLhs v (m + 1) k p a (.var 0)
=
inst0BinderClosedTarget v (m + 1) k p a (.var 0) := by
calc
inst0BinderClosedLhs v (m + 1) k p a (.var 0)
=
  closeRange v k m (.bvar 0) := by
    simp [inst0BinderClosedLhs, closeRange, buildEnv, quoteTmWith, liftSubN, subst, closeFVar
, envCons]
_ = .bvar 0 := by
  exact closeRange_bvar v k m 0
_ = inst0BinderClosedTarget v (m + 1) k p a (.var 0) := by
  simp [inst0BinderClosedTarget, buildEnv, quoteTmWith, closeFVar, envCons, closeRange_bvar
, applySubst]

```

theorem inst0BinderClosedLhs_prefix_var

```

v{ : Nat → String} (hij : Function.Injective v)
(m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
inst0BinderClosedLhs v m k p a (.var (prefixIdx (n + 1) i)) = .bvar 0 := by
rcases buildEnv_prefix_name v k p i with {r, hr, }hname
calc
inst0BinderClosedLhs v m k p a (.var (prefixIdx (n + 1) i))
=
  closeRange v k m (.fvar v( (k + r))) := by
    simp [inst0BinderClosedLhs, liftSubN_prefix_apply, quoteTmWith, hname]
_ = .bvar 0 := closeRange_bound_name hij k m r hr

```

theorem inst0BinderClosedTarget_prefix_var

```

v{ : Nat → String} (hij : Function.Injective v)
(m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
inst0BinderClosedTarget v m k p a (.var (prefixIdx (n + 1) i)) = .bvar 0 := by
rcases buildEnv_prefix_name v (k + 1) (envCons v( k) p) i with {r, hr, }hname
calc
inst0BinderClosedTarget v m k p a (.var (prefixIdx (n + 1) i))
=
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))

```

```

      (closeRange v (k + 1) m (.fvar v( (k + 1 + r)))) := by
        simp [inst0BinderClosedTarget, quoteTmWith, hname]
    _ =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
        (.bvar 0) := by
          rw [closeRange_bound_name hnj (k + 1) m r hr]
    _ = .bvar 0 := by
      simp [applySubst]

theorem inst0BinderTargetEq_prefix_var
  v{ : Nat → String} (hjn : Function.Injective v)
  (m k : Nat) p{ : QuoteEnv n} (a : PureTm n) (i : Fin m) :
  inst0BinderClosedLhs v m k p a (.var (prefixIdx (n + 1) i))
  =
  inst0BinderClosedTarget v m k p a (.var (prefixIdx (n + 1) i)) := by
  rw [inst0BinderClosedLhs_prefix_var hjn m k p a i,
    inst0BinderClosedTarget_prefix_var hjn m k p a i]

theorem fin_prefix_or_wkN
  (m : Nat) (i : Fin ((n + 1) + m)) :
  ∃( j : Fin m, i = prefixIdx (n + 1) j) v
  ∃( j : Fin (n + 1), i = wkN (n := n + 1) m j) := by
  by_cases h : i.1 < m
  · left
    refine ⟨{i.1, }h, ?⟩_
    apply Fin.ext
    simp [prefixIdx]
  · right
    have hm : m ≤ i.1 := by omega
    refine ⟨{i.1 - m, by }omega, ?⟩_
    apply Fin.ext
    simp [wkN_val, hm]

theorem inst0BinderTargetEq_var
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (m : Nat) (a : PureTm n) (i : Fin ((n + 1) + m)) :
  inst0BinderClosedLhs v m k p a (.var i)
  =
  inst0BinderClosedTarget v m k p a (.var i) := by
  rcases fin_prefix_or_wkN (n := n) m i with hprefix | hwk
  · rcases hprefix with {j, }rfl
    exact inst0BinderTargetEq_prefix_var hcompat.1 m k p a j
  · rcases hwk with {j, }rfl
    exact inst0BinderTargetEq_wkN_var hcompat m a j

theorem inst0BinderTargetEqDistinct_wkN_var
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (m : Nat) (a : PureTm n) (i : Fin (n + 1)) :
  inst0BinderClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) m i))
  =
  inst0BinderClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) m i)) := by
  have hL :
    inst0BinderClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) m i))
    =

```

```

quoteTmWith v (k + m) p (subst0 a i) := by
calc
inst0BinderClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) m i))
=
closeAmbient 0 v k m
  (quoteTmWith v (k + m) (buildEnv v k m p)
    (liftSubN m (subst0 a) (wkN (n := n + 1) m i))) := by
    simp [inst0BinderClosedLhsDistinct]
- =
closeAmbient 0 v k m
  (quoteTmWith v (k + m) (buildEnv v k m p)
    (renameWkN m (subst0 a i))) := by
    simp [liftSubN_wkN_apply]
_ = quoteTmWith v (k + m) p (subst0 a i) := by
    exact closeAmbient_quoteTmWith_renameWkN_buildEnv_any hcompat 0 m (k + m) (subst0 a i
)
cases i using Fin.cases with
| zero =>
  have hclose :
    closeAmbient 0 v (k + 1) m (.fvar v( k)) = .fvar v( k) := by
    apply closeAmbient_fvar_of_ne
    intro r hr hEq
    have hk : k ≠ k + 1 + r := by omega
    exact hk (hcompat.1 hEq)
  calc
    inst0BinderClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) m 0))
      = quoteTmWith v (k + m) p a := by
        simpa using hL
  - =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient 0 v (k + 1) m
        (quoteTmWith v (k + m + 1)
          (buildEnv v (k + 1) m (envCons v( k) p))
          (.var (wkN (n := n + 1) m 0)))) := by
        simp [quoteTmWith, buildEnv_wkN_apply, envCons]
        rw [hclose]
        simp [applySubst, SubstEnv.find_extend_eq]
  _ = inst0BinderClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) m 0)) := by
    rfl
| succ j =>
  have hclose :
    closeAmbient 0 v (k + 1) m (.fvar p( j)) = .fvar p( j) := by
    apply closeAmbient_fvar_of_ne
    intro r hr
    exact hcompat.2 j (k + 1 + r) (by omega)
  have hne : p j ≠ v k := hcompat.2 j k (by omega)
  calc
    inst0BinderClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) m j.succ))
      = .fvar p( j) := by
        simpa [quoteTmWith] using hL
  - =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient 0 v (k + 1) m
        (quoteTmWith v (k + m + 1)

```

```

      (buildEnv v (k + 1) m (envCons v( k) p))
      (.var (wkN (n := n + 1) m j.succ))) := by
        simp [quoteTmWith, buildEnv_wkN_apply, envCons]
        rw [hclose]
        rw [applySubst, SubstEnv.find_extend_ne hne.symm]
        simp [SubstEnv.find, SubstEnv.empty]
    _ = inst0BinderClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) m j.succ)) := by
      rfl

theorem inst0BinderClosedLhsDistinct_prefix_var
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderClosedLhsDistinct v m k p a (.var (prefixIdx (n + 1) i)) = .bvar i.1 := by
have hname :
  buildEnv v k m p (prefixIdx n i) = v (k + (m - 1 - i.1)) :=
  buildEnv_prefix_name_exact v k p i
have hr : m - 1 - i.1 < m := by omega
calc
  inst0BinderClosedLhsDistinct v m k p a (.var (prefixIdx (n + 1) i))
  =
  closeAmbient 0 v k m (.fvar v( (k + (m - 1 - i.1)))) := by
    simp [inst0BinderClosedLhsDistinct, liftSubN_prefix_apply, quoteTmWith, hname]
  _ = .bvar (0 + (m - 1 - (m - 1 - i.1))) := closeAmbient_bound_name hij 0 k m (m - 1 - i.1)
  hr
  _ = .bvar i.1 := by
    congr 1
    omega

theorem inst0BinderClosedTargetDistinct_prefix_var
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderClosedTargetDistinct v m k p a (.var (prefixIdx (n + 1) i)) = .bvar i.1 := by
have hname :
  buildEnv v (k + 1) m (envCons v( k) p) (prefixIdx (n + 1) i)
  = v ((k + 1) + (m - 1 - i.1)) :=
  buildEnv_prefix_name_exact v (k + 1) (envCons v( k) p) i
have hr : m - 1 - i.1 < m := by omega
calc
  inst0BinderClosedTargetDistinct v m k p a (.var (prefixIdx (n + 1) i))
  =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient 0 v (k + 1) m (.fvar v( ((k + 1) + (m - 1 - i.1))))) := by
    simp [inst0BinderClosedTargetDistinct, quoteTmWith, hname]
  _ =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (.bvar (0 + (m - 1 - (m - 1 - i.1)))) := by
    rw [closeAmbient_bound_name hij 0 (k + 1) m (m - 1 - i.1) hr]
  _ = .bvar i.1 := by
    have hidx : 0 + (m - 1 - (m - 1 - i.1)) = i.1 := by omega
    simp [applySubst, hidx]

theorem inst0BinderTargetEqDistinct_prefix_var
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :

```

```

    inst0BinderClosedLhsDistinct v m k p a (.var (prefixIdx (n + 1) i))
    =
    inst0BinderClosedTargetDistinct v m k p a (.var (prefixIdx (n + 1) i)) := by
rw [inst0BinderClosedLhsDistinct_prefix_var hnj m k p a i,
    inst0BinderClosedTargetDistinct_prefix_var hnj m k p a i]

theorem inst0BinderTargetEqDistinct_var
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ) (m : Nat) (a : PureTm n) (i : Fin ((n + 1) + m)) :
  inst0BinderClosedLhsDistinct v m k p a (.var i)
  =
  inst0BinderClosedTargetDistinct v m k p a (.var i) := by
rcases fin_prefix_or_wkN (n := n) m i with hprefix | hwk
· rcases hprefix with {j, } rfl
  exact inst0BinderTargetEqDistinct_prefix_var hcompat.1 m k p a j
· rcases hwk with {j, } rfl
  exact inst0BinderTargetEqDistinct_wkN_var hcompat m a j

@[simp] theorem applySubst_u0 (env : SubstEnv) :
  applySubst env Mettapedia.Languages.MeTTa.Pure.Core.u0 = Mettapedia.Languages.MeTTa.Pure.Core.
  u0 := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.u0, applySubst]

@[simp] theorem applySubst_u1 (env : SubstEnv) :
  applySubst env Mettapedia.Languages.MeTTa.Pure.Core.u1 = Mettapedia.Languages.MeTTa.Pure.Core.
  u1 := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.u1, applySubst]

@[simp] theorem applySubst_quoteConst (env : SubstEnv) (c : DeclName) :
  applySubst env (quoteConst c) = quoteConst c := by
simp [quoteConst, applySubst]

@[simp] theorem closeRange_u0 v( : Nat → String) (k m : Nat) :
  closeRange v k m Mettapedia.Languages.MeTTa.Pure.Core.u0 = Mettapedia.Languages.MeTTa.Pure.
  Core.u0 := by
induction m generalizing k with
| zero =>
  simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.u0]
| succ m ih =>
  simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.u0] using ih (k := k)

@[simp] theorem closeRange_u1 v( : Nat → String) (k m : Nat) :
  closeRange v k m Mettapedia.Languages.MeTTa.Pure.Core.u1 = Mettapedia.Languages.MeTTa.Pure.
  Core.u1 := by
induction m generalizing k with
| zero =>
  simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.u1]
| succ m ih =>
  simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.u1] using ih (k := k)

@[simp] theorem applySubst_mkId
  (env : SubstEnv) (A a b : Pattern) :
  applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkId A a b)
  =

```

```

    Mettapedia.Languages.MeTTa.Pure.Core.mkId
      (applySubst env A) (applySubst env a) (applySubst env b) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkId, applySubst]

@[simp] theorem applySubst_mkApp
  (env : SubstEnv) (f a : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkApp f a)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkApp (applySubst env f) (applySubst env a) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkApp, applySubst]

@[simp] theorem applySubst_mkPair
  (env : SubstEnv) (a b : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkPair a b)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkPair (applySubst env a) (applySubst env b) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPair, applySubst]

@[simp] theorem applySubst_mkFst
  (env : SubstEnv) (p : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkFst p)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkFst (applySubst env p) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkFst, applySubst]

@[simp] theorem applySubst_mkSnd
  (env : SubstEnv) (p : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkSnd p)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkSnd (applySubst env p) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd, applySubst]

@[simp] theorem applySubst_mkRefl
  (env : SubstEnv) (a : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkRefl a)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkRefl (applySubst env a) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl, applySubst]

@[simp] theorem applySubst_mkPi
  (env : SubstEnv) (A B : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkPi A B)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkPi (applySubst env A) (applySubst env B) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPi, applySubst]

@[simp] theorem applySubst_mkSigma
  (env : SubstEnv) (A B : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma A B)
      =
    Mettapedia.Languages.MeTTa.Pure.Core.mkSigma (applySubst env A) (applySubst env B) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSigma, applySubst]

@[simp] theorem applySubst_mgLam

```



```

    (env : SubstEnv) (body : Pattern) :
    applySubst env (Mettapedia.Languages.MeTTa.Pure.Core.mkLam body)
    =
    Mettapedia.Languages.MeTTa.Pure.Core.mkLam (applySubst env body) := by
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkLam, applySubst]

@[simp] theorem closeRangeAt_u0
  (d : Nat) v( : Nat → String) (k m : Nat) :
  closeRangeAt d v k m Mettapedia.Languages.MeTTa.Pure.Core.u0 = Mettapedia.Languages.MeTTa.
  Pure.Core.u0 := by
  induction m generalizing k with
  | zero =>
    simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.u0]
  | succ m ih =>
    simp [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.u0] using ih (k := k)

@[simp] theorem closeRangeAt_u1
  (d : Nat) v( : Nat → String) (k m : Nat) :
  closeRangeAt d v k m Mettapedia.Languages.MeTTa.Pure.Core.u1 = Mettapedia.Languages.MeTTa.
  Pure.Core.u1 := by
  induction m generalizing k with
  | zero =>
    simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.u1]
  | succ m ih =>
    simp [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.u1] using ih (k := k)

@[simp] theorem closeAmbient_u0
  (d : Nat) v( : Nat → String) (k m : Nat) :
  closeAmbient d v k m Mettapedia.Languages.MeTTa.Pure.Core.u0 = Mettapedia.Languages.MeTTa.
  Pure.Core.u0 := by
  induction m generalizing d k with
  | zero =>
    simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.u0]
  | succ m ih =>
    simp [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.u0] using ih (d := d +
    1) (k := k)

@[simp] theorem closeAmbient_u1
  (d : Nat) v( : Nat → String) (k m : Nat) :
  closeAmbient d v k m Mettapedia.Languages.MeTTa.Pure.Core.u1 = Mettapedia.Languages.MeTTa.
  Pure.Core.u1 := by
  induction m generalizing d k with
  | zero =>
    simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.u1]
  | succ m ih =>
    simp [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.u1] using ih (d := d +
    1) (k := k)

@[simp] theorem closeAmbient_quoteConst
  (d : Nat) v( : Nat → String) (k m : Nat) (c : DeclName) :
  closeAmbient d v k m (quoteConst c) = quoteConst c := by
  induction m generalizing d k with
  | zero =>
    simp [closeAmbient, quoteConst]

```

```

| succ m ih =>
  simpa [closeAmbient, closeFVar, quoteConst] using ih (d := d + 1) (k := k)

@[simp] theorem closeAmbient_mkId
  (d : Nat) v( : Nat → String) (k m : Nat) (A a b : Pattern) :
  closeAmbient d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkId A a b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkId
    (closeAmbient d v k m A) (closeAmbient d v k m a) (closeAmbient d v k m b) := by
induction m generalizing d k A a b with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
| succ m ih =>
  simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    ih (d := d + 1) (k := k)
    (A := closeFVar d v( (k + m)) A)
    (a := closeFVar d v( (k + m)) a)
    (b := closeFVar d v( (k + m)) b)

@[simp] theorem closeAmbient_mkApp
  (d : Nat) v( : Nat → String) (k m : Nat) (f a : Pattern) :
  closeAmbient d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkApp f a)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp (closeAmbient d v k m f) (closeAmbient d v k m a)
  := by
induction m generalizing d k f a with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkApp]
| succ m ih =>
  simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkApp] using
    ih (d := d + 1) (k := k)
    (f := closeFVar d v( (k + m)) f)
    (a := closeFVar d v( (k + m)) a)

@[simp] theorem closeAmbient_mkPair
  (d : Nat) v( : Nat → String) (k m : Nat) (a b : Pattern) :
  closeAmbient d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkPair a b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair (closeAmbient d v k m a) (closeAmbient d v k m b)
  := by
induction m generalizing d k a b with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
| succ m ih =>
  simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
    ih (d := d + 1) (k := k)
    (a := closeFVar d v( (k + m)) a)
    (b := closeFVar d v( (k + m)) b)

@[simp] theorem closeAmbient_mkFst
  (d : Nat) v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeAmbient d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkFst p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst (closeAmbient d v k m p) := by

```

```

induction m generalizing d k p with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
| succ m ih =>
  simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
    ih (d := d + 1) (k := k) (p := closeFVar d v( (k + m)) p)

@[simp] theorem closeAmbient_mkSnd
  (d : Nat) v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeAmbient d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkSnd p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSnd (closeAmbient d v k m p) := by
induction m generalizing d k p with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
| succ m ih =>
  simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
    ih (d := d + 1) (k := k) (p := closeFVar d v( (k + m)) p)

@[simp] theorem closeAmbient_mkRefl
  (d : Nat) v( : Nat → String) (k m : Nat) (a : Pattern) :
  closeAmbient d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkRefl a)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl (closeAmbient d v k m a) := by
induction m generalizing d k a with
| zero =>
  simp [closeAmbient, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl]
| succ m ih =>
  simpa [closeAmbient, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
    ih (d := d + 1) (k := k) (a := closeFVar d v( (k + m)) a)

@[simp] theorem closeRange_mkId
  v( : Nat → String) (k m : Nat) (A a b : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkId A a b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkId
    (closeRange v k m A) (closeRange v k m a) (closeRange v k m b) := by
induction m generalizing k A a b with
| zero =>
  simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
| succ m ih =>
  simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    ih (k := k)
      (A := closeFVar 0 v( (k + m)) A)
      (a := closeFVar 0 v( (k + m)) a)
      (b := closeFVar 0 v( (k + m)) b)

@[simp] theorem closeRange_mkApp
  v( : Nat → String) (k m : Nat) (f a : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkApp f a)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp (closeRange v k m f) (closeRange v k m a) := by
induction m generalizing k f a with
| zero =>

```

```

    simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.mkApp]
| succ m ih =>
    simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkApp] using
      ih (k := k)
      (f := closeFVar 0 v( (k + m)) f)
      (a := closeFVar 0 v( (k + m)) a)

@[simp] theorem closeRange_mkPair
  v( : Nat → String) (k m : Nat) (a b : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkPair a b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair (closeRange v k m a) (closeRange v k m b) := by
induction m generalizing k a b with
| zero =>
    simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
| succ m ih =>
    simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
      ih (k := k)
      (a := closeFVar 0 v( (k + m)) a)
      (b := closeFVar 0 v( (k + m)) b)

@[simp] theorem closeRange_mkFst
  v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkFst p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst (closeRange v k m p) := by
induction m generalizing k p with
| zero =>
    simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
| succ m ih =>
    simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
      ih (k := k) (p := closeFVar 0 v( (k + m)) p)

@[simp] theorem closeRange_mkSnd
  v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkSnd p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSnd (closeRange v k m p) := by
induction m generalizing k p with
| zero =>
    simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
| succ m ih =>
    simpa [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
      ih (k := k) (p := closeFVar 0 v( (k + m)) p)

@[simp] theorem closeRange_mkRefl
  v( : Nat → String) (k m : Nat) (a : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkRefl a)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl (closeRange v k m a) := by
induction m generalizing k a with
| zero =>
    simp [closeRange, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl]
| succ m ih =>

```

```

    simp [closeRange, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
      ih (k := k) (a := closeFVar 0 v( (k + m)) a)

```

theorem closeRangeAt_mkLam

```

  (d : Nat) v( : Nat → String) (k m : Nat) (body : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkLam body)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (closeRangeAt (d + 1) v k m body) := by
induction m generalizing k body with
| zero =>
  simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkLam]
| succ m ih =>
  calc
    closeRangeAt d v k (m + 1) (Mettapedia.Languages.MeTTa.Pure.Core.mkLam body)
    =
    closeRangeAt d v k m
      (closeFVar d v( (k + m)) (Mettapedia.Languages.MeTTa.Pure.Core.mkLam body)) := by
      simp [closeRangeAt]
  - =
    closeRangeAt d v k m
      (Mettapedia.Languages.MeTTa.Pure.Core.mkLam
        (closeFVar (d + 1) v( (k + m)) body)) := by
      simp [Mettapedia.Languages.MeTTa.Pure.Core.mkLam, closeFVar]
  - =
    Mettapedia.Languages.MeTTa.Pure.Core.mkLam
      (closeRangeAt (d + 1) v k m
        (closeFVar (d + 1) v( (k + m)) body)) := by
      simp using ih (k := k) (body := closeFVar (d + 1) v( (k + m)) body)
  - =
    Mettapedia.Languages.MeTTa.Pure.Core.mkLam
      (closeRangeAt (d + 1) v k (m + 1) body) := by
      simp [closeRangeAt]

```

theorem closeRangeAt_mkPi

```

  (d : Nat) v( : Nat → String) (k m : Nat) (A B : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkPi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (closeRangeAt d v k m A)
    (closeRangeAt (d + 1) v k m B) := by
induction m generalizing k A B with
| zero =>
  simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkPi]
| succ m ih =>
  calc
    closeRangeAt d v k (m + 1) (Mettapedia.Languages.MeTTa.Pure.Core.mkPi A B)
    =
    closeRangeAt d v k m
      (closeFVar d v( (k + m)) (Mettapedia.Languages.MeTTa.Pure.Core.mkPi A B)) := by
      simp [closeRangeAt]
  - =
    closeRangeAt d v k m
      (Mettapedia.Languages.MeTTa.Pure.Core.mkPi

```

```

      (closeFVar d v( (k + m)) A)
      (closeFVar (d + 1) v( (k + m)) B)) := by
        simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPi, closeFVar]
    - =
      Mettapedia.Languages.MeTTa.Pure.Core.mkPi
      (closeRangeAt d v k m (closeFVar d v( (k + m)) A))
      (closeRangeAt (d + 1) v k m (closeFVar (d + 1) v( (k + m)) B)) := by
        simpa using ih (k := k)
        (A := closeFVar d v( (k + m)) A)
        (B := closeFVar (d + 1) v( (k + m)) B)
    - =
      Mettapedia.Languages.MeTTa.Pure.Core.mkPi
      (closeRangeAt d v k (m + 1) A)
      (closeRangeAt (d + 1) v k (m + 1) B) := by
        simp [closeRangeAt]

theorem closeRangeAt_mkSigma
  (d : Nat) v( : Nat → String) (k m : Nat) (A B : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (closeRangeAt d v k m A)
    (closeRangeAt (d + 1) v k m B) := by
  induction m generalizing k A B with
  | zero =>
    simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkSigma]
  | succ m ih =>
    calc
      closeRangeAt d v k (m + 1) (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma A B)
      =
      closeRangeAt d v k m
        (closeFVar d v( (k + m)) (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma A B)) := by
        simp [closeRangeAt]
    - =
      closeRangeAt d v k m
        (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
          (closeFVar d v( (k + m)) A)
          (closeFVar (d + 1) v( (k + m)) B)) := by
        simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSigma, closeFVar]
    - =
      Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
        (closeRangeAt d v k m (closeFVar d v( (k + m)) A))
        (closeRangeAt (d + 1) v k m (closeFVar (d + 1) v( (k + m)) B)) := by
        simpa using ih (k := k)
        (A := closeFVar d v( (k + m)) A)
        (B := closeFVar (d + 1) v( (k + m)) B)
    - =
      Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
        (closeRangeAt d v k (m + 1) A)
        (closeRangeAt (d + 1) v k (m + 1) B) := by
        simp [closeRangeAt]

@[simp] theorem closeRangeAt_mkId
  (d : Nat) v( : Nat → String) (k m : Nat) (A a b : Pattern) :

```

```

closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkId A a b)
=
Mettapedia.Languages.MeTTa.Pure.Core.mkId
  (closeRangeAt d v k m A) (closeRangeAt d v k m a) (closeRangeAt d v k m b) := by
induction m generalizing k A a b with
| zero =>
  simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
| succ m ih =>
  simp [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    ih (k := k)
    (A := closeFVar d v (k + m)) A)
    (a := closeFVar d v (k + m)) a)
    (b := closeFVar d v (k + m)) b)

@[simp] theorem closeRangeAt_mkApp
  (d : Nat) v( : Nat → String) (k m : Nat) (f a : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkApp f a)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp (closeRangeAt d v k m f) (closeRangeAt d v k m a)
  := by
induction m generalizing k f a with
| zero =>
  simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkApp]
| succ m ih =>
  simp [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkApp] using
    ih (k := k)
    (f := closeFVar d v (k + m)) f)
    (a := closeFVar d v (k + m)) a)

@[simp] theorem closeRangeAt_mkPair
  (d : Nat) v( : Nat → String) (k m : Nat) (a b : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkPair a b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair (closeRangeAt d v k m a) (closeRangeAt d v k m b)
  := by
induction m generalizing k a b with
| zero =>
  simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
| succ m ih =>
  simp [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
    ih (k := k)
    (a := closeFVar d v (k + m)) a)
    (b := closeFVar d v (k + m)) b)

@[simp] theorem closeRangeAt_mkFst
  (d : Nat) v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkFst p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst (closeRangeAt d v k m p) := by
induction m generalizing k p with
| zero =>
  simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
| succ m ih =>
  simp [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using

```

```

      ih (k := k) (p := closeFVar d v (k + m)) p)

@[simp] theorem closeRangeAt_mkSnd
  (d : Nat) v( : Nat → String) (k m : Nat) (p : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkSnd p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSnd (closeRangeAt d v k m p) := by
  induction m generalizing k p with
  | zero =>
    simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
  | succ m ih =>
    simpa [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
      ih (k := k) (p := closeFVar d v (k + m)) p)

@[simp] theorem closeRangeAt_mkRefl
  (d : Nat) v( : Nat → String) (k m : Nat) (a : Pattern) :
  closeRangeAt d v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkRefl a)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl (closeRangeAt d v k m a) := by
  induction m generalizing k a with
  | zero =>
    simp [closeRangeAt, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl]
  | succ m ih =>
    simpa [closeRangeAt, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
      ih (k := k) (a := closeFVar d v (k + m)) a)

theorem closeRange_mkLam_shifted
  v( : Nat → String) (k m : Nat) (body : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkLam body)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (closeRangeAt 1 v k m body) := by
  rw [closeRange_eq_closeRangeAt_zero]
  simpa using closeRangeAt_mkLam 0 v k m body

theorem closeRange_mkPi_shifted
  v( : Nat → String) (k m : Nat) (A B : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkPi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (closeRangeAt 0 v k m A)
    (closeRangeAt 1 v k m B) := by
  rw [closeRange_eq_closeRangeAt_zero]
  simpa using closeRangeAt_mkPi 0 v k m A B

theorem closeRange_mkSigma_shifted
  v( : Nat → String) (k m : Nat) (A B : Pattern) :
  closeRange v k m (Mettapedia.Languages.MeTTa.Pure.Core.mkSigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (closeRangeAt 0 v k m A)
    (closeRangeAt 1 v k m B) := by
  rw [closeRange_eq_closeRangeAt_zero]
  simpa using closeRangeAt_mkSigma 0 v k m A B

```



```

theorem inst0BinderClosedLhs_lam_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedLhs v m k p a (.lam b)
  =
  Mettapedia.Languages.Metta.Pure.Core.mkLam
    (closeRangeAt 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
          (subst (liftSubN (m + 1) (subst0 a)) b)))) := by
simp [inst0BinderClosedLhs, quoteTmWith, subst, closeRange_mkLam_shifted, buildEnv, liftSubN,
  Nat.add_assoc, Nat.add_comm]

theorem inst0BinderClosedTarget_lam_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedTarget v m k p a (.lam b)
  =
  Mettapedia.Languages.Metta.Pure.Core.mkLam
    (applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (closeRangeAt 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2)
            (buildEnv v (k + 1) (m + 1) (envCons v( k) p)) b)))) := by
simp [inst0BinderClosedTarget, quoteTmWith, closeRange_mkLam_shifted, applySubst_mkLam,
  buildEnv,
  Nat.add_assoc, Nat.add_comm, Nat.add_left_comm]

def inst0BinderBodyClosedLhs
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (b : PureTm (((n + 1) + m) + 1)) : Pattern :=
  closeRangeAt 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
        (subst (liftSubN (m + 1) (subst0 a)) b)))

def inst0BinderBodyClosedTarget
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (b : PureTm (((n + 1) + m) + 1)) : Pattern :=
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (buildEnv v (k + 1) (m + 1) (envCons v( k) p)) b)))

def inst0BinderBodyTargetEq
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (b : PureTm (((n + 1) + m) + 1)) : Prop :=
  inst0BinderBodyClosedLhs v m k p a b = inst0BinderBodyClosedTarget v m k p a b

theorem inst0AmbientClosedLhs_one_eq
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :

```

```

    inst0AmbientClosedLhs 1 v m k p a b = inst0BinderBodyClosedLhs v m k p a b := by
simp [inst0AmbientClosedLhs, inst0BinderBodyClosedLhs]

theorem inst0AmbientClosedTarget_one_eq
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
  inst0AmbientClosedTarget 1 v m k p a b = inst0BinderBodyClosedTarget v m k p a b := by
simp [inst0AmbientClosedTarget, inst0BinderBodyClosedTarget]

theorem inst0AmbientTargetEq_one_iff
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
  inst0AmbientTargetEq 1 v m k p a b ↔ inst0BinderBodyTargetEq v m k p a b := by
simp [inst0AmbientTargetEq, inst0BinderBodyTargetEq, inst0AmbientClosedLhs_one_eq,
  inst0AmbientClosedTarget_one_eq]

def inst0BinderBodyClosedLhsDistinct
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (b : PureTm (((n + 1) + m) + 1)) : Pattern :=
  closeAmbient 1 v k m
  (closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (subst (liftSubN (m + 1) (subst0 a)) b)))

def inst0BinderBodyClosedTargetDistinct
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (b : PureTm (((n + 1) + m) + 1)) : Pattern :=
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2)
        (buildEnv v (k + 1) (m + 1) (envCons v( k) p)) b)))

def inst0BinderBodyTargetEqDistinct
  v( : Nat → String) {n : Nat}
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (b : PureTm (((n + 1) + m) + 1)) : Prop :=
  inst0BinderBodyClosedLhsDistinct v m k p a b =
  inst0BinderBodyClosedTargetDistinct v m k p a b

theorem inst0AmbientDistinctClosedLhs_one_eq
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
  inst0AmbientDistinctClosedLhs 1 v m k p a b =
  inst0BinderBodyClosedLhsDistinct v m k p a b := by
simp [inst0AmbientDistinctClosedLhs, inst0BinderBodyClosedLhsDistinct]

theorem inst0AmbientDistinctClosedTarget_one_eq
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
  inst0AmbientDistinctClosedTarget 1 v m k p a b =
  inst0BinderBodyClosedTargetDistinct v m k p a b := by
simp [inst0AmbientDistinctClosedTarget, inst0BinderBodyClosedTargetDistinct]

theorem inst0AmbientDistinctTargetEq_one_iff

```

```

v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
(b : PureTm ((n + 1) + m) + 1)) :
inst0AmbientDistinctTargetEq 1 v m k p a b ↔
  inst0BinderBodyTargetEqDistinct v m k p a b := by
simp [inst0AmbientDistinctTargetEq, inst0BinderBodyTargetEqDistinct,
  inst0AmbientDistinctClosedLhs_one_eq, inst0AmbientDistinctClosedTarget_one_eq]

theorem inst0BinderBodyTargetEq_zero
v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderBodyTargetEq v m k p a (.var 0) := by
unfold inst0BinderBodyTargetEq inst0BinderBodyClosedLhs inst0BinderBodyClosedTarget
have hhead :
  closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (subst (liftSubN (m + 1) (subst0 a)) (.var 0)))
  = .bvar 0 := by
simp [quoteTmWith, buildEnv, subst, liftSubN, envCons, closeFVar, Nat.add_assoc, Nat.add_comm]
]
calc
closeRangeAt 1 v k m
  (closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (subst (liftSubN (m + 1) (subst0 a)) (.var 0))))
  = closeRangeAt 1 v k m (.bvar 0) := by
    rw [hhead]
_ = .bvar 0 := by
  simp [closeRangeAt_bvar]
_ = applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2)
        (buildEnv v (k + 1) (m + 1) (envCons v( k) p)) (.var 0)))) := by
  simp [quoteTmWith, buildEnv, envCons, closeFVar, closeRangeAt_bvar,
    Nat.add_assoc, Nat.add_comm, applySubst]

theorem inst0BinderBodyTargetEqDistinct_zero
v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderBodyTargetEqDistinct v m k p a (.var 0) := by
unfold inst0BinderBodyTargetEqDistinct inst0BinderBodyClosedLhsDistinct
  inst0BinderBodyClosedTargetDistinct
have hhead :
  closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (subst (liftSubN (m + 1) (subst0 a)) (.var 0)))
  = .bvar 0 := by
simp [quoteTmWith, buildEnv, subst, liftSubN, envCons, closeFVar, Nat.add_assoc, Nat.add_comm]
]
calc
closeAmbient 1 v k m
  (closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (subst (liftSubN (m + 1) (subst0 a)) (.var 0))))
  = closeAmbient 1 v k m (.bvar 0) := by
    rw [hhead]

```

```

_ = .bvar 0 := by
  simp [closeAmbient_bvar]
_ = applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2)
        (buildEnv v (k + 1) (m + 1) (envCons v( k) p)) (.var 0)))) := by
  simp [quoteTmWith, buildEnv, envCons, closeFVar, closeAmbient_bvar,
    Nat.add_assoc, Nat.add_comm, applySubst]

theorem inst0BinderBodyClosedLhs_prefix_succ
  v{ : Nat → String} (hinj : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderBodyClosedLhs v m k p a (.var (Fin.succ (prefixIdx (n + 1) i))) = .bvar 1 := by
rcases buildEnv_prefix_name v k p i with (r, hr, )hname
have hneq : v (k + r) ≠ v (k + m) := by
  intro hEq
  have : k + r = k + m := hinj hEq
  omega
calc
inst0BinderBodyClosedLhs v m k p a (.var (Fin.succ (prefixIdx (n + 1) i)))
=
  closeRangeAt 1 v k m
    (closeFVar 0 v( (k + m)) (.fvar (buildEnv v k m p (prefixIdx n i)))) := by
    simp [inst0BinderBodyClosedLhs, liftSubN_prefix_apply, quoteTmWith, buildEnv, envCons,
      rename, wk, Nat.add_assoc, Nat.add_comm]
_ = closeRangeAt 1 v k m (.fvar (buildEnv v k m p (prefixIdx n i))) := by
  simp [closeFVar, hneq, hname]
_ = closeRangeAt 1 v k m (.fvar v( (k + r))) := by
  simp [hname]
_ = .bvar 1 := closeRangeAt_bound_name hinj 1 k m r hr

theorem inst0BinderBodyClosedTarget_prefix_succ
  v{ : Nat → String} (hinj : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderBodyClosedTarget v m k p a (.var (Fin.succ (prefixIdx (n + 1) i))) = .bvar 1 := by
rcases buildEnv_prefix_name v (k + 1) (envCons v( k) p) i with (r, hr, )hname
have hneq : v (k + 1 + r) ≠ v (k + m + 1) := by
  intro hEq
  have : k + 1 + r = k + m + 1 := hinj hEq
  omega
calc
inst0BinderBodyClosedTarget v m k p a (.var (Fin.succ (prefixIdx (n + 1) i)))
=
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (closeRangeAt 1 v (k + 1) m (.fvar v( (k + 1 + r)))) := by
    simp [inst0BinderBodyClosedTarget, quoteTmWith, buildEnv, hname, closeFVar, hneq,
      envCons]
_ =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a)) (.bvar 1) :=
  by
    rw [closeRangeAt_bound_name hinj 1 (k + 1) m r hr]
_ = .bvar 1 := by
  simp [applySubst]

```

```

theorem inst0BinderBodyTargetEq_prefix_succ
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderBodyTargetEq v m k p a (.var (Fin.succ (prefixIdx (n + 1) i))) := by
  unfold inst0BinderBodyTargetEq
  rw [inst0BinderBodyClosedLhs_prefix_succ hij m k p a i,
      inst0BinderBodyClosedTarget_prefix_succ hij m k p a i]

theorem inst0BinderBodyClosedLhsDistinct_prefix_succ
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderBodyClosedLhsDistinct v m k p a (.var (Fin.succ (prefixIdx (n + 1) i)))
  = .bvar (i.1 + 1) := by
have hname :
  buildEnv v k m p (prefixIdx n i) = v (k + (m - 1 - i.1)) :=
  buildEnv_prefix_name_exact v k p i
have hneq : v (k + (m - 1 - i.1)) ≠ v (k + m) := by
  intro hEq
  have : k + (m - 1 - i.1) = k + m := hij hEq
  omega
have hr : m - 1 - i.1 < m := by
  omega
calc
  inst0BinderBodyClosedLhsDistinct v m k p a (.var (Fin.succ (prefixIdx (n + 1) i)))
  =
  closeAmbient 1 v k m
  (closeFVar 0 v( (k + m)) (.fvar (buildEnv v k m p (prefixIdx n i)))) := by
  simp [inst0BinderBodyClosedLhsDistinct, liftSubN_prefix_apply, quoteTmWith, buildEnv,
  envCons,
  rename, wk, Nat.add_assoc, Nat.add_comm]
  _ = closeAmbient 1 v k m (.fvar (buildEnv v k m p (prefixIdx n i))) := by
  simp [closeFVar, hneq, hname]
  _ = closeAmbient 1 v k m (.fvar v( (k + (m - 1 - i.1)))) := by
  simp [hname]
  _ = .bvar (1 + (m - 1 - (m - 1 - i.1))) := closeAmbient_bound_name hij 1 k m (m - 1 - i.1)
  hr
  _ = .bvar (i.1 + 1) := by
  have hidx : 1 + (m - 1 - (m - 1 - i.1)) = i.1 + 1 := by
  omega
  simp [hidx]

theorem inst0BinderBodyClosedTargetDistinct_prefix_succ
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderBodyClosedTargetDistinct v m k p a (.var (Fin.succ (prefixIdx (n + 1) i)))
  = .bvar (i.1 + 1) := by
have hname :
  buildEnv v (k + 1) m (envCons v( k) p) (prefixIdx (n + 1) i)
  = v ((k + 1) + (m - 1 - i.1)) :=
  buildEnv_prefix_name_exact v (k + 1) (envCons v( k) p) i
have hneq : v ((k + 1) + (m - 1 - i.1)) ≠ v (k + m + 1) := by
  intro hEq
  have : (k + 1) + (m - 1 - i.1) = k + m + 1 := hij hEq

```

```

    omega
  have hr : m - 1 - i.1 < m := by
    omega
  calc
    inst0BinderBodyClosedTargetDistinct v m k p a (.var (Fin.succ (prefixIdx (n + 1) i)))
    =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient 1 v (k + 1) m (.fvar v( ((k + 1) + (m - 1 - i.1))))) := by
      simp [inst0BinderBodyClosedTargetDistinct, quoteTmWith, buildEnv, hname, closeFVar,
hneq,
envCons]
    - =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (.bvar (1 + (m - 1 - (m - 1 - i.1)))) := by
      rw [closeAmbient_bound_name hnj 1 (k + 1) m (m - 1 - i.1) hr]
    - = .bvar (i.1 + 1) := by
      have hidx : 1 + (m - 1 - (m - 1 - i.1)) = i.1 + 1 := by
        omega
      simp [applySubst, hidx]

theorem inst0BinderBodyTargetEqDistinct_prefix_succ
  v{ : Nat → String} (hij : Function.Injective v)
  (m k : Nat) p( : QuoteEnv n) (a : PureTm n) (i : Fin m) :
  inst0BinderBodyTargetEqDistinct v m k p a (.var (Fin.succ (prefixIdx (n + 1) i))) := by
  unfold inst0BinderBodyTargetEqDistinct
  rw [inst0BinderBodyClosedLhsDistinct_prefix_succ hij m k p a i,
inst0BinderBodyClosedTargetDistinct_prefix_succ hij m k p a i]

theorem inst0BinderBodyTargetEqDistinct_wkN_succ
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) (j : Fin n) :
  inst0BinderBodyTargetEqDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ)) := by
  have hrename :
    liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) j.succ)
    = renameWkN (m + 1) (.var j) := by
    simp using
      (liftSubN_wkN_apply (m := m + 1) σ( := subst0 a) (i := j.succ))
  have hL :
    inst0BinderBodyClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ))
    = .fvar p( j) := by
  calc
    inst0BinderBodyClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ))
    =
    closeAmbient 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
          (liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) j.succ)))) := by
      simp [inst0BinderBodyClosedLhsDistinct]
    - =
    closeAmbient 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
          (renameWkN (m + 1) (.var j))))) := by

```

```

      rw [hrename]
    - =
      closeAmbient 1 v k m
      (quoteTmWith v (k + m + 1) (buildEnv v k m p) (renameWkN m (.var j))) := by
        rw [closeFVar_quoteTmWith_renameWkN_buildEnv_head hcompat]
    - = .fvar p( j) := by
      rw [closeAmbient_quoteTmWith_renameWkN_buildEnv_any hcompat]
      simp [quoteTmWith]
have hclose :
  closeAmbient 1 v (k + 1) m (.fvar p( j)) = .fvar p( j) := by
  apply closeAmbient_fvar_of_ne
  intro r hr
  exact hcompat.2 j (k + 1 + r) (by omega)
have hne : p j ≠ v k := hcompat.2 j k (by omega)
have hR :
  inst0BinderBodyClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ))
  = .fvar p( j) := by
  have hlookup :
    buildEnv v (k + 1) (m + 1) (envCons v( k) p)
    (wkN (n := n + 1) (m + 1) j.succ) = p j := by
  simp [envCons] using
    (buildEnv_wkN_apply v (k + 1) (m + 1) (envCons v( k) p) j.succ)
  have hlookup' :
    envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))
    (wk (wkN (n := n + 1) m j.succ)) = p j := by
  simp [buildEnv, wkN, envCons, Nat.add_assoc] using hlookup
  have hvar :
    quoteTmWith v (k + m + 2)
    (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
    (.var (wkN (n := n + 1) (m + 1) j.succ))
    = .fvar p( j) := by
    simp [quoteTmWith, hlookup']
  have hfutue : p j ≠ v (k + m + 1) := hcompat.2 j (k + m + 1) (by omega)
  calc
    inst0BinderBodyClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ))
    =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient 1 v (k + 1) m (.fvar p( j))) := by
      unfold inst0BinderBodyClosedTargetDistinct
      rw [hvar]
      simp [closeFVar, hfutue]
    - =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (.fvar p( j)) := by
      rw [hclose]
    - = .fvar p( j) := by
      rw [applySubst, SubstEnv.find_extend_ne hne.symm]
      simp [SubstEnv.find, SubstEnv.empty]
  unfold inst0BinderBodyTargetEqDistinct
  rw [hL, hR]

theorem inst0BinderBodyClosedLhsDistinct_wkN_zero
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)

```

```

(m : Nat) (a : PureTm n) :
inst0BinderBodyClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
= quoteTmWith v (k + m + 1) p a := by
have hrename :
  liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) 0)
  = renameWkN (m + 1) a := by
simp using
  (liftSubN_wkN_apply (m := m + 1) σ( := subst0 a) (i := (0 : Fin (n + 1))))
calc
inst0BinderBodyClosedLhsDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
=
  closeAmbient 1 v k m
  (closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) 0)))) := by
    simp [inst0BinderBodyClosedLhsDistinct]
- =
  closeAmbient 1 v k m
  (closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
      (renameWkN (m + 1) a))) := by
    rw [hrename]
- =
  closeAmbient 1 v k m
  (quoteTmWith v (k + m + 1) (buildEnv v k m p) (renameWkN m a)) := by
    rw [closeFVar_quoteTmWith_renameWkN_buildEnv_head hcompat]
_ = quoteTmWith v (k + m + 1) p a := by
  rw [closeAmbient_quoteTmWith_renameWkN_buildEnv_any hcompat]

theorem inst0BinderBodyClosedTargetDistinct_wkN_zero
v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
(hcompat : QuoteCompat v k p)
(m : Nat) (a : PureTm n) :
inst0BinderBodyClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
= quoteTmWith v (k + m) p a := by
have hlookup :
  buildEnv v (k + 1) (m + 1) (envCons v( k) p)
  (wkN (n := n + 1) (m + 1) 0) = v k := by
simp [envCons] using
  (buildEnv_wkN_apply v (k + 1) (m + 1) (envCons v( k) p) (0 : Fin (n + 1)))
have hlookup' :
  envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))
  (wk (wkN (n := n + 1) m 0)) = v k := by
simp [buildEnv, wkN, envCons, Nat.add_assoc] using hlookup
have hvar :
  quoteTmWith v (k + m + 2)
  (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
  (.var (wkN (n := n + 1) (m + 1) 0))
  = .fvar v( k) := by
  simp [quoteTmWith, hlookup']
have hfutur : v k ≠ v (k + m + 1) := by
  intro hEq
  have : k = k + m + 1 := hcompat.1 hEq
  omega

```



```

have hclose :
  closeAmbient 1 v (k + 1) m (.fvar v( k)) = .fvar v( k) := by
apply closeAmbient_fvar_of_ne
intro r hr hEq
have : k = k + 1 + r := hcompat.1 hEq
omega
calc
  inst0BinderBodyClosedTargetDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
    =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient 1 v (k + 1) m (.fvar v( k))) := by
      unfold inst0BinderBodyClosedTargetDistinct
      rw [hvar]
      simp [closeFVar, hfuture]
  _ =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (.fvar v( k)) := by
    rw [hclose]
  _ = quoteTmWith v (k + m) p a := by
    rw [applySubst, SubstEnv.find_extend_eq]

theorem inst0BinderBodyTargetEqDistinct_wkN_zero_of
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n)
  (hstep : quoteTmWith v (k + m + 1) p a = quoteTmWith v (k + m) p a) :
  inst0BinderBodyTargetEqDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) 0)) := by
  unfold inst0BinderBodyTargetEqDistinct
  rw [inst0BinderBodyClosedLhsDistinct_wkN_zero hcompat m a,
    inst0BinderBodyClosedTargetDistinct_wkN_zero hcompat m a]
exact hstep

theorem inst0BinderBodyTargetEqDistinct_wkN_zero
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) :
  inst0BinderBodyTargetEqDistinct v m k p a (.var (wkN (n := n + 1) (m + 1) 0)) := by
apply inst0BinderBodyTargetEqDistinct_wkN_zero_of hcompat m a
exact quoteTmWith_depth_indep_staging v (k + m + 1) (k + m) p a
  (quoteCompat_mono hcompat (j := k + m + 1) (by omega))
  (quoteCompat_mono hcompat (j := k + m) (by omega))

theorem inst0BinderBodyTargetEq_wkN_succ
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) (j : Fin n) :
  inst0BinderBodyTargetEq v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ)) := by
have hrename :
  liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) j.succ)
    = renameWkN (m + 1) (.var j) := by
  simp using
    (liftSubN_wkN_apply (m := m + 1) σ( := subst0 a) (i := j.succ))
have hL :
  inst0BinderBodyClosedLhs v m k p a (.var (wkN (n := n + 1) (m + 1) j.succ))

```

```

    = .fvar p( j ) := by
calc
  inst0BinderBodyClosedLhs v m k p a ( .var (wkN (n := n + 1) (m + 1) j.succ))
    =
    closeRangeAt 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
          (liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) j.succ)))) := by
        simp [inst0BinderBodyClosedLhs]
  - =
  closeRangeAt 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
        (renameWkN (m + 1) ( .var j)))) := by
        rw [hrename]
  - =
  closeRangeAt 1 v k m
    (quoteTmWith v (k + m + 1) (buildEnv v k m p) (renameWkN m ( .var j))) := by
        rw [closeFVar_quoteTmWith_renameWkN_buildEnv_head hcompat]
  - = .fvar p( j ) := by
        rw [closeRangeAt_quoteTmWith_renameWkN_buildEnv_any hcompat]
        simp [quoteTmWith]
have hclose :
  closeRangeAt 1 v (k + 1) m ( .fvar p( j )) = .fvar p( j ) := by
  apply closeRangeAt_fvar_of_ne
  intro r hr
  exact hcompat.2 j (k + 1 + r) (by omega)
have hne : p j ≠ v k := hcompat.2 j k (by omega)
have hR :
  inst0BinderBodyClosedTarget v m k p a ( .var (wkN (n := n + 1) (m + 1) j.succ))
    = .fvar p( j ) := by
  have hlookup :
    buildEnv v (k + 1) (m + 1) (envCons v( k ) p)
      (wkN (n := n + 1) (m + 1) j.succ) = p j := by
    simp [envCons] using
      (buildEnv_wkN_apply v (k + 1) (m + 1) (envCons v( k ) p) j.succ)
  have hlookup' :
    envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k ) p))
      (wk (wkN (n := n + 1) m j.succ)) = p j := by
    simp [buildEnv, wkN, envCons, Nat.add_assoc] using hlookup
  have hvar :
    quoteTmWith v (k + m + 2)
      (buildEnv v (k + 1) (m + 1) (envCons v( k ) p))
      ( .var (wkN (n := n + 1) (m + 1) j.succ))
    = .fvar p( j ) := by
    simp [quoteTmWith, hlookup']
  have hfuture : p j ≠ v (k + m + 1) := hcompat.2 j (k + m + 1) (by omega)
calc
  inst0BinderBodyClosedTarget v m k p a ( .var (wkN (n := n + 1) (m + 1) j.succ))
    =
    applySubst (SubstEnv.extend SubstEnv.empty v( k ) (quoteTmWith v (k + m) p a))
      (closeRangeAt 1 v (k + 1) m ( .fvar p( j ))) := by
      unfold inst0BinderBodyClosedTarget
      rw [hvar]

```

```

      simp [closeFVar, hfuture]
    - =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
        (.fvar p( j)) := by
          rw [hclose]
    - = .fvar p( j) := by
          rw [applySubst, SubstEnv.find_extend_ne hne.symm]
          simp [SubstEnv.find, SubstEnv.empty]
  unfold inst0BinderBodyTargetEq
  rw [hL, hR]

```

theorem inst0BinderBodyClosedLhs_wkN_zero

```

  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) :
  inst0BinderBodyClosedLhs v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
    = quoteTmWith v (k + m + 1) p a := by
  have hrename :
    liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) 0)
      = renameWkN (m + 1) a := by
  simp using
    (liftSubN_wkN_apply (m := m + 1) σ( := subst0 a) (i := (0 : Fin (n + 1))))
  calc
    inst0BinderBodyClosedLhs v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
      =
      closeRangeAt 1 v k m
        (closeFVar 0 v( (k + m))
          (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
            (liftSubN (m + 1) (subst0 a) (wkN (n := n + 1) (m + 1) 0)))) := by
          simp [inst0BinderBodyClosedLhs]
    - =
      closeRangeAt 1 v k m
        (closeFVar 0 v( (k + m))
          (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
            (renameWkN (m + 1) a))) := by
          rw [hrename]
    - =
      closeRangeAt 1 v k m
        (quoteTmWith v (k + m + 1) (buildEnv v k m p) (renameWkN m a)) := by
          rw [closeFVar_quoteTmWith_renameWkN_buildEnv_head hcompat]
    - = quoteTmWith v (k + m + 1) p a := by
          rw [closeRangeAt_quoteTmWith_renameWkN_buildEnv_any hcompat]

```

theorem inst0BinderBodyClosedTarget_wkN_zero

```

  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) :
  inst0BinderBodyClosedTarget v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
    = quoteTmWith v (k + m) p a := by
  have hlookup :
    buildEnv v (k + 1) (m + 1) (envCons v( k) p)
      (wkN (n := n + 1) (m + 1) 0) = v k := by
  simp [envCons] using
    (buildEnv_wkN_apply v (k + 1) (m + 1) (envCons v( k) p) (0 : Fin (n + 1)))

```

```

have hlookup' :
  envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))
    (wk (wkN (n := n + 1) m 0)) = v k := by
  simp [buildEnv, wkN, envCons, Nat.add_assoc] using hlookup
have hvar :
  quoteTmWith v (k + m + 2)
    (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
    (.var (wkN (n := n + 1) (m + 1) 0))
  = .fvar v( k) := by
  simp [quoteTmWith, hlookup']
have hfutur : v k ≠ v (k + m + 1) := by
  intro hEq
  have : k = k + m + 1 := hcompat.1 hEq
  omega
have hclose :
  closeRangeAt 1 v (k + 1) m (.fvar v( k)) = .fvar v( k) := by
  apply closeRangeAt_fvar_of_ne
  intro r hr hEq
  have : k = k + 1 + r := hcompat.1 hEq
  omega
calc
  inst0BinderBodyClosedTarget v m k p a (.var (wkN (n := n + 1) (m + 1) 0))
    =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
      (closeRangeAt 1 v (k + 1) m (.fvar v( k))) := by
      unfold inst0BinderBodyClosedTarget
      rw [hvar]
      simp [closeFVar, hfutur]
  _ =
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v (k + m) p a))
    (.fvar v( k)) := by
    rw [hclose]
  _ = quoteTmWith v (k + m) p a := by
    rw [applySubst, SubstEnv.find_extend_eq]

theorem inst0BinderBodyTargetEq_wkN_zero_of
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n)
  (hstep : quoteTmWith v (k + m + 1) p a = quoteTmWith v (k + m) p a) :
  inst0BinderBodyTargetEq v m k p a (.var (wkN (n := n + 1) (m + 1) 0)) := by
  unfold inst0BinderBodyTargetEq
  rw [inst0BinderBodyClosedLhs_wkN_zero hcompat m a,
    inst0BinderBodyClosedTarget_wkN_zero hcompat m a]
  exact hstep

theorem inst0BinderBodyTargetEq_wkN_zero
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) :
  inst0BinderBodyTargetEq v m k p a (.var (wkN (n := n + 1) (m + 1) 0)) := by
  apply inst0BinderBodyTargetEq_wkN_zero_of hcompat m a
  exact quoteTmWith_depth_indep_staging v (k + m + 1) (k + m) p a
    (quoteCompat_mono hcompat (j := k + m + 1) (by omega))

```

```

    (quoteCompat_mono hcompat (j := k + m) (by omega))

@[simp] theorem succ_wkN_zero_eq_wkN_zero
  (m : Nat) :
    Fin.succ (wkN (n := n + 1) m 0) = wkN (n := n + 1) (m + 1) 0 := by
  apply Fin.ext
  simp [wk, wkN_val]

@[simp] theorem succ_wkN_succ_eq_wkN_succ
  (m : Nat) (j : Fin n) :
    Fin.succ (wkN (n := n + 1) m j.succ) = wkN (n := n + 1) (m + 1) j.succ := by
  apply Fin.ext
  simp [wk, wkN_val, Nat.add_assoc]

theorem fin_body_cases
  (m : Nat) (i : Fin (((n + 1) + m) + 1)) :
  i = 0 ∨
    ∃(j : Fin m, i = Fin.succ (prefixIdx (n + 1) j)) ∨
    i = wkN (n := n + 1) (m + 1) 0 ∨
    ∃(j : Fin n, i = wkN (n := n + 1) (m + 1) j.succ) := by
  cases i using Fin.cases with
  | zero =>
    exact Or.inl rfl
  | succ i =>
    rcases fin_prefix_or_wkN (n := n) m i with hprefix | hwk
    · right
      left
      rcases hprefix with ⟨j, ⟩rfl
      exact ⟨j, ⟩rfl
    · rcases hwk with ⟨j, ⟩rfl
      cases j using Fin.cases with
      | zero =>
        right
        right
        left
        simp
      | succ j =>
        right
        right
        right
        exact ⟨j, by ⟩simp

theorem inst0BinderBodyTargetEqDistinct_var
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ)
  (m : Nat) (a : PureTm n) (i : Fin (((n + 1) + m) + 1)) :
  inst0BinderBodyTargetEqDistinct v m k ρ a (.var i) := by
  rcases fin_body_cases (n := n) m i with rfl | hprefix | hwk0 | hwk
  · exact inst0BinderBodyTargetEqDistinct_zero v m k ρ a
  · rcases hprefix with ⟨j, ⟩rfl
    exact inst0BinderBodyTargetEqDistinct_prefix_succ hcompat.1 m k ρ a j
  · rw [hwk0]
    exact inst0BinderBodyTargetEqDistinct_wkN_zero hcompat m a
  · rcases hwk with ⟨j, ⟩rfl

```

```

exact inst0BinderBodyTargetEqDistinct_wkN_succ hcompat m a j

theorem inst0BinderBodyTargetEq_var
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p)
  (m : Nat) (a : PureTm n) (i : Fin ((n + 1) + m) + 1) :
  inst0BinderBodyTargetEq v m k p a (.var i) := by
rcases fin_body_cases (n := n) m i with rfl | hprefix | hwk0 | hwk
· exact inst0BinderBodyTargetEq_zero v m k p a
· rcases hprefix with {j, }rfl
  exact inst0BinderBodyTargetEq_prefix_succ hcompat.1 m k p a j
· rw [hwk0]
  exact inst0BinderBodyTargetEq_wkN_zero hcompat m a
· rcases hwk with {j, }rfl
  exact inst0BinderBodyTargetEq_wkN_succ hcompat m a j

theorem inst0BinderBodyTargetEq_u0
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderBodyTargetEq v m k p a .u0 := by
unfold inst0BinderBodyTargetEq inst0BinderBodyClosedLhs inst0BinderBodyClosedTarget
calc
  closeRangeAt 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
        (subst (liftSubN (m + 1) (subst0 a)) PureTm.u0)))
    = Mettapedial.Languages.MeTTa.Pure.Core.u0 := by
      simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.
        Core.u0] using
        (closeRangeAt_u0 1 v k m)
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      Mettapedial.Languages.MeTTa.Pure.Core.u0 := by
      simp [Mettapedial.Languages.MeTTa.Pure.Core.u0, applySubst]
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      (closeRangeAt 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
            PureTm.u0))) := by
      simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.
        Core.u0] using
        (congrArg
          (applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a)))
          (closeRangeAt_u0 1 v (k + 1) m)).symm

theorem inst0BinderBodyTargetEq_u1
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderBodyTargetEq v m k p a .u1 := by
unfold inst0BinderBodyTargetEq inst0BinderBodyClosedLhs inst0BinderBodyClosedTarget
calc
  closeRangeAt 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
        (subst (liftSubN (m + 1) (subst0 a)) PureTm.u1)))
    = Mettapedial.Languages.MeTTa.Pure.Core.u1 := by
      simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.

```

```

Core.u1] using
  (closeRangeAt_u1 1 v k m)
_ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
  Mettapedial.Languages.MeTTa.Pure.Core.u1 := by
  simp [Mettapedial.Languages.MeTTa.Pure.Core.u1, applySubst]
_ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
        PureTm.u1))) := by
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.
Core.u1] using
  (congrArg
    (applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      (closeRangeAt_u1 1 v (k + 1) m)).symm

theorem inst0BinderBodyTargetEqDistinct_u0
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderBodyTargetEqDistinct v m k p a .u0 := by
unfold inst0BinderBodyTargetEqDistinct inst0BinderBodyClosedLhsDistinct
  inst0BinderBodyClosedTargetDistinct
calc
  closeAmbient 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
        (subst (liftSubN (m + 1) (subst0 a)) PureTm.u0)))
  = Mettapedial.Languages.MeTTa.Pure.Core.u0 := by
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.
Core.u0] using
  (closeAmbient_u0 1 v k m)
_ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
  Mettapedial.Languages.MeTTa.Pure.Core.u0 := by
  simp [Mettapedial.Languages.MeTTa.Pure.Core.u0, applySubst]
_ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
        PureTm.u0))) := by
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.
Core.u0] using
  (congrArg
    (applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient_u0 1 v (k + 1) m)).symm

theorem inst0BinderBodyTargetEqDistinct_u1
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderBodyTargetEqDistinct v m k p a .u1 := by
unfold inst0BinderBodyTargetEqDistinct inst0BinderBodyClosedLhsDistinct
  inst0BinderBodyClosedTargetDistinct
calc
  closeAmbient 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (buildEnv v k (m + 1) p)
        (subst (liftSubN (m + 1) (subst0 a)) PureTm.u1)))

```

```

    = Mettapedia.Languages.MeTTa.Pure.Core.u1 := by
      simpa [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.
Core.u1] using
        (closeAmbient_u1 1 v k m)
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      Mettapedia.Languages.MeTTa.Pure.Core.u1 := by
        simp [Mettapedia.Languages.MeTTa.Pure.Core.u1, applySubst]
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (buildEnv v (k + 1) (m + 1) (envCons v( k) p))
            PureTm.u1))) := by
        simpa [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.
Core.u1] using
          (congrArg
            (applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a)))
            (closeAmbient_u1 1 v (k + 1) m)).symm

```

theorem inst0BinderBodyClosedLhs_app_expand

```

v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
(f b : PureTm (((n + 1) + m) + 1)) :
inst0BinderBodyClosedLhs v m k p a (.app f b)
=

```

```

Mettapedia.Languages.MeTTa.Pure.Core.mkApp
  (inst0BinderBodyClosedLhs v m k p a f)
  (inst0BinderBodyClosedLhs v m k p a b) := by

```

unfold inst0BinderBodyClosedLhs

```

simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkApp]
simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkApp] using
  closeRangeAt_mkApp 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) f)))
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) b)))

```

theorem inst0BinderBodyClosedTarget_app_expand

```

v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
(f b : PureTm (((n + 1) + m) + 1)) :
inst0BinderBodyClosedTarget v m k p a (.app f b)
=

```

```

Mettapedia.Languages.MeTTa.Pure.Core.mkApp
  (inst0BinderBodyClosedTarget v m k p a f)
  (inst0BinderBodyClosedTarget v m k p a b) := by

```

unfold inst0BinderBodyClosedTarget

```

simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkApp]
rw [show closeRangeAt 1 v (k + 1) m
  (Pattern.apply "App"
    [closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(
k) p))) f),
    closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(

```



```

k) p))) b]))
=
Mettapedial.Languages.MeTTa.Pure.Core.mkApp
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) f)))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) b))) by
    simp [Mettapedial.Languages.MeTTa.Pure.Core.mkApp] using
      closeRangeAt_mkApp 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) f))
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) b)))
    simp [Mettapedial.Languages.MeTTa.Pure.Core.mkApp] using
      applySubst_mkApp
        (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
        (closeRangeAt 1 v (k + 1) m
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2)
              (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) f)))
        (closeRangeAt 1 v (k + 1) m
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2)
              (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) b)))
theorem inst0BinderBodyTargetEq_app_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {f b : PureTm (((n + 1) + m) + 1)}
  (hf : inst0BinderBodyTargetEq v m k p a f)
  (hb : inst0BinderBodyTargetEq v m k p a b) :
  inst0BinderBodyTargetEq v m k p a (.app f b) := by
  unfold inst0BinderBodyTargetEq at hf hb ⊢
  rw [inst0BinderBodyClosedLhs_app_expand, inst0BinderBodyClosedTarget_app_expand]
  simp using >congrArg Mettapedial.Languages.MeTTa.Pure.Core.mkApp hf hb
theorem inst0BinderBodyClosedLhsDistinct_app_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (f b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhsDistinct v m k p a (.app f b)
  =
  Mettapedial.Languages.MeTTa.Pure.Core.mkApp
    (inst0BinderBodyClosedLhsDistinct v m k p a f)
    (inst0BinderBodyClosedLhsDistinct v m k p a b) := by
  unfold inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedial.Languages.MeTTa.Pure.Core.mkApp]
  simp [Mettapedial.Languages.MeTTa.Pure.Core.mkApp] using
    closeAmbient_mkApp 1 v k m
      (closeFVar 0 v( (k + m))

```

```

      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) f)))
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) b)))

theorem inst0BinderBodyClosedTargetDistinct_app_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (f b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTargetDistinct v m k p a (.app f b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp
    (inst0BinderBodyClosedTargetDistinct v m k p a f)
    (inst0BinderBodyClosedTargetDistinct v m k p a b) := by
  unfold inst0BinderBodyClosedTargetDistinct
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkApp]
  rw [show closeAmbient 1 v (k + 1) m
    (Pattern.apply "App"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) f),
        closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) b)]]
    =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) f)))
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) b))) by
    simp [Mettapedia.Languages.MeTTa.Pure.Core.mkApp] using
      closeAmbient_mkApp 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) f))
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) b))]
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkApp] using
  applySubst_mkApp
    (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) f)))
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) b)))

```

```

theorem inst0BinderBodyTargetEqDistinct_app_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {f b : PureTm (((n + 1) + m) + 1)}
  (hf : inst0BinderBodyTargetEqDistinct v m k p a f)
  (hb : inst0BinderBodyTargetEqDistinct v m k p a b) :
  inst0BinderBodyTargetEqDistinct v m k p a (.app f b) := by
  unfold inst0BinderBodyTargetEqDistinct at hf hb ⊢
  rw [inst0BinderBodyClosedLhsDistinct_app_expand, inst0BinderBodyClosedTargetDistinct_app_expand]
  simp using >congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkApp hf hb

theorem inst0BinderBodyClosedLhs_id_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A 1a 1b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhs v m k p a (.id A 1a 1b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkId
    (inst0BinderBodyClosedLhs v m k p a A)
    (inst0BinderBodyClosedLhs v m k p a 1a)
    (inst0BinderBodyClosedLhs v m k p a 1b) := by
  unfold inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    closeRangeAt_mkId 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) A)))
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) 1a)))
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) 1b)))

theorem inst0BinderBodyClosedTarget_id_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A 1a 1b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTarget v m k p a (.id A 1a 1b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkId
    (inst0BinderBodyClosedTarget v m k p a A)
    (inst0BinderBodyClosedTarget v m k p a 1a)
    (inst0BinderBodyClosedTarget v m k p a 1b) := by
  unfold inst0BinderBodyClosedTarget
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
  rw [show closeRangeAt 1 v (k + 1) m
    (Pattern.apply "Id"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(
          k) p))) A),
        closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(
            k) p))) 1a),
        closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(
            k) p))) 1b)
      ]
    )
  ]

```

```

      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p)))) 1b)))
    =
Mettapedia.Languages.MeTTa.Pure.Core.mkId
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A)))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a)))
      (closeRangeAt 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b)))) by
    simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
      closeRangeAt_mkId 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A))
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a))
            (closeFVar 0 v( (k + m + 1))
              (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b))))
    simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
      applySubst_mkId
        (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
        (closeRangeAt 1 v (k + 1) m
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2)
              (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A)))
          (closeRangeAt 1 v (k + 1) m
            (closeFVar 0 v( (k + m + 1))
              (quoteTmWith v (k + m + 2)
                (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a)))
            (closeRangeAt 1 v (k + 1) m
              (closeFVar 0 v( (k + m + 1))
                (quoteTmWith v (k + m + 2)
                  (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b))))

theorem inst0BinderBodyTargetEq_id_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A 1a 1b : PureTm (((n + 1) + m) + 1)}
  (hA : inst0BinderBodyTargetEq v m k p a A)
  (1ha : inst0BinderBodyTargetEq v m k p a 1a)
  (1hb : inst0BinderBodyTargetEq v m k p a 1b) :
  inst0BinderBodyTargetEq v m k p a (.id A 1a 1b) := by
  unfold inst0BinderBodyTargetEq at hA 1ha 1hb ⊢
  rw [inst0BinderBodyClosedLhs_id_expand, inst0BinderBodyClosedTarget_id_expand]
  rw [hA, 1ha, 1hb]

```

```

theorem inst0BinderBodyClosedLhs_pair_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p q : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhs v m k p a (.pair p q)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair
    (inst0BinderBodyClosedLhs v m k p a p)
    (inst0BinderBodyClosedLhs v m k p a q) := by
  unfold inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
    closeRangeAt_mkPair 1 v k m
    (closeFVar 0 v( (k + m)))
    (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
      (subst (liftSub (liftSubN m (subst0 a))) p)))
    (closeFVar 0 v( (k + m)))
    (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
      (subst (liftSub (liftSubN m (subst0 a))) q)))

theorem inst0BinderBodyClosedTarget_pair_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p q : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTarget v m k p a (.pair p q)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair
    (inst0BinderBodyClosedTarget v m k p a p)
    (inst0BinderBodyClosedTarget v m k p a q) := by
  unfold inst0BinderBodyClosedTarget
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
  rw [show closeRangeAt 1 v (k + 1) m
    (Pattern.apply "Pair"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p),
        closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) q)]))]
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) q))) by
    simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
      closeRangeAt_mkPair 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) p))
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) q))

```

```

      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) q)))
simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
  applySubst_mkPair
    (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) q)))

```

```

theorem inst0BinderBodyTargetEq_pair_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p q : PureTm (((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEq v m k p a p)
  (hq : inst0BinderBodyTargetEq v m k p a q) :
  inst0BinderBodyTargetEq v m k p a (.pair p q) := by
  unfold inst0BinderBodyTargetEq at hp hq ⊢
  rw [inst0BinderBodyClosedLhs_pair_expand, inst0BinderBodyClosedTarget_pair_expand]
  simpa using >congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkPair hp hq

```

```

theorem inst0BinderBodyClosedLhs_fst_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhs v m k p a (.fst p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst
    (inst0BinderBodyClosedLhs v m k p a p) := by
  unfold inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
    closeRangeAt_mkFst 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
          (subst (liftSub (liftSubN m (subst0 a))) p)))

```

```

theorem inst0BinderBodyClosedTarget_fst_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTarget v m k p a (.fst p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst
    (inst0BinderBodyClosedTarget v m k p a p) := by
  unfold inst0BinderBodyClosedTarget
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
  rw [show closeRangeAt 1 v (k + 1) m
    (Pattern.apply "Fst"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k)
) p))) p])]
  =

```

```

Mettapedia.Languages.MeTTa.Pure.Core.mkFst
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) p))) by
    simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
      closeRangeAt_mkFst 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) p)))
simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
  applySubst_mkFst
    (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeRangeAt 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))

theorem inst0BinderBodyTargetEq_fst_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm (((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEq v m k p a p) :
  inst0BinderBodyTargetEq v m k p a (.fst p) := by
  unfold inst0BinderBodyTargetEq at hp ⊢
  rw [inst0BinderBodyClosedLhs_fst_expand, inst0BinderBodyClosedTarget_fst_expand]
  simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkFst hp

theorem inst0BinderBodyClosedLhs_snd_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhs v m k p a (.snd p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSnd
    (inst0BinderBodyClosedLhs v m k p a p) := by
  unfold inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
    closeRangeAt_mkSnd 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
          (subst (liftSub (liftSubN m (subst0 a))) p)))

theorem inst0BinderBodyClosedTarget_snd_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTarget v m k p a (.snd p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSnd
    (inst0BinderBodyClosedTarget v m k p a p) := by
  unfold inst0BinderBodyClosedTarget
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
  rw [show closeRangeAt 1 v (k + 1) m
    (Pattern.apply "Snd"
      [closeFVar 0 v( (k + m + 1))

```

```

      (quoteTmWith v (k + m + 2) (envCons v (k + 1 + m)) (buildEnv v (k + 1) m (envCons v (k
) p))) p]))
    =
Mettapedia.Languages.MeTTa.Pure.Core.mkSnd
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v (k + m + 1))
    (quoteTmWith v (k + m + 2) (envCons v (k + 1 + m)) (buildEnv v (k + 1) m (envCons v (k
) p))) p))) by
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
    closeRangeAt_mkSnd 1 v (k + 1) m
    (closeFVar 0 v (k + m + 1))
    (quoteTmWith v (k + m + 2) (envCons v (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v (k) p))) p))]
simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
  applySubst_mkSnd
  (SubstEnv.empty.extend v (k) (quoteTmWith v (k + m) p a))
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v (k + m + 1))
    (quoteTmWith v (k + m + 2)
      (envCons v (k + 1 + m)) (buildEnv v (k + 1) m (envCons v (k) p))) p)))

theorem inst0BinderBodyTargetEq_snd_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm (((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEq v m k p a p) :
  inst0BinderBodyTargetEq v m k p a (.snd p) := by
  unfold inst0BinderBodyTargetEq at hp ⊢
  rw [inst0BinderBodyClosedLhs_snd_expand, inst0BinderBodyClosedTarget_snd_expand]
  simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkSnd hp

theorem inst0BinderBodyClosedLhs_refl_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhs v m k p a (.refl p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
    (inst0BinderBodyClosedLhs v m k p a p) := by
  unfold inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
  ]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
    closeRangeAt_mkRefl 1 v k m
    (closeFVar 0 v (k + m))
    (quoteTmWith v (k + m + 1) (envCons v (k + m)) (buildEnv v k m p))
    (subst (liftSub (liftSubN m (subst0 a))) p)))

theorem inst0BinderBodyClosedTarget_refl_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTarget v m k p a (.refl p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
    (inst0BinderBodyClosedTarget v m k p a p) := by
  unfold inst0BinderBodyClosedTarget

```



```

simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl]
rw [show closeRangeAt 1 v (k + 1) m
    (Pattern.apply "Refl"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
        ) p))) p))]
    =
Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
  (closeRangeAt 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
      ) p))) p))) by
    simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
      closeRangeAt_mkRefl 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
          v( k) p))) p))]
    simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
      applySubst_mkRefl
        (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
        (closeRangeAt 1 v (k + 1) m
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2)
              (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))

theorem inst0BinderBodyTargetEq_refl_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm (((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEq v m k p a p) :
  inst0BinderBodyTargetEq v m k p a (.refl p) := by
  unfold inst0BinderBodyTargetEq at hp ⊢
  rw [inst0BinderBodyClosedLhs_refl_expand, inst0BinderBodyClosedTarget_refl_expand]
  simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkRefl hp

theorem inst0BinderBodyClosedLhsDistinct_id_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A 1a 1b : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhsDistinct v m k p a (.id A 1a 1b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkId
    (inst0BinderBodyClosedLhsDistinct v m k p a A)
    (inst0BinderBodyClosedLhsDistinct v m k p a 1a)
    (inst0BinderBodyClosedLhsDistinct v m k p a 1b) := by
  unfold inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    closeAmbient_mkId 1 v k m
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
          (subst (liftSub (liftSubN m (subst0 a))) A)))
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
          (subst (liftSub (liftSubN m (subst0 a))) 1a)))
      (closeFVar 0 v( (k + m))
        (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
          (subst (liftSub (liftSubN m (subst0 a))) 1b)))

```

```

      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) 1b)))

```

theorem inst0BinderBodyClosedTargetDistinct_id_expand

```

v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
(A 1a 1b : PureTm (((n + 1) + m) + 1)) :
inst0BinderBodyClosedTargetDistinct v m k p a (.id A 1a 1b)
=
Mettapedia.Languages.MeTTa.Pure.Core.mkId
  (inst0BinderBodyClosedTargetDistinct v m k p a A)
  (inst0BinderBodyClosedTargetDistinct v m k p a 1a)
  (inst0BinderBodyClosedTargetDistinct v m k p a 1b) := by
unfold inst0BinderBodyClosedTargetDistinct
simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkId]
rw [show closeAmbient 1 v (k + 1) m
  (Pattern.apply "Id"
    [closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A),
      closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a),
      closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b))]]
=
Mettapedia.Languages.MeTTa.Pure.Core.mkId
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A)))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a)))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b)))) by
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    closeAmbient_mkId 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A))
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a))
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b)))
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkId] using
    applySubst_mkId
    (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient 1 v (k + 1) m

```

```

      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) A)))
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1a)))
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) 1b)))

```

theorem inst0BinderBodyTargetEqDistinct_id_of

```

  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A 1a 1b : PureTm (((n + 1) + m) + 1)}
  (hA : inst0BinderBodyTargetEqDistinct v m k p a A)
  (1ha : inst0BinderBodyTargetEqDistinct v m k p a 1a)
  (1hb : inst0BinderBodyTargetEqDistinct v m k p a 1b) :
  inst0BinderBodyTargetEqDistinct v m k p a (.id A 1a 1b) := by
  unfold inst0BinderBodyTargetEqDistinct at hA 1ha 1hb ⊢
  rw [inst0BinderBodyClosedLhsDistinct_id_expand, inst0BinderBodyClosedTargetDistinct_id_expand]
  rw [hA, 1ha, 1hb]

```

theorem inst0BinderBodyClosedLhsDistinct_pair_expand

```

  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p q : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhsDistinct v m k p a (.pair p q)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair
    (inst0BinderBodyClosedLhsDistinct v m k p a p)
    (inst0BinderBodyClosedLhsDistinct v m k p a q) := by
  unfold inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkPair] using
    closeAmbient_mkPair 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) p)))
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) q)))

```

theorem inst0BinderBodyClosedTargetDistinct_pair_expand

```

  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p q : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTargetDistinct v m k p a (.pair p q)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPair
    (inst0BinderBodyClosedTargetDistinct v m k p a p)
    (inst0BinderBodyClosedTargetDistinct v m k p a q) := by
  unfold inst0BinderBodyClosedTargetDistinct
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkPair]
  rw [show closeAmbient 1 v (k + 1) m

```

```

    (Pattern.apply "Pair"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(
k) p))) p),
        closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v(
k) p))) q)])
    =
Mettapedialanguages.Metta.Pure.Core.mkPair
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) p)))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) q))) by
  simpa [Mettapedialanguages.Metta.Pure.Core.mkPair] using
    closeAmbient_mkPair 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) p))
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) q))]
  simpa [Mettapedialanguages.Metta.Pure.Core.mkPair] using
    applySubst_mkPair
      (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
      (closeAmbient 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2)
            (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))
      (closeAmbient 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2)
            (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) q)))

theorem inst0BinderBodyTargetEqDistinct_pair_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p q : PureTm (((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEqDistinct v m k p a p)
  (hq : inst0BinderBodyTargetEqDistinct v m k p a q) :
  inst0BinderBodyTargetEqDistinct v m k p a (.pair p q) := by
  unfold inst0BinderBodyTargetEqDistinct at hp hq ⊢
  rw [inst0BinderBodyClosedLhsDistinct_pair_expand,
    inst0BinderBodyClosedTargetEqDistinct_pair_expand]
  simpa using >congrArg Mettapedialanguages.Metta.Pure.Core.mkPair hp hq

theorem inst0BinderBodyClosedLhsDistinct_fst_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhsDistinct v m k p a (.fst p)
  =
  Mettapedialanguages.Metta.Pure.Core.mkFst

```

```

      (inst0BinderBodyClosedLhsDistinct v m k p a p) := by
    unfold inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
    closeAmbient_mkFst 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) p)))

theorem inst0BinderBodyClosedTargetDistinct_fst_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm ((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTargetDistinct v m k p a (.fst p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst
    (inst0BinderBodyClosedTargetDistinct v m k p a p) := by
  unfold inst0BinderBodyClosedTargetDistinct
  simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkFst]
  rw [show closeAmbient 1 v (k + 1) m
    (Pattern.apply "Fst"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
        ) p))) p))]
    =
  Mettapedia.Languages.MeTTa.Pure.Core.mkFst
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
        ) p))) p))) by
    simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
      closeAmbient_mkFst 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
        v( k) p))) p))]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkFst] using
    applySubst_mkFst
    (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient 1 v (k + 1) m
      (closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2)
          (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))

theorem inst0BinderBodyTargetEqDistinct_fst_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm ((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEqDistinct v m k p a p) :
  inst0BinderBodyTargetEqDistinct v m k p a (.fst p) := by
  unfold inst0BinderBodyTargetEqDistinct at hp
  rw [inst0BinderBodyClosedLhsDistinct_fst_expand, inst0BinderBodyClosedTargetDistinct_fst_expand
    ]
  simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkFst hp

theorem inst0BinderBodyClosedLhsDistinct_snd_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)

```

```

(p : PureTm (((n + 1) + m) + 1)) :
inst0BinderBodyClosedLhsDistinct v m k p a (.snd p)
=
Mettapedia.Languages.MeTTa.Pure.Core.mkSnd
  (inst0BinderBodyClosedLhsDistinct v m k p a p) := by
unfold inst0BinderBodyClosedLhsDistinct
simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
  closeAmbient_mkSnd 1 v k m
  (closeFVar 0 v( (k + m))
    (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
      (subst (liftSub (liftSubN m (subst0 a))) p)))

theorem inst0BinderBodyClosedTargetDistinct_snd_expand
  v{ : Nat → String} (m k : Nat) p{ : QuoteEnv n} (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
inst0BinderBodyClosedTargetDistinct v m k p a (.snd p)
=
Mettapedia.Languages.MeTTa.Pure.Core.mkSnd
  (inst0BinderBodyClosedTargetDistinct v m k p a p) := by
unfold inst0BinderBodyClosedTargetDistinct
simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkSnd]
rw [show closeAmbient 1 v (k + 1) m
  (Pattern.apply "Snd"
    [closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) p))]
  =
Mettapedia.Languages.MeTTa.Pure.Core.mkSnd
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
) p))) p))) by
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
    closeAmbient_mkSnd 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
v( k) p))) p))]
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSnd] using
  applySubst_mkSnd
  (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
  (closeAmbient 1 v (k + 1) m
    (closeFVar 0 v( (k + m + 1))
      (quoteTmWith v (k + m + 2)
        (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))

theorem inst0BinderBodyTargetEqDistinct_snd_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm (((n + 1) + m) + 1)}
  (hp : inst0BinderBodyTargetEqDistinct v m k p a p) :
  inst0BinderBodyTargetEqDistinct v m k p a (.snd p) := by
unfold inst0BinderBodyTargetEqDistinct at hp ⊢
rw [inst0BinderBodyClosedLhsDistinct_snd_expand, inst0BinderBodyClosedTargetDistinct_snd_expand
]

```

simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkSnd hp

```

theorem inst0BinderBodyClosedLhsDistinct_refl_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedLhsDistinct v m k p a (.refl p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
    (inst0BinderBodyClosedLhsDistinct v m k p a p) := by
  unfold inst0BinderBodyClosedLhsDistinct
simp [quoteTmWith, subst, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl]
  ]
  simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
    closeAmbient_mkRefl 1 v k m
    (closeFVar 0 v( (k + m))
      (quoteTmWith v (k + m + 1) (envCons v( (k + m)) (buildEnv v k m p))
        (subst (liftSub (liftSubN m (subst0 a))) p)))

theorem inst0BinderBodyClosedTargetDistinct_refl_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (p : PureTm (((n + 1) + m) + 1)) :
  inst0BinderBodyClosedTargetDistinct v m k p a (.refl p)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
    (inst0BinderBodyClosedTargetDistinct v m k p a p) := by
  unfold inst0BinderBodyClosedTargetDistinct
simp [quoteTmWith, buildEnv_succ, closeFVar, Mettapedia.Languages.MeTTa.Pure.Core.mkRefl]
  rw [show closeAmbient 1 v (k + 1) m
    (Pattern.apply "Refl"
      [closeFVar 0 v( (k + m + 1))
        (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
        ) p))) p))]
    =
    Mettapedia.Languages.MeTTa.Pure.Core.mkRefl
      (closeAmbient 1 v (k + 1) m
        (closeFVar 0 v( (k + m + 1))
          (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k
          ) p))) p))) by
        simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
          closeAmbient_mkRefl 1 v (k + 1) m
          (closeFVar 0 v( (k + m + 1))
            (quoteTmWith v (k + m + 2) (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons
            v( k) p))) p))]
        simpa [Mettapedia.Languages.MeTTa.Pure.Core.mkRefl] using
          applySubst_mkRefl
            (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
            (closeAmbient 1 v (k + 1) m
              (closeFVar 0 v( (k + m + 1))
                (quoteTmWith v (k + m + 2)
                  (envCons v( (k + 1 + m)) (buildEnv v (k + 1) m (envCons v( k) p))) p)))

theorem inst0BinderBodyTargetEqDistinct_refl_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm (((n + 1) + m) + 1)}

```

```

    (hp : inst0BinderBodyTargetEqDistinct v m k p a p) :
      inst0BinderBodyTargetEqDistinct v m k p a (.refl p) := by
    unfold inst0BinderBodyTargetEqDistinct at hp ⊢
    rw [inst0BinderBodyClosedLhsDistinct_refl_expand,
        inst0BinderBodyClosedTargetDistinct_refl_expand]
    simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkRefl hp

theorem inst0BinderClosedLhsDistinct_lam_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
    inst0BinderClosedLhsDistinct v m k p a (.lam b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (inst0BinderBodyClosedLhsDistinct v m k p a b) := by
  unfold inst0BinderClosedLhsDistinct inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm]
  rw [closeAmbient_mkLam]

theorem inst0BinderClosedTargetDistinct_lam_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (b : PureTm (((n + 1) + m) + 1)) :
    inst0BinderClosedTargetDistinct v m k p a (.lam b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (inst0BinderBodyClosedTargetDistinct v m k p a b) := by
  unfold inst0BinderClosedTargetDistinct inst0BinderBodyClosedTargetDistinct
  simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm]
  rw [closeAmbient_mkLam]
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkLam, applySubst, Nat.add_left_comm]

theorem inst0BinderClosedLhsDistinct_pi_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
    inst0BinderClosedLhsDistinct v m k p a (.pi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (inst0BinderClosedLhsDistinct v m k p a A)
    (inst0BinderBodyClosedLhsDistinct v m k p a B) := by
  unfold inst0BinderClosedLhsDistinct inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm]
  rw [closeAmbient_mkPi]

theorem inst0BinderClosedTargetDistinct_pi_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
    inst0BinderClosedTargetDistinct v m k p a (.pi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (inst0BinderClosedTargetDistinct v m k p a A)
    (inst0BinderBodyClosedTargetDistinct v m k p a B) := by
  unfold inst0BinderClosedTargetDistinct inst0BinderBodyClosedTargetDistinct
  simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm]
  rw [closeAmbient_mkPi]
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPi, applySubst, Nat.add_left_comm]

```



```

theorem inst0BinderClosedLhsDistinct_sigma_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedLhsDistinct v m k p a (.sigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (inst0BinderClosedLhsDistinct v m k p a A)
    (inst0BinderBodyClosedLhsDistinct v m k p a B) := by
  unfold inst0BinderClosedLhsDistinct inst0BinderBodyClosedLhsDistinct
  simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm]
  rw [closeAmbient_mkSigma]

theorem inst0BinderClosedTargetDistinct_sigma_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedTargetDistinct v m k p a (.sigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (inst0BinderClosedTargetDistinct v m k p a A)
    (inst0BinderBodyClosedTargetDistinct v m k p a B) := by
  unfold inst0BinderClosedTargetDistinct inst0BinderBodyClosedTargetDistinct
  simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm]
  rw [closeAmbient_mkSigma]
  simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSigma, applySubst, Nat.add_left_comm]

theorem inst0BinderTargetEqDistinct_lam_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {b : PureTm (((n + 1) + m) + 1)}
  (hb : inst0BinderBodyTargetEqDistinct v m k p a b) :
  inst0BinderTargetEqDistinct v m k p a (.lam b) := by
  unfold inst0BinderTargetEqDistinct
  rw [inst0BinderClosedLhsDistinct_lam_expand, inst0BinderClosedTargetDistinct_lam_expand]
  simpa [inst0BinderBodyTargetEqDistinct, inst0BinderBodyClosedLhsDistinct,
    inst0BinderBodyClosedTargetDistinct] using
  congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkLam hb

theorem inst0BinderTargetEqDistinct_pi_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A : PureTm ((n + 1) + m)} {B : PureTm (((n + 1) + m) + 1)}
  (hA : inst0BinderTargetEqDistinct v m k p a A)
  (hB : inst0BinderBodyTargetEqDistinct v m k p a B) :
  inst0BinderTargetEqDistinct v m k p a (.pi A B) := by
  unfold inst0BinderTargetEqDistinct at hA ⊢
  rw [inst0BinderClosedLhsDistinct_pi_expand, inst0BinderClosedTargetDistinct_pi_expand]
  simpa [inst0BinderBodyTargetEqDistinct, inst0BinderBodyClosedLhsDistinct,
    inst0BinderBodyClosedTargetDistinct] using
  >congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkPi hA hB

theorem inst0BinderTargetEqDistinct_sigma_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A : PureTm ((n + 1) + m)} {B : PureTm (((n + 1) + m) + 1)}
  (hA : inst0BinderTargetEqDistinct v m k p a A)
  (hB : inst0BinderBodyTargetEqDistinct v m k p a B) :

```

```

    inst0BinderTargetEqDistinct v m k p a (.sigma A B) := by
  unfold inst0BinderTargetEqDistinct at hA ⊢
  rw [inst0BinderClosedLhsDistinct_sigma_expand, inst0BinderClosedTargetDistinct_sigma_expand]
  simp [inst0BinderBodyTargetEqDistinct, inst0BinderBodyClosedLhsDistinct,
    inst0BinderBodyClosedTargetDistinct] using
    zcongrArg Mettapedia.Languages.MetTtA.Pure.Core.mkSigma hA hB

theorem inst0BinderTargetEqDistinct_u0
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderTargetEqDistinct v m k p a .u0 := by
  unfold inst0BinderTargetEqDistinct inst0BinderClosedLhsDistinct inst0BinderClosedTargetDistinct
  simp [quoteTmWith, subst]

theorem inst0BinderTargetEqDistinct_u1
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderTargetEqDistinct v m k p a .u1 := by
  unfold inst0BinderTargetEqDistinct inst0BinderClosedLhsDistinct inst0BinderClosedTargetDistinct
  simp [quoteTmWith, subst]

theorem inst0BinderTargetEqDistinct_id_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A 1a 1b : PureTm ((n + 1) + m)}
  (hA : inst0BinderTargetEqDistinct v m k p a A)
  (1ha : inst0BinderTargetEqDistinct v m k p a 1a)
  (1hb : inst0BinderTargetEqDistinct v m k p a 1b) :
  inst0BinderTargetEqDistinct v m k p a (.id A 1a 1b) := by
  unfold inst0BinderTargetEqDistinct at hA 1ha 1hb ⊢
  simp [inst0BinderClosedLhsDistinct, inst0BinderClosedTargetDistinct, quoteTmWith, subst] at hA
    1ha 1hb ⊢
  rw [hA, 1ha, 1hb]

theorem inst0BinderTargetEqDistinct_app_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {f b : PureTm ((n + 1) + m)}
  (hf : inst0BinderTargetEqDistinct v m k p a f)
  (hb : inst0BinderTargetEqDistinct v m k p a b) :
  inst0BinderTargetEqDistinct v m k p a (.app f b) := by
  unfold inst0BinderTargetEqDistinct at hf hb ⊢
  simp [inst0BinderClosedLhsDistinct, inst0BinderClosedTargetDistinct, quoteTmWith, subst] at hf
    hb ⊢
  rw [hf, hb]

theorem inst0BinderTargetEqDistinct_pair_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p q : PureTm ((n + 1) + m)}
  (hp : inst0BinderTargetEqDistinct v m k p a p)
  (hq : inst0BinderTargetEqDistinct v m k p a q) :
  inst0BinderTargetEqDistinct v m k p a (.pair p q) := by
  unfold inst0BinderTargetEqDistinct at hp hq ⊢
  simp [inst0BinderClosedLhsDistinct, inst0BinderClosedTargetDistinct, quoteTmWith, subst] at hp
    hq ⊢
  rw [hp, hq]

theorem inst0BinderTargetEqDistinct_fst_of

```

```

v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
{p : PureTm ((n + 1) + m)}
(hp : inst0BinderTargetEqDistinct v m k p a p) :
  inst0BinderTargetEqDistinct v m k p a (.fst p) := by
unfold inst0BinderTargetEqDistinct at hp ⊢
simp [inst0BinderClosedLhsDistinct, inst0BinderClosedTargetDistinct, quoteTmWith, subst] at hp
⊢
rw [hp]

theorem inst0BinderTargetEqDistinct_snd_of
v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
{p : PureTm ((n + 1) + m)}
(hp : inst0BinderTargetEqDistinct v m k p a p) :
  inst0BinderTargetEqDistinct v m k p a (.snd p) := by
unfold inst0BinderTargetEqDistinct at hp ⊢
simp [inst0BinderClosedLhsDistinct, inst0BinderClosedTargetDistinct, quoteTmWith, subst] at hp
⊢
rw [hp]

theorem inst0BinderTargetEqDistinct_refl_of
v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
{p : PureTm ((n + 1) + m)}
(hp : inst0BinderTargetEqDistinct v m k p a p) :
  inst0BinderTargetEqDistinct v m k p a (.refl p) := by
unfold inst0BinderTargetEqDistinct at hp ⊢
simp [inst0BinderClosedLhsDistinct, inst0BinderClosedTargetDistinct, quoteTmWith, subst] at hp
⊢
rw [hp]

theorem inst0AmbientDistinctClosedLhs_lam_expand
v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
(b : PureTm (((n + 1) + (m + e)) + 1)) :
  inst0AmbientDistinctClosedLhs e v m k p a (.lam b)
=
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (inst0AmbientDistinctClosedLhs (e + 1) v m k p a (castAmbientBody b)) := by
unfold inst0AmbientDistinctClosedLhs
simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm,
  castAmbientBody, castPureTm]
rw [closeAmbient_mkLam, closeAmbient_mkLam]

theorem inst0AmbientDistinctClosedTarget_lam_expand
v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
(b : PureTm (((n + 1) + (m + e)) + 1)) :
  inst0AmbientDistinctClosedTarget e v m k p a (.lam b)
=
  Mettapedia.Languages.MeTTa.Pure.Core.mkLam
    (inst0AmbientDistinctClosedTarget (e + 1) v m k p a (castAmbientBody b)) := by
unfold inst0AmbientDistinctClosedTarget
simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm,
  castAmbientBody, castPureTm]
rw [closeAmbient_mkLam, closeAmbient_mkLam]
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkLam, applySubst]

```

```

theorem inst0AmbientDistinctClosedLhs_pi_expand
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + (m + e))) (B : PureTm (((n + 1) + (m + e)) + 1)) :
  inst0AmbientDistinctClosedLhs e v m k p a (.pi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (inst0AmbientDistinctClosedLhs e v m k p a A)
    (inst0AmbientDistinctClosedLhs (e + 1) v m k p a (castAmbientBody B)) := by
  unfold inst0AmbientDistinctClosedLhs
simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm,
  castAmbientBody, castPureTm]
  rw [closeAmbient_mkPi, closeAmbient_mkPi]

theorem inst0AmbientDistinctClosedTarget_pi_expand
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + (m + e))) (B : PureTm (((n + 1) + (m + e)) + 1)) :
  inst0AmbientDistinctClosedTarget e v m k p a (.pi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (inst0AmbientDistinctClosedTarget e v m k p a A)
    (inst0AmbientDistinctClosedTarget (e + 1) v m k p a (castAmbientBody B)) := by
  unfold inst0AmbientDistinctClosedTarget
simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm,
  castAmbientBody, castPureTm]
  rw [closeAmbient_mkPi, closeAmbient_mkPi]
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkPi, applySubst]

theorem inst0AmbientDistinctClosedLhs_sigma_expand
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + (m + e))) (B : PureTm (((n + 1) + (m + e)) + 1)) :
  inst0AmbientDistinctClosedLhs e v m k p a (.sigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (inst0AmbientDistinctClosedLhs e v m k p a A)
    (inst0AmbientDistinctClosedLhs (e + 1) v m k p a (castAmbientBody B)) := by
  unfold inst0AmbientDistinctClosedLhs
simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm,
  castAmbientBody, castPureTm]
  rw [closeAmbient_mkSigma, closeAmbient_mkSigma]

theorem inst0AmbientDistinctClosedTarget_sigma_expand
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + (m + e))) (B : PureTm (((n + 1) + (m + e)) + 1)) :
  inst0AmbientDistinctClosedTarget e v m k p a (.sigma A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkSigma
    (inst0AmbientDistinctClosedTarget e v m k p a A)
    (inst0AmbientDistinctClosedTarget (e + 1) v m k p a (castAmbientBody B)) := by
  unfold inst0AmbientDistinctClosedTarget
simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm,
  castAmbientBody, castPureTm]
  rw [closeAmbient_mkSigma, closeAmbient_mkSigma]
simp [Mettapedia.Languages.MeTTa.Pure.Core.mkSigma, applySubst]

```

```

theorem inst0AmbientDistinctTargetEq_lam_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {b : PureTm ((n + 1) + (m + e)) + 1)}
  (hb : inst0AmbientDistinctTargetEq (e + 1) v m k p a (castAmbientBody b)) :
  inst0AmbientDistinctTargetEq e v m k p a (.lam b) := by
  unfold inst0AmbientDistinctTargetEq at hb ⊢
  rw [inst0AmbientDistinctClosedLhs_lam_expand, inst0AmbientDistinctClosedTarget_lam_expand]
  simpa using congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkLam hb

theorem inst0AmbientDistinctTargetEq_pi_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A : PureTm ((n + 1) + (m + e))} {B : PureTm ((n + 1) + (m + e)) + 1)}
  (hA : inst0AmbientDistinctTargetEq e v m k p a A)
  (hB : inst0AmbientDistinctTargetEq (e + 1) v m k p a (castAmbientBody B)) :
  inst0AmbientDistinctTargetEq e v m k p a (.pi A B) := by
  unfold inst0AmbientDistinctTargetEq at hA hB ⊢
  rw [inst0AmbientDistinctClosedLhs_pi_expand, inst0AmbientDistinctClosedTarget_pi_expand]
  simpa using >congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkPi hA hB

theorem inst0AmbientDistinctTargetEq_sigma_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A : PureTm ((n + 1) + (m + e))} {B : PureTm ((n + 1) + (m + e)) + 1)}
  (hA : inst0AmbientDistinctTargetEq e v m k p a A)
  (hB : inst0AmbientDistinctTargetEq (e + 1) v m k p a (castAmbientBody B)) :
  inst0AmbientDistinctTargetEq e v m k p a (.sigma A B) := by
  unfold inst0AmbientDistinctTargetEq at hA hB ⊢
  rw [inst0AmbientDistinctClosedLhs_sigma_expand, inst0AmbientDistinctClosedTarget_sigma_expand]
  simpa using >congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkSigma hA hB

theorem inst0AmbientDistinctTargetEq_u0
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0AmbientDistinctTargetEq e v m k p a .u0 := by
  unfold inst0AmbientDistinctTargetEq inst0AmbientDistinctClosedLhs
  inst0AmbientDistinctClosedTarget
calc
  closeAmbient e v k m
    (closeAmbient 0 v (k + m) e
      (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
        (subst (liftSubN (m + e) (subst0 a)) PureTm.u0)))
  = Mettapedia.Languages.MeTTa.Pure.Core.u0 := by
    rw [show closeAmbient 0 v (k + m) e
      (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
        (subst (liftSubN (m + e) (subst0 a)) PureTm.u0))
      = Mettapedia.Languages.MeTTa.Pure.Core.u0 by
        simpa [quoteTmWith, subst, buildEnv_succ, closeFVar,
          Mettapedia.Languages.MeTTa.Pure.Core.u0] using
          (closeAmbient_u0 0 v (k + m) e)]
    simp [closeAmbient_u0]
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    Mettapedia.Languages.MeTTa.Pure.Core.u0 := by
    simp [Mettapedia.Languages.MeTTa.Pure.Core.u0, applySubst]
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient e v (k + 1) m
      (closeAmbient 0 v (k + m + 1) e

```

```

      (quoteTmWith v (k + m + e + 1)
        (buildEnv v (k + 1) (m + e) (envCons v( k) p)) PureTm.u0))) := by
rw [show closeAmbient e v (k + 1) m
  (closeAmbient 0 v (k + m + 1) e
    (quoteTmWith v (k + m + e + 1)
      (buildEnv v (k + 1) (m + e) (envCons v( k) p)) PureTm.u0))
  = Mettapedialanguages.Metta.Pure.Core.u0 by
  rw [show closeAmbient 0 v (k + m + 1) e
    (quoteTmWith v (k + m + e + 1)
      (buildEnv v (k + 1) (m + e) (envCons v( k) p)) PureTm.u0)
    = Mettapedialanguages.Metta.Pure.Core.u0 by
  simp [quoteTmWith, buildEnv_succ, closeFVar,
    Mettapedialanguages.Metta.Pure.Core.u0] using
    (closeAmbient_u0 0 v (k + m + 1) e)]
  simp [closeAmbient_u0]]

theorem inst0AmbientDistinctTargetEq_u1
  v( : Nat → String) (e m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0AmbientDistinctTargetEq e v m k p a .u1 := by
unfold inst0AmbientDistinctTargetEq inst0AmbientDistinctClosedLhs
  inst0AmbientDistinctClosedTarget
calc
  closeAmbient e v k m
    (closeAmbient 0 v (k + m) e
      (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
        (subst (liftSubN (m + e) (subst0 a)) PureTm.u1)))
  = Mettapedialanguages.Metta.Pure.Core.u1 := by
  rw [show closeAmbient 0 v (k + m) e
    (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
      (subst (liftSubN (m + e) (subst0 a)) PureTm.u1))
    = Mettapedialanguages.Metta.Pure.Core.u1 by
    simp [quoteTmWith, subst, buildEnv_succ, closeFVar,
      Mettapedialanguages.Metta.Pure.Core.u1] using
      (closeAmbient_u1 0 v (k + m) e)]
    simp [closeAmbient_u1]
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    Mettapedialanguages.Metta.Pure.Core.u1 := by
  simp [Mettapedialanguages.Metta.Pure.Core.u1, applySubst]
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient e v (k + 1) m
      (closeAmbient 0 v (k + m + 1) e
        (quoteTmWith v (k + m + e + 1)
          (buildEnv v (k + 1) (m + e) (envCons v( k) p)) PureTm.u1))) := by
rw [show closeAmbient e v (k + 1) m
  (closeAmbient 0 v (k + m + 1) e
    (quoteTmWith v (k + m + e + 1)
      (buildEnv v (k + 1) (m + e) (envCons v( k) p)) PureTm.u1))
  = Mettapedialanguages.Metta.Pure.Core.u1 by
  rw [show closeAmbient 0 v (k + m + 1) e
    (quoteTmWith v (k + m + e + 1)
      (buildEnv v (k + 1) (m + e) (envCons v( k) p)) PureTm.u1)
    = Mettapedialanguages.Metta.Pure.Core.u1 by
    simp [quoteTmWith, buildEnv_succ, closeFVar,
      Mettapedialanguages.Metta.Pure.Core.u1] using

```

```

      (closeAmbient_ul 0 v (k + m + 1) e)]
    simp [closeAmbient_ul]]

theorem inst0AmbientDistinctClosedLhs_app_expand
  v( : Nat → String) (e m k : Nat) ρ( : QuoteEnv n) (a : PureTm n)
  (f b : PureTm ((n + 1) + (m + e))) :
  inst0AmbientDistinctClosedLhs e v m k ρ a (.app f b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp
    (inst0AmbientDistinctClosedLhs e v m k ρ a f)
    (inst0AmbientDistinctClosedLhs e v m k ρ a b) := by
  unfold inst0AmbientDistinctClosedLhs
  simp [quoteTmWith, subst, Nat.add_comm]

theorem inst0AmbientDistinctClosedTarget_app_expand
  v( : Nat → String) (e m k : Nat) ρ( : QuoteEnv n) (a : PureTm n)
  (f b : PureTm ((n + 1) + (m + e))) :
  inst0AmbientDistinctClosedTarget e v m k ρ a (.app f b)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkApp
    (inst0AmbientDistinctClosedTarget e v m k ρ a f)
    (inst0AmbientDistinctClosedTarget e v m k ρ a b) := by
  unfold inst0AmbientDistinctClosedTarget
  simp [quoteTmWith, Nat.add_assoc, Nat.add_comm]

theorem inst0AmbientDistinctTargetEq_app_of
  v{ : Nat → String} {e m k : Nat} ρ{ : QuoteEnv n} {a : PureTm n}
  {f b : PureTm ((n + 1) + (m + e))}
  (hf : inst0AmbientDistinctTargetEq e v m k ρ a f)
  (hb : inst0AmbientDistinctTargetEq e v m k ρ a b) :
  inst0AmbientDistinctTargetEq e v m k ρ a (.app f b) := by
  unfold inst0AmbientDistinctTargetEq at hf hb ⊢
  rw [inst0AmbientDistinctClosedLhs_app_expand, inst0AmbientDistinctClosedTarget_app_expand]
  simpa using >congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkApp hf hb

theorem inst0AmbientDistinctTargetEq_var
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ)
  (e m : Nat) (a : PureTm n) (i : Fin ((n + 1) + (m + e))) :
  inst0AmbientDistinctTargetEq e v m k ρ a (.var i) := by
  rw [inst0AmbientDistinctTargetEq_total_iff _ _ _ _ _ hcompat]
  exact inst0BinderTargetEqDistinct_var hcompat (m + e) a i

theorem inst0AmbientDistinctTargetEq_id_of
  v{ : Nat → String} {e m k : Nat} ρ{ : QuoteEnv n} {a : PureTm n}
  (hcompat : QuoteCompat v k ρ)
  {A ιa ιb : PureTm ((n + 1) + (m + e))}
  (hA : inst0AmbientDistinctTargetEq e v m k ρ a A)
  (ιha : inst0AmbientDistinctTargetEq e v m k ρ a ιa)
  (ιhb : inst0AmbientDistinctTargetEq e v m k ρ a ιb) :
  inst0AmbientDistinctTargetEq e v m k ρ a (.id A ιa ιb) := by
  rw [inst0AmbientDistinctTargetEq_total_iff _ _ _ _ _ hcompat] at hA ιha ιhb ⊢
  exact inst0BinderTargetEqDistinct_id_of hA ιha ιhb

```

```

theorem inst0AmbientDistinctTargetEq_pair_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  (hcompat : QuoteCompat v k p)
  {p q : PureTm ((n + 1) + (m + e))}
  (hp : inst0AmbientDistinctTargetEq e v m k p a p)
  (hq : inst0AmbientDistinctTargetEq e v m k p a q) :
  inst0AmbientDistinctTargetEq e v m k p a (.pair p q) := by
  rw [inst0AmbientDistinctTargetEq_total_iff _ _ _ _ hcompat] at hp hq ⊢
exact inst0BinderTargetEqDistinct_pair_of hp hq

```

```

theorem inst0AmbientDistinctTargetEq_fst_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  (hcompat : QuoteCompat v k p)
  {p : PureTm ((n + 1) + (m + e))}
  (hp : inst0AmbientDistinctTargetEq e v m k p a p) :
  inst0AmbientDistinctTargetEq e v m k p a (.fst p) := by
  rw [inst0AmbientDistinctTargetEq_total_iff _ _ _ _ hcompat] at hp ⊢
exact inst0BinderTargetEqDistinct_fst_of hp

```

```

theorem inst0AmbientDistinctTargetEq_snd_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  (hcompat : QuoteCompat v k p)
  {p : PureTm ((n + 1) + (m + e))}
  (hp : inst0AmbientDistinctTargetEq e v m k p a p) :
  inst0AmbientDistinctTargetEq e v m k p a (.snd p) := by
  rw [inst0AmbientDistinctTargetEq_total_iff _ _ _ _ hcompat] at hp ⊢
exact inst0BinderTargetEqDistinct_snd_of hp

```

```

theorem inst0AmbientDistinctTargetEq_refl_of
  v{ : Nat → String} {e m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  (hcompat : QuoteCompat v k p)
  {p : PureTm ((n + 1) + (m + e))}
  (hp : inst0AmbientDistinctTargetEq e v m k p a p) :
  inst0AmbientDistinctTargetEq e v m k p a (.refl p) := by
  rw [inst0AmbientDistinctTargetEq_total_iff _ _ _ _ hcompat] at hp ⊢
exact inst0BinderTargetEqDistinct_refl_of hp

```

```

theorem inst0AmbientDistinctTargetEq_all
  v( : Nat → String) :
  ∀ {n : Nat} (e m k : Nat) p( : QuoteEnv n) (a : PureTm n)
    (t : PureTm ((n + 1) + (m + e))),
    QuoteCompat v k p →
    inst0AmbientDistinctTargetEq e v m k p a t
  | _, e, m, k, p, a, .var i, hcompat =>
    inst0AmbientDistinctTargetEq_var hcompat e m a i
  | _, e, m, k, p, a, .const c, _ =>
    by
    unfold inst0AmbientDistinctTargetEq inst0AmbientDistinctClosedLhs
    inst0AmbientDistinctClosedTarget
    have hInnerL :
      closeAmbient 0 v (k + m) e (Pattern.apply (Lean.Name.toString c) []) =
        Pattern.apply (Lean.Name.toString c) [] := by
    simp [quoteConst] using
      (closeAmbient_quoteConst (d := 0) v( := v) (k := k + m) (m := e) c)

```



```

have hLeft :
  closeAmbient e v k m
  (closeAmbient 0 v (k + m) e (Pattern.apply (Lean.Name.toString c) [])) =
    Pattern.apply (Lean.Name.toString c) [] := by
  rw [hInnerL]
  simp [quoteConst] using
    (closeAmbient_quoteConst (d := e) v( := v) (k := k) (m := m) c)
have hInnerR0 :
  closeAmbient 0 v (k + m + 1) e (Pattern.apply (Lean.Name.toString c) []) =
    Pattern.apply (Lean.Name.toString c) [] := by
  simp [quoteConst] using
    (closeAmbient_quoteConst (d := 0) v( := v) (k := k + m + 1) (m := e) c)
have hInnerR :
  closeAmbient e v (k + 1) m
  (closeAmbient 0 v (k + m + 1) e (Pattern.apply (Lean.Name.toString c) [])) =
    Pattern.apply (Lean.Name.toString c) [] := by
  rw [hInnerR0]
  simp [quoteConst] using
    (closeAmbient_quoteConst (d := e) v( := v) (k := k + 1) (m := m) c)
have hRightExpanded :
  closeAmbient e v (k + 1) m
  (closeAmbient 0 v (k + m + 1) e
    (quoteTmWith v (k + m + e + 1)
      (buildEnv v (k + 1) (m + e) (envCons v( k) p)) (.const c))) =
    quoteConst c := by
  simp [quoteTmWith, quoteConst] using hInnerR
calc
  closeAmbient e v k m
  (closeAmbient 0 v (k + m) e
    (quoteTmWith v (k + m + e) (buildEnv v k (m + e) p)
      (subst (liftSubN (m + e) (subst0 a)) (.const c))))
  = quoteConst c := by
    exact hLeft
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (quoteConst c) := by
    simp
  _ = applySubst (SubstEnv.empty.extend v( k) (quoteTmWith v (k + m) p a))
    (closeAmbient e v (k + 1) m
      (closeAmbient 0 v (k + m + 1) e
        (quoteTmWith v (k + m + e + 1)
          (buildEnv v (k + 1) (m + e) (envCons v( k) p)) (.const c)))) := by
    rw [hRightExpanded]
| _, e, m, k, p, a, .u0, _ =>
  inst0AmbientDistinctTargetEq_u0 v e m k p a
| _, e, m, k, p, a, .u1, _ =>
  inst0AmbientDistinctTargetEq_u1 v e m k p a
| _, e, m, k, p, a, .id A 1a 1b, hcompat =>
  inst0AmbientDistinctTargetEq_id_of hcompat
  (inst0AmbientDistinctTargetEq_all v e m k p a A hcompat)
  (inst0AmbientDistinctTargetEq_all v e m k p a 1a hcompat)
  (inst0AmbientDistinctTargetEq_all v e m k p a 1b hcompat)
| _, e, m, k, p, a, .app f b, hcompat =>
  inst0AmbientDistinctTargetEq_app_of
  (inst0AmbientDistinctTargetEq_all v e m k p a f hcompat)

```

```

      (inst0AmbientDistinctTargetEq_all v e m k p a b hcompat)
| _, e, m, k, p, a, .pair p q, hcompat =>
  inst0AmbientDistinctTargetEq_pair_of hcompat
  (inst0AmbientDistinctTargetEq_all v e m k p a p hcompat)
  (inst0AmbientDistinctTargetEq_all v e m k p a q hcompat)
| _, e, m, k, p, a, .fst p, hcompat =>
  inst0AmbientDistinctTargetEq_fst_of hcompat
  (inst0AmbientDistinctTargetEq_all v e m k p a p hcompat)
| _, e, m, k, p, a, .snd p, hcompat =>
  inst0AmbientDistinctTargetEq_snd_of hcompat
  (inst0AmbientDistinctTargetEq_all v e m k p a p hcompat)
| _, e, m, k, p, a, .refl p, hcompat =>
  inst0AmbientDistinctTargetEq_refl_of hcompat
  (inst0AmbientDistinctTargetEq_all v e m k p a p hcompat)
| _, e, m, k, p, a, .lam b, hcompat =>
  inst0AmbientDistinctTargetEq_lam_of
  (inst0AmbientDistinctTargetEq_all v (e + 1) m k p a (castAmbientBody b) hcompat)
| _, e, m, k, p, a, .pi A B, hcompat =>
  inst0AmbientDistinctTargetEq_pi_of
  (inst0AmbientDistinctTargetEq_all v e m k p a A hcompat)
  (inst0AmbientDistinctTargetEq_all v (e + 1) m k p a (castAmbientBody B) hcompat)
| _, e, m, k, p, a, .sigma A B, hcompat =>
  inst0AmbientDistinctTargetEq_sigma_of
  (inst0AmbientDistinctTargetEq_all v e m k p a A hcompat)
  (inst0AmbientDistinctTargetEq_all v (e + 1) m k p a (castAmbientBody B) hcompat)
termination_by _ _ _ _ _ t _ => sizeOf t
decreasing_by
  all_goals
    simp [castAmbientBody, castPureTm]
    try omega

theorem inst0BinderTargetEq_lam_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {b : PureTm ((n + 1) + m) + 1)}
  (hb : inst0BinderBodyTargetEq v m k p a b) :
  inst0BinderTargetEq v m k p a (.lam b) := by
  unfold inst0BinderTargetEq
  rw [inst0BinderClosedLhs_lam_expand, inst0BinderClosedTarget_lam_expand]
  simp [inst0BinderBodyTargetEq, inst0BinderBodyClosedLhs, inst0BinderBodyClosedTarget] using
    congrArg Mettapedia.Languages.MeTTa.Pure.Core.mkLam hb

theorem inst0BinderClosedLhs_pi_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm ((n + 1) + m) + 1) :
  inst0BinderClosedLhs v m k p a (.pi A B)
  =
  Mettapedia.Languages.MeTTa.Pure.Core.mkPi
    (inst0BinderClosedLhs v m k p a A)
    (inst0BinderBodyClosedLhs v m k p a B) := by
  unfold inst0BinderClosedLhs inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm]
  rw [closeRange_eq_closeRangeAt_zero, closeRangeAt_mkPi]
  simp [closeRange_eq_closeRangeAt_zero]

```

```

theorem inst0BinderClosedTarget_pi_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedTarget v m k p a (.pi A B)
  =
  Mettapedia.Languages.Metta.Pure.Core.mkPi
    (inst0BinderClosedTarget v m k p a A)
    (inst0BinderBodyClosedTarget v m k p a B) := by
  unfold inst0BinderClosedTarget inst0BinderBodyClosedTarget
  simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm]
  rw [closeRange_mkPi_shifted]
  simp [closeRange_eq_closeRangeAt_zero, Mettapedia.Languages.Metta.Pure.Core.mkPi, applySubst,
    Nat.add_left_comm]

theorem inst0BinderClosedLhs_sigma_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedLhs v m k p a (.sigma A B)
  =
  Mettapedia.Languages.Metta.Pure.Core.mkSigma
    (inst0BinderClosedLhs v m k p a A)
    (inst0BinderBodyClosedLhs v m k p a B) := by
  unfold inst0BinderClosedLhs inst0BinderBodyClosedLhs
  simp [quoteTmWith, subst, buildEnv, liftSubN, Nat.add_assoc, Nat.add_comm]
  rw [closeRange_eq_closeRangeAt_zero, closeRangeAt_mkSigma]
  simp [closeRange_eq_closeRangeAt_zero]

theorem inst0BinderClosedTarget_sigma_expand
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n)
  (A : PureTm ((n + 1) + m)) (B : PureTm (((n + 1) + m) + 1)) :
  inst0BinderClosedTarget v m k p a (.sigma A B)
  =
  Mettapedia.Languages.Metta.Pure.Core.mkSigma
    (inst0BinderClosedTarget v m k p a A)
    (inst0BinderBodyClosedTarget v m k p a B) := by
  unfold inst0BinderClosedTarget inst0BinderBodyClosedTarget
  simp [quoteTmWith, buildEnv, Nat.add_assoc, Nat.add_comm]
  rw [closeRange_mkSigma_shifted]
  simp [closeRange_eq_closeRangeAt_zero, Mettapedia.Languages.Metta.Pure.Core.mkSigma, applySubst,
    Nat.add_left_comm]

theorem inst0BinderTargetEq_pi_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A : PureTm ((n + 1) + m)} {B : PureTm (((n + 1) + m) + 1)}
  (hA : inst0BinderTargetEq v m k p a A)
  (hB : inst0BinderBodyTargetEq v m k p a B) :
  inst0BinderTargetEq v m k p a (.pi A B) := by
  unfold inst0BinderTargetEq at hA ⊢
  rw [inst0BinderClosedLhs_pi_expand, inst0BinderClosedTarget_pi_expand]
  simpa [inst0BinderBodyTargetEq, inst0BinderBodyClosedLhs, inst0BinderBodyClosedTarget] using
    zcongrArg Mettapedia.Languages.Metta.Pure.Core.mkPi hA hB

theorem inst0BinderTargetEq_sigma_of

```

```

v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
{A : PureTm ((n + 1) + m)} {B : PureTm (((n + 1) + m) + 1)}
(hA : inst0BinderTargetEq v m k p a A)
(hB : inst0BinderBodyTargetEq v m k p a B) :
inst0BinderTargetEq v m k p a (.sigma A B) := by
unfold inst0BinderTargetEq at hA ⊢
rw [inst0BinderClosedLhs_sigma_expand, inst0BinderClosedTarget_sigma_expand]
simp [inst0BinderBodyTargetEq, inst0BinderBodyClosedLhs, inst0BinderBodyClosedTarget] using
  >congrArg Mettapedia.Languages.MetTa.Pure.Core.mkSigma hA hB

theorem inst0BinderTargetEq_u0
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderTargetEq v m k p a .u0 := by
simp [inst0BinderTargetEq, inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst]

theorem inst0BinderTargetEq_u1
  v( : Nat → String) (m k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  inst0BinderTargetEq v m k p a .u1 := by
simp [inst0BinderTargetEq, inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst]

theorem inst0BinderTargetEq_id_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {A 1a 1b : PureTm ((n + 1) + m)}
  (hA : inst0BinderTargetEq v m k p a A)
  (1ha : inst0BinderTargetEq v m k p a 1a)
  (1hb : inst0BinderTargetEq v m k p a 1b) :
  inst0BinderTargetEq v m k p a (.id A 1a 1b) := by
unfold inst0BinderTargetEq at hA 1ha 1hb ⊢
simp [inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst] at hA 1ha 1hb ⊢
rw [hA, 1ha, 1hb]

theorem inst0BinderTargetEq_app_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {f b : PureTm ((n + 1) + m)}
  (hf : inst0BinderTargetEq v m k p a f)
  (hb : inst0BinderTargetEq v m k p a b) :
  inst0BinderTargetEq v m k p a (.app f b) := by
unfold inst0BinderTargetEq at hf hb ⊢
simp [inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst] at hf hb ⊢
rw [hf, hb]

theorem inst0BinderTargetEq_pair_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p q : PureTm ((n + 1) + m)}
  (hp : inst0BinderTargetEq v m k p a p)
  (hq : inst0BinderTargetEq v m k p a q) :
  inst0BinderTargetEq v m k p a (.pair p q) := by
unfold inst0BinderTargetEq at hp hq ⊢
simp [inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst] at hp hq ⊢
rw [hp, hq]

theorem inst0BinderTargetEq_fst_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm ((n + 1) + m)}

```

```

    (hp : inst0BinderTargetEq v m k p a p) :
      inst0BinderTargetEq v m k p a (.fst p) := by
    unfold inst0BinderTargetEq at hp ⊢
    simp [inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst] at hp ⊢
    rw [hp]

theorem inst0BinderTargetEq_snd_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm ((n + 1) + m)}
  (hp : inst0BinderTargetEq v m k p a p) :
    inst0BinderTargetEq v m k p a (.snd p) := by
  unfold inst0BinderTargetEq at hp ⊢
  simp [inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst] at hp ⊢
  rw [hp]

theorem inst0BinderTargetEq_refl_of
  v{ : Nat → String} {m k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  {p : PureTm ((n + 1) + m)}
  (hp : inst0BinderTargetEq v m k p a p) :
    inst0BinderTargetEq v m k p a (.refl p) := by
  unfold inst0BinderTargetEq at hp ⊢
  simp [inst0BinderClosedLhs, inst0BinderClosedTarget, quoteTmWith, subst] at hp ⊢
  rw [hp]

end Mettapedia.Languages.MeTTa.PureKernel.PatternBridge.Inst0BridgeProof

```

A.12 InductiveDecl.lean

Mettapedia/Languages/MeTTa/PureKernel/InductiveDecl.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec
import Mettapedia.Languages.MeTTa.PureKernel.UnitDecl
import Mettapedia.Languages.MeTTa.PureKernel.BoolDecl
import Mettapedia.Languages.MeTTa.PureKernel.NatDecl

/--! LLM primer:
- `DeclName` := Lean.Name` has `DecidableEq` and `BEq`
- `PureTm n` has `DecidableEq`
- `checkIndDecl` is the Lean-style `addInductive` analogue for our kernel
- No telescopes yet – this handles non-parametric, non-indexed families (Unit, Bool, Nat)
- Recursors are a separate concern (see RecursorDecl.lean)
-!-/

namespace Mettapedia.Languages.MeTTa.PureKernel.IndDecl

set_option linter.dupNamespace false

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

/--! ## Input types -!
/-- A single constructor specification for an inductive family. -!


```

```

structure CtorSpec where
  name : DeclName
  /-- Constructor type as a closed term. For a non-parametric type `T`,
     this has the form  $\prod (iA : \iota A) \dots (\kappa A : \kappa A). T$ . -/
  type : PureTm 0
deriving DecidableEq, Repr

/-- An inductive type declaration (non-parametric, non-indexed).

- positive example: `{ typeName := `Nat, ctors := [{name := `Nat.zero, type := const `Nat},
  {name := `Nat.succ, type := pi (const `Nat) (const `Nat)}] }`
- negative example: a declaration where a constructor argument type mentions the
  family name negatively, e.g., `{..., type := pi (pi (const `Bad) u0) (const `Bad)} ` -/
structure IndDecl where
  /-- Name of the type former (e.g., `Nat). -/
  typeName : DeclName
  /-- Constructor specifications. -/
  ctors : List CtorSpec
deriving Repr

/--! ## Validation predicates -/

/-- Check that the declaration name `c` does not occur as a `const` in `t`. -/
def notOccursConst (c : DeclName) : PureTm n → Bool
| .var _ => true
| .const d => d != c
| .u0 => true
| .u1 => true
| .pi A B => notOccursConst c A && notOccursConst c B
| .sigma A B => notOccursConst c A && notOccursConst c B
| .id A a b => notOccursConst c A && notOccursConst c a && notOccursConst c b
| .lam body => notOccursConst c body
| .app f a => notOccursConst c f && notOccursConst c a
| .pair a b => notOccursConst c a && notOccursConst c b
| .fst p => notOccursConst c p
| .snd p => notOccursConst c p
| .refl a => notOccursConst c a

/-- Check that the family name occurs only positively within a type.

In  $\prod (a : A). B$ : the family name must NOT occur in `A` (contravariant
position), but may occur positively in `B`.

At a non-Pi position, any occurrence is positive – returns `true`.

- positive example: `occursOnlyPositively `Nat (.const `Nat) = true`
- negative example: `occursOnlyPositively `Nat (.pi (.const `Nat) .u0) = false` -/
def occursOnlyPositively (familyName : DeclName) : PureTm n → Bool
| .pi A B => notOccursConst familyName A && occursOnlyPositively familyName B
| _ => true

/-- Check strict positivity of the family name in a constructor type.

For  $\prod (a : A). B$ : the family name may appear in `A` but only positively

```

(not on the left of any arrow within `A`), and must be strictly positive in `B`.

At the target position (non- Π head), anything is allowed.

```
- positive example:  $\Pi$  `(n : Nat). Nat` - `Nat` appears directly in arg, ok
- negative example:  $\Pi$  `(f : Nat  $\rightarrow$  U0). Nat` - `Nat` on left of arrow in arg -/
def strictlyPositive (familyName : DeclName) : PureTm n  $\rightarrow$  Bool
| .pi A B => occursOnlyPositively familyName A && strictlyPositive familyName B
| _ => true
```

Check that a constructor type targets the family: peel off all outermost Π -binders and verify the resulting head is `const familyName`. -/

```
def targetsFamily (familyName : DeclName) : PureTm n  $\rightarrow$  Bool
| .pi _ B => targetsFamily familyName B
| .const c => c == familyName
| _ => false
```

Check that all names in a list are distinct. -/

```
def namesDistinct : List DeclName  $\rightarrow$  Bool
| [] => true
| a :: rest => !rest.contains a && namesDistinct rest
```

#!/ ## The checker -/

Check an inductive declaration and emit `DeclSpec`s if valid.

Validates:

1. All names (type + constructors) are distinct
2. Each constructor type targets the family
3. Strict positivity: family name does not appear in constructor argument types

Returns the type spec (with type `U0`) followed by one spec per constructor.

Does NOT generate recursors or aliases – those are separate concerns.

```
- positive example: `checkIndDecl unitDecl = some [unitTySpec, unitCtorSpec]`
- negative example: a declaration with duplicate names returns `none` -/
```

```
def checkIndDecl (decl : IndDecl) : Option (List DeclSpec) :=
  let allNames := decl.typeName :: decl.ctors.map CtorSpec.name
  if !namesDistinct allNames then none
  else
    let ctorsValid := decl.ctors.all fun ctor =>
      targetsFamily decl.typeName ctor.type &&
      strictlyPositive decl.typeName ctor.type
    if !ctorsValid then none
    else
      let tySpec : DeclSpec := { name := decl.typeName, type := .u0 }
      let ctorSpecs : List DeclSpec := decl.ctors.map fun ctor =>
        { name := ctor.name, type := ctor.type }
      some (tySpec :: ctorSpecs)
```

#!/ ## Standard declarations as `IndDecl` -/

```
def unitDecl : IndDecl :=
  { typeName := `Unit
```

```

    ctors := [{ name := `Unit.unit, type := .const `Unit }] }

def boolDecl : IndDecl :=
{ typeName := `Bool
  ctors := [{ name := `Bool.true, type := .const `Bool },
    { name := `Bool.false, type := .const `Bool }] }

def natDecl : IndDecl :=
{ typeName := `Nat
  ctors := [{ name := `Nat.zero, type := .const `Nat },
    { name := `Nat.succ, type := .pi (.const `Nat) (.const `Nat) }] }

/-! ## Checker passes on standard declarations -/

theorem check_unitDecl :
  checkIndDecl unitDecl = some
    [{ name := `Unit, type := .u0 },
      { name := `Unit.unit, type := .const `Unit }] := by
decide

theorem check_boolDecl :
  checkIndDecl boolDecl = some
    [{ name := `Bool, type := .u0 },
      { name := `Bool.true, type := .const `Bool },
      { name := `Bool.false, type := .const `Bool }] := by
decide

theorem check_natDecl :
  checkIndDecl natDecl = some
    [{ name := `Nat, type := .u0 },
      { name := `Nat.zero, type := .const `Nat },
      { name := `Nat.succ, type := .pi (.const `Nat) (.const `Nat) }] := by
decide

/-! ## Negative examples -/

/-- A declaration with duplicate names is rejected. -/
def badDupDecl : IndDecl :=
{ typeName := `Bad
  ctors := [{ name := `Bad, type := .const `Bad }] }

theorem check_badDupDecl : checkIndDecl badDupDecl = none := by decide

/-- A declaration with a negatively-occurring family name is rejected. -/
def badNegDecl : IndDecl :=
{ typeName := `Bad
  ctors := [{ name := `Bad.mk,
    type := .pi (.pi (.const `Bad) .u0) (.const `Bad) }] }

theorem check_badNegDecl : checkIndDecl badNegDecl = none := by decide

/-- A constructor that doesn't target the family is rejected. -/
def badTargetDecl : IndDecl :=
{ typeName := `Bad

```



```

    ctors := [{ name := `Bad.mk, type := .u0 }] }

theorem check_badTargetDecl : checkIndDecl badTargetDecl = none := by decide

/-! ## Agreement with existing hand-written specs -/

open Mettapedia.Languages.MeTTa.PureKernel.UnitDecl in
theorem unitDecl_specs_agree :
  checkIndDecl unitDecl = some [unitTySpec, unitCtorSpec] := by decide

open Mettapedia.Languages.MeTTa.PureKernel.BoolDecl in
theorem boolDecl_specs_agree :
  checkIndDecl boolDecl = some [boolTySpec, boolTrueSpec, boolFalseSpec] := by decide

open Mettapedia.Languages.MeTTa.PureKernel.NatDecl in
theorem natDecl_specs_agree :
  checkIndDecl natDecl = some [natTySpec, natZeroSpec, natSuccSpec] := by decide

end Mettapedia.Languages.MeTTa.PureKernel.IndDecl

```

A.13 NatDecl.lean

Mettapedia/Languages/MeTTa/PureKernel/NatDecl.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

namespace Mettapedia.Languages.MeTTa.PureKernel.NatDecl

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

/-- Pilot family/type constant name for Nat in the declaration environment. -/
def natTyName : DeclName := `Nat

/-- Pilot constructor names for Nat (Lean-style naming). -/
def natZeroName : DeclName := `Nat.zero
def natSuccName : DeclName := `Nat.succ

/-- Pilot alias constant used to witness declaration-aware unfolding 6(`). -/
def natAliasName : DeclName := `Nat.alias

/-- `Nat` family/type declaration spec. -/
def natTySpec : DeclSpec :=
  { name := natTyName, type := .u0 }

/-- `Nat.zero` constructor declaration spec. -/
def natZeroSpec : DeclSpec :=
  { name := natZeroName, type := .const natTyName }

/-- `Nat.succ` constructor declaration spec. -/
def natSuccSpec : DeclSpec :=

```

```

{ name := natSuccName, type := .pi (.const natTyName) (.const natTyName) }

/-- `Nat.alias` declaration spec used for the first δ witness. -/
def natAliasSpec : DeclSpec :=
  { name := natAliasName, type := .const natTyName, value? := some (.const natZeroName) }

/-- `Nat` pilot declarations expressed through the shared scaffold. -/
def natSpecs : List DeclSpec :=
  [natTySpec, natZeroSpec, natSuccSpec, natAliasSpec]

/-- Minimal declaration environment for the Nat pilot.
`Nat`, `Nat.zero`, and `Nat.succ` are declared; `Nat.alias` unfolds to `Nat.zero`. -/
def natDeclEnv : DeclEnv := envOfSpecs natSpecs

@[simp] theorem typeOf_natTy :
  typeOf? natDeclEnv natTyName = some .u0 := by
  decide

@[simp] theorem typeOf_natZero :
  typeOf? natDeclEnv natZeroName = some (.const natTyName) := by
  decide

@[simp] theorem typeOf_natSucc :
  typeOf? natDeclEnv natSuccName = some (.pi (.const natTyName) (.const natTyName)) := by
  decide

@[simp] theorem typeOf_natAlias :
  typeOf? natDeclEnv natAliasName = some (.const natTyName) := by
  decide

@[simp] theorem valueOf_natAlias :
  valueOf? natDeclEnv natAliasName = some (.const natZeroName) := by
  decide

/-- Typed declaration witness:  $\vdash \text{Nat} : U0$ . -/
theorem hasType_natTy :
  HasTypeDecl natDeclEnv .nil ((.const natTyName : PureTm 0)) .u0 :=
  hasType_const_from_lookup (E := natDeclEnv)  $\Gamma$  (:= .nil) (c := natTyName) (A0 := .u0) (by
    simp)

/-- Typed declaration witness:  $\vdash \text{Nat.zero} : \text{Nat}$ . -/
theorem hasType_natZero :
  HasTypeDecl natDeclEnv .nil ((.const natZeroName : PureTm 0)) (.const natTyName) :=
  hasType_const_from_lookup (E := natDeclEnv)  $\Gamma$  (:= .nil) (c := natZeroName) (A0 := .const
    natTyName) (by
    simp)

/-- Typed declaration witness:  $\vdash \text{Nat.succ} : \text{Nat} \rightarrow \text{Nat}$ . -/
theorem hasType_natSucc :
  HasTypeDecl natDeclEnv .nil
    ((.const natSuccName : PureTm 0))
    (.pi (.const natTyName) (.const natTyName)) :=
  hasType_const_from_lookup
    (E := natDeclEnv)  $\Gamma$  (:= .nil) (c := natSuccName) (A0 := .pi (.const natTyName) (.const

```

```

    natTyName)) (by
      simp)

/-- Typed declaration witness:  $\vdash` \text{Nat.alias} : \text{Nat}`$ . -/
theorem hasType_natAlias :
  HasTypeDecl natDeclEnv .nil ((.const natAliasName : PureTm 0)) (.const natTyName) :=
  hasType_const_from_lookup (E := natDeclEnv)  $\Gamma$ ( := .nil) (c := natAliasName) (A0 := .const
    natTyName) (by
      simp)

/-- Reduction witness:  $\text{Nat.alias}$  unfolds to  $\text{Nat.zero}$ . -/
theorem red_natAlias_to_zero :
  RedDecl natDeclEnv
    ((.const natAliasName : PureTm 0))
    ((.const natZeroName : PureTm 0)) := by
  simp using
    red_const_from_unfold0 (E := natDeclEnv) (c := natAliasName) (v := (.const natZeroName)) (by
      simp)

/-- Small checked pilot package: typed source, one declaration-aware step, typed target. -/
theorem natAlias_checked_step :
   $\exists A : \text{PureTm } 0,$ 
  HasTypeDecl natDeclEnv .nil ((.const natAliasName : PureTm 0)) A  $\wedge$ 
  RedDecl natDeclEnv ((.const natAliasName : PureTm 0)) ((.const natZeroName : PureTm 0))  $\wedge$ 
  HasTypeDecl natDeclEnv .nil ((.const natZeroName : PureTm 0)) A := by
  simp using
    (checked_delta_step_closed_of_typed_value
      (E := natDeclEnv)
      (c := natAliasName)
      (A0 := (.const natTyName))
      (v0 := (.const natZeroName))
      (by simp)
      (by simp)
      (by simp using hasType_natZero))

/-- Reusable typed-value obligations for  $\text{natSpecs}$  in any declaration
environment where  $\text{Nat.zero} : \text{Nat}$  is available. -/
theorem natSpecs_hTyped_in
  (E : DeclEnv)
  (hZeroType : typeOf? E natZeroName = some (.const natTyName)) :
   $\forall s \in \text{natSpecs}, \forall v0 : \text{PureTm } 0,$ 
  s.value? = some v0  $\rightarrow$ 
  HasTypeDecl E .nil (liftClosed v0) (liftClosed s.type) := by
  intro s hs v0 hVal
  simp [natSpecs, natTySpec, natZeroSpec, natSuccSpec, natAliasSpec] at hs
  rcases hs with rfl | rfl | rfl | rfl
  · simp at hVal
  · simp at hVal
  · simp at hVal
  · have hv : v0 = (.const natZeroName) := by
      simp [natAliasSpec] using hVal.symm
    subst hv
    simp [liftClosed_zero] using
      (hasType_const_from_lookup

```

```

      (E := E)
       $\Gamma$ ( := .nil)
      (c := natZeroName)
      (A0 := (.const natTyName))
      hZeroType)

/-- Reusable no-self-unfolding obligations for `natSpecs`. -/
theorem natSpecs_hNoSelf :
   $\forall$  s  $\in$  natSpecs,  $\forall$  v0 : PureTm 0,
    s.value? = some v0  $\rightarrow$ 
    v0  $\neq$  (.const s.name) := by
  intro s hs v0 hVal
  simp [natSpecs, natTySpec, natZeroSpec, natSuccSpec, natAliasSpec] at hs
  rcases hs with rfl | rfl | rfl | rfl
  · simp at hVal
  · simp at hVal
  · simp at hVal
  · have hv : v0 = (.const natZeroName) := by
      simpa [natAliasSpec] using hVal.symm
    subst hv
    intro hEq
    have hNe : ((.const natZeroName : PureTm 0))  $\neq$  (.const natAliasName) := by
      decide
    exact hNe hEq

def natSpecs_obligations : DeclSpecObligations natSpecs where
  valuesWellTyped :=
    natSpecs_hTyped_in
    (E := envOfSpecs natSpecs)
    (hZeroType := by decide)
  noSelfDelta := natSpecs_hNoSelf

def natSpecs_signatureWellFormed : SignatureWellFormed natSpecs where
  noShadowing := by decide
  obligations := natSpecs_obligations

theorem natTySpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [] natTySpec where
  fresh := by simp [natTySpec]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames, natTySpec]
  valueUsesEarlier := by
    intro v0 hVal
    simp [natTySpec] at hVal
  valueWellTyped := by
    intro v0 hVal
    simp [natTySpec] at hVal
  noSelfDelta := by
    intro v0 hVal
    simp [natTySpec] at hVal

theorem natZeroSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [natTySpec] natZeroSpec where
  fresh := by simp [natTySpec, natZeroSpec, natTyName, natZeroName]

```

```

typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames, natTySpec, natZeroSpec, natTyName, natZeroName]
valueUsesEarlier := by
  intro v0 hVal
  simp [natZeroSpec] at hVal
valueWellTyped := by
  intro v0 hVal
  simp [natZeroSpec] at hVal
noSelfDelta := by
  intro v0 hVal
  simp [natZeroSpec] at hVal

theorem natSuccSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [natTySpec, natZeroSpec] natSuccSpec where
fresh := by
  simp [natTySpec, natZeroSpec, natSuccSpec, natTyName, natZeroName, natSuccName]
typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames,
    natTySpec, natZeroSpec, natSuccSpec, natTyName, natZeroName, natSuccName]
valueUsesEarlier := by
  intro v0 hVal
  simp [natSuccSpec] at hVal
valueWellTyped := by
  intro v0 hVal
  simp [natSuccSpec] at hVal
noSelfDelta := by
  intro v0 hVal
  simp [natSuccSpec] at hVal

theorem natAliasSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [natTySpec, natZeroSpec, natSuccSpec] natAliasSpec where
fresh := by
  simp [natTySpec, natZeroSpec, natSuccSpec, natAliasSpec,
    natTyName, natZeroName, natSuccName, natAliasName]
typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames,
    natTySpec, natZeroSpec, natSuccSpec, natAliasSpec,
    natTyName, natZeroName, natSuccName, natAliasName]
valueUsesEarlier := by
  intro v0 hVal
  have hv : v0 = (.const natZeroName) := by
    simpa [natAliasSpec] using hVal.symm
  subst hv
  simp [UsesOnlyDeclNamesFrom, prefixNames,
    natTySpec, natZeroSpec, natSuccSpec, natZeroName]
valueWellTyped := by
  intro v0 hVal
  have hv : v0 = (.const natZeroName) := by
    simpa [natAliasSpec] using hVal.symm
  subst hv
  have hLookup :
    typeOf? (envOfSpecs [natTySpec, natZeroSpec, natSuccSpec]) natZeroName =
      some (.const natTyName) :=
    typeOf_envOfSpecs_eq_of_mem_of_nodup

```

```

    (specs := [natTySpec, natZeroSpec, natSuccSpec])
    (s := natZeroSpec)
    (hNodup := by decide)
    (hs := by simp [natTySpec, natZeroSpec, natSuccSpec])
  simpa [liftClosed_zero] using
    (hasType_const_from_lookup
      (E := envOfSpecs [natTySpec, natZeroSpec, natSuccSpec])
      Γ( := .nil)
      (c := natZeroName)
      (A0 := (.const natTyName))
      hLookup)
  noSelfDelta := by
    intro v0 hVal
    have hv : v0 = (.const natZeroName) := by
      simpa [natAliasSpec] using hVal.symm
    subst hv
    intro hEq
    have hNe : ((.const natZeroName : PureTm 0)) ≠ (.const natAliasName) := by
      decide
    exact hNe hEq

theorem natSpecs_prefixWellFormed :
  SignatureWellFormedPrefix natSpecs := by
  refine And.intro natTySpec_prefixAdmissible ?_
  refine And.intro ?_ ?_
  · simpa [natSpecs, natTySpec, natZeroSpec]
    using natZeroSpec_prefixAdmissible
  · refine And.intro ?_ ?_
    · simpa [PrefixSignatureWellFormed, natSpecs, natTySpec, natZeroSpec, natSuccSpec]
      using natSuccSpec_prefixAdmissible
    · refine And.intro ?_ trivial
      simpa [PrefixSignatureWellFormed, natSpecs, natTySpec, natZeroSpec, natSuccSpec,
        natAliasSpec]
        using natAliasSpec_prefixAdmissible

theorem natSpecs_signatureWellFormed_fromPrefix :
  SignatureWellFormed natSpecs :=
  SignatureWellFormed.ofPrefix natSpecs_prefixWellFormed natSpecs_obligations

theorem natDeclEnv_wellFormed : DeclEnvWellFormed natDeclEnv := by
  unfold natDeclEnv
  exact natSpecs_signatureWellFormed.toDeclEnvWellFormed

end Mettapedata.Languages.MeTTa.PureKernel.NatDecl

```

A.14 Parallel.lean

Mettapedata/Languages/MeTTa/PureKernel/Parallel.lean

```

import Mettapedata.Languages.MeTTa.PureKernel.Reduction

namespace Mettapedata.Languages.MeTTa.PureKernel.Parallel

open Mettapedata.Languages.MeTTa.PureKernel.Syntax

```

```

open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.Reduction

/-- Parallel one-step reduction for the Pure kernel. -/
inductive ParRed : PureTm n → PureTm n → Prop where
| var (i : Fin n) : ParRed (.var i) (.var i)
| const (c : DeclName) : ParRed (.const c : PureTm n) (.const c)
| u0 : ParRed (.u0 : PureTm n) .u0
| u1 : ParRed (.u1 : PureTm n) .u1
| pi {A A' : PureTm n} {B B' : PureTm (n + 1)} :
  ParRed A A' → ParRed B B' → ParRed (.pi A B) (.pi A' B')
| sigma {A A' : PureTm n} {B B' : PureTm (n + 1)} :
  ParRed A A' → ParRed B B' → ParRed (.sigma A B) (.sigma A' B')
| id {A A' a a' b b' : PureTm n} :
  ParRed A A' → ParRed a a' → ParRed b b' → ParRed (.id A a b) (.id A' a' b')
| lam {b b' : PureTm (n + 1)} :
  ParRed b b' → ParRed (.lam b) (.lam b')
| app {f f' a a' : PureTm n} :
  ParRed f f' → ParRed a a' → ParRed (.app f a) (.app f' a')
| pair {a a' b b' : PureTm n} :
  ParRed a a' → ParRed b b' → ParRed (.pair a b) (.pair a' b')
| fst {p p' : PureTm n} :
  ParRed p p' → ParRed (.fst p) (.fst p')
| snd {p p' : PureTm n} :
  ParRed p p' → ParRed (.snd p) (.snd p')
| refl {a a' : PureTm n} :
  ParRed a a' → ParRed (.refl a) (.refl a')
| betaPi {body body' : PureTm (n + 1)} {a a' : PureTm n} :
  ParRed body body' → ParRed a a' →
  ParRed (.app (.lam body) a) (inst0 a' body')
| betaSigmaFst {a a' b b' : PureTm n} :
  ParRed a a' → ParRed b b' →
  ParRed (.fst (.pair a b)) a'
| betaSigmaSnd {a a' b b' : PureTm n} :
  ParRed a a' → ParRed b b' →
  ParRed (.snd (.pair a b)) b'

@[simp] theorem par_refl : ∀ t : PureTm n, ParRed t t := by
  intro t
  induction t with
  | var i => exact .var i
  | const c => exact .const c
  | u0 => exact .u0
  | u1 => exact .u1
  | pi A B ihA ihB => exact .pi ihA ihB
  | sigma A B ihA ihB => exact .sigma ihA ihB
  | id A a b ihA iha ihb => exact .id ihA iha ihb
  | lam b ih => exact .lam ih
  | app f a ihf iha => exact .app ihf iha
  | pair a b iha ihb => exact .pair iha ihb
  | fst p ih => exact .fst ih
  | snd p ih => exact .snd ih
  | refl a iha => exact .refl iha

```

```

theorem red_to_par {t u : PureTm n} (h : Red t u) : ParRed t u := by
  induction h with
  | betaPi body a =>
    exact .betaPi (par_refl body) (par_refl a)
  | betaSigmaFst a b =>
    exact .betaSigmaFst (par_refl a) (par_refl b)
  | betaSigmaSnd a b =>
    exact .betaSigmaSnd (par_refl a) (par_refl b)
  | congPiDom h ih =>
    exact .pi ih (par_refl _)
  | congPiCod h ih =>
    exact .pi (par_refl _) ih
  | congSigmaDom h ih =>
    exact .sigma ih (par_refl _)
  | congSigmaCod h ih =>
    exact .sigma (par_refl _) ih
  | congIdTy h ih =>
    exact .id ih (par_refl _) (par_refl _)
  | congIdLeft h ih =>
    exact .id (par_refl _) ih (par_refl _)
  | congIdRight h ih =>
    exact .id (par_refl _) (par_refl _) ih
  | congLam h ih =>
    exact .lam ih
  | congAppFun h ih =>
    exact .app ih (par_refl _)
  | congAppArg h ih =>
    exact .app (par_refl _) ih
  | congPairFst h ih =>
    exact .pair ih (par_refl _)
  | congPairSnd h ih =>
    exact .pair (par_refl _) ih
  | congFst h ih =>
    exact .fst ih
  | congSnd h ih =>
    exact .snd ih
  | congRefl h ih =>
    exact .refl ih

theorem par_rename {n m : Nat} p( : Ren n m) {t u : PureTm n}
  (h : ParRed t u) : ParRed (rename p t) (rename p u) := by
  induction h generalizing m with
  | var i =>
    exact .var p( i)
  | const c =>
    exact .const c
  | u0 =>
    exact .u0
  | u1 =>
    exact .u1
  | pi hA hB ihA ihB =>
    simpa [rename] using .pi (ihA p( := p)) (ihB p( := liftRen p))
  | sigma hA hB ihA ihB =>

```



```

    simpa [rename] using .sigma (ihA ρ( := ρ)) (ihB ρ( := liftRen ρ))
| id hA ha hb ihA iha ihb =>
    simpa [rename] using .id (ihA ρ( := ρ)) (iha ρ( := ρ)) (ihb ρ( := ρ))
| lam hb ih =>
    simpa [rename] using .lam (ih ρ( := liftRen ρ))
| app hf ha ihf iha =>
    simpa [rename] using .app (ihf ρ( := ρ)) (iha ρ( := ρ))
| pair ha hb iha ihb =>
    simpa [rename] using .pair (iha ρ( := ρ)) (ihb ρ( := ρ))
| fst hp ih =>
    simpa [rename] using .fst (ih ρ( := ρ))
| snd hp ih =>
    simpa [rename] using .snd (ih ρ( := ρ))
| refl ha ih =>
    simpa [rename] using .refl (ih ρ( := ρ))
| betaPi hbody ha ihbody iha =>
    simpa [rename, rename_inst0] using
      (ParRed.betaPi (ihbody ρ( := liftRen ρ)) (iha ρ( := ρ)))
| betaSigmaFst ha hb iha ihb =>
    simpa [rename] using
      (ParRed.betaSigmaFst (iha ρ( := ρ)) (ihb ρ( := ρ)))
| betaSigmaSnd ha hb iha ihb =>
    simpa [rename] using
      (ParRed.betaSigmaSnd (iha ρ( := ρ)) (ihb ρ( := ρ)))

```

```

theorem par_subst {n m : Nat} {σ' : Sub n m}
  (oh : ∀ i, ParRed σ( i) σ(' i))
  {t u : PureTm n} (h : ParRed t u) :
  ParRed (subst σ t) (subst σ' u) := by
induction h generalizing m with
| var i =>
  exact oh i
| const c =>
  exact .const c
| u0 =>
  exact .u0
| u1 =>
  exact .u1
| pi hA hB ihA ihB =>
  have ohlift : ∀ i, ParRed (liftSub σ i) (liftSub σ' i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .var 0
    · intro j
      simpa [liftSub] using par_rename ρ( := wk) (oh j)
      simpa [subst] using .pi (ihA oh) (ihB ohlift)
| sigma hA hB ihA ihB =>
  have ohlift : ∀ i, ParRed (liftSub σ i) (liftSub σ' i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .var 0
    · intro j
      simpa [liftSub] using par_rename ρ( := wk) (oh j)
      simpa [subst] using .sigma (ihA oh) (ihB ohlift)

```

```

| id hA ha hb ihA iha ihb =>
  simpa [subst] using .id (ihA oh) (iha oh) (ihb oh)
| lam hb ih =>
  have ohlift :  $\forall i$ , ParRed (liftSub  $\sigma$  i) (liftSub  $\sigma'$  i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .var 0
    · intro j
      simpa [liftSub] using par_rename  $\rho$ ( := wk) (oh j)
      simpa [subst] using .lam (ih ohlift)
| app hf ha ihf iha =>
  simpa [subst] using .app (ihf oh) (iha oh)
| pair ha hb iha ihb =>
  simpa [subst] using .pair (iha oh) (ihb oh)
| fst hp ih =>
  simpa [subst] using .fst (ih oh)
| snd hp ih =>
  simpa [subst] using .snd (ih oh)
| refl ha ih =>
  simpa [subst] using .refl (ih oh)
| betaPi hbody ha ihbody iha =>
  have ohlift :  $\forall i$ , ParRed (liftSub  $\sigma$  i) (liftSub  $\sigma'$  i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact .var 0
    · intro j
      simpa [liftSub] using par_rename  $\rho$ ( := wk) (oh j)
      simpa [subst, subst_inst0] using (ParRed.betaPi (ihbody ohlift) (iha oh))
| betaSigmaFst ha hb iha ihb =>
  simpa [subst] using (ParRed.betaSigmaFst (iha oh) (ihb oh))
| betaSigmaSnd ha hb iha ihb =>
  simpa [subst] using (ParRed.betaSigmaSnd (iha oh) (ihb oh))

theorem par_inst0 {a a' : PureTm n} {b b' : PureTm (n + 1)}
  (ha : ParRed a a') (hb : ParRed b b') :
  ParRed (inst0 a b) (inst0 a' b') := by
  have oh :  $\forall i$ , ParRed (subst0 a i) (subst0 a' i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · simpa using ha
    · intro j
      exact .var j
  simpa [inst0] using (par_subst  $\sigma$ ( := subst0 a)  $\sigma'$ ( := subst0 a') oh hb)

theorem par_to_redStar {t u : PureTm n} (h : ParRed t u) : RedStar t u := by
  induction h with
  | var i =>
    exact .refl _
  | const c =>
    exact .refl _
  | u0 =>
    exact .refl _
  | u1 =>
    exact .refl _

```

```

| pi hA hB ihA ihB =>
  exact .trans (.congPiDom ihA) (.congPiCod ihB)
| sigma hA hB ihA ihB =>
  exact .trans (.congSigmaDom ihA) (.congSigmaCod ihB)
| id hA ha hb ihA iha ihb =>
  exact .trans
    (.trans
      (RedStar.map (F := fun t => .id t _ _) (fun hstep => .congIdTy hstep) ihA)
      (RedStar.map (F := fun t => .id _ t _) (fun hstep => .congIdLeft hstep) iha))
      (RedStar.map (F := fun t => .id _ _ t) (fun hstep => .congIdRight hstep) ihb)
| lam hb ih =>
  exact .congLam ih
| app hf ha ihf iha =>
  exact .trans (.congAppFun ihf) (.congAppArg iha)
| pair ha hb iha ihb =>
  exact .trans (.congPairFst iha) (.congPairSnd ihb)
| fst hp ih =>
  exact .congFst ih
| snd hp ih =>
  exact .congSnd ih
| refl ha iha =>
  exact .congRefl iha
| betaPi hb ha ihb iha =>
  exact .trans
    (.trans
      (.congAppFun (.congLam ihb))
      (.congAppArg iha))
      (red_to_redStar (.betaPi _ _))
| betaSigmaFst ha hb iha ihb =>
  exact .trans
    (.trans
      (.congFst (.congPairFst iha))
      (.congFst (.congPairSnd ihb)))
      (red_to_redStar (.betaSigmaFst _ _))
| betaSigmaSnd ha hb iha ihb =>
  exact .trans
    (.trans
      (.congSnd (.congPairFst iha))
      (.congSnd (.congPairSnd ihb)))
      (red_to_redStar (.betaSigmaSnd _ _))

end Mettapedia.Languages.MeTTa.PureKernel.Parallel

```

A.15 PatternBridge.lean

Mettapedia/Languages/MeTTa/PureKernel/PatternBridge.lean

```

import Mettapedia.Languages.MeTTa.Pure.Core
import Mettapedia.Languages.MeTTa.PureKernel.Context
import Mettapedia.Languages.MeTTa.PureKernel.Substitution
import Mettapedia.OSLF.MeTTaIL.Substitution

namespace Mettapedia.Languages.MeTTa.PureKernel.PatternBridge

```

```

open Mettapedia.OSLF.MeTTaIL.Syntax
open Mettapedia.Languages.MeTTa.Pure.Core
open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.OSLF.MeTTaIL.Substitution

abbrev QuoteEnv (n : Nat) := Fin n → String

def envCons (x : String) p( : QuoteEnv n) : QuoteEnv (n + 1) :=
  Fin.cases x (fun i => p i)

/-- Quote a kernel declaration name as a nullary shared artifact node. -/
def quoteConst (c : DeclName) : Pattern :=
  .apply c.toString []

/-- Legacy raw quote (debug only): maps scoped vars directly to bvars. -/
def quoteRaw : PureTm n → Pattern
| .var i => .bvar i.1
| .const c => quoteConst c
| .u0 => u0
| .u1 => u1
| .pi A B => mkPi (quoteRaw A) (quoteRaw B)
| .sigma A B => mkSigma (quoteRaw A) (quoteRaw B)
| .id A a b => mkId (quoteRaw A) (quoteRaw a) (quoteRaw b)
| .lam b => mkLam (quoteRaw b)
| .app f a => mkApp (quoteRaw f) (quoteRaw a)
| .pair a b => mkPair (quoteRaw a) (quoteRaw b)
| .fst p => mkFst (quoteRaw p)
| .snd p => mkSnd (quoteRaw p)
| .refl a => mkRefl (quoteRaw a)

def defaultBinderName (k : Nat) : String := "__pk_" ++ toString k

/-- Contextual locally-nameless quote:
`PureTm` variables are mapped to `fvar`s via p``,
and binders are encoded by `closeFVar` with fresh binder names from v``. -/
def quoteTmWith v( : Nat → String) (k : Nat) p( : QuoteEnv n) : PureTm n → Pattern
| .var i => .fvar p( i)
| .const c => quoteConst c
| .u0 => u0
| .u1 => u1
| .pi A B =>
  let x := v k
  mkPi (quoteTmWith v k p A)
    (closeFVar 0 x (quoteTmWith v (k + 1) (envCons x p) B))
| .sigma A B =>
  let x := v k
  mkSigma (quoteTmWith v k p A)
    (closeFVar 0 x (quoteTmWith v (k + 1) (envCons x p) B))
| .id A a b => mkId (quoteTmWith v k p A) (quoteTmWith v k p a) (quoteTmWith v k p b)
| .lam b =>
  let x := v k

```

```

    mkLam (closeFVar 0 x (quoteTmWith v (k + 1) (envCons x p) b))
| .app f a => mkApp (quoteTmWith v k p f) (quoteTmWith v k p a)
| .pair a b => mkPair (quoteTmWith v k p a) (quoteTmWith v k p b)
| .fst p => mkFst (quoteTmWith v k p p)
| .snd p => mkSnd (quoteTmWith v k p p)
| .refl a => mkRefl (quoteTmWith v k p a)

@[simp] theorem envCons_comp_liftRen
  (x : String) p(dst : QuoteEnv m) p( : Ren n m) :
  (fun j : Fin (n + 1) => envCons x pdst (liftRen p j)) =
    envCons x (fun i => pdst p( i)) := by
  funext j
  refine Fin.cases ?_ ?_ j
  · rfl
  · intro i
    rfl

/-- Contextual quotation commutes with kernel renaming. -/
theorem quoteTmWith_rename v( : Nat → String) :
  ∀ {n m : Nat} (k : Nat) p(dst : QuoteEnv m) p( : Ren n m) (t : PureTm n),
  quoteTmWith v k pdst (rename p t) =
    quoteTmWith v k (fun i => pdst p( i)) t := by
  intro n m k pdst p t
  induction t generalizing m k pdst with
  | var i =>
    rfl
  | const c =>
    rfl
  | u0 =>
    rfl
  | u1 =>
    rfl
  | pi A B ihA ihB =>
    have hB :=
      ihB (k := k + 1) p(dst := envCons v( k) pdst) p( := liftRen p)
    calc
      quoteTmWith v k pdst (rename p (.pi A B))
      = mkPi (quoteTmWith v k pdst (rename p A))
        (closeFVar 0 v( k)
          (quoteTmWith v (k + 1) (envCons v( k) pdst) (rename (liftRen p) B))) := by
        rfl
    _ = mkPi (quoteTmWith v k (fun i => pdst p( i)) A)
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1) (envCons v( k) pdst) (rename (liftRen p) B))) := by
        simp [ihA (k := k) p(dst := pdst) p( := p)]
    _ = mkPi (quoteTmWith v k (fun i => pdst p( i)) A)
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1)
          (fun j => envCons v( k) pdst (liftRen p j)) B)) := by
        simpa using congrArg
        (fun q => mkPi (quoteTmWith v k (fun i => pdst p( i)) A)
          (closeFVar 0 v( k) q)) hB
    _ = mkPi (quoteTmWith v k (fun i => pdst p( i)) A)
      (closeFVar 0 v( k)

```

```

      (quoteTmWith v (k + 1)
        (envCons v( k) (fun i => pdst p( i))) B)) := by
      simp [envCons_comp_liftRen]
    _ = quoteTmWith v k (fun i => pdst p( i)) (.pi A B) := by
      rfl
| sigma A B ihA ihB =>
  have hB :=
    ihB (k := k + 1) p(dst := envCons v( k) pdst) p( := liftRen p)
  calc
    quoteTmWith v k pdst (rename p (.sigma A B))
      = mkSigma (quoteTmWith v k pdst (rename p A))
        (closeFVar 0 v( k)
          (quoteTmWith v (k + 1) (envCons v( k) pdst) (rename (liftRen p) B))) := by
          rfl
    _ = mkSigma (quoteTmWith v k (fun i => pdst p( i)) A)
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1) (envCons v( k) pdst) (rename (liftRen p) B))) := by
        simp [ihA (k := k) p(dst := pdst) p( := p)]
    _ = mkSigma (quoteTmWith v k (fun i => pdst p( i)) A)
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1)
          (fun j => envCons v( k) pdst (liftRen p j)) B)) := by
        simp using congrArg
        (fun q => mkSigma (quoteTmWith v k (fun i => pdst p( i)) A)
          (closeFVar 0 v( k) q)) hB
    _ = mkSigma (quoteTmWith v k (fun i => pdst p( i)) A)
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1)
          (envCons v( k) (fun i => pdst p( i))) B)) := by
        simp [envCons_comp_liftRen]
    _ = quoteTmWith v k (fun i => pdst p( i)) (.sigma A B) := by
      rfl
| id A a b ihA iha ihb =>
  simp [quoteTmWith, rename,
    ihA (k := k) p(dst := pdst) p( := p),
    iha (k := k) p(dst := pdst) p( := p),
    ihb (k := k) p(dst := pdst) p( := p)]
| lam b ih =>
  have hb := ih (k := k + 1) p(dst := envCons v( k) pdst) p( := liftRen p)
  calc
    quoteTmWith v k pdst (rename p (.lam b))
      = mkLam
        (closeFVar 0 v( k)
          (quoteTmWith v (k + 1)
            (envCons v( k) pdst) (rename (liftRen p) b))) := by
            rfl
    _ = mkLam
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1)
          (fun j => envCons v( k) pdst (liftRen p j)) b)) := by
        simp using congrArg (fun q => mkLam (closeFVar 0 v( k) q)) hb
    _ = mkLam
      (closeFVar 0 v( k)
        (quoteTmWith v (k + 1)

```

```

      (envCons v( k) (fun i => pdst p( i))) b)) := by
      simp [envCons_comp_liftRen]
    _ = quoteTmWith v k (fun i => pdst p( i)) (.lam b) := by
      rfl
| app f a ihf iha =>
  simp [quoteTmWith, rename,
    ihf (k := k) p(dst := pdst) p( := p),
    iha (k := k) p(dst := pdst) p( := p)]
| pair a b iha ihb =>
  simp [quoteTmWith, rename,
    iha (k := k) p(dst := pdst) p( := p),
    ihb (k := k) p(dst := pdst) p( := p)]
| fst p ih =>
  simp [quoteTmWith, rename] using congrArg mkFst (ih (k := k) p(dst := pdst) p( := p))
| snd p ih =>
  simp [quoteTmWith, rename] using congrArg mkSnd (ih (k := k) p(dst := pdst) p( := p))
| refl a iha =>
  simp [quoteTmWith, rename] using congrArg mkRefl (iha (k := k) p(dst := pdst) p( := p))

/-- Useful weakening corollary for bridge proofs. -/
theorem quoteTmWith_rename_wk_envCons
  v( : Nat → String) (k : Nat) (x : String) p( : QuoteEnv n) (t : PureTm n) :
  quoteTmWith v k (envCons x p) (rename wk t) =
  quoteTmWith v k p t := by
  simp [wk, envCons] using
    (quoteTmWith_rename v (k := k) p(dst := envCons x p) p( := wk) (t := t))

/-- Body-quotation mode under the current binder name v`k`. -/
def quoteBodyWith v( : Nat → String) (k : Nat) p( : QuoteEnv n)
  (b : PureTm (n + 1)) : Pattern :=
  quoteTmWith v (k + 1) (envCons v( k) p) b

/-- Under body-quotation mode, weakening by `wk` cancels the outer binder extension. -/
@[simp] theorem quoteBodyWith_wk
  v( : Nat → String) (k : Nat) p( : QuoteEnv n) (t : PureTm n) :
  quoteBodyWith v k p (rename wk t) = quoteTmWith v (k + 1) p t := by
  simp [quoteBodyWith] using quoteTmWith_rename_wk_envCons v (k + 1) v( k) p t

def quoteTm p( : QuoteEnv n) (t : PureTm n) : Pattern :=
  quoteTmWith defaultBinderName 0 p t

def emptyEnv : QuoteEnv 0 := fun i => False.elim (Nat.not_lt_zero _ i.isLt)

def quoteClosedTm (t : PureTm 0) : Pattern :=
  quoteTm p( := emptyEnv) t

def quoteCtx p( : QuoteEnv n) : Ctx n → Pattern
| .nil => mkCtxEmpty
| .snoc Γ A =>
  let pprev : QuoteEnv _ := fun i => p i.succ
  mkCtxExtend (quoteCtx pprev Γ) (quoteTm pprev A)

/-- Binder naming policy for contextual quotation. -/
structure BinderPolicy where

```

```

name : Nat → String
inj : Function.Injective name

/-- Compatibility assumptions for contextual quotation: v
` ` is injective and all future binder names v` j` (j ≥ k) are absent from p` `. -/
def QuoteCompat v( : Nat → String) (k : Nat) p( : QuoteEnv n) : Prop :=
  Function.Injective v ∧ ∀( i j, k ≤ j → p i ≠ v j)

/-- Strong quotation invariant for open terms:
the quote environment is injective and compatible with the binder naming policy. -/
structure QuoteEnvGood (P : BinderPolicy) (k : Nat) p( : QuoteEnv n) : Prop where
  env_inj : Function.Injective p
  compat : QuoteCompat P.name k p

/-- Bridge contract (open-form): kernel `inst0` commutes with contextual quotation. -/
def Inst0OpenBridge v( : Nat → String) : Prop :=
  ∀ {n : Nat} (k : Nat) p( : QuoteEnv n) (a : PureTm n) (body : PureTm (n + 1)),
    quoteTmWith v k p (inst0 a body) =
      openBVar 0 (quoteTmWith v k p a)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) p) body))

/-- Bridge contract (apply-form): kernel `inst0` commutes with singleton `applySubst`. -/
def Inst0ApplyBridge v( : Nat → String) : Prop :=
  ∀ {n : Nat} (k : Nat) p( : QuoteEnv n) (a : PureTm n) (body : PureTm (n + 1)),
    quoteTmWith v k p (inst0 a body) =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
        (quoteTmWith v (k + 1) (envCons v( k) p) body)

/-- Compatibility-aware bridge contract (open-form). -/
def Inst0OpenBridgeCompat v( : Nat → String) : Prop :=
  ∀ {n : Nat} (k : Nat) p( : QuoteEnv n) (a : PureTm n) (body : PureTm (n + 1)),
    QuoteCompat v k p →
    quoteTmWith v k p (inst0 a body) =
      openBVar 0 (quoteTmWith v k p a)
        (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) p) body))

/-- Compatibility-aware bridge contract (apply-form). -/
def Inst0ApplyBridgeCompat v( : Nat → String) : Prop :=
  ∀ {n : Nat} (k : Nat) p( : QuoteEnv n) (a : PureTm n) (body : PureTm (n + 1)),
    QuoteCompat v k p →
    quoteTmWith v k p (inst0 a body) =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
        (quoteTmWith v (k + 1) (envCons v( k) p) body)

theorem quoteCompat_empty v( : Nat → String) (vh : Function.Injective v) (k : Nat) :
  QuoteCompat v k emptyEnv := by
  refine {vh, ?}_
  intro i
  exact Fin.elim0 i

theorem QuoteEnvGood.empty (P : BinderPolicy) (k : Nat) :
  QuoteEnvGood P k emptyEnv := by
  refine {?, ?}_
  · intro i

```



```

    exact Fin.elim0 i
  · exact quoteCompat_empty P.name P.inj k

theorem QuoteCompat.envCons v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (vh : Function.Injective v) (ph : QuoteCompat v k p) :
  QuoteCompat v (k + 1) (envCons v( k) p) := by
  refine (vh, ?)_
  intro i j hj
  refine Fin.cases ?h0 ?hs i
  · intro hEq
    have hkj : k ≠ j := by omega
    exact hkj (vh hEq)
  · intro i'
    exact ph.2 i' j (by omega)

theorem QuoteEnvGood.envCons {P : BinderPolicy} {k : Nat} p{ : QuoteEnv n}
  (ph : QuoteEnvGood P k p) :
  QuoteEnvGood P (k + 1) (envCons (P.name k) p) := by
  refine {?, ?}_
  · intro i j hij
    cases i using Fin.cases with
    | zero =>
      cases j using Fin.cases with
      | zero =>
        rfl
      | succ j' =>
        exfalso
        have hneq : p j' ≠ P.name k := ph.compat.2 j' k (by omega)
        exact hneq hij.symm
    | succ i' =>
      cases j using Fin.cases with
      | zero =>
        exfalso
        have hneq : p i' ≠ P.name k := ph.compat.2 i' k (by omega)
        exact hneq hij
      | succ j' =>
        have hij' : p i' = p j' := by simpa [envCons] using hij
        have heq : i' = j' := ph.env_inj hij'
        cases heq
        rfl
  · exact QuoteCompat.envCons P.inj ph.compat

theorem envCons_injective_of_injective_of_compat
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (phinj : Function.Injective p) (hcompat : QuoteCompat v k p) :
  Function.Injective (envCons v( k) p) := by
  let P : BinderPolicy := { name := v, inj := hcompat.1 }
  have hgood : QuoteEnvGood P k p := (phinj, )hcompat
  exact (QuoteEnvGood.envCons (P := P) hgood).env_inj

/-- Quote a kernel substitution as a MeTTaIL substitution environment. -/
def quoteSubstEnv v{ : Nat → String} (k : Nat) p(src : QuoteEnv n)
  p(dst : QuoteEnv m) σ{ : Sub n m) : SubstEnv :=
  match n with

```

```

| 0 => SubstEnv.empty
| n + 1 =>
  let ptail : QuoteEnv n := fun i => psrc i.succ
  let otail : Sub n m := fun i => σ i.succ
  SubstEnv.extend (quoteSubstEnv v k ptail pdst otail)
    p(src 0) (quoteTmWith v k pdst σ( 0))
termination_by n

theorem SubstEnv.find_extend_eq {env : SubstEnv} {x : String} {q : Pattern} :
  (SubstEnv.extend env x q).find x = some q := by
simp [SubstEnv.extend, SubstEnv.find]

theorem SubstEnv.find_extend_ne {env : SubstEnv} {x y : String} {q : Pattern}
  (hxy : x ≠ y) :
  (SubstEnv.extend env x q).find y = env.find y := by
simp [SubstEnv.extend, SubstEnv.find, beq_eq_false_iff_ne.mpr hxy]

/-- Extending an environment with `x ↦ fvar x` is a no-op when `x` is not already bound. -/
theorem applySubst_extend_fvar_self_of_find_none
  (env : SubstEnv) (x : String) (p : Pattern)
  (hfind : env.find x = none) :
  applySubst (SubstEnv.extend env x (.fvar x)) p = applySubst env p := by
induction p using Pattern.inductionOn with
| hbvar n =>
  simp [applySubst]
| hfvar name =>
  by_cases hname : name = x
  · subst hname
    simp [applySubst, SubstEnv.find_extend_eq, hfind]
  · have hne : x ≠ name := by
    exact fun h => hname h.symm
    simp [applySubst, SubstEnv.find_extend_ne hne]
| happly c args ih =>
  simp [applySubst]
  intro a ha
  exact ih a ha
| hlambdas body ih =>
  simp [applySubst] using ih
| hmultilambda n body ih =>
  simp [applySubst] using ih
| hsubst body repl ihb ihr =>
  simp [applySubst, ihb, ihr]
| hcollection ct elems rest ih =>
  simp [applySubst]
  intro a ha
  exact ih a ha

theorem quoteSubstEnv_find
  v( : Nat → String) (k : Nat) p(src : QuoteEnv n) p(dst : QuoteEnv m) σ( : Sub n m)
  (hsrcInj : Function.Injective psrc) (i : Fin n) :
  (quoteSubstEnv v k psrc pdst σ).find p(src i) =
    some (quoteTmWith v k pdst σ( i)) := by
induction n with
| zero =>

```

```

    exact Fin.elim0 i
| succ n ih =>
    cases i using Fin.cases with
    | zero =>
        simp [quoteSubstEnv, SubstEnv.find_extend_eq]
    | succ j =>
        have hneq : psrc 0 ≠ psrc j.succ := by
            intro heq
            have : (0 : Fin (n + 1)) = j.succ := hsrcInj heq
            cases this
            have htailInj : Function.Injective (fun t : Fin n => psrc t.succ) := by
                intro a b hab
                have hEq : (a.succ : Fin (n + 1)) = b.succ := hsrcInj (by simp using hab)
                exact Fin.ext (by simp using congrArg Fin.val hEq)
            simp [quoteSubstEnv, SubstEnv.find_extend_ne hneq] using
            ih p(src := fun t : Fin n => psrc t.succ)
            σ( := fun t : Fin n => σ t.succ) htailInj j

theorem quoteSubstEnv_find_none_future
  (v : Nat → String) (k : Nat) p(src : QuoteEnv n) p(dst : QuoteEnv m) σ( : Sub n m)
  (hcompatSrc : QuoteCompat v k psrc) {j : Nat} (hj : k ≤ j) :
  (quoteSubstEnv v k psrc pdst σ).find v( j) = none := by
induction n with
| zero =>
    simp [quoteSubstEnv, SubstEnv.find, SubstEnv.empty]
| succ n ih =>
    have hheadneq : psrc 0 ≠ v j := hcompatSrc.2 0 j hj
    have htailCompat : QuoteCompat v k (fun t : Fin n => psrc t.succ) := by
        refine (hcompatSrc.1, ?)_
    intro i j' hk
    exact hcompatSrc.2 i.succ j' hk
    simp [quoteSubstEnv, SubstEnv.find_extend_ne hheadneq] using
    ih p(src := fun t : Fin n => psrc t.succ)
    σ( := fun t : Fin n => σ t.succ) htailCompat

theorem quoteSubstEnv_find_some_exists
  (v : Nat → String) (k : Nat) p(src : QuoteEnv n) p(dst : QuoteEnv m) σ( : Sub n m)
  {name : String} {r : Pattern}
  (hfind : (quoteSubstEnv v k psrc pdst σ).find name = some r) :
  ∃ i : Fin n, psrc i = name ∧ r = quoteTmWith v k pdst σ( i) := by
induction n with
| zero =>
    simp [quoteSubstEnv, SubstEnv.find, SubstEnv.empty] at hfind
| succ n ih =>
    by_cases hEq : psrc 0 = name
    · refine (0, hEq, ?)_
        have hhead :
            (quoteSubstEnv v k psrc pdst σ).find name =
            some (quoteTmWith v k pdst σ( 0)) := by
            simp [quoteSubstEnv, SubstEnv.find, hEq, SubstEnv.extend]
        rw [hhead] at hfind
        exact (Option.some.inj hfind).symm
    · have hEqBool : p(src 0 == name) = false := by
        exact beq_eq_false_iff_ne.mpr hEq

```

```

    have htail :
      (quoteSubstEnv v k (fun t : Fin n => psrc t.succ) pdst
        (fun t : Fin n => σ t.succ)).find name = some r := by
      simp [quoteSubstEnv, SubstEnv.find, hEqBool, SubstEnv.extend] using hfind
    rcases ih p(src := fun t : Fin n => psrc t.succ) σ( := fun t : Fin n => σ t.succ) htail
  with
    {i, hiName, }hiVal
  exact {i.succ, hiName, }hiVal

abbrev SubstEnvIsIdentity (env : SubstEnv) : Prop :=
  ∀ name r, env.find name = some r → r = .fvar name

private theorem list_map_eq_self_local {f : Pattern → Pattern} {l : List Pattern}
  (h : ∀ a ∈ l, f a = a) : l.map f = l := by
  induction l with
  | nil =>
    rfl
  | cons a as ih =>
    simp [h a List.mem_cons_self, ih (fun b hb => h b (List.mem_cons_of_mem _ hb))]

theorem applySubst_of_isIdentity
  (env : SubstEnv) (hId : SubstEnvIsIdentity env)
  (p : Pattern) (hnes : noExplicitSubst p = true) :
  applySubst env p = p := by
  induction p using Pattern.inductionOn with
  | hbvar n =>
    simp [applySubst]
  | hfvar name =>
    cases hfind : env.find name with
    | none =>
      simp [applySubst, hfind]
    | some r =>
      have hr : r = .fvar name := hId name r hfind
      simp [applySubst, hfind, hr]
  | happly c args ih =>
    simp [applySubst]
    exact list_map_eq_self_local (fun a ha => ih a ha (allNoExplicitSubst_mem hnes ha))
  | hlamba body ih =>
    simp [applySubst, noExplicitSubst] using ih (by simp [noExplicitSubst] using hnes)
  | hmultiLamba n body ih =>
    simp [applySubst, noExplicitSubst] using ih (by simp [noExplicitSubst] using hnes)
  | hsubst body repl ihb ihr =>
    exact absurd hnes Bool.false_ne_true
  | hcollection ct elems rest ih =>
    simp [applySubst]
    exact list_map_eq_self_local (fun a ha => ih a ha (allNoExplicitSubst_mem hnes ha))

theorem quoteSubstEnv_ids_isIdentity
  v( : Nat → String) (k : Nat) p( : QuoteEnv n) :
  SubstEnvIsIdentity (quoteSubstEnv v k p p ids) := by
  intro name r hfind
  rcases quoteSubstEnv_find_some_exists v k p p ids hfind with {i, hiName, }hiVal
  calc
    r = quoteTmWith v k p (ids i) := hiVal

```

```

_ = .fvar p( i) := by simp [ids, quoteTmWith]
_ = .fvar name := by simp [hiName]

theorem applySubst_quoteSubstEnv_ids
  v( : Nat → String) (k : Nat) p( : QuoteEnv n) (p : Pattern)
  (hnes : noExplicitSubst p = true) :
  applySubst (quoteSubstEnv v k p p ids) p = p := by
exact applySubst_of_isIdentity (quoteSubstEnv v k p p ids)
  (quoteSubstEnv_ids_isIdentity v k p) p hnes

theorem quoteSubstEnv_subst0_decompose
  v( : Nat → String) (k : Nat) p( : QuoteEnv n) (a : PureTm n) :
  quoteSubstEnv v k (envCons v( k) p) p (subst0 a) =
    SubstEnv.extend (quoteSubstEnv v k p p ids) v( k) (quoteTmWith v k p a) := by
simp [ids] using (by
  simp [quoteSubstEnv, envCons, subst0] :
    quoteSubstEnv v k (envCons v( k) p) p (subst0 a) =
      SubstEnv.extend (quoteSubstEnv v k (fun i => p i) p (fun i => PureTm.var i))
        v( k) (quoteTmWith v k p a))

theorem quoteSubstEnv_rename_wk_envCons_find
  v( : Nat → String) (k : Nat) (x : String)
  p(src : QuoteEnv n) p(dst : QuoteEnv m) σ( : Sub n m) (name : String) :
  (quoteSubstEnv v k psrc (envCons x pdst) (fun i => rename wk σ( i))).find name =
    (quoteSubstEnv v k psrc pdst σ).find name := by
induction n with
| zero =>
  simp [quoteSubstEnv]
| succ n ih =>
  by_cases hhead : psrc 0 = name
  · subst hhead
    have hheadVal :
      quoteTmWith v k (envCons x pdst) (rename wk σ( 0)) =
        quoteTmWith v k pdst σ( 0) := by
      simp using quoteTmWith_rename_wk_envCons v k x pdst σ( 0)
    simp [quoteSubstEnv, hheadVal, SubstEnv.find_extend_eq]
  · have hhead' : p(src 0 == name) = false := beq_eq_false_iff_ne.mpr hhead
    have hheadVal :
      quoteTmWith v k (envCons x pdst) (rename wk σ( 0)) =
        quoteTmWith v k pdst σ( 0) := by
      simp using quoteTmWith_rename_wk_envCons v k x pdst σ( 0)
    have htail := ih p(src := fun i => psrc i.succ) σ( := fun i => σ i.succ)
    let leftTail : SubstEnv :=
      quoteSubstEnv v k (fun i => psrc i.succ) (envCons x pdst) (fun i => rename wk σ( i.succ
    ))
    let rightTail : SubstEnv :=
      quoteSubstEnv v k (fun i => psrc i.succ) pdst (fun i => σ i.succ)
    have hleftExpand :
      (quoteSubstEnv v k psrc (envCons x pdst) (fun i => rename wk σ( i))).find name =
        (leftTail.extend p(src 0) (quoteTmWith v k pdst σ( 0))).find name := by
      simp [quoteSubstEnv, leftTail, hheadVal]
    have hrightExpand :
      (quoteSubstEnv v k psrc pdst σ).find name =
        (rightTail.extend p(src 0) (quoteTmWith v k pdst σ( 0))).find name := by

```

```

    simp [quoteSubstEnv, rightTail]
  have hleft :
    (leftTail.extend p(src 0) (quoteTmWith v k pdst σ( 0))).find name =
      leftTail.find name := by
    exact SubstEnv.find_extend_ne hhead
  have hright :
    (rightTail.extend p(src 0) (quoteTmWith v k pdst σ( 0))).find name =
      rightTail.find name := by
    exact SubstEnv.find_extend_ne hhead
  calc
    (quoteSubstEnv v k psrc (envCons x pdst) (fun i => rename wk σ( i))).find name
      = (leftTail.extend p(src 0) (quoteTmWith v k pdst σ( 0))).find name := hleftExpand
    _ = leftTail.find name := hleft
    _ = rightTail.find name := by simpa [leftTail, rightTail] using htail
    _ = (rightTail.extend p(src 0) (quoteTmWith v k pdst σ( 0))).find name := by
      symm; exact hright
    _ = (quoteSubstEnv v k psrc pdst σ).find name := hrightExpand.symm

theorem quoteSubstEnv_liftSub_find
  (v : Nat → String) (k : Nat) p(src : QuoteEnv n) p(dst : QuoteEnv m)
  σ( : Sub n m) (name : String) :
  (quoteSubstEnv v (k + 1) (envCons v( k) psrc) (envCons v( k) pdst) (liftSub σ)).find name =
    ((quoteSubstEnv v (k + 1) psrc pdst σ).extend v( k) (.fvar v( k))).find name := by
  by_cases hnk : v k = name
  · subst hnk
  · simp [quoteSubstEnv, envCons, quoteTmWith, SubstEnv.find_extend_eq]
  · have hnk' : v k ≠ name := hnk
  calc
    (quoteSubstEnv v (k + 1) (envCons v( k) psrc) (envCons v( k) pdst) (liftSub σ)).find name
      = (quoteSubstEnv v (k + 1) psrc (envCons v( k) pdst)
        (fun i => rename wk σ( i))).find name := by
        simp [quoteSubstEnv, envCons, liftSub, SubstEnv.find_extend_ne hnk']
    _ = (quoteSubstEnv v (k + 1) psrc pdst σ).find name :=
      quoteSubstEnv_rename_wk_envCons_find v (k + 1) v( k) psrc pdst σ name
    _ = ((quoteSubstEnv v (k + 1) psrc pdst σ).extend v( k) (.fvar v( k))).find name := by
      symm
      exact SubstEnv.find_extend_ne hnk'

mutual
theorem noExplicitSubst_closeFVar (k : Nat) (x : String) (p : Pattern) :
  noExplicitSubst (closeFVar k x p) = noExplicitSubst p := by
  induction p using Pattern.inductionOn generalizing k with
  | hvar _ =>
    simp [closeFVar, noExplicitSubst]
  | hvar y =>
    by_cases hy : y = x
    · simp [closeFVar, noExplicitSubst, hy]
    · simp [closeFVar, noExplicitSubst, hy]
  | happly c args ih =>
    simpa [closeFVar, noExplicitSubst] using
      allNoExplicitSubst_map_closeFVar k x args ih
  | hlambda body ih =>
    simpa [closeFVar, noExplicitSubst] using ih (k + 1)
  | hmultiLambda n body ih =>

```

```

      simpa [closeFVar, noExplicitSubst] using ih (k + n)
| hsubst body repl ihb ihr =>
  simp [closeFVar, noExplicitSubst]
| hcollection ct elems rest ih =>
  simpa [closeFVar, noExplicitSubst] using
    allNoExplicitSubst_map_closeFVar k x elems ih

theorem allNoExplicitSubst_map_closeFVar (k : Nat) (x : String)
  (ps : List Pattern)
  (ih : ∀ p, p ∈ ps → ∀ k, noExplicitSubst (closeFVar k x p) = noExplicitSubst p) :
  allNoExplicitSubst (ps.map (closeFVar k x)) = allNoExplicitSubst ps := by
induction ps with
| nil =>
  simp [allNoExplicitSubst]
| cons p ps ihps =>
  simp [allNoExplicitSubst, ih p List.mem_cons_self k,
    ihps (fun q hq => ih q (List.mem_cons_of_mem _ hq))]
end

theorem lc_at_closeFVar_of_lt {k l : Nat} (x : String) (p : Pattern)
  (hl : l < k) (hp : lc_at k p = true) :
  lc_at k (closeFVar l x p) = true := by
induction p using Pattern.inductionOn generalizing k l with
| hbvar n =>
  simpa [closeFVar, lc_at] using hp
| hfvar y =>
  by_cases hy : y = x
  · simp [closeFVar, lc_at, hy, hl]
  · simp [closeFVar, lc_at, hy]
| happly c args ih =>
  simp only [lc_at] at hp
  simp only [closeFVar, lc_at]
  induction args with
| nil =>
  simp [lc_at_list]
| cons a as ihas =>
  simp only [List.map_cons, lc_at_list, Bool.and_eq_true] at hp ⊢
  exact ⟨ih a List.mem_cons_self hl hp.1,
    ihas (fun q hq => ih q (List.mem_cons_of_mem _ hq)) hp⟩.2
| hlambda body ih =>
  simp only [closeFVar, lc_at] at hp ⊢
  exact ih (Nat.succ_lt_succ hl) hp
| hmultiLambda n body ih =>
  simp only [closeFVar, lc_at] at hp ⊢
  exact ih (Nat.add_lt_add_right hl n) hp
| hsubst body repl ihb ihr =>
  simp only [closeFVar, lc_at, Bool.and_eq_true] at hp ⊢
  exact ⟨ihb (Nat.succ_lt_succ hl) hp.1, ihr hl hp⟩.2
| hcollection ct elems rest ih =>
  simp only [lc_at] at hp
  simp only [closeFVar, lc_at]
  induction elems with
| nil =>
  simp [lc_at_list]

```

```

| cons a as ihas =>
  simp only [List.map_cons, lc_at_list, Bool.and_eq_true] at hp ⊢
  exact ⟨ih a List.mem_cons_self hl hp.1,
    ihas (fun q hq => ih q (List.mem_cons_of_mem _ hq)) hp⟩.2

theorem not_mem_freeVars_closeFVar_self (k : Nat) (x : String) (p : Pattern) :
  x ∉ freeVars (closeFVar k x p) := by
  induction p using Pattern.inductionOn generalizing k with
  | hbvar _ =>
    simp [closeFVar, freeVars]
  | hfvar y =>
    by_cases hy : y = x
    · simp [closeFVar, freeVars, hy]
    · have hxy : x ≠ y := fun h => hy h.symm
      simp [closeFVar, freeVars, hy, hxy]
  | happly c args ih =>
    intro hx
    simp [closeFVar, freeVars, List.mem_flatMap] at hx
    rcases hx with ⟨a, ha, ⟩hxa
    exact (ih a ha k) hxa
  | hlamba body ih =>
    simpa [closeFVar, freeVars] using ih (k + 1)
  | hmultiLamba n body ih =>
    simpa [closeFVar, freeVars] using ih (k + n)
  | hsubst body repl ihb ihr =>
    intro hx
    simp [closeFVar, freeVars, List.mem_append] at hx
    cases hx with
    | inl hxBody => exact (ihb (k + 1) hxBody)
    | inr hxRepl => exact (ihr k hxRepl)
  | hcollection ct elems rest ih =>
    intro hx
    simp [closeFVar, freeVars, List.mem_flatMap] at hx
    rcases hx with ⟨a, ha, ⟩hxa
    exact (ih a ha k) hxa

theorem isFresh_closeFVar_self (k : Nat) (x : String) (p : Pattern) :
  isFresh x (closeFVar k x p) = true := by
  have hnot : x ∉ freeVars (closeFVar k x p) := not_mem_freeVars_closeFVar_self k x p
  unfold isFresh
  rw [Bool.not_eq_true']
  apply Bool.eq_false_iff.mpr
  intro htrue
  exact hnot (List.contains_iff_mem.mp htrue)

private theorem list_map_eq_map {f g : Pattern → Pattern} {l : List Pattern}
  (h : ∀ a ∈ l, f a = g a) : l.map f = l.map g := by
  induction l with
  | nil => rfl
  | cons a as ih =>
    simp [h a List.mem_cons_self, ih (fun b hb => h b (List.mem_cons_of_mem _ hb))]

private theorem list_map_eq_self {f : Pattern → Pattern} {l : List Pattern}
  (h : ∀ a ∈ l, f a = a) : l.map f = l := by

```



```

simpa [List.map_id] using
  (list_map_eq_map (f := f) (g := fun a => a) (l := l) h)

theorem closeFVar_fresh_id (k : Nat) (x : String) (p : Pattern)
  (hfresh : isFresh x p = true) :
  closeFVar k x p = p := by
  suffices h : ∀ p : Pattern, ∀ k : Nat, isFresh x p = true → closeFVar k x p = p from
    h p k hfresh
  intro p
  induction p using Pattern.inductionOn with
  | hbvar n =>
    intro k _
    simp [closeFVar]
  | hfvar y =>
    intro k hfresh
    have hne : y ≠ x := Ne.symm (isFresh_fvar_neq (x := x) (y := y) hfresh)
    simp [closeFVar, hne]
  | happly c args ih =>
    intro k hfresh
    simp [closeFVar]
    exact list_map_eq_self (fun a ha => by
      simpa using ih a ha k (isFresh_mem_of_flatMap hfresh ha))
  | hlamba body ih =>
    intro k hfresh
    simpa [closeFVar, isFresh_lambda_iff] using ih (k + 1) (by simpa [isFresh_lambda_iff] using
      hfresh)
  | hmultiLambda n body ih =>
    intro k hfresh
    have hbody : isFresh x body = true := by simpa [isFresh, freeVars] using hfresh
    simpa [closeFVar] using ih (k + n) hbody
  | hsubst body repl ihb ihr =>
    intro k hfresh
    have hparts : (freeVars body).contains x = false ∧ (freeVars repl).contains x = false := by
      simpa [isFresh, freeVars, Bool.not_eq_true', List.contains_append, Bool.or_eq_false_iff]
    using hfresh
    have hbody : isFresh x body = true := by simpa [isFresh, Bool.not_eq_true'] using hparts.1
    have hrepl : isFresh x repl = true := by simpa [isFresh, Bool.not_eq_true'] using hparts.2
    simp [closeFVar, ihb (k + 1) hbody, ihr k hrepl]
  | hcollection ct elems rest ih =>
    intro k hfresh
    simp [closeFVar]
    exact list_map_eq_self (fun a ha => by
      simpa using ih a ha k (isFresh_collection_mem hfresh ha))

/-- Closing two distinct free names commutes (same binder depth). -/
theorem closeFVar_comm (k : Nat) {x y : String} (hxy : x ≠ y) (p : Pattern) :
  closeFVar k x (closeFVar k y p) = closeFVar k y (closeFVar k x p) := by
  induction p using Pattern.inductionOn generalizing k with
  | hbvar n =>
    simp [closeFVar]
  | hfvar z =>
    by_cases hzx : z = x
    · subst hzx
    · simp [closeFVar, hxy]

```

```

    · by_cases hzy : z = y
      · subst hzy
        simp [closeFVar, hzx]
      · simp [closeFVar, hzx, hzy]
| happly c args ih =>
  simp [closeFVar]
  intro a ha
  exact ih a ha k
| hlambda body ih =>
  simp [closeFVar] using ih (k + 1)
| hmultiLambda n body ih =>
  simp [closeFVar] using ih (k + n)
| hsubst body repl ihb ihr =>
  simp [closeFVar, ihb (k + 1), ihr k]
| hcollection ct elems rest ih =>
  simp [closeFVar]
  intro a ha
  exact ih a ha k

theorem applySubst_single_closeFVar_comm
  {x y : String} {q p : Pattern} {k : Nat}
  (hxy : y ≠ x) (hfresh : isFresh y q = true) (hnes : noExplicitSubst p = true) :
  applySubst (SubstEnv.extend SubstEnv.empty x q) (closeFVar k y p) =
    closeFVar k y (applySubst (SubstEnv.extend SubstEnv.empty x q) p) := by
  suffices h : ∀ p : Pattern, ∀ k : Nat, noExplicitSubst p = true →
    applySubst (SubstEnv.extend SubstEnv.empty x q) (closeFVar k y p) =
      closeFVar k y (applySubst (SubstEnv.extend SubstEnv.empty x q) p) from
    h p k hnes
  intro p
  induction p using Pattern.inductionOn with
  | hbvar n =>
    intro k _
    simp [applySubst, closeFVar]
  | hfvar name =>
    intro k _
    by_cases hny : name = y
      · subst hny
        simp [closeFVar, applySubst, SubstEnv.find_extend_empty_ne (Ne.symm hxy)]
      · by_cases hnx : name = x
        · subst hnx
          simp [closeFVar, applySubst, hny, SubstEnv.find_extend_empty_eq] using
            (closeFVar_fresh_id k y q hfresh).symm
        · have hxn' : x ≠ name := fun h => hnx h.symm
          simp [closeFVar, applySubst, hny, SubstEnv.find_extend_empty_ne hxn']
  | happly c args ih =>
    intro k hnes
    simp [applySubst, closeFVar]
    intro a ha
    simp using ih a ha k (allNoExplicitSubst_mem hnes ha)
  | hlambda body ih =>
    intro k hnes
    simp [applySubst, closeFVar] using ih (k + 1) (by simp [noExplicitSubst] using hnes)
  | hmultiLambda n body ih =>
    intro k hnes

```

```

    simp [applySubst, closeFVar] using ih (k + n) (by simp [noExplicitSubst] using hnes)
| hsubst body repl _ _ =>
  intro _ hnes
  exact absurd hnes Bool.false_ne_true
| hcollection ct elems rest ih =>
  intro k hnes
  simp [applySubst, closeFVar]
  intro a ha
  simp using ih a ha k (allNoExplicitSubst_mem hnes ha)

theorem freeVars_closeFVar_mem_of_ne (k : Nat) {x z : String} (p : Pattern)
  (hneq : z ≠ x) :
  z ∈ freeVars (closeFVar k x p) → z ∈ freeVars p := by
induction p using Pattern.inductionOn generalizing k with
| hbvar _ =>
  intro hz
  simp [closeFVar, freeVars] at hz
| hfvar y =>
  intro hz
  by_cases hy : y = x
  · subst hy
    simp [closeFVar, freeVars] at hz
  · simp [closeFVar, freeVars, hy] using hz
| happly c args ih =>
  intro hz
  simp [closeFVar, freeVars, List.mem_flatMap] at hz ⊢
  rcases hz with ⟨a, ha, ⟩hza
  exact ⟨a, ha, ih a ha k ⟩hza
| hlamba body ih =>
  intro hz
  simp [closeFVar, freeVars] at hz
  simp [freeVars] using ih (k + 1) hz
| hmultiLamba n body ih =>
  intro hz
  simp [closeFVar, freeVars] at hz
  simp [freeVars] using ih (k + n) hz
| hsubst body repl ihb ihr =>
  intro hz
  simp [closeFVar, freeVars, List.mem_append] at hz ⊢
  cases hz with
  | inl hzBody => exact Or.inl (ihb (k + 1) hzBody)
  | inr hzRepl => exact Or.inr (ihr k hzRepl)
| hcollection ct elems rest ih =>
  intro hz
  simp [closeFVar, freeVars, List.mem_flatMap] at hz ⊢
  rcases hz with ⟨a, ha, ⟩hza
  exact ⟨a, ha, ih a ha k ⟩hza

theorem isFresh_closeFVar_ne (k : Nat) {x z : String} {p : Pattern}
  (hneq : z ≠ x) (hfresh : isFresh z p = true) :
  isFresh z (closeFVar k x p) = true := by
have hcontFalse : (freeVars p).contains z = false := by
  by_cases hcz : (freeVars p).contains z = false
  · exact hcz

```

```

· have htmp : z ∉ freeVars p := by simp [isFresh] using hfresh
have hctrue : (freeVars p).contains z = true := by
  cases hc : (freeVars p).contains z with
  | false => exact False.elim (hcz hc)
  | true => rfl
have hz : z ∈ freeVars p := List.contains_iff_mem.mp hctrue
exact False.elim (htmp hz)
unfold isFresh
rw [Bool.not_eq_true']
apply Bool.eq_false_iff.mpr
intro htrue
have hzClose : z ∈ freeVars (closeFVar k x p) := List.contains_iff_mem.mp htrue
have hz : z ∈ freeVars p := freeVars_closeFVar_mem_of_ne k p hneq hzClose
have hpTrue : (freeVars p).contains z = true := List.contains_iff_mem.mpr hz
have hcontra : false = true := (Eq.symm hcontFalse).trans hpTrue
exact Bool.false_ne_true hcontra

/-- Every free variable in a contextual quote comes from the context environment p`. -/
theorem freeVars_quoteTmWith_mem_env
  v( : Nat → String) (k : Nat) ρ( : QuoteEnv n) (t : PureTm n)
  {z : String} :
  z ∈ freeVars (quoteTmWith v k p t) → ∃ i : Fin n, ρ i = z := by
induction t generalizing k with
| var i =>
  intro hz
  simp [quoteTmWith, freeVars] at hz
  exact ⟨i, hz.⟩.symm
| const c =>
  intro hz
  simp [quoteTmWith, quoteConst, freeVars] at hz
| u0 =>
  intro hz
  simp [quoteTmWith, u0, freeVars] at hz
| u1 =>
  intro hz
  simp [quoteTmWith, u1, freeVars] at hz
| pi A B ihA ihB =>
  intro hz
  simp [quoteTmWith, mkPi, freeVars, List.mem_append] at hz
  cases hz with
  | inl hzA =>
    exact ihA (k := k) ρ( := ρ) hzA
  | inr hzClose =>
    let x := v k
    have hzNeX : z ≠ x := by
      intro hzx
      subst hzx
      exact (not_mem_freeVars_closeFVar_self 0 x
        (quoteTmWith v (k + 1) (envCons x ρ) B)) hzClose
    have hzBody : z ∈ freeVars (quoteTmWith v (k + 1) (envCons x ρ) B) := by
      exact freeVars_closeFVar_mem_of_ne 0
      (p := quoteTmWith v (k + 1) (envCons x ρ) B) hzNeX hzClose
    rcases ihB (k := k + 1) ρ( := envCons x ρ) hzBody with ⟨i, ⟩hi
    exact Fin.cases

```

```

      (motive := fun i => envCons x p i = z → ∃ j, p j = z)
      (fun h0 => False.elim (hzNeX (by simp [envCons] using h0.symm)))
      (fun j hs => ⟨j, by simp [envCons] using >hs⟩)
    i hi
| sigma A B ihA ihB =>
  intro hz
  simp [quoteTmWith, mkSigma, freeVars, List.mem_append] at hz
  cases hz with
| inl hzA =>
  exact ihA (k := k) ρ( := ρ) hzA
| inr hzClose =>
  let x := v k
  have hzNeX : z ≠ x := by
    intro hzx
    subst hzx
    exact (not_mem_freeVars_closeFVar_self 0 x
      (quoteTmWith v (k + 1) (envCons x ρ) B)) hzClose
  have hzBody : z ∈ freeVars (quoteTmWith v (k + 1) (envCons x ρ) B) := by
    exact freeVars_closeFVar_mem_of_ne 0
      (p := quoteTmWith v (k + 1) (envCons x ρ) B) hzNeX hzClose
  rcases ihB (k := k + 1) ρ( := envCons x ρ) hzBody with ⟨i, >hi
  exact Fin.cases
    (motive := fun i => envCons x p i = z → ∃ j, p j = z)
    (fun h0 => False.elim (hzNeX (by simp [envCons] using h0.symm)))
    (fun j hs => ⟨j, by simp [envCons] using >hs⟩)
    i hi
| id A a b ihA iha ihb =>
  intro hz
  simp [quoteTmWith, mkId, freeVars, List.mem_append] at hz
  rcases hz with hz | hz
  · exact ihA (k := k) ρ( := ρ) hz
  · rcases hz with hz | hz
    · exact iha (k := k) ρ( := ρ) hz
    · exact ihb (k := k) ρ( := ρ) hz
| lam b ih =>
  intro hz
  simp [quoteTmWith, mkLam, freeVars] at hz
  let x := v k
  have hzNeX : z ≠ x := by
    intro hzx
    subst hzx
    exact (not_mem_freeVars_closeFVar_self 0 x
      (quoteTmWith v (k + 1) (envCons x ρ) b)) hz
  have hzBody : z ∈ freeVars (quoteTmWith v (k + 1) (envCons x ρ) b) := by
    exact freeVars_closeFVar_mem_of_ne 0
      (p := quoteTmWith v (k + 1) (envCons x ρ) b) hzNeX hz
  rcases ih (k := k + 1) ρ( := envCons x ρ) hzBody with ⟨i, >hi
  exact Fin.cases
    (motive := fun i => envCons x p i = z → ∃ j, p j = z)
    (fun h0 => False.elim (hzNeX (by simp [envCons] using h0.symm)))
    (fun j hs => ⟨j, by simp [envCons] using >hs⟩)
    i hi
| app f a ihf iha =>
  intro hz

```

```

    simp [quoteTmWith, mkApp, freeVars, List.mem_append] at hz
    cases hz with
    | inl hzf => exact ihf (k := k) ρ( := ρ) hzf
    | inr hza => exact iha (k := k) ρ( := ρ) hza
| pair a b iha ihb =>
    intro hz
    simp [quoteTmWith, mkPair, freeVars, List.mem_append] at hz
    cases hz with
    | inl hza => exact iha (k := k) ρ( := ρ) hza
    | inr hzb => exact ihb (k := k) ρ( := ρ) hzb
| fst p ih =>
    intro hz
    simp [quoteTmWith, mkFst, freeVars] at hz
    exact ih (k := k) ρ( := ρ) hz
| snd p ih =>
    intro hz
    simp [quoteTmWith, mkSnd, freeVars] at hz
    exact ih (k := k) ρ( := ρ) hz
| refl a iha =>
    intro hz
    simp [quoteTmWith, mkRefl, freeVars] at hz
    exact iha (k := k) ρ( := ρ) hz

/-- Any future binder name  $v`j`$  ( $j \geq k$ ) is fresh in the quote under `QuoteCompat`. -/
theorem isFresh_quoteTmWith_future
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ) (t : PureTm n) {j : Nat} (hj : k ≤ j) :
  isFresh v( j) (quoteTmWith v k ρ t) = true := by
  unfold isFresh
  rw [Bool.not_eq_true']
  apply Bool.eq_false_iff.mpr
  intro hcontains
  have hz : v j ∈ freeVars (quoteTmWith v k ρ t) := List.contains_iff_mem.mp hcontains
  rcases freeVars_quoteTmWith_mem_env v k ρ t hz with {i, }hi
  exact (hcompat.2 i j hj) hi

/-- Specialization of `isFresh_quoteTmWith_future` at the next binder index. -/
theorem isFresh_quoteTmWith_next
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ) (t : PureTm n) :
  isFresh v( (k + 1)) (quoteTmWith v k ρ t) = true :=
  isFresh_quoteTmWith_future hcompat t (j := k + 1) (by omega)

/-- At quote depth  $k + 1$ , the previous binder name  $v`k`$  remains fresh under `QuoteCompat v k`. -/
theorem isFresh_quoteTmWith_prev
  v{ : Nat → String} {k : Nat} ρ{ : QuoteEnv n}
  (hcompat : QuoteCompat v k ρ) (t : PureTm n) :
  isFresh v( k) (quoteTmWith v (k + 1) ρ t) = true := by
  unfold isFresh
  rw [Bool.not_eq_true']
  apply Bool.eq_false_iff.mpr
  intro hcontains
  have hz : v k ∈ freeVars (quoteTmWith v (k + 1) ρ t) := List.contains_iff_mem.mp hcontains

```

```

rcases freeVars_quoteTmWith_mem_env v (k + 1) p t hz with (i, )hi
exact (hcompat.2 i k (by omega)) hi

/-- Closing the current binder name after weakening-quote through `envCons` is inert:
`rename wk` removes references to the newly consed head, and freshness discharges the close. -/
theorem closeFVar_quoteTmWith_rename_wk_envCons
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n}
  (hcompat : QuoteCompat v k p) (t : PureTm n) :
  closeFVar 0 v( k)
    (quoteTmWith v (k + 1) (envCons v( k) p) (rename wk t))
  =
  quoteTmWith v (k + 1) p t := by
calc
  closeFVar 0 v( k)
    (quoteTmWith v (k + 1) (envCons v( k) p) (rename wk t))
  =
  closeFVar 0 v( k) (quoteTmWith v (k + 1) p t) := by
    simpa using congrArg (closeFVar 0 v( k))
      (quoteTmWith_rename_wk_envCons v (k + 1) v( k) p t)
  _ = quoteTmWith v (k + 1) p t := by
    exact closeFVar_fresh_id 0 v( k) (quoteTmWith v (k + 1) p t)
      (isFresh_quoteTmWith_prev hcompat t)

/-- Commute singleton substitution past closing the *next* binder name, for quoted arguments. -/
theorem applySubst_single_closeFVar_comm_quote_next
  v{ : Nat → String} {k : Nat} p{ : QuoteEnv n} {a : PureTm n}
  (hcompat : QuoteCompat v k p) {p : Pattern}
  (hnes : noExplicitSubst p = true) :
  applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
    (closeFVar 0 v( (k + 1)) p) =
  closeFVar 0 v( (k + 1))
    (applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a)) p) := by
  refine applySubst_single_closeFVar_comm ?hneq ?hfresh hnes
  · intro hEq
    have hk : k = k + 1 := hcompat.1 hEq.symm
    omega
  · exact isFresh_quoteTmWith_next hcompat a

theorem quoteSubstEnv_find_some_fresh_future
  v( : Nat → String) (k : Nat) p(src : QuoteEnv n) p(dst : QuoteEnv m) σ( : Sub n m)
  (hcompatDst : QuoteCompat v k pdst) {name : String} {r : Pattern} {j : Nat} (hj : k ≤ j)
  (hfind : (quoteSubstEnv v k psrc pdst σ).find name = some r) :
  isFresh v( j) r = true := by
rcases quoteSubstEnv_find_some_exists v k psrc pdst σ hfind with (i, _, )rfl
exact isFresh_quoteTmWith_future hcompatDst σ( i) (j := j) hj

theorem applySubst_quoteSubstEnv_closeFVar_comm_future
  v( : Nat → String) (k : Nat) p(src : QuoteEnv n) p(dst : QuoteEnv m) σ( : Sub n m)
  (hcompatSrc : QuoteCompat v k psrc) (hcompatDst : QuoteCompat v k pdst)
  {j : Nat} (hj : k ≤ j) {p : Pattern} []{ : Nat}
  (hnes : noExplicitSubst p = true) :
  applySubst (quoteSubstEnv v k psrc pdst σ) (closeFVar [] v( j) p) =
  closeFVar [] v( j) (applySubst (quoteSubstEnv v k psrc pdst σ) p) := by
  suffices h :

```

```

    ∀ p : Pattern, ∀ j : Nat, noExplicitSubst p = true →
      applySubst (quoteSubstEnv v k psrc pdst σ) (closeFVar j v( j) p) =
        closeFVar j v( j) (applySubst (quoteSubstEnv v k psrc pdst σ) p) from
  h p hnes
intro p
induction p using Pattern.inductionOn with
| hbvar n =>
  intro _ _
  simp [applySubst, closeFVar]
| hfvar name =>
  intro _ _
  by_cases hname : name = v j
  · subst hname
  have hnone :
    (quoteSubstEnv v k psrc pdst σ).find v( j) = none :=
      quoteSubstEnv_find_none_future v k psrc pdst σ hcompatSrc hj
  simp [applySubst, closeFVar, hnone]
  · by_cases hfind : (quoteSubstEnv v k psrc pdst σ).find name = none
  · simp [applySubst, closeFVar, hname, hfind]
  · rcases Option.ne_none_iff_exists'.mp hfind with {r, }hr
    have hfreshR : isFresh v( j) r = true :=
      quoteSubstEnv_find_some_fresh_future v k psrc pdst σ hcompatDst hj hr
    have hclose : closeFVar j v( j) r = r := closeFVar_fresh_id j v( j) r hfreshR
    simp [applySubst, closeFVar, hname, hr, hclose]
| happly c args ih =>
  intro _ hnes
  simp [applySubst, closeFVar]
  intro a ha
  simpa using ih a ha (allNoExplicitSubst_mem hnes ha)
| hlamba body ih =>
  intro _ hnes
  simpa [applySubst, closeFVar] using ih (j + 1) (by simpa [noExplicitSubst] using hnes)
| hmultiLamba n body ih =>
  intro _ hnes
  simpa [applySubst, closeFVar] using ih (j + n) (by simpa [noExplicitSubst] using hnes)
| hsubst body repl _ =>
  intro _ hnes
  exact absurd hnes Bool.false_ne_true
| hcollection ct elems rest ih =>
  intro _ hnes
  simp [applySubst, closeFVar]
  intro a ha
  simpa using ih a ha (allNoExplicitSubst_mem hnes ha)

theorem noExplicitSubst_quoteTmWith v( : Nat → String) (k : Nat) p( : QuoteEnv n)
  (t : PureTm n) :
  noExplicitSubst (quoteTmWith v k p t) = true := by
induction t generalizing k with
| var =>
  simp [quoteTmWith, noExplicitSubst]
| const c =>
  simp [quoteTmWith, quoteConst, noExplicitSubst, allNoExplicitSubst]
| u0 =>
  simp [quoteTmWith, u0, noExplicitSubst, allNoExplicitSubst]

```



```

| u1 =>
  simp [quoteTmWith, u1, noExplicitSubst, allNoExplicitSubst]
| pi A B ihA ihB =>
  simp [quoteTmWith, mkPi, noExplicitSubst, allNoExplicitSubst,
    ihA (k := k), noExplicitSubst_closeFVar,
    ihB (k := k + 1) ρ( := envCons v( k) ρ)]
| sigma A B ihA ihB =>
  simp [quoteTmWith, mkSigma, noExplicitSubst, allNoExplicitSubst,
    ihA (k := k), noExplicitSubst_closeFVar,
    ihB (k := k + 1) ρ( := envCons v( k) ρ)]
| id A a b ihA iha ihb =>
  simp [quoteTmWith, mkId, noExplicitSubst, allNoExplicitSubst,
    ihA (k := k), iha (k := k), ihb (k := k)]
| lam b ih =>
  simp [quoteTmWith, mkLam, noExplicitSubst, allNoExplicitSubst,
    noExplicitSubst_closeFVar, ih (k := k + 1) ρ( := envCons v( k) ρ)]
| app f a ihf iha =>
  simp [quoteTmWith, mkApp, noExplicitSubst, allNoExplicitSubst,
    ihf (k := k), iha (k := k)]
| pair a b iha ihb =>
  simp [quoteTmWith, mkPair, noExplicitSubst, allNoExplicitSubst,
    iha (k := k), ihb (k := k)]
| fst p ih =>
  simp [quoteTmWith, mkFst, noExplicitSubst, allNoExplicitSubst, ih (k := k)]
| snd p ih =>
  simp [quoteTmWith, mkSnd, noExplicitSubst, allNoExplicitSubst, ih (k := k)]
| refl a iha =>
  simp [quoteTmWith, mkRefl, noExplicitSubst, allNoExplicitSubst, iha (k := k)]

theorem lc_quoteTmWith v( : Nat → String) (k : Nat) ρ( : QuoteEnv n) (t : PureTm n) :
  lc_at 0 (quoteTmWith v k ρ t) = true := by
  induction t generalizing k with
  | var =>
    simp [quoteTmWith, lc_at]
  | const c =>
    simp [quoteTmWith, quoteConst, lc_at, lc_at_list]
  | u0 =>
    simp [quoteTmWith, u0, lc_at, lc_at_list]
  | u1 =>
    simp [quoteTmWith, u1, lc_at, lc_at_list]
  | pi A B ihA ihB =>
    have hA := ihA (k := k)
    have hB0 := ihB (k := k + 1) ρ( := envCons v( k) ρ)
    have hB1 : lc_at 1 (quoteTmWith v (k + 1) (envCons v( k) ρ) B) = true :=
      lc_at_mono hB0 (Nat.zero_le 1)
    have hBClosed : lc_at 1 (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) ρ) B)) =
      true :=
      lc_at_closeFVar_of_lt (x := v k) (p := quoteTmWith v (k + 1) (envCons v( k) ρ) B)
      (by omega) hB1
    simp [quoteTmWith, mkPi, lc_at, lc_at_list, hA, hBClosed]
  | sigma A B ihA ihB =>
    have hA := ihA (k := k)
    have hB0 := ihB (k := k + 1) ρ( := envCons v( k) ρ)
    have hB1 : lc_at 1 (quoteTmWith v (k + 1) (envCons v( k) ρ) B) = true :=

```

```

    lc_at_mono hb0 (Nat.zero_le 1)
  have hBClosed : lc_at 1 (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) p) B)) =
  true :=
    lc_at_closeFVar_of_lt (x := v k) (p := quoteTmWith v (k + 1) (envCons v( k) p) B)
    (by omega) hb1
  simp [quoteTmWith, mkSigma, lc_at, lc_at_list, hA, hBClosed]
| id A a b ihA iha ihb =>
  simp [quoteTmWith, mkId, lc_at, lc_at_list, iha (k := k), iha (k := k), ihb (k := k)]
| lam b ih =>
  have hb0 := ih (k := k + 1) p( := envCons v( k) p)
  have hb1 : lc_at 1 (quoteTmWith v (k + 1) (envCons v( k) p) b) = true :=
    lc_at_mono hb0 (Nat.zero_le 1)
  have hBClosed : lc_at 1 (closeFVar 0 v( k) (quoteTmWith v (k + 1) (envCons v( k) p) b)) =
  true :=
    lc_at_closeFVar_of_lt (x := v k) (p := quoteTmWith v (k + 1) (envCons v( k) p) b)
    (by omega) hb1
  simp [quoteTmWith, mkLam, lc_at, lc_at_list, hBClosed]
| app f a ihf iha =>
  simp [quoteTmWith, mkApp, lc_at, lc_at_list, ihf (k := k), iha (k := k)]
| pair a b iha ihb =>
  simp [quoteTmWith, mkPair, lc_at, lc_at_list, iha (k := k), ihb (k := k)]
| fst p ih =>
  simp [quoteTmWith, mkFst, lc_at, lc_at_list, ih (k := k)]
| snd p ih =>
  simp [quoteTmWith, mkSnd, lc_at, lc_at_list, ih (k := k)]
| refl a iha =>
  simp [quoteTmWith, mkRefl, lc_at, lc_at_list, iha (k := k)]

/--  $\beta$ -critical bridge step for contextual LN quotation:
opening the closed quoted body by an argument equals singleton substitution on the quoted body.
-/
theorem quoteTmWith_open_close_as_applySubst
  v( : Nat → String) (k : Nat) p( : QuoteEnv n)
  (a : PureTm n) (body : PureTm (n + 1)) :
  let x := v k
  let qb := quoteTmWith v (k + 1) (envCons x p) body
  openBVar 0 (quoteTmWith v k p a) (closeFVar 0 x qb) =
  applySubst (SubstEnv.extend SubstEnv.empty x (quoteTmWith v k p a)) qb := by
  simp only
  let x := v k
  let qb := quoteTmWith v (k + 1) (envCons x p) body
  have hLcQb : lc_at 0 qb = true := by
    simp [qb] using lc_quoteTmWith v (k + 1) (envCons x p) body
  have hFreshClosed : isFresh x (closeFVar 0 x qb) = true := by
    simp [qb] using isFresh_closeFVar_self 0 x qb
  have hNesClosed : noExplicitSubst (closeFVar 0 x qb) = true := by
    have hNesQb : noExplicitSubst qb = true := by
      simp [qb] using noExplicitSubst_quoteTmWith v (k + 1) (envCons x p) body
    simp [noExplicitSubst_closeFVar] using hNesQb
  have hSubstIntro :=
    subst_intro (z := x) (u := quoteTmWith v k p a) (p := closeFVar 0 x qb)
    hFreshClosed hNesClosed
  have hOpenClose : openBVar 0 (.fvar x) (closeFVar 0 x qb) = qb := by
    simp [qb] using open_close_id 0 x qb hLcQb

```

```

rw [hOpenClose] at hSubstIntro
simpa using hSubstIntro.symm

/-- Contextual quotation commutes with kernel `inst0` into LN opening form. -/
theorem quoteTmWith_inst0_open
  (v : Nat → String) (k : Nat) (p : QuoteEnv n)
  (a : PureTm n) (body : PureTm (n + 1))
  (hinst0 :
    quoteTmWith v k p (inst0 a body) =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
        (quoteTmWith v (k + 1) (envCons v( k) p) body)) :
    quoteTmWith v k p (inst0 a body) =
      openBVar 0 (quoteTmWith v k p a)
        (closeFVar 0 v( k)
          (quoteTmWith v (k + 1) (envCons v( k) p) body)) := by
calc
  quoteTmWith v k p (inst0 a body)
    = applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
      (quoteTmWith v (k + 1) (envCons v( k) p) body) := hinst0
_ = openBVar 0 (quoteTmWith v k p a)
  (closeFVar 0 v( k)
    (quoteTmWith v (k + 1) (envCons v( k) p) body)) := by
  symm
  simpa using quoteTmWith_open_close_as_applySubst v k p a body

theorem inst0ApplyBridge_to_openBridge
  (v{ : Nat → String} (hinst0 : Inst0ApplyBridge v) :
    Inst0OpenBridge v := by
intro n k p a body
exact quoteTmWith_inst0_open v k p a body (hinst0 k p a body)

theorem inst0ApplyBridgeCompat_to_openBridgeCompat
  (v{ : Nat → String} (hinst0 : Inst0ApplyBridgeCompat v) :
    Inst0OpenBridgeCompat v := by
intro n k p a body hcompat
exact quoteTmWith_inst0_open v k p a body (hinst0 k p a body hcompat)

/-- Canonical `inst0` bridge at the quoted-pattern layer:
kernel substitution commutes with contextual LN quotation via singleton `applySubst`. -/
theorem quoteTmWith_inst0
  (v : Nat → String) (k : Nat) (p : QuoteEnv n)
  (a : PureTm n) (body : PureTm (n + 1))
  (hinst0 :
    quoteTmWith v k p (inst0 a body) =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
        (quoteTmWith v (k + 1) (envCons v( k) p) body)) :
    quoteTmWith v k p (inst0 a body) =
      applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k p a))
        (quoteTmWith v (k + 1) (envCons v( k) p) body) := by
exact hinst0

/-- Explicit-compat specialization requested by bridge clients.
Carries an explicit `inst0` bridge hypothesis until the unconditional theorem is proved. -/
theorem quoteTmWith_defaultBinderName_inst0_open

```

```

(k : Nat) ρ( : QuoteEnv n) (hcompat : QuoteCompat defaultBinderName k ρ)
(a : PureTm n) (body : PureTm (n + 1))
(hinst0 :
  quoteTmWith defaultBinderName k ρ (inst0 a body) =
    applySubst (SubstEnv.extend SubstEnv.empty (defaultBinderName k)
      (quoteTmWith defaultBinderName k ρ a))
      (quoteTmWith defaultBinderName (k + 1)
        (envCons (defaultBinderName k) ρ) body)) :
  quoteTmWith defaultBinderName k ρ (inst0 a body) =
    openBVar 0 (quoteTmWith defaultBinderName k ρ a)
      (closeFVar 0 (defaultBinderName k)
        (quoteTmWith defaultBinderName (k + 1)
          (envCons (defaultBinderName k) ρ) body))) := by
have _ := hcompat
simp using quoteTmWith_inst0_open defaultBinderName k ρ a body hinst0

theorem quoteTmWith_defaultBinderName_inst0_open_assuming_inst0Compat
  (hinst0 : Inst0ApplyBridgeCompat defaultBinderName)
  (k : Nat) ρ( : QuoteEnv n) (hcompat : QuoteCompat defaultBinderName k ρ)
  (a : PureTm n) (body : PureTm (n + 1)) :
  quoteTmWith defaultBinderName k ρ (inst0 a body) =
    openBVar 0 (quoteTmWith defaultBinderName k ρ a)
      (closeFVar 0 (defaultBinderName k)
        (quoteTmWith defaultBinderName (k + 1)
          (envCons (defaultBinderName k) ρ) body))) := by
exact quoteTmWith_inst0_open defaultBinderName k ρ a body
  (hinst0 k ρ a body hcompat)

@[simp] theorem quoteTmWith_inst0_var_zero
  v( : Nat → String) (k : Nat) ρ( : QuoteEnv n) (a : PureTm n) :
  quoteTmWith v k ρ (inst0 a (.var 0)) =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k ρ a))
      (quoteTmWith v (k + 1) (envCons v( k) ρ) (.var 0))) := by
simp [inst0, quoteTmWith, envCons, applySubst, SubstEnv.find_extend_empty_eq]

@[simp] theorem quoteTmWith_inst0_var_succ
  v( : Nat → String) (k : Nat) ρ( : QuoteEnv n) (hcompat : QuoteCompat v k ρ)
  (a : PureTm n) (i : Fin n) :
  quoteTmWith v k ρ (inst0 a (.var i.succ)) =
    applySubst (SubstEnv.extend SubstEnv.empty v( k) (quoteTmWith v k ρ a))
      (quoteTmWith v (k + 1) (envCons v( k) ρ) (.var i.succ))) := by
have hne : ρ i ≠ v k := hcompat.2 i k (by omega)
have hne' : v k ≠ ρ i := fun h => hne h.symm
simp [inst0, quoteTmWith, envCons, applySubst, SubstEnv.find_extend_empty_ne hne']

/--
Counterexample: scoped `inst0` and ambient `openBVar 0` do not agree under quoting.
This exposes the non-shifting behavior of `openBVar` in the ambient Pattern layer.
-/
theorem quoteRaw_inst0_ne_openBVar_witness :
  quoteRaw (inst0 (.u0 : PureTm 1) (.var (Fin.succ (0 : Fin 1)) : PureTm 2))
    ≠ openBVar 0 (quoteRaw (.u0 : PureTm 1))
      (quoteRaw (.var (Fin.succ (0 : Fin 1)) : PureTm 2)) := by
have hleft :

```

```

      quoteRaw (inst0 (.u0 : PureTm 1) (.var (Fin.succ (0 : Fin 1)) : PureTm 2))
        = Pattern.bvar 0 := by
    rfl
  have hright :
    openBVar 0 (quoteRaw (.u0 : PureTm 1))
      (quoteRaw (.var (Fin.succ (0 : Fin 1)) : PureTm 2))
        = Pattern.bvar 1 := by
    simp [openBVar, quoteRaw]
  rw [hleft, hright]
  intro h
  cases h
end Mettapedia.Languages.MeTTa.PureKernel.PatternBridge

```

A.16 PatternBridgeSubst.lean

Mettapedia/Languages/MeTTa/PureKernel/PatternBridgeSubst.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.PatternBridge

/-!
# PatternBridgeSubst (Workbench)

This module is intentionally lightweight and non-imported.
It hosts bridge-oriented aliases and local scaffolding while the main
substitution transport theorem is developed directly in `PatternBridge.lean`.
-/

namespace Mettapedia.Languages.MeTTa.PureKernel.PatternBridge

open Mettapedia.OSLF.MeTTaIL.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.OSLF.MeTTaIL.Substitution

/-- Workbench alias of the generic renaming bridge. -/
theorem quoteTmWith_rename_workbench
  (v : Nat → String) {n m : Nat} (k : Nat)
  (p : QuoteEnv m) (p' : Ren n m) (t : PureTm n) :
  quoteTmWith v k p (p' t) =
    quoteTmWith v k (fun i => p (i)) t :=
  quoteTmWith_rename v k p p' t

/-- Workbench alias for weakening through an `envCons` quote environment. -/
theorem quoteTmWith_rename_wk_envCons_workbench
  (v : Nat → String) (k : Nat) (x : String) (p : QuoteEnv n) (t : PureTm n) :
  quoteTmWith v k (envCons x p) (p' t) =
    quoteTmWith v k p t :=
  quoteTmWith_rename_wk_envCons v k x p t

end Mettapedia.Languages.MeTTa.PureKernel.PatternBridge

```

A.17 ProfileTheory.lean

Mettapedia/Languages/MeTTa/PureKernel/ProfileTheory.lean

```

import Mettapedia.Languages.MeTTa.Pure.Core
import Mettapedia.OSLF.Framework.TypeSynthesis
import Mettapedia.OSLF.MeTTaIL.Substitution

/-!
# PureKernel Profile-Theory Layer (C1)

This file defines the profile-side theory closure for MeTTa-Pure:

- a sealed base step relation with exactly the three pure  $\beta$  rules
- pure `Pattern` contexts (`PurePatCtx`)
- contextual closure (`PureProfileTheoryStep`)
- reflexive-transitive closure (`PureProfileTheoryStepStar`)

This is the C1 layer in the A/B/C architecture.
-/

namespace Mettapedia.Languages.MeTTa.PureKernel.ProfileTheory

open Mettapedia.OSLF.MeTTaIL.Syntax
open Mettapedia.OSLF.Framework.TypeSynthesis
open Mettapedia.OSLF.MeTTaIL.Match
open Mettapedia.OSLF.MeTTaIL.Engine
open Mettapedia.OSLF.MeTTaIL.Substitution
open Mettapedia.Languages.MeTTa.Pure.Core

/-- Reflexive-transitive closure of profile-level one-step reduction for `mettaPure`. -/
abbrev PureProfileStepStar (p q : Pattern) : Prop :=
  Relation.ReflTransGen (langReduces mettaPure) p q

/-- Sealed base one-step relation for the pure profile:
exactly  $\beta\eta$ ,  $\beta\Sigma$ -fst,  $\beta\Sigma$ -snd at the `Pattern` level. -/
inductive PureProfileBaseStep : Pattern → Pattern → Prop where
| betaPi (body a : Pattern) :
  PureProfileBaseStep (mkApp (mkLam body) a) (openBVar 0 a body)
| betaSigmaFst (a b : Pattern) :
  PureProfileBaseStep (mkFst (mkPair a b)) a
| betaSigmaSnd (a b : Pattern) :
  PureProfileBaseStep (mkSnd (mkPair a b)) b

private def betaPiRule : RewriteRule :=
{ name := "BetaPi",
  typeContext := [ ("body", .base "Tm"), ("a", .base "Tm") ],
  premises := [],
  left := .apply "App" [.apply "Lam" [.lambda (.fvar "body")], .fvar "a"],
  right := .subst (.fvar "body") (.fvar "a") }

private def betaSigmaFstRule : RewriteRule :=
{ name := "BetaSigmaFst",
  typeContext := [ ("a", .base "Tm"), ("b", .base "Tm") ],
  premises := [],

```

```

left := .apply "Fst" [.apply "Pair" [.fvar "a", .fvar "b"]],
right := .fvar "a" }

private def betaSigmaSndRule : RewriteRule :=
{ name := "BetaSigmaSnd",
  typeContext := [("a", .base "Tm"), ("b", .base "Tm")],
  premises := [],
  left := .apply "Snd" [.apply "Pair" [.fvar "a", .fvar "b"]],
  right := .fvar "b" }

/-- Sealed base steps are sound for profile one-step reduction. -/
theorem pureProfileBaseStep_sound_langReduces {s t : Pattern}
(h : PureProfileBaseStep s t) :
langReduces mettaPure s t := by
cases h with
| betaPi body a =>
  apply exec_to_langReducesUsing (relEnv := RelationEnv.empty) (lang := mettaPure)
  unfold langReducesExecUsing rewriteWithContextWithPremisesUsing
  rewriteStepWithPremisesUsing
  apply List.mem_append.mpr
  left
  rw [List.mem_flatMap]
  refine {betaPiRule, ?_, ?}_
  · simp [betaPiRule, mettaPure]
  · unfold applyRuleWithPremisesUsing
    rw [List.mem_flatMap]
    let bs : Bindings := [("a", a), ("body", body)]
    refine {bs, ?_, ?}_
    · simp [bs, betaPiRule, mkApp, mkLam, matchPattern, matchArgs, mergeBindings]
    · rw [List.mem_map]
      refine {bs, ?_, ?}_
      · simp [applyPremisesWithEnv, bs, betaPiRule]
      · simp [bs, betaPiRule, applyBindings]
| @betaSigmaFst aa bb =>
  apply exec_to_langReducesUsing (relEnv := RelationEnv.empty) (lang := mettaPure)
  unfold langReducesExecUsing rewriteWithContextWithPremisesUsing
  rewriteStepWithPremisesUsing
  apply List.mem_append.mpr
  left
  rw [List.mem_flatMap]
  refine {betaSigmaFstRule, ?_, ?}_
  · simp [betaSigmaFstRule, mettaPure]
  · unfold applyRuleWithPremisesUsing
    rw [List.mem_flatMap]
    let bs : Bindings := [("b", bb), ("a", t)]
    refine {bs, ?_, ?}_
    · simp [bs, betaSigmaFstRule, mkFst, mkPair, matchPattern, matchArgs, mergeBindings]
    · rw [List.mem_map]
      refine {bs, ?_, ?}_
      · simp [applyPremisesWithEnv, bs, betaSigmaFstRule]
      · simp [bs, betaSigmaFstRule, applyBindings]
| @betaSigmaSnd aa bb =>
  apply exec_to_langReducesUsing (relEnv := RelationEnv.empty) (lang := mettaPure)
  unfold langReducesExecUsing rewriteWithContextWithPremisesUsing

```

```

rewriteStepWithPremisesUsing
  apply List.mem_append.mpr
  left
  rw [List.mem_flatMap]
  refine (betaSigmaSndRule, ?_, ?)_
  · simp [betaSigmaSndRule, mettaPure]
  · unfold applyRuleWithPremisesUsing
    rw [List.mem_flatMap]
    let bs : Bindings := [("b", t), ("a", aa)]
    refine (bs, ?_, ?)_
    · simp [bs, betaSigmaSndRule, mkSnd, mkPair, matchPattern, matchArgs, mergeBindings]
    · rw [List.mem_map]
      refine (bs, ?_, ?)_
      · simp [applyPremisesWithEnv, bs, betaSigmaSndRule]
      · simp [bs, betaSigmaSndRule, applyBindings]

/-- Pure term-contexts on the `Pattern` side (C1): exactly the constructor positions
corresponding to kernel congruence (`Red`). -/
inductive PurePatCtx : Type where
| hole : PurePatCtx
| piDom (K : PurePatCtx) (B : Pattern) : PurePatCtx
| piCod (A : Pattern) (K : PurePatCtx) : PurePatCtx
| sigmaDom (K : PurePatCtx) (B : Pattern) : PurePatCtx
| sigmaCod (A : Pattern) (K : PurePatCtx) : PurePatCtx
| idTy (K : PurePatCtx) (a b : Pattern) : PurePatCtx
| idLeft (A : Pattern) (K : PurePatCtx) (b : Pattern) : PurePatCtx
| idRight (A a : Pattern) (K : PurePatCtx) : PurePatCtx
| lam (K : PurePatCtx) : PurePatCtx
| appFun (K : PurePatCtx) (a : Pattern) : PurePatCtx
| appArg (f : Pattern) (K : PurePatCtx) : PurePatCtx
| pairFst (K : PurePatCtx) (b : Pattern) : PurePatCtx
| pairSnd (a : Pattern) (K : PurePatCtx) : PurePatCtx
| fst (K : PurePatCtx) : PurePatCtx
| snd (K : PurePatCtx) : PurePatCtx
| refl (K : PurePatCtx) : PurePatCtx
| close (x : String) (K : PurePatCtx) : PurePatCtx

/-- Plug a `Pattern` into a pure term-context. -/
def plugPurePatCtx : PurePatCtx → Pattern → Pattern
| .hole, p => p
| .piDom K B, p => mkPi (plugPurePatCtx K p) B
| .piCod A K, p => mkPi A (plugPurePatCtx K p)
| .sigmaDom K B, p => mkSigma (plugPurePatCtx K p) B
| .sigmaCod A K, p => mkSigma A (plugPurePatCtx K p)
| .idTy K a b, p => mkId (plugPurePatCtx K p) a b
| .idLeft A K b, p => mkId A (plugPurePatCtx K p) b
| .idRight A a K, p => mkId A a (plugPurePatCtx K p)
| .lam K, p => mkLam (plugPurePatCtx K p)
| .appFun K a, p => mkApp (plugPurePatCtx K p) a
| .appArg f K, p => mkApp f (plugPurePatCtx K p)
| .pairFst K b, p => mkPair (plugPurePatCtx K p) b
| .pairSnd a K, p => mkPair a (plugPurePatCtx K p)
| .fst K, p => mkFst (plugPurePatCtx K p)
| .snd K, p => mkSnd (plugPurePatCtx K p)

```



```

| .refl K, p => mkRefl (plugPurePatCtx K p)
| .close x K, p => closeFVar 0 x (plugPurePatCtx K p)

/-- C1 one-step profile-theory relation:
least contextual closure (under pure contexts) of the sealed base  $\beta$  rules. -/
inductive PureProfileTheoryStep : Pattern → Pattern → Prop where
| base {s t : Pattern} :
  PureProfileBaseStep s t →
  PureProfileTheoryStep s t
| ctx {K : PurePatCtx} {s t : Pattern} :
  PureProfileTheoryStep s t →
  PureProfileTheoryStep (plugPurePatCtx K s) (plugPurePatCtx K t)

/-- C1 star closure. -/
abbrev PureProfileTheoryStepStar (p q : Pattern) : Prop :=
  Relation.ReflTransGen PureProfileTheoryStep p q

/-- Any sealed base step is a C1 step. -/
theorem pureProfileBaseStep_to_pureProfileTheoryStep {s t : Pattern}
  (h : PureProfileBaseStep s t) :
  PureProfileTheoryStep s t :=
  .base h

/-- Any sealed base step is a C1-star step. -/
theorem pureProfileBaseStep_to_pureProfileTheoryStepStar {s t : Pattern}
  (h : PureProfileBaseStep s t) :
  PureProfileTheoryStepStar s t :=
  Relation.ReflTransGen.tail Relation.ReflTransGen.refl (.base h)

end Mettapedia.Languages.MeTTa.PureKernel.ProfileTheory

```

A.18 RecursorDecl.lean

Mettapedia/Languages/MeTTa/PureKernel/RecursorDecl.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.UnitDecl
import Mettapedia.Languages.MeTTa.PureKernel.BoolDecl
import Mettapedia.Languages.MeTTa.PureKernel.NatDecl

namespace Mettapedia.Languages.MeTTa.PureKernel.RecursorDecl

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics
open Mettapedia.Languages.MeTTa.PureKernel.Reduction
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec
open Mettapedia.Languages.MeTTa.PureKernel.UnitDecl
open Mettapedia.Languages.MeTTa.PureKernel.BoolDecl
open Mettapedia.Languages.MeTTa.PureKernel.NatDecl

/-- Minimal declaration-level recursor contract record for the current kernel pilot layer. -/
structure FamilyRecursorDeclContract where
  familyName : DeclName

```

```

recursorName : DeclName
recursorType : PureTm 0
deriving Repr

def unitRecName : DeclName := `Unit.rec
def boolRecName : DeclName := `Bool.rec
def natRecName : DeclName := `Nat.rec

/-- Pilot executable body for non-dependent `Unit.rec`: `fun _R r _u => r`. -/
def unitRecValue : PureTm 0 :=
  .lam (.lam (.lam (.var (1 : Fin 3))))

/-- Non-dependent Unit recursor shape:  $\forall` R : U0, R \rightarrow Unit \rightarrow R`$ . -/
def unitRecType : PureTm 0 :=
  .pi .u0
    (.pi (.var 0)
      (.pi (.const unitTyName) (.var 2)))

/-- Non-dependent Bool recursor shape:  $\forall` R : U0, R \rightarrow R \rightarrow Bool \rightarrow R`$ . -/
def boolRecType : PureTm 0 :=
  .pi .u0
    (.pi (.var 0)
      (.pi (.var 1)
        (.pi (.const boolTyName) (.var 3))))

/-- Non-dependent Nat step premise shape in context `(R : U0), (base : R)` -/
def natStepType : PureTm 2 :=
  .pi (.const natTyName)
    (.pi (.var 2) (.var 3))

/-- Non-dependent Nat recursor shape:  $\forall` R : U0, R \rightarrow (Nat \rightarrow R \rightarrow R) \rightarrow Nat \rightarrow R`$ . -/
def natRecType : PureTm 0 :=
  .pi .u0
    (.pi (.var 0)
      (.pi natStepType
        (.pi (.const natTyName) (.var 3))))

def unitRecContract : FamilyRecursorDeclContract :=
  { familyName := unitTyName, recursorName := unitRecName, recursorType := unitRecType }

def boolRecContract : FamilyRecursorDeclContract :=
  { familyName := boolTyName, recursorName := boolRecName, recursorType := boolRecType }

def natRecContract : FamilyRecursorDeclContract :=
  { familyName := natTyName, recursorName := natRecName, recursorType := natRecType }

theorem unitRecContract_name : unitRecContract.recursorName = `Unit.rec := rfl
theorem boolRecContract_name : boolRecContract.recursorName = `Bool.rec := rfl
theorem natRecContract_name : natRecContract.recursorName = `Nat.rec := rfl

def unitRecSpec : DeclSpec :=
  { name := unitRecName, type := unitRecType, value? := some unitRecValue }

def boolRecSpec : DeclSpec :=

```

```

    { name := boolRecName, type := boolRecType }

def natRecSpec : DeclSpec :=
  { name := natRecName, type := natRecType }

def unitRecSpecs : List DeclSpec :=
  unitSpecs ++ [unitRecSpec]

def boolRecSpecs : List DeclSpec :=
  boolSpecs ++ [boolRecSpec]

def natRecSpecs : List DeclSpec :=
  natSpecs ++ [natRecSpec]

def unitRecDeclEnv : DeclEnv := envOfSpecs unitRecSpecs
def boolRecDeclEnv : DeclEnv := envOfSpecs boolRecSpecs
def natRecDeclEnv : DeclEnv := envOfSpecs natRecSpecs

@[simp] theorem typeOf_unitRec :
  typeOf? unitRecDeclEnv unitRecName = some unitRecType := by
  decide

@[simp] theorem typeOf_boolRec :
  typeOf? boolRecDeclEnv boolRecName = some boolRecType := by
  decide

@[simp] theorem typeOf_natRec :
  typeOf? natRecDeclEnv natRecName = some natRecType := by
  decide

theorem hasType_unitRec :
  HasTypeDecl unitRecDeclEnv .nil ((.const unitRecName : PureTm 0)) unitRecType :=
  hasType_const_from_lookup (E := unitRecDeclEnv)  $\Gamma$ ( := .nil) (c := unitRecName) (A0 :=
    unitRecType) (by
    simp)

theorem hasType_boolRec :
  HasTypeDecl boolRecDeclEnv .nil ((.const boolRecName : PureTm 0)) boolRecType :=
  hasType_const_from_lookup (E := boolRecDeclEnv)  $\Gamma$ ( := .nil) (c := boolRecName) (A0 :=
    boolRecType) (by
    simp)

theorem hasType_natRec :
  HasTypeDecl natRecDeclEnv .nil ((.const natRecName : PureTm 0)) natRecType :=
  hasType_const_from_lookup (E := natRecDeclEnv)  $\Gamma$ ( := .nil) (c := natRecName) (A0 := natRecType)
  (by
    simp)

private theorem boolRecSpecs_hTyped :
   $\forall s \in \text{boolRecSpecs}, \forall v0 : \text{PureTm } 0,$ 
  s.value? = some v0  $\rightarrow$ 
  HasTypeDecl (envOfSpecs boolRecSpecs) .nil (liftClosed v0) (liftClosed s.type) := by
  intro s hs v0 hVal
  simp [boolRecSpecs] at hs

```

```

rcases hs with hs | rfl
· exact
  boolSpecs_hTyped_in
    (E := envOfSpecs boolRecSpecs)
    (hTrueType := by decide)
    s hs v0 hVal
· simp [boolRecSpec] at hVal

private theorem boolRecSpecs_hNoSelf :
  ∀ s ∈ boolRecSpecs, ∀ v0 : PureTm 0,
  s.value? = some v0 →
  v0 ≠ (.const s.name) := by
  intro s hs v0 hVal
  simp [boolRecSpecs] at hs
  rcases hs with hs | rfl
  · exact boolSpecs_hNoSelf s hs v0 hVal
  · simp [boolRecSpec] at hVal

private def boolRecSpecs_obligations : DeclSpecObligations boolRecSpecs where
  valuesWellTyped := boolRecSpecs_hTyped
  noSelfDelta := boolRecSpecs_hNoSelf

private def boolRecSpecs_signatureWellFormed : SignatureWellFormed boolRecSpecs where
  noShadowing := by decide
  obligations := boolRecSpecs_obligations

theorem boolRecDeclEnv_wellFormed : DeclEnvWellFormed boolRecDeclEnv := by
  unfold boolRecDeclEnv
  exact boolRecSpecs_signatureWellFormed.toDeclEnvWellFormed

theorem boolRecSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible boolSpecs boolRecSpec where
  fresh := by
    simp [boolSpecs, boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec,
      boolRecSpec, boolTyName, boolTrueName, boolFalseName, boolAliasName, boolRecName]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames, boolRecSpec, boolRecType,
      boolSpecs, boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec,
      boolTyName, boolTrueName, boolFalseName, boolAliasName, boolRecName]
  valueUsesEarlier := by
    intro v0 hVal
    simp [boolRecSpec] at hVal
  valueWellTyped := by
    intro v0 hVal
    simp [boolRecSpec] at hVal
  noSelfDelta := by
    intro v0 hVal
    simp [boolRecSpec] at hVal

theorem boolRecSpecs_prefixWellFormed :
  SignatureWellFormedPrefix boolRecSpecs := by
  refine And.intro boolTySpec_prefixAdmissible ?_
  refine And.intro ?_ ?_
  · simp [boolRecSpecs, boolSpecs, boolTySpec, boolTrueSpec]

```

```

    using boolTrueSpec_prefixAdmissible
  · refine And.intro ?_ ?_
  · simp [PrefixSignatureWellFormed, boolRecSpecs, boolSpecs, boolTySpec, boolTrueSpec,
    boolFalseSpec]
    using boolFalseSpec_prefixAdmissible
  · refine And.intro ?_ ?_
  · simp [PrefixSignatureWellFormed, boolRecSpecs, boolSpecs,
    boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec]
    using boolAliasSpec_prefixAdmissible
  · refine And.intro ?_ trivial
    simp [PrefixSignatureWellFormed, boolRecSpecs, boolSpecs,
    boolTySpec, boolTrueSpec, boolFalseSpec, boolAliasSpec, boolRecSpec]
    using boolRecSpec_prefixAdmissible

private theorem boolRecSpecs_signatureWellFormed_fromPrefix :
  SignatureWellFormed boolRecSpecs :=
  SignatureWellFormed.ofPrefix boolRecSpecs_prefixWellFormed boolRecSpecs_obligations

private theorem natRecSpecs_hTyped :
  ∀ s ∈ natRecSpecs, ∀ v0 : PureTm 0,
  s.value? = some v0 →
  HasTypeDecl (envOfSpecs natRecSpecs) .nil (liftClosed v0) (liftClosed s.type) := by
  intro s hs v0 hVal
  simp [natRecSpecs] at hs
  rcases hs with hs | rfl
  · exact
    natSpecs_hTyped_in
    (E := envOfSpecs natRecSpecs)
    (hZeroType := by decide)
    s hs v0 hVal
  · simp [natRecSpec] at hVal

private theorem natRecSpecs_hNoSelf :
  ∀ s ∈ natRecSpecs, ∀ v0 : PureTm 0,
  s.value? = some v0 →
  v0 ≠ (.const s.name) := by
  intro s hs v0 hVal
  simp [natRecSpecs] at hs
  rcases hs with hs | rfl
  · exact natSpecs_hNoSelf s hs v0 hVal
  · simp [natRecSpec] at hVal

private def natRecSpecs_obligations : DeclSpecObligations natRecSpecs where
  valuesWellTyped := natRecSpecs_hTyped
  noSelfDelta := natRecSpecs_hNoSelf

private def natRecSpecs_signatureWellFormed : SignatureWellFormed natRecSpecs where
  noShadowing := by decide
  obligations := natRecSpecs_obligations

theorem natRecDeclEnv_wellFormed : DeclEnvWellFormed natRecDeclEnv := by
  unfold natRecDeclEnv
  exact natRecSpecs_signatureWellFormed.toDeclEnvWellFormed

```

```

theorem natRecSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible natSpecs natRecSpec where
  fresh := by
    simp [natSpecs, natTySpec, natZeroSpec, natSuccSpec, natAliasSpec,
      natRecSpec, natTyName, natZeroName, natSuccName, natAliasName, natRecName]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames, natRecSpec, natRecType, natStepType,
      natSpecs, natTySpec, natZeroSpec, natSuccSpec, natAliasSpec,
      natTyName, natZeroName, natSuccName, natAliasName, natRecName]
  valueUsesEarlier := by
    intro v0 hVal
    simp [natRecSpec] at hVal
  valueWellTyped := by
    intro v0 hVal
    simp [natRecSpec] at hVal
  noSelfDelta := by
    intro v0 hVal
    simp [natRecSpec] at hVal

theorem natRecSpecs_prefixWellFormed :
  SignatureWellFormedPrefix natRecSpecs := by
  refine And.intro natTySpec_prefixAdmissible ?_
  refine And.intro ?_ ?_
  · simp [natRecSpecs, natSpecs, natTySpec, natZeroSpec]
    using natZeroSpec_prefixAdmissible
  · refine And.intro ?_ ?_
    · simp [PrefixSignatureWellFormed, natRecSpecs, natSpecs, natTySpec, natZeroSpec,
      natSuccSpec]
      using natSuccSpec_prefixAdmissible
    · refine And.intro ?_ ?_
      · simp [PrefixSignatureWellFormed, natRecSpecs, natSpecs,
        natTySpec, natZeroSpec, natSuccSpec, natAliasSpec]
        using natAliasSpec_prefixAdmissible
      · refine And.intro ?_ trivial
        simp [PrefixSignatureWellFormed, natRecSpecs, natSpecs,
          natTySpec, natZeroSpec, natSuccSpec, natAliasSpec, natRecSpec]
          using natRecSpec_prefixAdmissible

private theorem natRecSpecs_signatureWellFormed_fromPrefix :
  SignatureWellFormed natRecSpecs :=
  SignatureWellFormed.ofPrefix natRecSpecs_prefixWellFormed natRecSpecs_obligations

@[simp] theorem valueOf_unitRec :
  valueOf? unitRecDeclEnv unitRecName = some unitRecValue := by
  decide

/-- `Unit.rec` executable body is well-typed at its declaration type. -/
theorem hasType_unitRecValue :
  HasTypeDecl unitRecDeclEnv .nil unitRecValue unitRecType := by
  unfold unitRecValue unitRecType
  refine .lam_intro ?_
  refine .lam_intro ?_
  refine .lam_intro ?_
  exact .var (i := (1 : Fin 3))

```

```

theorem hasType_unitRecValue_inUnitPrefix :
  HasTypeDecl (envOfSpecs unitSpecs) .nil unitRecValue unitRecType := by
  unfold unitRecValue unitRecType
  refine .lam_intro ?_
  refine .lam_intro ?_
  refine .lam_intro ?_
  exact .var (i := (1 : Fin 3))

/-- Generic closed checked-step for unfolding `Unit.rec` via declaration semantics. -/
theorem unitRec_checked_delta :
  ∃ A : PureTm 0,
    HasTypeDecl unitRecDeclEnv .nil ((.const unitRecName : PureTm 0)) A ∧
    RedDecl unitRecDeclEnv ((.const unitRecName : PureTm 0)) unitRecValue ∧
    HasTypeDecl unitRecDeclEnv .nil unitRecValue A := by
  simp using
    (checked_delta_step_closed_of_typed_value
      (E := unitRecDeclEnv)
      (c := unitRecName)
      (A0 := unitRecType)
      (v0 := unitRecValue)
      (by simp)
      (by simp)
      hasType_unitRecValue)

@[simp] theorem typeOf_unitTy_inRecEnv :
  typeOf? unitRecDeclEnv unitTyName = some .u0 := by
  decide

@[simp] theorem typeOf_unitCtor_inRecEnv :
  typeOf? unitRecDeclEnv unitCtorName = some (.const unitTyName) := by
  decide

@[simp] theorem typeOf_unitAlias_inRecEnv :
  typeOf? unitRecDeclEnv unitAliasName = some (.const unitTyName) := by
  decide

@[simp] theorem valueOf_unitAlias_inRecEnv :
  valueOf? unitRecDeclEnv unitAliasName = some (.const unitCtorName) := by
  decide

theorem hasType_unitTy_inRecEnv :
  HasTypeDecl unitRecDeclEnv .nil ((.const unitTyName : PureTm 0)) .u0 :=
  hasType_const_from_lookup (E := unitRecDeclEnv) Γ( := .nil) (c := unitTyName) (A0 := .u0) (by
    simp)

theorem hasType_unitCtor_inRecEnv :
  HasTypeDecl unitRecDeclEnv .nil ((.const unitCtorName : PureTm 0)) (.const unitTyName) :=
  hasType_const_from_lookup (E := unitRecDeclEnv) Γ( := .nil) (c := unitCtorName) (A0 := .const
    unitTyName) (by
    simp)

theorem hasType_unitAlias_inRecEnv :
  HasTypeDecl unitRecDeclEnv .nil ((.const unitAliasName : PureTm 0)) (.const unitTyName) :=

```

```

hasType_const_from_lookup (E := unitRecDeclEnv)  $\Gamma$ ( := .nil) (c := unitAliasName) (A0 := .const
  unitTyName) (by
    simp)

private theorem unitRecSpecs_hTyped :
   $\forall$  s  $\in$  unitRecSpecs,  $\forall$  v0 : PureTm 0,
    s.value? = some v0  $\rightarrow$ 
    HasTypeDecl (envOfSpecs unitRecSpecs) .nil (liftClosed v0) (liftClosed s.type) := by
intro s hs v0 hVal
simp [unitRecSpecs] at hs
rcases hs with hs | rfl
· exact
  unitSpecs_hTyped_in
  (E := envOfSpecs unitRecSpecs)
  (hCtorType := by decide)
  s hs v0 hVal
· have hv : v0 = unitRecValue := by
  simpa [unitRecSpec] using hVal.symm
  subst hv
  simpa [liftClosed_zero] using hasType_unitRecValue

private theorem unitRecSpecs_hNoSelf :
   $\forall$  s  $\in$  unitRecSpecs,  $\forall$  v0 : PureTm 0,
    s.value? = some v0  $\rightarrow$ 
    v0  $\neq$  (.const s.name) := by
intro s hs v0 hVal
simp [unitRecSpecs] at hs
rcases hs with hs | rfl
· exact unitSpecs_hNoSelf s hs v0 hVal
· have hv : v0 = unitRecValue := by
  simpa [unitRecSpec] using hVal.symm
  subst hv
  intro hEq
  have hNe : unitRecValue  $\neq$  (.const unitRecName) := by
    decide
  exact hNe hEq

private def unitRecSpecs_obligations : DeclSpecObligations unitRecSpecs where
  valuesWellTyped := unitRecSpecs_hTyped
  noSelfDelta := unitRecSpecs_hNoSelf

private def unitRecSpecs_signatureWellFormed : SignatureWellFormed unitRecSpecs where
  noShadowing := by decide
  obligations := unitRecSpecs_obligations

theorem unitRecDeclEnv_wellFormed : DeclEnvWellFormed unitRecDeclEnv := by
  unfold unitRecDeclEnv
  exact unitRecSpecs_signatureWellFormed.toDeclEnvWellFormed

theorem unitRecSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible unitSpecs unitRecSpec where
  fresh := by
    simp [unitSpecs, unitTySpec, unitCtorSpec, unitAliasSpec,
      unitRecSpec, unitTyName, unitCtorName, unitAliasName, unitRecName]

```



```

typeUsesEarlier := by
  simp [UsesOnlyDeclNamesFrom, prefixNames, unitRecSpec, unitRecType,
    unitSpecs, unitTySpec, unitCtorSpec, unitAliasSpec,
    unitTyName, unitCtorName, unitAliasName, unitRecName]
valueUsesEarlier := by
  intro v0 hVal
  have hv : v0 = unitRecValue := by
    simpa [unitRecSpec] using hVal.symm
  subst hv
  simp [UsesOnlyDeclNamesFrom, prefixNames, unitRecValue, unitSpecs,
    unitTySpec, unitCtorSpec, unitAliasSpec]
valueWellTyped := by
  intro v0 hVal
  have hv : v0 = unitRecValue := by
    simpa [unitRecSpec] using hVal.symm
  subst hv
  simpa [liftClosed_zero] using hasType_unitRecValue_inUnitPrefix
noSelfDelta := by
  intro v0 hVal
  have hv : v0 = unitRecValue := by
    simpa [unitRecSpec] using hVal.symm
  subst hv
  intro hEq
  have hNe : unitRecValue ≠ (.const unitRecName) := by
    decide
  exact hNe hEq

theorem unitRecSpecs_prefixWellFormed :
  SignatureWellFormedPrefix unitRecSpecs := by
  refine And.intro unitTySpec_prefixAdmissible ?_
  refine And.intro ?_ ?_
  · simpa [unitRecSpecs, unitSpecs, unitTySpec, unitCtorSpec]
    using unitCtorSpec_prefixAdmissible
  · refine And.intro ?_ ?_
    · simpa [PrefixSignatureWellFormed, unitRecSpecs, unitSpecs, unitTySpec, unitCtorSpec,
      unitAliasSpec]
      using unitAliasSpec_prefixAdmissible
    · refine And.intro ?_ trivial
      simpa [PrefixSignatureWellFormed, unitRecSpecs, unitSpecs,
        unitTySpec, unitCtorSpec, unitAliasSpec, unitRecSpec]
        using unitRecSpec_prefixAdmissible

private theorem unitRecSpecs_signatureWellFormed_fromPrefix :
  SignatureWellFormed unitRecSpecs :=
  SignatureWellFormed.ofPrefix unitRecSpecs_prefixWellFormed unitRecSpecs_obligations

def unitRecTerm : PureTm 0 := .const unitRecName
def unitTyTerm : PureTm 0 := .const unitTyName
def unitCtorTerm : PureTm 0 := .const unitCtorName

/- Closed Unit recursor application used for the first iota-style pilot witness:
`((Unit.rec Unit) Unit.unit) Unit.unit`. -/
def unitRecOnCtor : PureTm 0 :=
  .app (.app (.app unitRecTerm unitTyTerm) unitCtorTerm) unitCtorTerm

```

```

def unitRecAfterTy : PureTm 0 := .lam (.lam (.var (1 : Fin 2)))
def unitRecAfterTyCtor : PureTm 0 := .lam (.const unitCtorName)

private theorem redDecl_to_star {E : DeclEnv} {t u : PureTm n} (h : RedDecl E t u) :
  RedStarDecl E t u :=
  Relation.ReflTransGen.tail Relation.ReflTransGen.refl h

/-- First iota-style computation witness in the declaration-aware layer:
`((Unit.rec Unit) Unit.unit) Unit.unit → Unit.unit`
via  $\delta$ -unfolding of `Unit.rec` plus  $\beta$ -reduction. -/
theorem unitRecOnCtor_iota :
  RedStarDecl unitRecDeclEnv unitRecOnCtor unitCtorTerm := by
  let t0 : PureTm 0 := unitRecOnCtor
  let t1 : PureTm 0 := .app (.app (.app unitRecValue unitTyTerm) unitCtorTerm) unitCtorTerm
  let t2 : PureTm 0 := .app (.app unitRecAfterTy unitCtorTerm) unitCtorTerm
  let t3 : PureTm 0 := .app unitRecAfterTyCtor unitCtorTerm
  let t4 : PureTm 0 := unitCtorTerm

  have h01 : RedDecl unitRecDeclEnv t0 t1 := by
    exact .congAppFun (.congAppFun (.congAppFun (.deltaConst (E := unitRecDeclEnv) (c :=
      unitRecName)
      (v := unitRecValue) valueOf_unitRec))))
  have h12 : RedDecl unitRecDeclEnv t1 t2 := by
    exact .congAppFun (.congAppFun (.core (.betaPi (.lam (.lam (.var (1 : Fin 3)))) unitTyTerm)))
  have h23 : RedDecl unitRecDeclEnv t2 t3 := by
    exact .congAppFun (.core (.betaPi (.lam (.var (1 : Fin 2)))) unitCtorTerm))
  have h34 : RedDecl unitRecDeclEnv t3 t4 := by
    exact .core (.betaPi (.const unitCtorName) unitCtorTerm)

  have hs01 : RedStarDecl unitRecDeclEnv t0 t1 := redDecl_to_star h01
  have hs12 : RedStarDecl unitRecDeclEnv t1 t2 := redDecl_to_star h12
  have hs23 : RedStarDecl unitRecDeclEnv t2 t3 := redDecl_to_star h23
  have hs34 : RedStarDecl unitRecDeclEnv t3 t4 := redDecl_to_star h34
  exact Relation.ReflTransGen.trans
    (Relation.ReflTransGen.trans
      (Relation.ReflTransGen.trans hs01 hs12)
      hs23)
    hs34

theorem hasType_unitRecOnCtor :
  HasTypeDecl unitRecDeclEnv .nil unitRecOnCtor (.const unitTyName) := by
  unfold unitRecOnCtor unitRecTerm unitTyTerm unitCtorTerm
  exact .app_elim
    (.app_elim
      (.app_elim hasType_unitRec hasType_unitTy_inRecEnv)
      hasType_unitCtor_inRecEnv)
    hasType_unitCtor_inRecEnv

/-- The Unit pilot run is now wired through the generic declaration-aware
star-preservation service boundary from `DeclarationSemantics`. -/
theorem hasType_unitRecOnCtor_result_of_preservation
  (piInjective :
     $\forall \{k : \mathbf{Nat}\} \{A A' : \text{PureTm } k\} \{B B' : \text{PureTm } (k + 1)\},$ 

```

```

      ConvDecl unitRecDeclEnv (.pi A B) (.pi A' B') →
      ConvDecl unitRecDeclEnv A A' ∧ ConvDecl unitRecDeclEnv B B')
(sigmaInjective :
  ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl unitRecDeclEnv (.sigma A B) (.sigma A' B') →
  ConvDecl unitRecDeclEnv A A' ∧ ConvDecl unitRecDeclEnv B B') :
HasTypeDecl unitRecDeclEnv .nil unitCtorTerm (.const unitTyName) := by
exact redStarDecl_preserves_type_of_injective
(E := unitRecDeclEnv)
(piInjective := piInjective)
(sigmaInjective := sigmaInjective)
(hWf := unitRecDeclEnv_wellFormed)
(ht := hasType_unitRecOnCtor)
(hs := unitRecOnCtor_iota)

/-- Preservation-backed pilot package: the target typing is obtained through
the generic declaration-aware star-preservation service. -/
theorem unitRecOnCtor_preserves_type_to_result_of_preservation
(piInjective :
  ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl unitRecDeclEnv (.pi A B) (.pi A' B') →
  ConvDecl unitRecDeclEnv A A' ∧ ConvDecl unitRecDeclEnv B B')
(sigmaInjective :
  ∀ {k : Nat} {A A' : PureTm k} {B B' : PureTm (k + 1)},
  ConvDecl unitRecDeclEnv (.sigma A B) (.sigma A' B') →
  ConvDecl unitRecDeclEnv A A' ∧ ConvDecl unitRecDeclEnv B B') :
HasTypeDecl unitRecDeclEnv .nil unitRecOnCtor (.const unitTyName) ∧
RedStarDecl unitRecDeclEnv unitRecOnCtor unitCtorTerm ∧
HasTypeDecl unitRecDeclEnv .nil unitCtorTerm (.const unitTyName) := by
exact
(hasType_unitRecOnCtor, unitRecOnCtor_iota,
 hasType_unitRecOnCtor_result_of_preservation
  (piInjective := piInjective)
  (sigmaInjective := sigmaInjective))

/-- Minimal env-aware preservation witness for the first iota-style recursor run. -/
theorem unitRecOnCtor_preserves_type_to_result :
HasTypeDecl unitRecDeclEnv .nil unitRecOnCtor (.const unitTyName) ∧
RedStarDecl unitRecDeclEnv unitRecOnCtor unitCtorTerm ∧
HasTypeDecl unitRecDeclEnv .nil unitCtorTerm (.const unitTyName) := by
exact (hasType_unitRecOnCtor, unitRecOnCtor_iota, )hasType_unitCtor_inRecEnv

end Mettapedia.Languages.MeTTa.PureKernel.RecursorDecl

```

A.19 Reduction.lean

Mettapedia/Languages/MeTTa/PureKernel/Reduction.lean

```

import Mathlib.Logic.Relation
import Mettapedia.Languages.MeTTa.PureKernel.Substitution

namespace Mettapedia.Languages.MeTTa.PureKernel.Reduction

open Mettapedia.Languages.MeTTa.PureKernel.Syntax

```

open Mettapedia.Languages.MeTTa.PureKernel.Substitution

-- One-step β -reduction with congruence for the Pure kernel syntax. --

inductive Red : PureTm n $\rightarrow$ PureTm n $\rightarrow$ **Prop** where

| betaPi (body : PureTm (n + 1)) (a : PureTm n) :
 Red (.app (.lam body) a) (inst0 a body)
| betaSigmaFst (a b : PureTm n) :
 Red (.fst (.pair a b)) a
| betaSigmaSnd (a b : PureTm n) :
 Red (.snd (.pair a b)) b
| congPiDom {A A' : PureTm n} {B : PureTm (n + 1)} :
 Red A A' $\rightarrow$ Red (.pi A B) (.pi A' B)
| congPiCod {A : PureTm n} {B B' : PureTm (n + 1)} :
 Red B B' $\rightarrow$ Red (.pi A B) (.pi A B')
| congSigmaDom {A A' : PureTm n} {B : PureTm (n + 1)} :
 Red A A' $\rightarrow$ Red (.sigma A B) (.sigma A' B)
| congSigmaCod {A : PureTm n} {B B' : PureTm (n + 1)} :
 Red B B' $\rightarrow$ Red (.sigma A B) (.sigma A B')
| congIdTy {A A' a b : PureTm n} :
 Red A A' $\rightarrow$ Red (.id A a b) (.id A' a b)
| congIdLeft {A a a' b : PureTm n} :
 Red a a' $\rightarrow$ Red (.id A a b) (.id A a' b)
| congIdRight {A a b b' : PureTm n} :
 Red b b' $\rightarrow$ Red (.id A a b) (.id A a b')
| congLam {b b' : PureTm (n + 1)} :
 Red b b' $\rightarrow$ Red (.lam b) (.lam b')
| congAppFun {f f' a : PureTm n} :
 Red f f' $\rightarrow$ Red (.app f a) (.app f' a)
| congAppArg {f a a' : PureTm n} :
 Red a a' $\rightarrow$ Red (.app f a) (.app f a')
| congPairFst {a a' b : PureTm n} :
 Red a a' $\rightarrow$ Red (.pair a b) (.pair a' b)
| congPairSnd {a b b' : PureTm n} :
 Red b b' $\rightarrow$ Red (.pair a b) (.pair a b')
| congFst {p p' : PureTm n} :
 Red p p' $\rightarrow$ Red (.fst p) (.fst p')
| congSnd {p p' : PureTm n} :
 Red p p' $\rightarrow$ Red (.snd p) (.snd p')
| congRefl {a a' : PureTm n} :
 Red a a' $\rightarrow$ Red (.refl a) (.refl a')

abbrev RedStar (t u : PureTm n) : **Prop** := Relation.ReflTransGen Red t u

namespace RedStar

theorem refl (t : PureTm n) : RedStar t t :=
 Relation.ReflTransGen.refl

theorem tail {t u v : PureTm n} (ih : RedStar t u) (zh : Red u v) : RedStar t v :=
 Relation.ReflTransGen.tail ih zh

theorem trans {t u v : PureTm n} (ih : RedStar t u) (zh : RedStar u v) : RedStar t v :=
 Relation.ReflTransGen.trans ih zh

```

theorem map {F : PureTm n → PureTm n}
  (hF : ∀ {x y}, Red x y → Red (F x) (F y))
  {t u : PureTm n} (h : RedStar t u) : RedStar (F t) (F u) := by
induction h with
| refl =>
  exact Relation.ReflTransGen.refl
| tail hxy hyz ih =>
  exact Relation.ReflTransGen.tail ih (hF hyz)

theorem congPiDom {A A' : PureTm n} {B : PureTm (n + 1)}
  (h : RedStar A A') : RedStar (.pi A B) (.pi A' B) :=
map (F := fun t => .pi t B) (fun hstep => .congPiDom hstep) h

theorem congPiCod {A : PureTm n} {B B' : PureTm (n + 1)}
  (h : RedStar B B') : RedStar (.pi A B) (.pi A B') := by
induction h with
| refl =>
  exact Relation.ReflTransGen.refl
| tail hxy hyz ih =>
  exact .tail ih (.congPiCod hyz)

theorem congSigmaDom {A A' : PureTm n} {B : PureTm (n + 1)}
  (h : RedStar A A') : RedStar (.sigma A B) (.sigma A' B) :=
map (F := fun t => .sigma t B) (fun hstep => .congSigmaDom hstep) h

theorem congSigmaCod {A : PureTm n} {B B' : PureTm (n + 1)}
  (h : RedStar B B') : RedStar (.sigma A B) (.sigma A B') := by
induction h with
| refl =>
  exact Relation.ReflTransGen.refl
| tail hxy hyz ih =>
  exact .tail ih (.congSigmaCod hyz)

theorem congLam {b b' : PureTm (n + 1)} (h : RedStar b b') : RedStar (.lam b) (.lam b') := by
induction h with
| refl =>
  exact Relation.ReflTransGen.refl
| tail hxy hyz ih =>
  exact .tail ih (.congLam hyz)

theorem congAppFun {f f' a : PureTm n} (h : RedStar f f') : RedStar (.app f a) (.app f' a) :=
map (F := fun t => .app t a) (fun hstep => .congAppFun hstep) h

theorem congAppArg {f a a' : PureTm n} (h : RedStar a a') : RedStar (.app f a) (.app f a') :=
map (F := fun t => .app f t) (fun hstep => .congAppArg hstep) h

theorem congPairFst {a a' b : PureTm n} (h : RedStar a a') :
  RedStar (.pair a b) (.pair a' b) :=
map (F := fun t => .pair t b) (fun hstep => .congPairFst hstep) h

theorem congPairSnd {a b b' : PureTm n} (h : RedStar b b') :
  RedStar (.pair a b) (.pair a b') :=
map (F := fun t => .pair a t) (fun hstep => .congPairSnd hstep) h

```

```

theorem congFst {p p' : PureTm n} (h : RedStar p p') : RedStar (.fst p) (.fst p') :=
  map (F := fun t => .fst t) (fun hstep => .congFst hstep) h

theorem congSnd {p p' : PureTm n} (h : RedStar p p') : RedStar (.snd p) (.snd p') :=
  map (F := fun t => .snd t) (fun hstep => .congSnd hstep) h

theorem congRefl {a a' : PureTm n} (h : RedStar a a') : RedStar (.refl a) (.refl a') :=
  map (F := fun t => .refl t) (fun hstep => .congRefl hstep) h

end RedStar

theorem red_to_redStar {t u : PureTm n} (h : Red t u) : RedStar t u :=
  Relation.ReflTransGen.tail Relation.ReflTransGen.refl h

end Mettapedia.Languages.MeTTa.PureKernel.Reduction

```

A.20 Renaming.lean

Mettapedia/Languages/MeTTa/PureKernel/Renaming.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.Syntax

namespace Mettapedia.Languages.MeTTa.PureKernel.Renaming

open Mettapedia.Languages.MeTTa.PureKernel.Syntax

abbrev Ren (n m : Nat) := Fin n → Fin m

def idRen : Ren n n := fun i => i

def wk : Ren n (n + 1) := Fin.succ

def liftRen p( : Ren n m) : Ren (n + 1) (m + 1) :=
  Fin.cases 0 (fun i => Fin.succ p( i))

def rename p( : Ren n m) : PureTm n → PureTm m
| .var i => .var p( i)
| .const c => .const c
| .u0 => .u0
| .u1 => .u1
| .pi A B => .pi (rename p A) (rename (liftRen p) B)
| .sigma A B => .sigma (rename p A) (rename (liftRen p) B)
| .id A a b => .id (rename p A) (rename p a) (rename p b)
| .lam b => .lam (rename (liftRen p) b)
| .app f a => .app (rename p f) (rename p a)
| .pair a b => .pair (rename p a) (rename p b)
| .fst p => .fst (rename p p)
| .snd p => .snd (rename p p)
| .refl a => .refl (rename p a)

@[simp] theorem liftRen_id : liftRen (idRen (n := n)) = idRen := by
  funext i
  refine Fin.cases ?_ ?_ i
  · rfl

```

```

· intro j
  rfl

@[simp] theorem liftRen_comp_apply p₂( : Ren m k) p₁( : Ren n m) (i : Fin (n + 1)) :
  liftRen p₂ (liftRen p₁ i) = liftRen (fun j => p₂ p₁( j)) i := by
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    rfl

@[simp] theorem liftRen_ext p{ ξ : Ren n m} (ph : ∀ i, p i = ξ i) :
  ∀ i : Fin (n + 1), liftRen p i = liftRen ξ i := by
  intro i
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    simp [liftRen, ph j]

theorem rename_ext p{ ξ : Ren n m} (ph : ∀ i, p i = ξ i) :
  ∀ t : PureTm n, rename p t = rename ξ t := by
  intro t
  induction t generalizing m with
  | var i =>
    simp [rename, ph i]
  | const c =>
    rfl
  | u0 =>
    rfl
  | u1 =>
    rfl
  | pi A B ihA ihB =>
    simp [rename, ihA p( := p) ξ( := ξ) ph]
    exact ihB p( := liftRen p) ξ( := liftRen ξ) (liftRen_ext ph)
  | sigma A B ihA ihB =>
    simp [rename, ihA p( := p) ξ( := ξ) ph]
    exact ihB p( := liftRen p) ξ( := liftRen ξ) (liftRen_ext ph)
  | id A a b ihA iha ihb =>
    simp [rename, ihA p( := p) ξ( := ξ) ph, iha p( := p) ξ( := ξ) ph,
      ihb p( := p) ξ( := ξ) ph]
  | lam b ih =>
    simp [rename]
    exact ih p( := liftRen p) ξ( := liftRen ξ) (liftRen_ext ph)
  | app f a ihf iha =>
    simp [rename, ihf p( := p) ξ( := ξ) ph, iha p( := p) ξ( := ξ) ph]
  | pair a b iha ihb =>
    simp [rename, iha p( := p) ξ( := ξ) ph, ihb p( := p) ξ( := ξ) ph]
  | fst p ih =>
    simp [rename] using ih p( := p) ξ( := ξ) ph
  | snd p ih =>
    simp [rename] using ih p( := p) ξ( := ξ) ph
  | refl a iha =>
    simp [rename] using iha p( := p) ξ( := ξ) ph

@[simp] theorem rename_id : ∀ t : PureTm n, rename idRen t = t := by

```

```

intro t
induction t with
| var i =>
  rfl
| const c =>
  rfl
| u0 =>
  rfl
| u1 =>
  rfl
| pi A B ihA ihB =>
  simp [rename, ihA, ihB]
| sigma A B ihA ihB =>
  simp [rename, ihA, ihB]
| id A a b ihA iha ihb =>
  simp [rename, ihA, iha, ihb]
| lam b ih =>
  simp [rename, ih]
| app f a ihf iha =>
  simp [rename, ihf, iha]
| pair a b iha ihb =>
  simp [rename, iha, ihb]
| fst p ih =>
  simp [rename, ih]
| snd p ih =>
  simp [rename, ih]
| refl a iha =>
  simp [rename, iha]

@[simp] theorem rename_comp :
  ∀ {n m k} p₂ ( : Ren m k) p₁ ( : Ren n m) (t : PureTm n),
    rename p₂ (rename p₁ t) = rename (fun i => p₂ p₁( i)) t := by
intro n m k p₂ p₁ t
induction t generalizing m k p₂ with
| var i =>
  rfl
| const c =>
  rfl
| u0 =>
  rfl
| u1 =>
  rfl
| pi A B ihA ihB =>
  simp [rename, ihA p₂( := p₂) p₁( := p₁)]
  calc
    rename (liftRen p₂) (rename (liftRen p₁) B)
      = rename (fun i => liftRen p₂ (liftRen p₁ i)) B := by
        simp using ihB p₂( := liftRen p₂) p₁( := liftRen p₁)
    _ = rename (liftRen (fun i => p₂ p₁( i))) B := by
      exact rename_ext
        p( := fun i => liftRen p₂ (liftRen p₁ i))
        ξ( := liftRen (fun i => p₂ p₁( i)))
        (fun i => liftRen_comp_apply p₂ p₁ i)
    B

```



```

| sigma A B ihA ihB =>
  simp [rename, ihA ρ₂( := ρ₂) ρ₁( := ρ₁)]
  calc
    rename (liftRen ρ₂) (rename (liftRen ρ₁) B)
      = rename (fun i => liftRen ρ₂ (liftRen ρ₁ i)) B := by
        simpa using ihB ρ₂( := liftRen ρ₂) ρ₁( := liftRen ρ₁)
    _ = rename (liftRen (fun i => ρ₂ ρ₁( i))) B := by
      exact rename_ext
        ρ( := fun i => liftRen ρ₂ (liftRen ρ₁ i))
        ξ( := liftRen (fun i => ρ₂ ρ₁( i)))
        (fun i => liftRen_comp_apply ρ₂ ρ₁ i)
        B
| id A a b ihA iha ihb =>
  simp [rename, ihA ρ₂( := ρ₂) ρ₁( := ρ₁), iha ρ₂( := ρ₂) ρ₁( := ρ₁),
    ihb ρ₂( := ρ₂) ρ₁( := ρ₁)]
| lam b ih =>
  simp [rename]
  calc
    rename (liftRen ρ₂) (rename (liftRen ρ₁) b)
      = rename (fun i => liftRen ρ₂ (liftRen ρ₁ i)) b := by
        simpa using ih ρ₂( := liftRen ρ₂) ρ₁( := liftRen ρ₁)
    _ = rename (liftRen (fun i => ρ₂ ρ₁( i))) b := by
      exact rename_ext
        ρ( := fun i => liftRen ρ₂ (liftRen ρ₁ i))
        ξ( := liftRen (fun i => ρ₂ ρ₁( i)))
        (fun i => liftRen_comp_apply ρ₂ ρ₁ i)
        b
| app f a ihf iha =>
  simp [rename, ihf ρ₂( := ρ₂) ρ₁( := ρ₁), iha ρ₂( := ρ₂) ρ₁( := ρ₁)]
| pair a b iha ihb =>
  simp [rename, iha ρ₂( := ρ₂) ρ₁( := ρ₁), ihb ρ₂( := ρ₂) ρ₁( := ρ₁)]
| fst p ih =>
  simpa [rename] using ih ρ₂( := ρ₂) ρ₁( := ρ₁)
| snd p ih =>
  simpa [rename] using ih ρ₂( := ρ₂) ρ₁( := ρ₁)
| refl a iha =>
  simpa [rename] using iha ρ₂( := ρ₂) ρ₁( := ρ₁)

end Mettapedia.Languages.MeTTa.PureKernel.Renaming

```

A.21 SubjectReduction.lean

Mettapedia/Languages/MeTTa/PureKernel/SubjectReduction.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.Confluence

namespace Mettapedia.Languages.MeTTa.PureKernel.SubjectReduction

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.Reduction
open Mettapedia.Languages.MeTTa.PureKernel.Parallel

```

```

open Mettapedia.Languages.MeTTa.PureKernel.Typing
open Mettapedia.Languages.MeTTa.PureKernel.Confluence

private theorem subst0_wk_cancel (a t : PureTm n) :
  subst (subst0 a) (rename wk t) = t := by
calc
  subst (subst0 a) (rename wk t)
    = subst (fun i => subst0 a (wk i)) t := by
      simpa using (subst_rename σ( := subst0 a) ρ( := wk) (t := t))
  _ = subst ids t := by
    apply subst_ext
    intro i
    rfl
  _ = t := by
    exact subst_ids (t := t)

private theorem ctxMor_subst0 {Γ : Ctx n} {A a : PureTm n}
  (ha : HasType Γ a A) :
  CtxMor (.snoc Γ A) Γ (subst0 a) := by
intro i
refine Fin.cases ?_ ?_ i
· simpa [CtxMor, lookup_snoc_zero, subst0_wk_cancel] using ha
· intro j
  simpa [CtxMor, lookup_snoc_succ, subst0_wk_cancel] using
    (HasType.var Γ( := Γ) (i := j))

private theorem typing_inst0 {Γ : Ctx n} {A a : PureTm n} {body B : PureTm (n + 1)}
  (hbody : HasType (.snoc Γ A) body B) (ha : HasType Γ a A) :
  HasType Γ (inst0 a body) (inst0 a B) := by
simpa [inst0] using
  (typing_subst Γ( := .snoc Γ A) Δ( := Γ) σ( := subst0 a) hbody (ctxMor_subst0 ha))

private def tmOf {Γ : Ctx n} {t A : PureTm n} ( _ : HasType Γ t A) : PureTm n := t

private theorem parStar_inst0_arg {a a' : PureTm n} {B : PureTm (n + 1)}
  (h : ParStar a a') : ParStar (inst0 a B) (inst0 a' B) := by
induction h with
| refl =>
  exact Relation.ReflTransGen.refl
| tail hxy hyz ih =>
  exact Relation.ReflTransGen.tail ih (par_inst0 hyz (par_refl B))

private theorem redStar_inst0_arg {a a' : PureTm n} {B : PureTm (n + 1)}
  (h : RedStar a a') : RedStar (inst0 a B) (inst0 a' B) := by
  exact parStar_to_redStar (parStar_inst0_arg (redStar_to_parStar h))

private theorem conv_inst0_arg {a a' : PureTm n} {B : PureTm (n + 1)}
  (h : Conv a a') : Conv (inst0 a B) (inst0 a' B) := by
  rcases church_rosser_conv h with ⟨u, ha, ha'⟩
  have h1 : RedStar (inst0 a B) (inst0 u B) := redStar_inst0_arg ha
  have h2 : RedStar (inst0 a' B) (inst0 u B) := redStar_inst0_arg ha'
  have hc1 : Conv (inst0 a B) (inst0 u B) := redStar_implies_conv h1
  have hc2 : Conv (inst0 a' B) (inst0 u B) := redStar_implies_conv h2
  exact Relation.EqvGen.trans _ _ hc1 (Relation.EqvGen.symm _ _ hc2)

```

```

private theorem app_generation_aux  $\Gamma\{ : \text{Ctx } n\} \{t C : \text{PureTm } n\}$ 
  (ht : HasType  $\Gamma$  t C) :
   $\forall \{f a : \text{PureTm } n\}, t = .\text{app } f \ a \rightarrow$ 
   $\exists A B, \text{HasType } \Gamma \ f \ (. \text{pi } A \ B) \wedge \text{HasType } \Gamma \ a \ A \wedge \text{Conv } (\text{inst0 } a \ B) \ C := \text{by}$ 
  induction ht with
  | app_elim hf ha =>
    intro f a hEq
    cases hEq
    exact  $(\_, \_, hf, ha, \text{conv\_refl } \_)$ 
  | conv ht hconv ih =>
    intro f a hEq
    rcases ih hEq with  $\langle A, B, hf, ha, \rangle hC$ 
    exact  $\langle A, B, hf, ha, \text{Relation.EqvGen.trans } \_ \_ \_ hC \rangle hconv$ 
  | u0_type =>
    intro f a hEq
    cases hEq
  | var =>
    intro f a hEq
    cases hEq
  | pi_form hA hB ihA ihB =>
    intro f a hEq
    cases hEq
  | sigma_form hA hB ihA ihB =>
    intro f a hEq
    cases hEq
  | lam_intro hBody ihBody =>
    intro f a hEq
    cases hEq
  | pair_intro ha hb iha ihb =>
    intro f a hEq
    cases hEq
  | fst_elim hp ihp =>
    intro f a hEq
    cases hEq
  | snd_elim hp ihp =>
    intro f a hEq
    cases hEq
  | id_form hA ha hb ihA iha ihb =>
    intro f a hEq
    cases hEq
  | refl_intro ha iha =>
    intro f a hEq
    cases hEq

theorem app_generation  $\Gamma\{ : \text{Ctx } n\} \{f a C : \text{PureTm } n\}$ 
  (ht : HasType  $\Gamma$   $(.\text{app } f \ a) \ C$ ) :
   $\exists A B, \text{HasType } \Gamma \ f \ (. \text{pi } A \ B) \wedge \text{HasType } \Gamma \ a \ A \wedge \text{Conv } (\text{inst0 } a \ B) \ C :=$ 
  app_generation_aux ht rfl

private theorem lam_generation_aux  $\Gamma\{ : \text{Ctx } n\} \{t C : \text{PureTm } n\}$ 
  (ht : HasType  $\Gamma$  t C) :
   $\forall \{body : \text{PureTm } (n + 1)\}, t = .\text{lam } body \rightarrow$ 
   $\exists A B, \text{HasType } (. \text{snoc } \Gamma \ A) \ body \ B \wedge \text{Conv } (. \text{pi } A \ B) \ C := \text{by}$ 

```

```

induction ht with
| lam_intro hBody =>
  intro body hEq
  cases hEq
  exact (_, _, hBody, conv_refl )_
| conv ht hconv ih =>
  intro body hEq
  rcases ih hEq with ⟨A, B, hBody, ⟩hPi
  exact ⟨A, B, hBody, Relation.EqvGen.trans _ _ _ hPi ⟩hconv
| u0_type =>
  intro body hEq
  cases hEq
| var =>
  intro body hEq
  cases hEq
| pi_form hA hB ihA ihB =>
  intro body hEq
  cases hEq
| sigma_form hA hB ihA ihB =>
  intro body hEq
  cases hEq
| app_elim hf ha ihf iha =>
  intro body hEq
  cases hEq
| pair_intro ha hb iha ihb =>
  intro body hEq
  cases hEq
| fst_elim hp ihp =>
  intro body hEq
  cases hEq
| snd_elim hp ihp =>
  intro body hEq
  cases hEq
| id_form hA ha hb ihA iha ihb =>
  intro body hEq
  cases hEq
| refl_intro ha iha =>
  intro body hEq
  cases hEq

theorem lam_generation Γ{ : Ctx n} {body : PureTm (n + 1)} {C : PureTm n}
  (ht : HasType Γ (.lam body) C) :
  ∃ A B, HasType (.snoc Γ A) body B ∧ Conv (.pi A B) C :=
  lam_generation_aux ht rfl

private theorem pair_generation_aux Γ{ : Ctx n} {t C : PureTm n}
  (ht : HasType Γ t C) :
  ∀ {a b : PureTm n}, t = .pair a b →
  ∃ A B, HasType Γ a A ∧ HasType Γ b (inst0 a B) ∧ Conv (.sigma A B) C := by
  induction ht with
| pair_intro ha hb =>
  intro a b hEq
  cases hEq
  exact (_, _, ha, hb, conv_refl )_

```

```

| conv ht hconv ih =>
  intro a b hEq
  rcases ih hEq with (A, B, ha, hb, )hSigma
  exact (A, B, ha, hb, Relation.EqvGen.trans _ _ _ hSigma )hconv
| u0_type =>
  intro a b hEq
  cases hEq
| var =>
  intro a b hEq
  cases hEq
| pi_form hA hB ihA ihB =>
  intro a b hEq
  cases hEq
| sigma_form hA hB ihA ihB =>
  intro a b hEq
  cases hEq
| lam_intro hBody ihBody =>
  intro a b hEq
  cases hEq
| app_elim hf ha ihf iha =>
  intro a b hEq
  cases hEq
| fst_elim hp ihp =>
  intro a b hEq
  cases hEq
| snd_elim hp ihp =>
  intro a b hEq
  cases hEq
| id_form hA ha hb ihA iha ihb =>
  intro a b hEq
  cases hEq
| refl_intro ha iha =>
  intro a b hEq
  cases hEq

theorem pair_generation  $\Gamma\{ : \text{Ctx } n\} \{a \ b \ C : \text{PureTm } n\}$ 
  (ht : HasType  $\Gamma$  (.pair a b) C) :
   $\exists \ A \ B, \text{HasType } \Gamma \ a \ A \wedge \text{HasType } \Gamma \ b \ (\text{inst0 } a \ B) \wedge \text{Conv } (.sigma \ A \ B) \ C :=$ 
  pair_generation_aux ht rfl

/-- One-step subject reduction for the Pure kernel. -/
theorem subject_reduction  $\Gamma\{ : \text{Ctx } n\} \{t \ t' \ A : \text{PureTm } n\}$ 
  (ht : HasType  $\Gamma \ t \ A$ ) (hr : Red t t') :
  HasType  $\Gamma \ t' \ A :=$  by
  induction ht with
| u0_type =>
  cases hr
| var =>
  cases hr
| @pi_form n  $\Gamma \ A \ B \ hA \ hB \ ihA \ ihB =>$ 
  cases hr with
| congPiDom hred =>
  have hA' : HasType  $\Gamma \ _ \ .u1 := ihA \ hred$ 
  have hB' : HasType  $(.snoc \ \Gamma \ _) \ B \ .u1 := \text{context\_conv\_head } (\text{red\_implies\_conv } hred) \ hB$ 

```

```

    exact .pi_form hA' hB'
  | congPiCod hred =>
    exact .pi_form hA (ihB hred)
| @sigma_form n  $\Gamma$  A B hA hB ihA ihB =>
  cases hr with
  | congSigmaDom hred =>
    have hA' : HasType  $\Gamma$  _ .u1 := ihA hred
    have hB' : HasType (.snoc  $\Gamma$  _) B .u1 := context_conv_head (red_implies_conv hred) hB
    exact .sigma_form hA' hB'
  | congSigmaCod hred =>
    exact .sigma_form hA (ihB hred)
| @lam_intro n  $\Gamma$  A body B hBody ihBody =>
  cases hr with
  | congLam hred =>
    exact .lam_intro (ihBody hred)
| @app_elim n  $\Gamma$  f a A B hf ha ihf iha =>
  cases hr with
  | betaPi body _ =>
    have hfLam : HasType  $\Gamma$  (.lam body) (.pi A B) := by
      simpa using hf
    have haArg : HasType  $\Gamma$  a A := by
      simpa using ha
    rcases lam_generation hfLam with (A1, B1, hBody, )hPiConv
    have hAB : Conv A1 A  $\wedge$  Conv B1 B := pi_injectivity hPiConv
    have ha1 : HasType  $\Gamma$  a A1 :=
      .conv haArg (Relation.EqvGen.symm _ _ hAB.1)
    have hSub : HasType  $\Gamma$  (inst0 a body) (inst0 a B1) :=
      typing_inst0 hBody ha1
    have hTy : Conv (inst0 a B1) (inst0 a B) :=
      conv_subst  $\sigma$ ( := subst0 a) hAB.2
    have hRes : HasType  $\Gamma$  (inst0 a body) (inst0 a B) :=
      .conv hSub hTy
    simpa using hRes
  | congAppFun hred =>
    exact .app_elim (ihf hred) ha
  | congAppArg hred =>
    have ha' : HasType  $\Gamma$  _ A := iha hred
    have hApp' : HasType  $\Gamma$  (.app f _) (inst0 _ B) := .app_elim hf ha'
    have hArgConv : Conv a _ := red_implies_conv hred
    have hTy : Conv (inst0 _ B) (inst0 a B) :=
      conv_inst0_arg (Relation.EqvGen.symm _ _ hArgConv)
    exact .conv hApp' hTy
| @pair_intro n  $\Gamma$  a b A B ha hb iha ihb =>
  cases hr with
  | congPairFst hred =>
    have ha' : HasType  $\Gamma$  _ A := iha hred
    have hArgConv : Conv a _ := red_implies_conv hred
    have hTy : Conv (inst0 a B) (inst0 _ B) := conv_inst0_arg hArgConv
    have hb' : HasType  $\Gamma$  b (inst0 _ B) := .conv hb hTy
    exact .pair_intro ha' hb'
  | congPairSnd hred =>
    exact .pair_intro ha (ihb hred)
| @fst_elim n  $\Gamma$  p A B hp ihp =>
  cases hr with

```

```

| betaSigmaFst _ _ =>
  rcases pair_generation hp with (A1, B1, ha1, hb1, )hSigma
  have hAB : Conv A1 A  $\wedge$  Conv B1 B := sigma_injectivity hSigma
  exact .conv ha1 hAB.1
| congFst hred =>
  exact .fst_elim (ihp hred)
| @snd_elim n  $\Gamma$  p A B hp ihp =>
  cases hr with
  | betaSigmaSnd _ _ =>
    rcases pair_generation hp with (A1, B1, ha1, hb1, )hSigma
    have hAB : Conv A1 A  $\wedge$  Conv B1 B := sigma_injectivity hSigma
    let a0 : PureTm n := tmOf ha1
    let b0 : PureTm n := tmOf hb1
    have hb1' : HasType  $\Gamma$  b0 (inst0 a0 B1) := by
      simpa [a0, b0, tmOf] using hb1
    have hCod : Conv (inst0 a0 B1) (inst0 a0 B) :=
      conv_subst  $\sigma$  (:= subst0 a0) hAB.2
    have hFst : Conv a0 (.fst (.pair a0 b0)) :=
      Relation.EqvGen.symm _ _ (red_implies_conv (Red.betaSigmaFst a0 b0))
    have hArg : Conv (inst0 a0 B) (inst0 (.fst (.pair a0 b0)) B) :=
      conv_inst0_arg hFst
    exact .conv hb1' (Relation.EqvGen.trans _ _ _ hCod hArg)
  | congSnd hred =>
    have hp' : HasType  $\Gamma$  _ (.sigma A B) := ihp hred
    have hsnd' : HasType  $\Gamma$  (.snd _) (inst0 (.fst _) B) := .snd_elim hp'
    have hFst : Conv (.fst _) (.fst p) :=
      Relation.EqvGen.symm _ _ (red_implies_conv (Red.congFst hred))
    have hTy : Conv (inst0 (.fst _) B) (inst0 (.fst p) B) :=
      conv_inst0_arg hFst
    exact .conv hsnd' hTy
| @id_form n  $\Gamma$  A a b hA ha hb ihA iha ihb =>
  cases hr with
  | congIdTy hred =>
    have hA' : HasType  $\Gamma$  _ .u1 := ihA hred
    have hAA' : Conv A _ := red_implies_conv hred
    have ha' : HasType  $\Gamma$  a _ := .conv ha hAA'
    have hb' : HasType  $\Gamma$  b _ := .conv hb hAA'
    exact .id_form hA' ha' hb'
  | congIdLeft hred =>
    exact .id_form hA (iha hred) hb
  | congIdRight hred =>
    exact .id_form hA ha (ihb hred)
| @refl_intro n  $\Gamma$  a A ha iha =>
  cases hr with
  | congRefl hred =>
    have ha' : HasType  $\Gamma$  _ A := iha hred
    let a' : PureTm n := tmOf ha'
    have hidL : Conv (.id A a a) (.id A a' a) :=
      red_implies_conv (Red.congIdLeft hred)
    have hidR : Conv (.id A a' a) (.id A a' a') :=
      red_implies_conv (Red.congIdRight (A := A) (a := a') hred)
    have hid : Conv (.id A a a) (.id A a' a') :=
      Relation.EqvGen.trans _ _ _ hidL hidR
    exact .conv (.refl_intro ha') (Relation.EqvGen.symm _ _ hid)

```

```
| @conv n  $\Gamma$  t A B ht hAB ih =>
  exact .conv (ih hr) hAB
```

```
end Mettapedia.Languages.MeTTa.PureKernel.SubjectReduction
```

A.22 Telescope.lean

Mettapedia/Languages/MeTTa/PureKernel/Telescope.lean

```
import Mettapedia.Languages.MeTTa.PureKernel.InductiveDecl

/-! LLM primer:
- `Ctx n` (from Context.lean) IS the telescope: `nil : Ctx 0`, `snoc  $\Gamma$  A : Ctx (n+1)`
- `telescopePi` wraps a body in  $\Pi$ 's from a context (innermost first)
- `piArity` counts leading  $\Pi$ 's; `resultIsU0 k t` peels k  $\Pi$ 's and checks body = U0
- De Bruijn for parametric families: after peeling `k` total  $\Pi$ 's with `p` params,
  param `j` (0-indexed outermost) has de Bruijn index `k - 1 - j`.
- `isParamApp` strips apps right-to-left starting at index `k - p`.
- `ParamIndDecl` subsumes `IndDecl`: set `typeFormerType := .u0` for 0-parameter families
- `targetsFamilyP` uses `targetsFamilyPAux` (standalone, not `where`) for kernel reducibility
- Checker validates `strictlyPositive` first (fast early-exit), then `matchingParamDomains`,
  then `targetsFamilyP` – this order prevents reduction budget exhaustion on negative examples
-/

namespace Mettapedia.Languages.MeTTa.PureKernel.Telescope

set_option linter.dupNamespace false

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec
open Mettapedia.Languages.MeTTa.PureKernel.IndDecl

/-! ## Telescope operations -/

/-- Wrap a body in  $\Pi$ -binders from a context (innermost binder first).

- positive example: `telescopePi (snoc nil .u0) (.var 0) = .pi .u0 (.var 0)`
  (i.e.,  $\Pi (A : U0). A$ )
- negative example: `telescopePi nil .u0 = .u0` (no binders) -/
def telescopePi : Ctx n  $\rightarrow$  PureTm n  $\rightarrow$  PureTm 0
| .nil, body => body
| .snoc  $\Gamma$  A, body => telescopePi  $\Gamma$  (.pi A body)

/-- Wrap a body in  $\lambda$ -binders from a context (innermost binder first).

- positive example: `telescopeLam (snoc nil .u0) (.var 0) = .lam (.var 0)`
- negative example: `telescopeLam nil .u0 = .u0` (no binders) -/
def telescopeLam : Ctx n  $\rightarrow$  PureTm n  $\rightarrow$  PureTm 0
| .nil, body => body
| .snoc  $\Gamma$  _, body => telescopeLam  $\Gamma$  (.lam body)

/-! ## Checker predicates -/
```



```

/-- Count leading  $\Pi$ -binders.

- positive example: `piArity (.pi .u0 .u0) = 1`
- negative example: `piArity .u0 = 0` -/
def piArity : PureTm n → Nat
  | .pi _ B => piArity B + 1
  | _ => 0

/-- After peeling `k`  $\Pi$ -binders, is the body `U0`?

- positive example: `resultIsU0 1 (.pi .u0 .u0) = true`
- negative example: `resultIsU0 0 (.pi .u0 .u0) = false` -/
def resultIsU0 : Nat → PureTm n → Bool
  | 0, .u0 => true
  | k + 1, .pi _ B => resultIsU0 k B
  | _, _ => false

/-- Check that the first `k`  $\Pi$ -domains of two terms match (by `BEq`).

- positive example: `matchingParamDomains 1 (.pi .u0 .u0) (.pi .u0 X) = true`
- negative example: `matchingParamDomains 1 (.pi .u0 .u0) (.pi .u1 X) = false` -/
def matchingParamDomains : Nat → PureTm n → PureTm n → Bool
  | 0, _, _ => true
  | k + 1, .pi 1A 1B, .pi 2A 2B => 1A == 2A && matchingParamDomains k 1B 2B
  | _, _, _ => false

/-- Check that a term is `const c` applied to consecutive de Bruijn variables.

Strips applications right-to-left, checking each argument is `var nextVar`,
then `var (nextVar + 1)`, etc.

- positive example: for `List` with `totalBinders = 1`, `numParams = 1`,
  `isParamApp `List 1 0 (.app (.const `List) (.var 0)) = true`
- negative example: `isParamApp `List 1 0 (.const `List) = false` (expected 1 app) -/
def isParamApp (c : DeclName) : (remaining : Nat) → (nextVar : Nat) → PureTm n → Bool
  | 0, _, .const d => d == c
  | k + 1, v, .app f (.var i) => i.val == v && isParamApp c k (v + 1) f
  | _, _, _ => false

/-- Auxiliary for `targetsFamilyP`: peel  $\Pi$ -binders while counting depth. -/
private def targetsFamilyPAux (familyName : DeclName) (numParams : Nat) (bindersSoFar : Nat) :
  PureTm m → Bool
  | .pi _ B => targetsFamilyPAux familyName numParams (bindersSoFar + 1) B
  | other => isParamApp familyName numParams (bindersSoFar - numParams) other

/-- After peeling ALL  $\Pi$ -binders, check the target is `familyName` applied to
the parameter variables (the first `numParams` outermost-bound vars).

- positive example: `targetsFamilyP `List 1 (.pi .u0 (.app (.const `List) (.var 0))) = true`
  (i.e.,  $\Pi (A : U0). List A$  targets `List` with 1 param)
- negative example: `targetsFamilyP `List 1 (.pi .u0 (.const `List)) = false`
  (missing the parameter application) -/
def targetsFamilyP (familyName : DeclName) (numParams : Nat) (t : PureTm n) : Bool :=

```

```

    targetsFamilyPAux familyName numParams 0 t

/-! ## Parametric inductive declaration -/

/-- An inductive type declaration with parameters.

`numParams` is derived from `typeFormerType` (= `piArity typeFormerType`).
For non-parametric families, use `typeFormerType := .u0`.

- positive example: `{ typeName := `List, typeFormerType := .pi .u0 .u0, ctors := [...] }`
- negative example: `{ typeName := `Nat, typeFormerType := .u0, ctors := [...] }` (0-param) -/
structure ParamIndDecl where
  /-- Name of the type former (e.g., `List`). -/
  typeName : DeclName
  /-- Full type of the type former (e.g.,  $\Pi (A : U0). U0$  for List). -/
  typeFormerType : PureTm 0
  /-- Constructor specifications (fully closed types). -/
  ctors : List CtorSpec
deriving Repr

/-! ## The parametric checker -/

/-- Check a parametric inductive declaration and emit `DeclSpec`s if valid.

Validates:
1. All names (type + constructors) are distinct
2. Type former result is  $U0$  after peeling parameter  $\Pi$ 's
3. Each constructor: strict positivity (checked first for fast early-exit),
   then matching param domains, then correct target
4. Constructors' first `numParams`  $\Pi$ -domains match the type former's

Returns the type spec followed by one spec per constructor.
Does NOT generate recursors.

- positive example: `checkParamIndDecl listDecl = some [listTySpec, nilSpec, consSpec]`
- negative example: a declaration with mismatched parameter types returns `none` -/
def checkParamIndDecl (decl : ParamIndDecl) : Option (List DeclSpec) :=
  let numParams := piArity decl.typeFormerType
  let allNames := decl.typeName :: decl.ctors.map CtorSpec.name
  if !namesDistinct allNames then none
  else if !resultIsU0 numParams decl.typeFormerType then none
  else
    let ctorsValid := decl.ctors.all fun ctor =>
      -- Check strictlyPositive first: fast early-exit prevents budget exhaustion
      strictlyPositive decl.typeName ctor.type &&
      matchingParamDomains numParams decl.typeFormerType ctor.type &&
      targetsFamilyP decl.typeName numParams ctor.type
    if !ctorsValid then none
    else
      let tySpec : DeclSpec := { name := decl.typeName, type := decl.typeFormerType }
      let ctorSpecs : List DeclSpec := decl.ctors.map fun ctor =>
        { name := ctor.name, type := ctor.type }
      some (tySpec :: ctorSpecs)

```

```

/-! ## Standard parametric declarations -/

def listDecl : ParamIndDecl :=
{ typeName := `List
  typeFormerType := .pi .u0 .u0
  ctors :=
    [{ name := `List.nil,
      type := .pi .u0 (.app (.const `List) (.var 0)) },
    { name := `List.cons,
      type := .pi .u0
        (.pi (.var 0)
          (.pi (.app (.const `List) (.var 1))
            (.app (.const `List) (.var 2)))) } ] }

/-! ## Backward-compatible 0-parameter declarations -/

def unitDeclP : ParamIndDecl :=
{ typeName := `Unit
  typeFormerType := .u0
  ctors := [{ name := `Unit.unit, type := .const `Unit } ] }

def boolDeclP : ParamIndDecl :=
{ typeName := `Bool
  typeFormerType := .u0
  ctors := [{ name := `Bool.true, type := .const `Bool },
    { name := `Bool.false, type := .const `Bool } ] }

def natDeclP : ParamIndDecl :=
{ typeName := `Nat
  typeFormerType := .u0
  ctors := [{ name := `Nat.zero, type := .const `Nat },
    { name := `Nat.succ, type := .pi (.const `Nat) (.const `Nat) } ] }

/-! ## Checker passes on standard declarations -/

theorem check_listDecl :
  checkParamIndDecl listDecl = some
    [{ name := `List, type := .pi .u0 .u0 },
    { name := `List.nil, type := .pi .u0 (.app (.const `List) (.var 0)) },
    { name := `List.cons,
      type := .pi .u0
        (.pi (.var 0)
          (.pi (.app (.const `List) (.var 1))
            (.app (.const `List) (.var 2)))) } ] := by
decide

theorem check_unitDeclP :
  checkParamIndDecl unitDeclP = some
    [{ name := `Unit, type := .u0 },
    { name := `Unit.unit, type := .const `Unit } ] := by
decide

theorem check_boolDeclP :
  checkParamIndDecl boolDeclP = some

```

```

    [{ name := `Bool, type := .u0 },
      { name := `Bool.true, type := .const `Bool },
      { name := `Bool.false, type := .const `Bool }] := by
decide

theorem check_natDeclP :
  checkParamIndDecl natDeclP = some
    [{ name := `Nat, type := .u0 },
      { name := `Nat.zero, type := .const `Nat },
      { name := `Nat.succ, type := .pi (.const `Nat) (.const `Nat) }] := by
decide

/-! ## Negative examples -/

/-- A declaration with mismatched parameter types is rejected. -/
def badParamMismatch : ParamIndDecl :=
  { typeName := `Bad
    typeFormerType := .pi .u0 .u0
    ctors := [{ name := `Bad.mk,
      type := .pi .u1 (.app (.const `Bad) (.var 0)) }] }

theorem check_badParamMismatch : checkParamIndDecl badParamMismatch = none := by decide

/-- A constructor that doesn't apply the family to parameter vars is rejected. -/
def badParamTarget : ParamIndDecl :=
  { typeName := `Bad
    typeFormerType := .pi .u0 .u0
    ctors := [{ name := `Bad.mk,
      type := .pi .u0 (.const `Bad) }] }

theorem check_badParamTarget : checkParamIndDecl badParamTarget = none := by decide

/-- A parametric declaration with a negatively-occurring family name is rejected. -/
def badParamNeg : ParamIndDecl :=
  { typeName := `Bad
    typeFormerType := .pi .u0 .u0
    ctors := [{ name := `Bad.mk,
      type := .pi .u0 (.pi (.pi (.const `Bad) .u0) (.app (.const `Bad) (.var 1))) }] }

theorem check_badParamNeg : checkParamIndDecl badParamNeg = none := by decide

/-! ## Lifting from `IndDecl` -/

/-- Lift a non-parametric `IndDecl` to a `ParamIndDecl` with 0 parameters. -/
def liftToParam (d : IndDecl) : ParamIndDecl :=
  { typeName := d.typeName, typeFormerType := .u0, ctors := d.ctors }

/-! Agreement: the parametric checker on a lifted declaration gives
the same result as the non-parametric checker. -/

theorem toParam_unitDecl_agree :
  checkParamIndDecl (liftToParam unitDecl) = checkIndDecl unitDecl := by decide

theorem toParam_boolDecl_agree :

```

```

    checkParamIndDecl (liftToParam boolDecl) = checkIndDecl boolDecl := by decide

theorem toParam_natDecl_agree :
    checkParamIndDecl (liftToParam natDecl) = checkIndDecl natDecl := by decide

/-! ## Agreement with existing hand-written specs -/

open Mettapedia.Languages.MeTTa.PureKernel.UnitDecl in
theorem unitDeclP_specs_agree :
    checkParamIndDecl unitDeclP = some [unitTySpec, unitCtorSpec] := by
simp only [unitTySpec, unitCtorSpec, unitTyName, unitCtorName]
exact check_unitDeclP

open Mettapedia.Languages.MeTTa.PureKernel.BoolDecl in
theorem boolDeclP_specs_agree :
    checkParamIndDecl boolDeclP = some [boolTySpec, boolTrueSpec, boolFalseSpec] := by
simp only [boolTySpec, boolTrueSpec, boolFalseSpec, boolTyName, boolTrueName, boolFalseName]
exact check_boolDeclP

open Mettapedia.Languages.MeTTa.PureKernel.NatDecl in
theorem natDeclP_specs_agree :
    checkParamIndDecl natDeclP = some [natTySpec, natZeroSpec, natSuccSpec] := by
simp only [natTySpec, natZeroSpec, natSuccSpec, natTyName, natZeroName, natSuccName]
exact check_natDeclP

end Mettapedia.Languages.MeTTa.PureKernel.Telescope

```

A.23 Substitution.lean

Mettapedia/Languages/MeTTa/PureKernel/Substitution.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.Renaming

namespace Mettapedia.Languages.MeTTa.PureKernel.Substitution

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming

abbrev Sub (n m : Nat) := Fin n → PureTm m

def ids : Sub n n := fun i => .var i

def liftSub σ ( : Sub n m) : Sub (n + 1) (m + 1) :=
    Fin.cases (.var 0) (fun i => rename wk σ ( i))

def subst σ ( : Sub n m) : PureTm n → PureTm m
| .var i => σ i
| .const c => .const c
| .u0 => .u0
| .u1 => .u1
| .pi A B => .pi (subst σ A) (subst (liftSub σ) B)
| .sigma A B => .sigma (subst σ A) (subst (liftSub σ) B)
| .id A a b => .id (subst σ A) (subst σ a) (subst σ b)
| .lam b => .lam (subst (liftSub σ) b)

```

```

| .app f a => .app (subst σ f) (subst σ a)
| .pair a b => .pair (subst σ a) (subst σ b)
| .fst p => .fst (subst σ p)
| .snd p => .snd (subst σ p)
| .refl a => .refl (subst σ a)

def subst0 (u : PureTm n) : Sub (n + 1) n :=
  Fin.cases u (fun i => .var i)

def inst0 (u : PureTm n) (t : PureTm (n + 1)) : PureTm n :=
  subst (subst0 u) t

@[simp] theorem subst0_zero (u : PureTm n) :
  subst0 u 0 = u := rfl

@[simp] theorem subst0_succ (u : PureTm n) (i : Fin n) :
  subst0 u i.succ = .var i := rfl

@[simp] theorem inst0_var_zero (u : PureTm n) :
  inst0 u (.var 0) = u := rfl

@[simp] theorem inst0_var_succ (u : PureTm n) (i : Fin n) :
  inst0 u (.var i.succ) = .var i := rfl

@[simp] theorem subst_var σ( : Sub n m) (i : Fin n) :
  subst σ (.var i) = σ i := rfl

@[simp] theorem liftSub_zero σ( : Sub n m) :
  liftSub σ 0 = (.var 0 : PureTm (m + 1)) := rfl

@[simp] theorem liftSub_succ σ( : Sub n m) (i : Fin n) :
  liftSub σ i.succ = rename wk σ( i) := rfl

theorem subst_ext σ{ τ : Sub n m} (σth : ∀ i, σ i = τ i) :
  ∀ t : PureTm n, subst σ t = subst τ t := by
  intro t
  induction t generalizing m with
  | var i =>
    simp [subst, σth i]
  | const c =>
    rfl
  | u0 =>
    rfl
  | u1 =>
    rfl
  | pi A B ihA ihB =>
    simp [subst, ihA σ( := σ) τ( := τ) σth]
    exact ihB σ( := liftSub σ) τ( := liftSub τ) (by
      intro i
      refine Fin.cases ?_ ?_ i
      · rfl
      · intro j
        simp [liftSub, σth j])
  | sigma A B ihA ihB =>

```

```

simp [subst, ihA σ( := σ) τ( := τ) σth]
exact ihB σ( := liftSub σ) τ( := liftSub τ) (by
  intro i
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    simp [liftSub, σth j])
| id A a b ihA iha ihb =>
simp [subst, ihA σ( := σ) τ( := τ) σth, iha σ( := σ) τ( := τ) σth,
  ihb σ( := σ) τ( := τ) σth]
| lam b ih =>
simp [subst]
exact ih σ( := liftSub σ) τ( := liftSub τ) (by
  intro i
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    simp [liftSub, σth j])
| app f a ihf iha =>
simp [subst, ihf σ( := σ) τ( := τ) σth, iha σ( := σ) τ( := τ) σth]
| pair a b iha ihb =>
simp [subst, iha σ( := σ) τ( := τ) σth, ihb σ( := σ) τ( := τ) σth]
| fst p ih =>
  simp [subst] using ih σ( := σ) τ( := τ) σth
| snd p ih =>
  simp [subst] using ih σ( := σ) τ( := τ) σth
| refl a iha =>
  simp [subst] using iha σ( := σ) τ( := τ) σth

@[simp] theorem liftSub_ids : liftSub (ids (n := n)) = ids := by
  funext i
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    rfl

@[simp] theorem subst_ids : ∀ t : PureTm n, subst ids t = t := by
  intro t
  induction t with
  | var i =>
    rfl
  | const c =>
    rfl
  | u0 =>
    rfl
  | u1 =>
    rfl
  | pi A B ihA ihB =>
    simp [subst, ihA, ihB, liftSub_ids]
  | sigma A B ihA ihB =>
    simp [subst, ihA, ihB, liftSub_ids]
  | id A a b ihA iha ihb =>
    simp [subst, ihA, iha, ihb]
  | lam b ih =>

```

```

    simp [subst, ih, liftSub_ids]
| app f a ihf iha =>
    simp [subst, ihf, iha]
| pair a b iha ihb =>
    simp [subst, iha, ihb]
| fst p ih =>
    simp [subst, ih]
| snd p ih =>
    simp [subst, ih]
| refl a iha =>
    simp [subst, iha]

@[simp] theorem rename_liftSub ρ( : Ren m k) σ( : Sub n m) (i : Fin (n + 1)) :
  rename (liftRen ρ) (liftSub σ i) = liftSub (fun j => rename ρ σ( j)) i := by
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    calc
      rename (liftRen ρ) (liftSub σ j.succ)
        = rename (liftRen ρ) (rename wk σ( j)) := by rfl
    _ = rename (fun x => liftRen ρ (wk x)) σ( j) := by
      simp [rename_comp]
    _ = rename (fun x => wk ρ( x)) σ( j) := by
      exact rename_ext
        ρ( := fun x => liftRen ρ (wk x))
        ξ( := fun x => wk ρ( x))
        (by intro x; simp [wk, liftRen])
        σ( j)
    _ = rename wk (rename ρ σ( j)) := by
      simp [rename_comp]
    _ = liftSub (fun x => rename ρ σ( x)) j.succ := by rfl

theorem rename_subst :
  ∀ {n m k} ρ( : Ren m k) σ( : Sub n m) (t : PureTm n),
  rename ρ (subst σ t) = subst (fun i => rename ρ σ( i)) t := by
  intro n m k ρ σ t
  induction t generalizing m k ρ with
  | var i =>
    rfl
  | const c =>
    rfl
  | u0 =>
    rfl
  | u1 =>
    rfl
  | pi A B iha ihB =>
    simp [rename, subst, iha ρ( := ρ) σ( := σ)]
    calc
      rename (liftRen ρ) (subst (liftSub σ) B)
        = subst (fun i => rename (liftRen ρ) (liftSub σ i)) B := by
          simpa using ihB ρ( := liftRen ρ) σ( := liftSub σ)
    _ = subst (liftSub (fun i => rename ρ σ( i))) B := by
      apply subst_ext
      intro i

```



```

    exact rename_liftSub p σ i
| sigma A B ihA ihB =>
  simp [rename, subst, ihA ρ( := ρ) σ( := σ)]
calc
  rename (liftRen p) (subst (liftSub σ) B)
    = subst (fun i => rename (liftRen p) (liftSub σ i)) B := by
      simpa using ihB ρ( := liftRen p) σ( := liftSub σ)
  _ = subst (liftSub (fun i => rename p σ( i))) B := by
    apply subst_ext
    intro i
    exact rename_liftSub p σ i
| id A a b ihA iha ihb =>
  simp [rename, subst, ihA ρ( := ρ) σ( := σ), iha ρ( := ρ) σ( := σ),
    ihb ρ( := ρ) σ( := σ)]
| lam b ih =>
  simp [rename, subst]
calc
  rename (liftRen p) (subst (liftSub σ) b)
    = subst (fun i => rename (liftRen p) (liftSub σ i)) b := by
      simpa using ih ρ( := liftRen p) σ( := liftSub σ)
  _ = subst (liftSub (fun i => rename p σ( i))) b := by
    apply subst_ext
    intro i
    exact rename_liftSub p σ i
| app f a ihf iha =>
  simp [rename, subst, ihf ρ( := ρ) σ( := σ), iha ρ( := ρ) σ( := σ)]
| pair a b iha ihb =>
  simp [rename, subst, iha ρ( := ρ) σ( := σ), ihb ρ( := ρ) σ( := σ)]
| fst p ih =>
  simpa [rename, subst] using ih ρ( := ρ) σ( := σ)
| snd p ih =>
  simpa [rename, subst] using ih ρ( := ρ) σ( := σ)
| refl a iha =>
  simpa [rename, subst] using iha ρ( := ρ) σ( := σ)

@[simp] theorem liftSub_liftRen_apply σ( : Sub m k) ρ( : Ren n m) (i : Fin (n + 1)) :
  liftSub σ (liftRen p i) = liftSub (fun j => σ ρ( j)) i := by
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    rfl

theorem subst_rename :
  ∀ {n m k} σ( : Sub m k) ρ( : Ren n m) (t : PureTm n),
    subst σ (rename p t) = subst (fun i => σ ρ( i)) t := by
  intro n m k σ ρ t
  induction t generalizing m k σ with
  | var i =>
    rfl
  | const c =>
    rfl
  | u0 =>
    rfl
  | u1 =>

```

```

rfl
| pi A B ihA ihB =>
  simp [subst, rename, ihA σ( := σ) ρ( := ρ)]
  calc
    subst (liftSub σ) (rename (liftRen ρ) B)
      = subst (fun i => liftSub σ (liftRen ρ i)) B := by
        simpa using ihB σ( := liftSub σ) ρ( := liftRen ρ)
    _ = subst (liftSub (fun i => σ ρ( i))) B := by
      apply subst_ext
      intro i
      exact liftSub_liftRen_apply σ ρ i
| sigma A B ihA ihB =>
  simp [subst, rename, ihA σ( := σ) ρ( := ρ)]
  calc
    subst (liftSub σ) (rename (liftRen ρ) B)
      = subst (fun i => liftSub σ (liftRen ρ i)) B := by
        simpa using ihB σ( := liftSub σ) ρ( := liftRen ρ)
    _ = subst (liftSub (fun i => σ ρ( i))) B := by
      apply subst_ext
      intro i
      exact liftSub_liftRen_apply σ ρ i
| id A a b ihA iha ihb =>
  simp [subst, rename, ihA σ( := σ) ρ( := ρ), iha σ( := σ) ρ( := ρ),
    ihb σ( := σ) ρ( := ρ)]
| lam b ih =>
  simp [subst, rename]
  calc
    subst (liftSub σ) (rename (liftRen ρ) b)
      = subst (fun i => liftSub σ (liftRen ρ i)) b := by
        simpa using ih σ( := liftSub σ) ρ( := liftRen ρ)
    _ = subst (liftSub (fun i => σ ρ( i))) b := by
      apply subst_ext
      intro i
      exact liftSub_liftRen_apply σ ρ i
| app f a ihf iha =>
  simp [subst, rename, ihf σ( := σ) ρ( := ρ), iha σ( := σ) ρ( := ρ)]
| pair a b iha ihb =>
  simp [subst, rename, iha σ( := σ) ρ( := ρ), ihb σ( := σ) ρ( := ρ)]
| fst p ih =>
  simpa [subst, rename] using ih σ( := σ) ρ( := ρ)
| snd p ih =>
  simpa [subst, rename] using ih σ( := σ) ρ( := ρ)
| refl a iha =>
  simpa [subst, rename] using iha σ( := σ) ρ( := ρ)

@[simp] theorem subst_liftSub_wk σ( : Sub n m) (t : PureTm n) :
  subst (liftSub σ) (rename wk t) = rename wk (subst σ t) := by
  calc
    subst (liftSub σ) (rename wk t)
      = subst (fun i => liftSub σ (wk i)) t := by
        simpa using (subst_rename σ( := liftSub σ) ρ( := wk) (t := t))
    _ = subst (fun i => rename wk σ( i)) t := by
      rfl
    _ = rename wk (subst σ t) := by

```

```

      symm
      simp using (rename_subst p( := wk) σ( := σ) (t := t))

@[simp] theorem rename_inst0 p( : Ren n m) (a : PureTm n) (b : PureTm (n + 1)) :
  rename p (inst0 a b) = inst0 (rename p a) (rename (liftRen p) b) := by
calc
  rename p (inst0 a b)
    = subst (fun i => rename p (subst0 a i)) b := by
      simp [inst0] using
        (rename_subst p( := p) σ( := subst0 a) (t := b))
  _ = subst (fun i => subst0 (rename p a) (liftRen p i)) b := by
    apply subst_ext
    intro i
    refine Fin.cases ?_ ?_ i
    · rfl
    · intro j
      rfl
  _ = subst (subst0 (rename p a) (rename (liftRen p) b) := by
    symm
    simp using
      (subst_rename σ( := subst0 (rename p a)) p( := liftRen p) (t := b))
  _ = inst0 (rename p a) (rename (liftRen p) b) := by
    rfl

@[simp] theorem liftSub_comp_apply τ( : Sub m k) σ( : Sub n m) (i : Fin (n + 1)) :
  subst (liftSub τ) (liftSub σ i) = liftSub (fun x => subst τ σ( x)) i := by
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    calc
      subst (liftSub τ) (liftSub σ j.succ)
        = subst (liftSub τ) (rename wk σ( j)) := by rfl
      _ = subst (fun x => liftSub τ (wk x)) σ( j) := by
        simp using (subst_rename σ( := liftSub τ) p( := wk) (t := σ j))
      _ = subst (fun x => rename wk τ( x)) σ( j) := by rfl
      _ = rename wk (subst τ σ( j)) := by
        symm
        simp using (rename_subst p( := wk) σ( := τ) (t := σ j))
      _ = liftSub (fun x => subst τ σ( x)) j.succ := by rfl

@[simp] theorem subst_comp :
  ∀ {n m k} τ( : Sub m k) σ( : Sub n m) (t : PureTm n),
  subst τ (subst σ t) = subst (fun i => subst τ σ( i)) t := by
intro n m k τ σ t
induction t generalizing m k τ with
| var i =>
  rfl
| const c =>
  rfl
| u0 =>
  rfl
| u1 =>
  rfl
| pi A B ihA ihB =>

```

```

simp [subst, ihA  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
calc
  subst (liftSub  $\tau$ ) (subst (liftSub  $\sigma$ ) B)
    = subst (fun i => subst (liftSub  $\tau$ ) (liftSub  $\sigma$  i)) B := by
      simpa using ihB  $\tau$ ( := liftSub  $\tau$ )  $\sigma$ ( := liftSub  $\sigma$ )
  _ = subst (liftSub (fun i => subst  $\tau$   $\sigma$ ( i))) B := by
    apply subst_ext
    intro i
    exact liftSub_comp_apply  $\tau$   $\sigma$  i
| sigma A B ihA ihB =>
simp [subst, ihA  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
calc
  subst (liftSub  $\tau$ ) (subst (liftSub  $\sigma$ ) B)
    = subst (fun i => subst (liftSub  $\tau$ ) (liftSub  $\sigma$  i)) B := by
      simpa using ihB  $\tau$ ( := liftSub  $\tau$ )  $\sigma$ ( := liftSub  $\sigma$ )
  _ = subst (liftSub (fun i => subst  $\tau$   $\sigma$ ( i))) B := by
    apply subst_ext
    intro i
    exact liftSub_comp_apply  $\tau$   $\sigma$  i
| id A a b ihA iha ihb =>
simp [subst, ihA  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ ), iha  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ ),
  ihb  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
| lam b ih =>
simp [subst]
calc
  subst (liftSub  $\tau$ ) (subst (liftSub  $\sigma$ ) b)
    = subst (fun i => subst (liftSub  $\tau$ ) (liftSub  $\sigma$  i)) b := by
      simpa using ih  $\tau$ ( := liftSub  $\tau$ )  $\sigma$ ( := liftSub  $\sigma$ )
  _ = subst (liftSub (fun i => subst  $\tau$   $\sigma$ ( i))) b := by
    apply subst_ext
    intro i
    exact liftSub_comp_apply  $\tau$   $\sigma$  i
| app f a ihf iha =>
simp [subst, ihf  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ ), iha  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
| pair a b iha ihb =>
simp [subst, iha  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ ), ihb  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
| fst p ih =>
simp [subst, ih  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
| snd p ih =>
simp [subst, ih  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]
| refl a iha =>
simp [subst, iha  $\tau$ ( :=  $\tau$ )  $\sigma$ ( :=  $\sigma$ )]

@[simp] theorem subst_inst0  $\sigma$ ( : Sub n m) (a : PureTm n) (b : PureTm (n + 1)) :
  subst  $\sigma$  (inst0 a b) = inst0 (subst  $\sigma$  a) (subst (liftSub  $\sigma$ ) b) := by
calc
  subst  $\sigma$  (inst0 a b)
    = subst (fun i => subst  $\sigma$  (subst0 a i)) b := by
      simp [inst0, subst_comp]
  _ = subst (fun i => subst (subst0 (subst  $\sigma$  a)) (liftSub  $\sigma$  i)) b := by
    apply subst_ext
    intro i
    refine Fin.cases ?_ ?_ i
    · rfl

```

```

· intro j
  calc
    subst σ (subst0 a j.succ) = subst σ (.var j) := by
      rfl
    _ = σ j := by
      rfl
    _ = subst (subst0 (subst σ a)) (rename wk σ( j)) := by
      symm
    calc
      subst (subst0 (subst σ a)) (rename wk σ( j))
        = subst (fun i => subst0 (subst σ a) (wk i)) σ( j) := by
          simpa using
            (subst_rename σ( := subst0 (subst σ a)) ρ( := wk) (t := σ j))
      _ = subst ids σ( j) := by
        apply subst_ext
        intro i
        rfl
      _ = σ j := by
        exact subst_ids (t := σ j)
      _ = subst (subst0 (subst σ a)) (liftSub σ j.succ) := by
        rfl
    _ = subst (subst0 (subst σ a)) (subst (liftSub σ) b) := by
      symm
    exact
      (subst_comp τ( := subst0 (subst σ a)) σ( := liftSub σ) (t := b))
    _ = inst0 (subst σ a) (subst (liftSub σ) b) := by
      rfl
end Mettapedia.Languages.MeTTa.PureKernel.Substitution

```

A.24 Syntax.lean

Mettapedia/Languages/MeTTa/PureKernel/Syntax.lean

```

import Lean

namespace Mettapedia.Languages.MeTTa.PureKernel.Syntax

/-- Kernel-level global declaration name. -/
abbrev DeclName := Lean.Name

/-- Core MeTTa-Pure term syntax, scoped by de Bruijn depth. -/
inductive PureTm : Nat → Type where
| var : Fin n → PureTm n
| const : DeclName → PureTm n
| u0 : PureTm n
| u1 : PureTm n
| pi : PureTm n → PureTm (n + 1) → PureTm n
| sigma : PureTm n → PureTm (n + 1) → PureTm n
| id : PureTm n → PureTm n → PureTm n → PureTm n
| lam : PureTm (n + 1) → PureTm n
| app : PureTm n → PureTm n → PureTm n
| pair : PureTm n → PureTm n → PureTm n
| fst : PureTm n → PureTm n

```

```

| snd : PureTm n → PureTm n
| refl : PureTm n → PureTm n
deriving DecidableEq, Repr

end Mettapedia.Languages.MeTTa.PureKernel.Syntax

```

A.25 UnitDecl.lean

Mettapedia/Languages/MeTTa/PureKernel/UnitDecl.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

namespace Mettapedia.Languages.MeTTa.PureKernel.UnitDecl

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationEnv
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSemantics
open Mettapedia.Languages.MeTTa.PureKernel.DeclarationSpec

/-- Pilot family/type constant name for Unit in the declaration environment. -/
def unitTyName : DeclName := `Unit

/-- Pilot constructor constant name for Unit (Lean-style naming). -/
def unitCtorName : DeclName := `Unit.unit

/-- Pilot alias constant used to witness declaration-aware unfolding δ(`). -/
def unitAliasName : DeclName := `Unit.alias

/-- `Unit` family/type declaration spec. -/
def unitTySpec : DeclSpec :=
{ name := unitTyName, type := .u0 }

/-- `Unit.unit` constructor declaration spec. -/
def unitCtorSpec : DeclSpec :=
{ name := unitCtorName, type := .const unitTyName }

/-- `Unit.alias` declaration spec used for the first δ`` witness. -/
def unitAliasSpec : DeclSpec :=
{ name := unitAliasName, type := .const unitTyName, value? := some (.const unitCtorName) }

/-- `Unit` pilot declarations expressed through the shared scaffold. -/
def unitSpecs : List DeclSpec :=
[unitTySpec, unitCtorSpec, unitAliasSpec]

/-- Minimal declaration environment for the Unit pilot.
`Unit` and `Unit.unit` are declared; `Unit.alias` unfolds to `Unit.unit`. -/
def unitDeclEnv : DeclEnv := envOfSpecs unitSpecs

@[simp] theorem typeOf_unitTy : typeOf? unitDeclEnv unitTyName = some .u0 := by
  decide

@[simp] theorem typeOf_unitCtor :
  typeOf? unitDeclEnv unitCtorName = some (.const unitTyName) := by

```

```

decide

@[simp] theorem typeOf_unitAlias :
  typeOf? unitDeclEnv unitAliasName = some (.const unitTyName) := by
decide

@[simp] theorem valueOf_unitAlias :
  valueOf? unitDeclEnv unitAliasName = some (.const unitCtorName) := by
decide

/-- Typed declaration witness:  $\vdash`Unit : U0`.$  -/
theorem hasType_unitTy :
  HasTypeDecl unitDeclEnv .nil (.const unitTyName) .u0 :=
  hasType_const_from_lookup (E := unitDeclEnv)  $\Gamma$ ( := .nil) (c := unitTyName) (A0 := .u0) (by
    simp)

/-- Typed declaration witness:  $\vdash`Unit.unit : Unit`.$  -/
theorem hasType_unitCtor :
  HasTypeDecl unitDeclEnv .nil (.const unitCtorName) (.const unitTyName) :=
  hasType_const_from_lookup (E := unitDeclEnv)  $\Gamma$ ( := .nil) (c := unitCtorName) (A0 := .const
    unitTyName) (by
      simp)

/-- Typed declaration witness:  $\vdash`Unit.alias : Unit`.$  -/
theorem hasType_unitAlias :
  HasTypeDecl unitDeclEnv .nil (.const unitAliasName) (.const unitTyName) :=
  hasType_const_from_lookup (E := unitDeclEnv)  $\Gamma$ ( := .nil) (c := unitAliasName) (A0 := .const
    unitTyName) (by
      simp)

/-- Reduction witness:  $`Unit.alias`$  unfolds to  $`Unit.unit`.$  -/
theorem red_unitAlias_to_ctor :
  RedDecl unitDeclEnv
    ((.const unitAliasName : PureTm 0))
    ((.const unitCtorName : PureTm 0)) := by
  simpa using
    red_const_from_unfold0 (E := unitDeclEnv) (c := unitAliasName) (v := (.const unitCtorName)) (
      by
        simp)

/-- Small checked pilot package: typed source, one declaration-aware step, typed target. -/
theorem unitAlias_checked_step :
   $\exists A : \text{PureTm } 0,$ 
  HasTypeDecl unitDeclEnv .nil ((.const unitAliasName : PureTm 0)) A  $\wedge$ 
  RedDecl unitDeclEnv ((.const unitAliasName : PureTm 0)) ((.const unitCtorName : PureTm 0))
   $\wedge$ 
  HasTypeDecl unitDeclEnv .nil ((.const unitCtorName : PureTm 0)) A := by
  simpa using
    (checked_delta_step_closed_of_typed_value
      (E := unitDeclEnv)
      (c := unitAliasName)
      (A0 := (.const unitTyName))
      (v0 := (.const unitCtorName))
      (by simp))

```

```

    (by simp)
    (by simp using hasType_unitCtor))

/-- Reusable typed-value obligations for `unitSpecs` in any declaration
environment where `Unit.unit : Unit` is available. -/
theorem unitSpecs_hTyped_in
  (E : DeclEnv)
  (hCtorType : typeOf? E unitCtorName = some (.const unitTyName)) :
  ∀ s ∈ unitSpecs, ∀ v0 : PureTm 0,
  s.value? = some v0 →
  HasTypeDecl E .nil (liftClosed v0) (liftClosed s.type) := by
  intro s hs v0 hVal
  simp [unitSpecs, unitTySpec, unitCtorSpec, unitAliasSpec] at hs
  rcases hs with rfl | rfl | rfl
  · simp at hVal
  · simp at hVal
  · have hv : v0 = (.const unitCtorName) := by
      simp [unitAliasSpec] using hVal.symm
    subst hv
    simp [liftClosed_zero] using
      (hasType_const_from_lookup
        (E := E)
        Γ( := .nil)
        (c := unitCtorName)
        (A0 := (.const unitTyName))
        hCtorType))

/-- Reusable no-self-unfolding obligations for `unitSpecs`. -/
theorem unitSpecs_hNoSelf :
  ∀ s ∈ unitSpecs, ∀ v0 : PureTm 0,
  s.value? = some v0 →
  v0 ≠ (.const s.name) := by
  intro s hs v0 hVal
  simp [unitSpecs, unitTySpec, unitCtorSpec, unitAliasSpec] at hs
  rcases hs with rfl | rfl | rfl
  · simp at hVal
  · simp at hVal
  · have hv : v0 = (.const unitCtorName) := by
      simp [unitAliasSpec] using hVal.symm
    subst hv
    intro hEq
    have hNe : ((.const unitCtorName : PureTm 0)) ≠ (.const unitAliasName) := by
      decide
    exact hNe hEq

def unitSpecs_obligations : DeclSpecObligations unitSpecs where
  valuesWellTyped :=
    unitSpecs_hTyped_in
    (E := envOfSpecs unitSpecs)
    (hCtorType := by decide)
  noSelfDelta := unitSpecs_hNoSelf

def unitSpecs_signatureWellFormed : SignatureWellFormed unitSpecs where
  noShadowing := by decide

```



```

obligations := unitSpecs_obligations

theorem unitTySpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [] unitTySpec where
  fresh := by simp [unitTySpec]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames, unitTySpec]
  valueUsesEarlier := by
    intro v0 hVal
    simp [unitTySpec] at hVal
  valueWellTyped := by
    intro v0 hVal
    simp [unitTySpec] at hVal
  noSelfDelta := by
    intro v0 hVal
    simp [unitTySpec] at hVal

theorem unitCtorSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [unitTySpec] unitCtorSpec where
  fresh := by simp [unitTySpec, unitCtorSpec, unitTyName, unitCtorName]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames, unitTySpec, unitCtorSpec, unitTyName, unitCtorName]
  valueUsesEarlier := by
    intro v0 hVal
    simp [unitCtorSpec] at hVal
  valueWellTyped := by
    intro v0 hVal
    simp [unitCtorSpec] at hVal
  noSelfDelta := by
    intro v0 hVal
    simp [unitCtorSpec] at hVal

theorem unitAliasSpec_prefixAdmissible :
  PrefixDeclSpecAdmissible [unitTySpec, unitCtorSpec] unitAliasSpec where
  fresh := by
    simp [unitTySpec, unitCtorSpec, unitAliasSpec, unitTyName, unitCtorName, unitAliasName]
  typeUsesEarlier := by
    simp [UsesOnlyDeclNamesFrom, prefixNames,
      unitTySpec, unitCtorSpec, unitAliasSpec, unitTyName, unitCtorName, unitAliasName]
  valueUsesEarlier := by
    intro v0 hVal
    have hv : v0 = (.const unitCtorName) := by
      simpa [unitAliasSpec] using hVal.symm
    subst hv
    simp [UsesOnlyDeclNamesFrom, prefixNames, unitTySpec, unitCtorSpec, unitCtorName]
  valueWellTyped := by
    intro v0 hVal
    have hv : v0 = (.const unitCtorName) := by
      simpa [unitAliasSpec] using hVal.symm
    subst hv
    have hLookup :
      typeOf? (envOfSpecs [unitTySpec, unitCtorSpec]) unitCtorName = some (.const unitTyName)
    :=
      typeOf_envOfSpecs_eq_of_mem_of_nodup

```

```

    (specs := [unitTySpec, unitCtorSpec])
    (s := unitCtorSpec)
    (hNodup := by decide)
    (hs := by simp [unitTySpec, unitCtorSpec])
  simpa [liftClosed_zero] using
    (hasType_const_from_lookup
      (E := envOfSpecs [unitTySpec, unitCtorSpec])
      Γ( := .nil)
      (c := unitCtorName)
      (A0 := (.const unitTyName))
      hLookup)
  noSelfDelta := by
    intro v0 hVal
    have hv : v0 = (.const unitCtorName) := by
      simpa [unitAliasSpec] using hVal.symm
    subst hv
    intro hEq
    have hNe : ((.const unitCtorName : PureTm 0)) ≠ (.const unitAliasName) := by
      decide
    exact hNe hEq

theorem unitSpecs_prefixWellFormed :
  SignatureWellFormedPrefix unitSpecs := by
  refine And.intro unitTySpec_prefixAdmissible ?_
  refine And.intro ?_ ?_
  · simpa [unitSpecs, unitTySpec, unitCtorSpec]
    using unitCtorSpec_prefixAdmissible
  · refine And.intro ?_ trivial
    simpa [PrefixSignatureWellFormed, unitSpecs, unitTySpec, unitCtorSpec, unitAliasSpec]
    using unitAliasSpec_prefixAdmissible

theorem unitSpecs_signatureWellFormed_fromPrefix :
  SignatureWellFormed unitSpecs :=
  SignatureWellFormed.ofPrefix unitSpecs_prefixWellFormed unitSpecs_obligations

theorem unitDeclEnv_wellFormed : DeclEnvWellFormed unitDeclEnv := by
  unfold unitDeclEnv
  exact unitSpecs_signatureWellFormed.toDeclEnvWellFormed

end Mettapedia.Languages.MeTTa.PureKernel.UnitDecl

```

A.26 TypedLangDef.lean

Mettapedia/Languages/MeTTa/PureKernel/TypedLangDef.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.SubjectReduction

namespace Mettapedia.Languages.MeTTa.PureKernel.Assembly

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.Reduction
open Mettapedia.Languages.MeTTa.PureKernel.Typing
open Mettapedia.Languages.MeTTa.PureKernel.SubjectReduction

```

```

/-- Kernel-level typed language bundle over scoped terms (`PureTm`). -/
structure TypedKernelDef where
  hasType : ∀ {n}, Ctx n → PureTm n → PureTm n → Prop
  reduces : ∀ {n}, PureTm n → PureTm n → Prop
  subject_reduction :
    ∀ {n} Γ{ : Ctx n} {t t' A : PureTm n},
      hasType Γ t A → reduces t t' → hasType Γ t' A

/-- MeTTa-Pure kernel packaged as a typed language definition. -/
def mettaPureKernelTyped : TypedKernelDef where
  hasType := @HasType
  reduces := @Red
  subject_reduction := by
    intro n Γ t t' A ht hr
    exact subject_reduction Γ( := Γ) (t := t) (t' := t') (A := A) ht hr

theorem mettaPureKernel_subject_reduction
  {n : Nat} Γ{ : Ctx n} {t t' A : PureTm n}
  (ht : mettaPureKernelTyped.hasType Γ t A)
  (hr : mettaPureKernelTyped.reduces t t') :
  mettaPureKernelTyped.hasType Γ t' A :=
  mettaPureKernelTyped.subject_reduction ht hr

end Mettapedia.Languages.MeTTa.PureKernel.Assembly

```

A.27 Typing.lean

Mettapedia/Languages/MeTTa/PureKernel/Typing.lean

```

import Mettapedia.Languages.MeTTa.PureKernel.Context
import Mettapedia.Languages.MeTTa.PureKernel.Reduction

namespace Mettapedia.Languages.MeTTa.PureKernel.Typing

open Mettapedia.Languages.MeTTa.PureKernel.Syntax
open Mettapedia.Languages.MeTTa.PureKernel.Renaming
open Mettapedia.Languages.MeTTa.PureKernel.Context
open Mettapedia.Languages.MeTTa.PureKernel.Substitution
open Mettapedia.Languages.MeTTa.PureKernel.Reduction

abbrev Conv (t u : PureTm n) : Prop := Relation.EqvGen (@Red n) t u

inductive HasType : Ctx n → PureTm n → PureTm n → Prop where
| u0_type Γ( : Ctx n) :
  HasType Γ .u0 .u1
| var Γ{ : Ctx n} (i : Fin n) :
  HasType Γ (.var i) (lookup Γ i)
| pi_form Γ{ : Ctx n} {A : PureTm n} {B : PureTm (n + 1)} :
  HasType Γ A .u1 →
  HasType (.snoc Γ A) B .u1 →
  HasType Γ (.pi A B) .u1
| sigma_form Γ{ : Ctx n} {A : PureTm n} {B : PureTm (n + 1)} :
  HasType Γ A .u1 →

```

```

      HasType (.snoc  $\Gamma$  A) B .u1 →
      HasType  $\Gamma$  (.sigma A B) .u1
| lam_intro  $\Gamma\{ : \text{Ctx } n\}$  {A : PureTm n} {body B : PureTm (n + 1)} :
  HasType (.snoc  $\Gamma$  A) body B →
  HasType  $\Gamma$  (.lam body) (.pi A B)
| app_elim  $\Gamma\{ : \text{Ctx } n\}$  {f a A : PureTm n} {B : PureTm (n + 1)} :
  HasType  $\Gamma$  f (.pi A B) →
  HasType  $\Gamma$  a A →
  HasType  $\Gamma$  (.app f a) (inst0 a B)
| pair_intro  $\Gamma\{ : \text{Ctx } n\}$  {a b A : PureTm n} {B : PureTm (n + 1)} :
  HasType  $\Gamma$  a A →
  HasType  $\Gamma$  b (inst0 a B) →
  HasType  $\Gamma$  (.pair a b) (.sigma A B)
| fst_elim  $\Gamma\{ : \text{Ctx } n\}$  {p A : PureTm n} {B : PureTm (n + 1)} :
  HasType  $\Gamma$  p (.sigma A B) →
  HasType  $\Gamma$  (.fst p) A
| snd_elim  $\Gamma\{ : \text{Ctx } n\}$  {p A : PureTm n} {B : PureTm (n + 1)} :
  HasType  $\Gamma$  p (.sigma A B) →
  HasType  $\Gamma$  (.snd p) (inst0 (.fst p) B)
| id_form  $\Gamma\{ : \text{Ctx } n\}$  {A a b : PureTm n} :
  HasType  $\Gamma$  A .u1 →
  HasType  $\Gamma$  a A →
  HasType  $\Gamma$  b A →
  HasType  $\Gamma$  (.id A a b) .u1
| refl_intro  $\Gamma\{ : \text{Ctx } n\}$  {a A : PureTm n} :
  HasType  $\Gamma$  a A →
  HasType  $\Gamma$  (.refl a) (.id A a a)
| conv  $\Gamma\{ : \text{Ctx } n\}$  {t A B : PureTm n} :
  HasType  $\Gamma$  t A →
  Conv A B →
  HasType  $\Gamma$  t B

theorem conv_refl (t : PureTm n) : Conv t t :=
  Relation.EqvGen.refl t

theorem red_implies_conv {t u : PureTm n} (h : Red t u) : Conv t u :=
  Relation.EqvGen.rel t u h

def renToSub p( : Ren n m) : Sub n m := fun i => .var p( i)

@[simp] theorem liftSub_renToSub p( : Ren n m) :
  liftSub (renToSub p) = renToSub (liftRen p) := by
  funext i
  refine Fin.cases ?_ ?_ i
  · rfl
  · intro j
    rfl

@[simp] theorem subst_renToSub p( : Ren n m) (t : PureTm n) :
  subst (renToSub p) t = rename p t := by
  induction t generalizing m with
  | var i =>
    rfl
  | const c =>

```

```

    rfl
| u0 =>
    rfl
| u1 =>
    rfl
| pi A B ihA ihB =>
    simp [subst, rename, ihA, ihB, liftSub_renToSub]
| sigma A B ihA ihB =>
    simp [subst, rename, ihA, ihB, liftSub_renToSub]
| id A a b ihA iha ihb =>
    simp [subst, rename, ihA, iha, ihb]
| lam b ih =>
    simp [subst, rename, ih, liftSub_renToSub]
| app f a ihf iha =>
    simp [subst, rename, ihf, iha]
| pair a b iha ihb =>
    simp [subst, rename, iha, ihb]
| fst p ih =>
    simp [subst, rename, ih]
| snd p ih =>
    simp [subst, rename, ih]
| refl a iha =>
    simp [subst, rename, iha]

theorem red_rename {t u : PureTm n} (h : Red t u) :
  ∀ {m : Nat} p( : Ren n m), Red (rename p t) (rename p u) := by
  induction h with
| betaPi body a =>
    intro m p
    simp [rename, rename_inst0] using
      (Red.betaPi (rename (liftRen p) body) (rename p a))
| betaSigmaFst a b =>
    intro m p
    simp [rename] using (Red.betaSigmaFst (rename p a) (rename p b))
| betaSigmaSnd a b =>
    intro m p
    simp [rename] using (Red.betaSigmaSnd (rename p a) (rename p b))
| congPiDom hA ih =>
    intro m p
    exact .congPiDom (ih p)
| congPiCod hB ih =>
    intro m p
    exact .congPiCod (ih (liftRen p))
| congSigmaDom hA ih =>
    intro m p
    exact .congSigmaDom (ih p)
| congSigmaCod hB ih =>
    intro m p
    exact .congSigmaCod (ih (liftRen p))
| congIdTy hA ih =>
    intro m p
    exact .congIdTy (ih p)
| congIdLeft ha ih =>
    intro m p

```

```

    exact .congIdLeft (ih ρ)
| congIdRight hb ih =>
    intro m ρ
    exact .congIdRight (ih ρ)
| congLam hb ih =>
    intro m ρ
    exact .congLam (ih (liftRen ρ))
| congAppFun hf ih =>
    intro m ρ
    exact .congAppFun (ih ρ)
| congAppArg ha ih =>
    intro m ρ
    exact .congAppArg (ih ρ)
| congPairFst ha ih =>
    intro m ρ
    exact .congPairFst (ih ρ)
| congPairSnd hb ih =>
    intro m ρ
    exact .congPairSnd (ih ρ)
| congFst hp ih =>
    intro m ρ
    exact .congFst (ih ρ)
| congSnd hp ih =>
    intro m ρ
    exact .congSnd (ih ρ)
| congRefl ha ih =>
    intro m ρ
    exact .congRefl (ih ρ)

theorem conv_rename ρ( : Ren n m) {t u : PureTm n} (h : Conv t u) :
  Conv (rename ρ t) (rename ρ u) := by
  induction h with
  | rel x y hred =>
    exact .rel _ _ (red_rename hred ρ)
  | refl x =>
    exact .refl _
  | symm x y hxy ih =>
    exact .symm _ _ ih
  | trans x y z hxy hyz ihxy ihyz =>
    exact .trans _ _ _ ihxy ihyz

theorem typing_rename Γ{ : Ctx n} {t A : PureTm n} (ht : HasType Γ t A) :
  ∀ {m : Nat} Δ{ : Ctx m} ρ{ : Ren n m},
  CtxRen Γ Δ ρ → HasType Δ (rename ρ t) (rename ρ A) := by
  induction ht with
  | u0_type Γ =>
    intro m Δ ρ ph
    simpa [rename] using (HasType.u0_type Γ( := Δ))
  | var i =>
    intro m Δ ρ ph
    simpa [ph i] using (HasType.var Γ( := Δ) (i := ρ i))
  | @pi_form n Γ A B hA hB ihA ihB =>
    intro m Δ ρ ph
    have hA' := ihA (m := m) Δ( := Δ) ρ( := ρ) ph

```

```

    have hB' := ihB (m := m + 1) Δ( := .snoc Δ (rename p A)) p( := liftRen p)
      (CtxRen.snoc ph A)
    simp [rename] using (HasType.pi_form hA' hB')
| @sigma_form n Γ A B hA hB ihA ihB =>
  intro m Δ p ph
  have hA' := ihA (m := m) Δ( := Δ) p( := p) ph
  have hB' := ihB (m := m + 1) Δ( := .snoc Δ (rename p A)) p( := liftRen p)
    (CtxRen.snoc ph A)
  simp [rename] using (HasType.sigma_form hA' hB')
| @lam_intro n Γ A body B hBody ihBody =>
  intro m Δ p ph
  have hBody' := ihBody (m := m + 1) Δ( := .snoc Δ (rename p A)) p( := liftRen p)
    (CtxRen.snoc ph A)
  simp [rename] using (HasType.lam_intro hBody')
| @app_elim n Γ f a A B hf ha ihf iha =>
  intro m Δ p ph
  simp [rename, rename_inst0] using
    (HasType.app_elim
      (ihf (m := m) Δ( := Δ) p( := p) ph)
      (iha (m := m) Δ( := Δ) p( := p) ph))
| @pair_intro n Γ a b A B ha hb iha ihb =>
  intro m Δ p ph
  have ha' := iha (m := m) Δ( := Δ) p( := p) ph
  have hb' : HasType Δ (rename p b) (inst0 (rename p a) (rename (liftRen p) B)) := by
    simp [rename_inst0] using (ihb (m := m) Δ( := Δ) p( := p) ph)
  simp [rename] using (HasType.pair_intro ha' hb')
| @fst_elim n Γ p A B hp ihp =>
  intro m Δ p ph
  simp [rename] using
    (HasType.fst_elim (ihp (m := m) Δ( := Δ) p( := p) ph))
| @snd_elim n Γ p A B hp ihp =>
  intro m Δ p ph
  simp [rename, rename_inst0] using
    (HasType.snd_elim (ihp (m := m) Δ( := Δ) p( := p) ph))
| @id_form n Γ A a b hA ha hb ihA iha ihb =>
  intro m Δ p ph
  simp [rename] using
    (HasType.id_form
      (ihA (m := m) Δ( := Δ) p( := p) ph)
      (iha (m := m) Δ( := Δ) p( := p) ph)
      (ihb (m := m) Δ( := Δ) p( := p) ph))
| @refl_intro n Γ a A ha iha =>
  intro m Δ p ph
  simp [rename] using
    (HasType.refl_intro (iha (m := m) Δ( := Δ) p( := p) ph))
| @conv n Γ t A B ht hAB iht =>
  intro m Δ p ph
  exact HasType.conv
    (iht (m := m) Δ( := Δ) p( := p) ph)
    (conv_rename p hAB)

/-- Weakening by one binder. -/
theorem weakening Γ{ : Ctx n} {t A U : PureTm n}
  (ht : HasType Γ t A) :

```

```

    HasType (.snoc  $\Gamma$  U) (rename wk t) (rename wk A) := by
have hwk : CtxRen  $\Gamma$  (.snoc  $\Gamma$  U) wk := by
  intro i
  simp [wk]
simpa using (typing_rename ht (m := n + 1)  $\Delta$ ( := .snoc  $\Gamma$  U)  $\rho$ ( := wk) hwk)

/-- Typed context morphism via simultaneous substitution. -/
def CtxMor  $\Gamma$ ( : Ctx n)  $\Delta$ ( : Ctx m)  $\sigma$ ( : Sub n m) : Prop :=
   $\forall$  i : Fin n, HasType  $\Delta$   $\sigma$ ( i) (subst  $\sigma$  (lookup  $\Gamma$  i))

theorem CtxMor.lift  $\Gamma\{ : Ctx n\}$   $\Delta\{ : Ctx m\}$   $\sigma\{ : Sub n m\}$ 
  (oh : CtxMor  $\Gamma$   $\Delta$   $\sigma$ ) (A : PureTm n) :
  CtxMor (.snoc  $\Gamma$  A) (.snoc  $\Delta$  (subst  $\sigma$  A)) (liftSub  $\sigma$ ) := by
intro i
refine Fin.cases ?_ ?_ i
· simpa [CtxMor, lookup_snoc_zero, liftSub, subst_liftSub_wk] using
  (HasType.var  $\Gamma$ ( := .snoc  $\Delta$  (subst  $\sigma$  A)) (i := (0 : Fin (m + 1))))
· intro j
  have hj : HasType (.snoc  $\Delta$  (subst  $\sigma$  A))
    (rename wk  $\sigma$ ( j))
    (rename wk (subst  $\sigma$  (lookup  $\Gamma$  j))) := by
    simpa using (weakening  $\Gamma$ ( :=  $\Delta$ ) (U := subst  $\sigma$  A) (ht := oh j))
  simpa [CtxMor, lookup_snoc_succ, liftSub, subst_liftSub_wk] using hj

theorem red_subst {t u : PureTm n} (h : Red t u) :
   $\forall$  {m : Nat}  $\sigma$ ( : Sub n m), Red (subst  $\sigma$  t) (subst  $\sigma$  u) := by
induction h with
| betaPi body a =>
  intro m  $\sigma$ 
  simpa [subst, subst_inst0] using
    (Red.betaPi (subst (liftSub  $\sigma$ ) body) (subst  $\sigma$  a))
| betaSigmaFst a b =>
  intro m  $\sigma$ 
  simpa [subst] using (Red.betaSigmaFst (subst  $\sigma$  a) (subst  $\sigma$  b))
| betaSigmaSnd a b =>
  intro m  $\sigma$ 
  simpa [subst] using (Red.betaSigmaSnd (subst  $\sigma$  a) (subst  $\sigma$  b))
| congPiDom hA ih =>
  intro m  $\sigma$ 
  exact .congPiDom (ih  $\sigma$ )
| congPiCod hB ih =>
  intro m  $\sigma$ 
  exact .congPiCod (ih (liftSub  $\sigma$ ))
| congSigmaDom hA ih =>
  intro m  $\sigma$ 
  exact .congSigmaDom (ih  $\sigma$ )
| congSigmaCod hB ih =>
  intro m  $\sigma$ 
  exact .congSigmaCod (ih (liftSub  $\sigma$ ))
| congIdTy hA ih =>
  intro m  $\sigma$ 
  exact .congIdTy (ih  $\sigma$ )
| congIdLeft ha ih =>
  intro m  $\sigma$ 

```



```

    exact .congIdLeft (ih σ)
| congIdRight hb ih =>
    intro m σ
    exact .congIdRight (ih σ)
| congLam hb ih =>
    intro m σ
    exact .congLam (ih (liftSub σ))
| congAppFun hf ih =>
    intro m σ
    exact .congAppFun (ih σ)
| congAppArg ha ih =>
    intro m σ
    exact .congAppArg (ih σ)
| congPairFst ha ih =>
    intro m σ
    exact .congPairFst (ih σ)
| congPairSnd hb ih =>
    intro m σ
    exact .congPairSnd (ih σ)
| congFst hp ih =>
    intro m σ
    exact .congFst (ih σ)
| congSnd hp ih =>
    intro m σ
    exact .congSnd (ih σ)
| congRefl ha ih =>
    intro m σ
    exact .congRefl (ih σ)

theorem conv_subst σ( : Sub n m) {t u : PureTm n} (h : Conv t u) :
  Conv (subst σ t) (subst σ u) := by
  induction h with
| rel x y hred =>
    exact .rel _ _ (red_subst hred σ)
| refl x =>
    exact .refl _
| symm x y hxy ih =>
    exact .symm _ _ ih
| trans x y z hxy hyz ihxy ihyz =>
    exact .trans _ _ _ ihxy ihyz

/-- Generic simultaneous typing substitution along a typed context morphism. -/
theorem typing_subst Γ{ : Ctx n} {t A : PureTm n} (ht : HasType Γ t A) :
  ∀ {m : Nat} Δ{ : Ctx m} σ{ : Sub n m},
  CtxMor Γ Δ σ → HasType Δ (subst σ t) (subst σ A) := by
  induction ht with
| u0_type Γ =>
    intro m Δ σ σh
    simp [subst] using (HasType.u0_type Γ( := Δ))
| var i =>
    intro m Δ σ σh
    simp [CtxMor] using σh i
| @pi_form n Γ A B hA hB ihA ihB =>
    intro m Δ σ σh

```

```

have hA' := ihA (m := m) Δ( := Δ) σ( := σ) oh
have hB' := ihB (m := m + 1) Δ( := .snoc Δ (subst σ A)) σ( := liftSub σ)
  (CtxMor.lift oh A)
simpa [subst] using (HasType.pi_form hA' hB')
| @sigma_form n Γ A B hA hB ihA ihB =>
  intro m Δ σ oh
  have hA' := ihA (m := m) Δ( := Δ) σ( := σ) oh
  have hB' := ihB (m := m + 1) Δ( := .snoc Δ (subst σ A)) σ( := liftSub σ)
    (CtxMor.lift oh A)
  simpa [subst] using (HasType.sigma_form hA' hB')
| @lam_intro n Γ A body B hBody ihBody =>
  intro m Δ σ oh
  have hBody' := ihBody (m := m + 1) Δ( := .snoc Δ (subst σ A)) σ( := liftSub σ)
    (CtxMor.lift oh A)
  simpa [subst] using (HasType.lam_intro hBody')
| @app_elim n Γ f a A B hf ha ihf iha =>
  intro m Δ σ oh
  simpa [subst, subst_inst0] using
    (HasType.app_elim
      (ihf (m := m) Δ( := Δ) σ( := σ) oh)
      (iha (m := m) Δ( := Δ) σ( := σ) oh))
| @pair_intro n Γ a b A B ha hb iha ihb =>
  intro m Δ σ oh
  have ha' := iha (m := m) Δ( := Δ) σ( := σ) oh
  have hb' : HasType Δ (subst σ b) (inst0 (subst σ a) (subst (liftSub σ) B)) := by
    simpa [subst_inst0] using (ihb (m := m) Δ( := Δ) σ( := σ) oh)
  simpa [subst] using (HasType.pair_intro ha' hb')
| @fst_elim n Γ p A B hp ihp =>
  intro m Δ σ oh
  simpa [subst] using
    (HasType.fst_elim (ihp (m := m) Δ( := Δ) σ( := σ) oh))
| @snd_elim n Γ p A B hp ihp =>
  intro m Δ σ oh
  simpa [subst, subst_inst0] using
    (HasType.snd_elim (ihp (m := m) Δ( := Δ) σ( := σ) oh))
| @id_form n Γ A a b hA ha hb ihA iha ihb =>
  intro m Δ σ oh
  simpa [subst] using
    (HasType.id_form
      (ihA (m := m) Δ( := Δ) σ( := σ) oh)
      (iha (m := m) Δ( := Δ) σ( := σ) oh)
      (ihb (m := m) Δ( := Δ) σ( := σ) oh))
| @refl_intro n Γ a A ha iha =>
  intro m Δ σ oh
  simpa [subst] using
    (HasType.refl_intro (iha (m := m) Δ( := Δ) σ( := σ) oh))
| @conv n Γ t A B ht hAB iht =>
  intro m Δ σ oh
  exact HasType.conv
    (iht (m := m) Δ( := Δ) σ( := σ) oh)
    (conv_subst σ hAB)

```

/-- Transport typing along pointwise context conversion. -/

theorem context_conv Γ{ Δ : Ctx n} {t T : PureTm n}

```

(hctx : ∀ i : Fin n, Conv (lookup Γ i) (lookup Δ i))
(ht : HasType Γ t T) :
  HasType Δ t T := by
induction ht with
| u0_type Γ =>
  exact .u0_type _
| var i =>
  exact .conv
    (HasType.var Γ( := Δ) (i := i))
    (Relation.EqvGen.symm _ _ (hctx i))
| @pi_form n Γ A B hA hB ihA ihB =>
  have hA' : HasType Δ A .u1 := ihA hctx
  have hctx' : ∀ i : Fin (n + 1), Conv (lookup (.snoc Γ A) i) (lookup (.snoc Δ A) i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact conv_refl _
    · intro j
      simp [lookup_snoc_succ] using (conv_rename p( := wk) (hctx j))
  have hB' : HasType (.snoc Δ A) B .u1 := ihB hctx'
  exact .pi_form hA' hB'
| @sigma_form n Γ A B hA hB ihA ihB =>
  have hA' : HasType Δ A .u1 := ihA hctx
  have hctx' : ∀ i : Fin (n + 1), Conv (lookup (.snoc Γ A) i) (lookup (.snoc Δ A) i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact conv_refl _
    · intro j
      simp [lookup_snoc_succ] using (conv_rename p( := wk) (hctx j))
  have hB' : HasType (.snoc Δ A) B .u1 := ihB hctx'
  exact .sigma_form hA' hB'
| @lam_intro n Γ A body B hBody ihBody =>
  have hctx' : ∀ i : Fin (n + 1), Conv (lookup (.snoc Γ A) i) (lookup (.snoc Δ A) i) := by
    intro i
    refine Fin.cases ?_ ?_ i
    · exact conv_refl _
    · intro j
      simp [lookup_snoc_succ] using (conv_rename p( := wk) (hctx j))
  exact .lam_intro (ihBody hctx')
| @app_elim n Γ f a A B hf ha ihf iha =>
  exact .app_elim (ihf hctx) (iha hctx)
| @pair_intro n Γ a b A B ha hb iha ihb =>
  exact .pair_intro (iha hctx) (ihb hctx)
| @fst_elim n Γ p A B hp ihp =>
  exact .fst_elim (ihp hctx)
| @snd_elim n Γ p A B hp ihp =>
  exact .snd_elim (ihp hctx)
| @id_form n Γ A a b hA ha hb ihA iha ihb =>
  exact .id_form (ihA hctx) (iha hctx) (ihb hctx)
| @refl_intro n Γ a A ha iha =>
  exact .refl_intro (iha hctx)
| @conv n Γ t A B ht hAB iht =>
  exact .conv (iht hctx) hAB

```

/-- Specialization of `context\_conv` to changing only the head binder type. -/

```

theorem context_conv_head  $\Gamma\{ : \text{Ctx } n\} \{A \ A' : \text{PureTm } n\} \{t \ T : \text{PureTm } (n + 1)\}$ 
  (hAA' : Conv A A') (ht : HasType (.snoc  $\Gamma$  A) t T) :
  HasType (.snoc  $\Gamma$  A') t T := by
have hctx :  $\forall i : \text{Fin } (n + 1),$ 
  Conv (lookup (.snoc  $\Gamma$  A) i) (lookup (.snoc  $\Gamma$  A') i) := by
intro i
  refine Fin.cases ?_ ?_ i
  · simpa [lookup_snoc_zero] using (conv_rename p( := wk) hAA')
  · intro j
    simpa [lookup_snoc_succ] using (conv_refl (rename wk (lookup  $\Gamma$  j)))
exact context_conv hctx ht

end Mettapedia.Languages.MeTTa.PureKernel.Typing

```